Catálogo de ideias

Gatilho → ideia → técnica → código, para programação competitiva e ICPC

Partes: mapa rápido · catálogo completo · índice da lib
Blocos marcados "adicionado … não revisado" foram gerados automaticamente e ainda não foram conferidos.

Sumário

Mapa rápido
- Reduções
- Limites escondidos
- Por tópico

Catálogo completo
- Como usar quando travar
- Leia as constraints
- Reformulações universais
- Cadeias de pulos (gatilho → observação → técnica)
- Arrays e guloso
- Busca binária e otimização
- DP
- Teoria dos números e divisores
- Combinatória, contagem e probabilidade
- Bits e XOR
- Strings
- Grafos
- Árvores
- Estruturas de dados e queries
- Construtivos e interativos
- Teoria dos jogos
- Geometria
- Aleatorização
- Álgebra, polinômios e FFT
- Matemática de olimpíada e math.stackexchange
- Otimização, programação linear e matroides
- Amostras de problemas e padrões de ICPC
- Quando nada funciona

Mapa rápido

Uma linha por salto: gatilho → técnica. O porquê está em `catalogo-completo.md`.

Reduções

Universais

- $\Sigma_i \Sigma_j$ difícil → troque a ordem dos fors; contribuição de cada elemento
- $\Sigma a_i \to \Sigma_t \#\{a_i \geq t\}$ (fatiar por camadas); $E[X] = \Sigma P(X \geq k)$
- "Pelo menos um" → complemento; vários defeitos → inclusão-exclusão
- Remoções → adições ao contrário (DSU)
- Min do max / max do min → binária na resposta
- "Exatamente k" com custo convexo → Aliens (λ)
- Operação reversível → forma canônica dos dois lados
- "Sempre existe" → pombal / extremo / invariante viram a construção
- Min difícil → dual (corte = fluxo, König, Dilworth)

Arrays e guloso

- Soma = 0 → prefixos iguais; média ≥ k → subtraia k; mediana ≥ x → ±1
- Op. em intervalo → 2 pontos no array de diferenças
- Ordem ótima de tarefas → exchange argument (t/w, prazo); jobs com prazo e lucro → heap por prazo; código de Huffman → heap
- Estritamente crescente → $a_i - i$ não decrescente
- Minimizar $\Sigma|a_i - b_i|$ com b monótono → slope trick (heap)
- Escolher k não adjacentes → heap + lista ligada (arrependimento)
- Trocas em permutação → ciclos (n − #ciclos) / inversões (adjacentes)
- Pares $i < j$ com condição → ordene + BIT; 3 dimensões → CDQ
- Cíclico → duplique; $n \leq 40$ → meet in the middle
- Cobertura mínima por crescentes → maior não crescente (Dilworth)

Busca

- k-ésimo de conjunto implícito → binária + contar ≤ x
- Razão máxima $\Sigma a/\Sigma b$ → binária em r ou Dinkelbach
- Primeiro índice com prefixo ≥ x → descida em BIT/segtree
- Primeiro instante em que conecta → Kruskal reconstruction ou binária paralela
- Binária sobre racionais / fração mais simples → Stern–Brocot

DP

- Valor grande, resposta pequena → troque estado e valor (dp[resposta] = melhor parâmetro)
- $dp[i] = \min_j (dp[j] + (S_i - S_j)^2)$ → CHT / Li Chao
- opt monótono (Monge) → D&C DP; intervalo → Knuth
- Transição linear com n enorme → matriz / Kitamasa; recorrência desconhecida → Berlekamp–Massey
- dp(n) polinomial em n → Lagrange com d+1 pontos
- Árvore + escolher k → knapsack $O(n^2)$ por tamanho ou ordem DFS pegar/pular
- Estado = profundidade em árvore → long-path decomposition
- Permutação com restrição de adjacência → connected components DP / insertion DP
- Esperança com ciclos → sistema linear ou $E[i] = a_i \cdot E[0] + b_i$
- Grid estreito → broken profile; com conectividade → plug DP; $n \leq 20$ → máscara / SOS / IE + SOS
- Custos ≥ 0 com ciclos nos estados → Dijkstra em vez de DP
- Mochila com remoção ou itens com validade → dp inversa (modular) ou segtree no tempo

Números

- Pares com gcd = g, $\Sigma \gcd(i, j)$ → Möbius/φ + blocos de $\lfloor n/d \rfloor$
- Soma de multiplicativa até 10^{11} → Dirichlet sieve / Lucy / Min_25
- $\Sigma \lfloor (ai+b)/m \rfloor$, pontos sob reta → floor sum
- Congruências → CRT (não coprimo: g | diferença); $a^x \equiv b$ → BSGS (se $\gcd(a,m) > 1$, exBSGS); $x^2 \equiv a$ → Tonelli–Shanks
- $a^{b^{c...}} \bmod m$ → cadeia $\varphi(\varphi(\cdot))$ (não coprimos: use b mod $\varphi(m) + \varphi(m)$); $v_p(x^n \pm y^n)$ → LTE
- Muitas moedas, menor pequena → Dijkstra nos restos

Contagem

- "Exatamente k defeitos" → inversão binomial de "pelo menos k"
- Rotações/reflexões → Burnside
- Conexos rotulados → total − componente do vértice 1 (ln de EGF)

- Árvores rotuladas → Prüfer/Cayley; geradoras → Matrix-Tree; circuitos eulerianos → BEST
- Caminhos disjuntos em DAG/grid → LGV (determinante; exige fontes/sumidouros compatíveis, senão há sinais)
- Extensões lineares de floresta → n!/∏ sz
- Caminhos que não cruzam uma linha → reflexão / Catalan
- Tempo esperado até padrão → Σ q^k sobre as bordas, incluindo o padrão inteiro
- Subconjuntos com soma ≡ r (mod m) → ∏(1+x^{a_i}) mod ($x^m - 1$)

Bits

- Soma/contagem com AND/OR/XOR → bit a bit
- Max xor → trie; com intervalo/versões → trie persistente ou base com índice mais recente
- Max xor de passeio em grafo → px + base linear dos ciclos
- Operação a_i ^= a_j, lights out, paridades → Gauss em GF(2)
- Pares com a_i & a_j = 0 → SOS; convolução xor/and/or → Walsh / zeta

Strings

- Substring de s em t → s + '#' + t; rotação/potência → s + s
- Ocorrências de vários padrões → Aho-Corasick + soma na link tree
- Substrings de um padrão → intervalo no suffix array; LCP de sufixos → RMQ no lcp
- Suffix tree de s → SAM de rev(s)
- Partição em palíndromos → eertree + série link; palíndromos por posição → Manacher
- LCS de permutações → LIS; subsequência palindrômica → LCS(s, rev s)
- Casamento com erros/curinga → FFT por letra ou kangaroo (LCE)
- Período d → DSU em classes i mod d
- Árvore ↔ parênteses → subárvore = intervalo, LCA = RMQ do balanço

Grafos

- Grafo direcionado → DAG (SCC); 2 opções por item + conflitos → 2-SAT
- Restrições x_j − x_i ≤ c → caminho mínimo com Bellman-Ford (ciclo negativo = inviável)
- Atribuir/emparelhar → matching/fluxo; lucro com dependências → corte mínimo (closure)
- Rótulo 0/1 com custo por par submodular → corte mínimo
- Bipartido: independente máximo = n − matching; cobertura de caminhos em DAG = n − matching
- Antichain máxima = cadeias mínimas (Dilworth) → matching no fecho transitivo
- Caminhos disjuntos em vértices → fluxo com v_in/v_out; em arestas → capacidade 1 na aresta; custo convexo → arestas unitárias marginais
- Matching geral (não bipartido) → Blossom; dominadores → Lengauer–Tarjan; árvore dinâmica → link-cut
- Corte mínimo global → Stoer–Wagner; com limites inferiores → circulação viável; interseção de 2 matroides → matroid intersection
- Carteiro chinês → Euler + matching nos vértices ímpares
- Minimax de caminho → MST + LCA / árvore de Kruskal
- Árvore + poucas arestas extras → Dijkstra dos extremos + LCA
- Orientar/decompor arestas, de Bruijn → circuito euleriano (vértice fantasma para ímpares)
- Vizinhos = intervalo de vértices → segtree de arestas; "todos os não visitados" → set/DSU
- Corte mínimo planar s–t → adicione a aresta st e ache o caminho mínimo entre as duas faces novas no dual (sem usar o dual de st)
- Grafo funcional → ciclos + árvores; k passos → binary lifting

Árvores

- Subárvore → intervalo (Euler tour); caminho → HLD ou prefixos da raiz + LCA (prefixos só para operações inversíveis)
- Todos os caminhos com condição → centroide; resposta para toda raiz → rerooting
- Update de vértice em caminho, leitura no fim → diff[u]++, diff[v]++, diff[lca]−−, diff[pai(lca)]−−; update de aresta → diff[lca] −= 2
- Soma de distâncias sobre pares de vértices → cada aresta contribui sz·(n − sz)
- Query com k vértices → virtual tree; cores na subárvore → small-to-large
- Isomorfismo → hash AHU a partir dos centros

Estruturas

- Queries sem updates → offline + sweep, Mo
- Arestas/itens com tempo de vida → segtree no tempo + rollback
- Janela com operação sem inverso → duas pilhas
- k-ésimo em intervalo → wavelet / persistente; mex → last[v] em segtree + descida
- Reta mais alta em x → Li Chao; no caminho da raiz → Li Chao com rollback
- Só insere + estrutura estática → Bentley–Saxe
- Estado de autômato com updates → segtree de funções

Geometria

- Manhattan → (x + y, x − y)
- Ponto acima/abaixo de retas → dualidade + CHT/hull
- Ordenar por ângulo → semiplano + cross; "contém a origem" → complemento por semiplano
- União de retângulos → sweep + segtree; retângulo → 4 prefixos + BIT
- Interseção de convexos → origem ∈ Minkowski(A, −B); LP 2D → semiplanos
- Espelhos/bilhar → refletir o mundo; bolas idênticas colidindo → atravessam

Jogos

- Configuração simétrica → espelho / pareamento
- Duração fixa → paridade de lances
- Jogos independentes somados → xor dos Grundy
- Lance que o outro desfaz → ignore
- Ciclos com empate → BFS retrógrada
- Pontuação → dp de diferença (minimax)

Álgebra

- Pares com soma/diferença em valores pequenos → produto de polinômios (FFT/NTT)
- Total = exp(conexas) → ln/exp de série
- Σ k^d e contagens polinomiais → Lagrange
- Igualdade de polinômios/matrizes → ponto aleatório (Schwartz–Zippel, Freivalds)
- Matching perfeito existe? → posto da matriz de Tutte aleatória

Limites escondidos

Raiz e log

- Σ tamanhos ≤ S → ≤ √(2S) tamanhos distintos; itens > √S são ≤ √S
- m arestas → ≤ 2√m vértices de grau ≥ √m; grau de saída ≤ √(2m) orientando por grau
- Σ ⌊n/i⌋ → 2√n valores; 1 + 2 + … + k ≤ n → k ≤ √(2n)
- gcd/OR/AND de subarray → ≤ log mudanças; x mod y → cai pela metade
- Caminho raiz–folha → ≤ log arestas leves; cadeia de divisibilidade ≤ $\log_2 N + 1$
- φ(φ(…m)) chega a 1 em ≤ 2 $\log_2 m$ passos
- Bordas de string → O(log n) PAs; ocorrências de p em janela 2|p| → uma PA
- "Pelo menos metade" → aleatorizar (30–60 tentativas)

Números para decorar

- Divisores máx.: 10^6 → 240; 10^9 → 1344; 10^{12} → 6720; 10^{18} → 103680
- Primos distintos: 10^6 → 7; 10^9 → 9; 10^{18} → 15
- π(10^6) = 78498; π(10^7) = 664579; π(10^9) ≈ $5{,}08 \cdot 10^7$
- Maior gap de primos: < 10^9 → 282; < 10^{18} → < 1500
- 20! < 2^{63} < 21!; lcm(1..43) estoura 2^{63}; ~86 Fibonacci ≤ 10^{18}
- double guarda inteiros até 2^{53}; FFT double ok se n·$coef^2$ ≲ 10^{14}–10^{15}
- NTT em 998244353 → tamanho ≤ 2^{23}; módulo arbitrário → 3 NTTs + CRT
- Miller–Rabin 64 bits → 12 primeiras bases primas
- Pares com n = $2 \cdot 10^5$ → $4 \cdot 10^{10}$ (long long); a·d vs b·c com 10^9 → __int128

Complexidades-chave

- Subconjuntos 3^n: n ≤ 16–18; 2^n·n: n ≤ 20–22; meet in the middle: n ≤ 40
- Knapsack em árvore → $O(n^2)$, ou O(nK) com limite K
- Mo → n√q; com updates → n^(5/3)
- Triângulos e C4 → m√m; bitset → ÷64
- Hopcroft–Karp → E√V; Hungarian → n^3; Gauss → n^3 (GF(2): $n^3/64$)
- Berlekamp–Massey precisa de 2k termos; Σ k^d → d + 2 pontos
- Interativo: ⌈log_b(#respostas)⌉ queries; 10^{18} → 60 perguntas binárias

Estrutura garantida

- Planar → m ≤ 3n − 6, degenerescência ≤ 5
- Sem triângulo → m ≤ $n^2/4$; sem C4 → m = O(n√n)
- Torneio → sempre tem caminho hamiltoniano
- Grau mínimo ≥ k → caminho de comprimento ≥ k; ≥ n/2 → ciclo hamiltoniano
- n + 1 prefixos mod n → bloco com soma ≡ 0; LIS · LDS ≥ n
- n > nº de bits → subconjunto de xor 0; base linear tem ≤ (nº de bits) vetores
- Centroide → componentes ≤ n/2; virtual tree → ≤ 2k − 1 vértices
- SAM → ≤ 2n − 1 estados; palíndromos distintos ≤ n; runs < n
- Dim. do espaço de ciclos → m − n + c; Euler: V − E + F = 1 + c

Por tópico

Arrays/guloso: ordenar · trás pra frente · two pointers · pilha monotônica · deque · heap com arrependimento · diferenças · mediana · contribuição · exchange argument · slope trick

Busca binária: na resposta · k-ésimo · razão · paralela · descida na segtree · ternária · Aliens · Dinkelbach

DP: prefixo · intervalo · subárvore · máscara · dígitos · trocar estado e valor · última operação · CHT · D&C · Knuth · matriz · SOS · bitset · rerooting · small-to-large · broken profile · long-path · connected components · Lagrange · BM

Números: √ divisores · múltiplos (harmônico) · fatorar e separar primos · Möbius · gcd(a, b − a) · CRT · Euclides estendido · crivo linear · blocos de ⌊n/d⌋ · floor sum · BSGS · Miller–Rabin/Pollard · LTE

Contagem: complemento · IE · bijeção · reflexão · contribuição · linearidade da esperança · Burnside · OEIS · inversão binomial · Prüfer · Matrix-Tree · LGV · EGF · tail sum

Bits: bit a bit · trie · base linear · prefix xor · submáscaras · SOS · Walsh · Gauss GF(2) · Gray · pigeonhole n > bits

Strings: hash · Z/KMP · Aho-Corasick · SA + LCP · SAM · trie · Manacher · eertree · frequências 26 · máscara de paridade · run-length · s + s · Fine–Wilf · Duval · next[i][c]

Grafos: grafo de estados · 0-1 BFS · super-fonte · SCC · 2-SAT · pontes · DSU · MST · bipartido · funcional + lifting · Euler · fluxo/corte · König/Hall/Dilworth · árvore de Kruskal · base de ciclos · degenerescência

Árvores: enraizar bem · rerooting · centroide · diâmetro · LCA · HLD · Euler tour · small-to-large · virtual tree · pesado/leve · long-path · diferenças em caminho · AHU

Estruturas: offline + sweep · Mo · segtree (o que o nó guarda?) · lazy · persistente · beats · sparse · ODT · √ decomposition · CDQ · DSU com info · segtree no tempo · duas pilhas · Li Chao · wavelet · treap implícito

Jogos: brute force pequeno · espelho · primeira escolha decide · roubo de estratégia · pares · paridade de lances · invariante · Grundy · Nim · BFS retrógrada · minimax em intervalo

Construtivos: impossível primeiro · n pequeno à mão · indução n → n + 2 · forma canônica · paridade/xadrez · pares · sentinela · Gray · de Bruijn · Havel–Hakimi · interativo: binária/centroide

Aleatório: sortear e verificar · hash de conjunto · xor hashing · shuffle · color coding · Schwartz–Zippel · Freivalds · annealing

Geometria: cross/dot · ordem por ângulo · hull · calipers · Minkowski · semiplanos · sweep · Pick · dualidade · closest pair · Welzl

Álgebra/otimização: FFT/NTT · série formal · Lagrange · BM/Kitamasa · Gauss · Kirchhoff · dualidade LP · matroide (guloso) · fluxo convexo · Hungarian

Travado: brute force n ≤ 15 · imprimir a escolha ótima · releia o enunciado · o que o autor quer cortar? · trocar a ordem dos fors · casos de borda · stress test

Catálogo completo

> Aviso: os blocos 'adicionado em 2026-11-10 por agentes' estão marcados como 'revisado', mas a auditoria de 2026-11-11 encontrou erros neles; este catálogo já foi corrigido. Ver docs/AUDITORIA-2026-11-11.md.

Nov 10, 2026 · @Walfrido

Use como checklist de perguntas, não como lista de técnicas: cada linha é um gatilho no enunciado → uma pergunta para se fazer.

Como usar quando travar

Forçar ideia funciona quando é rápido e barato: gaste 1–2 minutos por seção relevante, testando cada pergunta contra um exemplo pequeno. Se nenhuma encaixa, volte a observar o problema em vez de forçar mais.

1. Releia o enunciado e as constraints. Anote o que é estranho (um limite pequeno demais, uma soma garantida, uma condição "distintos").
2. Resolva à mão 3–4 casos pequenos. Se possível, escreva um brute force e imprima respostas para n = 1..10.
3. Abra a seção das constraints (abaixo) e decida a complexidade-alvo.
4. Passe pelas Reformulações universais. Elas destravam mais do que qualquer técnica.
5. Só então abra a seção do tipo do problema e teste cada gatilho contra o seu exemplo.
6. Travou 10+ minutos sem progresso: troque de problema e volte depois.

Leia as constraints

O limite de n quase sempre entrega a complexidade-alvo; um limite estranho quase sempre entrega a técnica.

Limite	Complexidade-alvo	O que tentar
$n \le 10$–12	n!, n!·n (este só até n = 10)	Permutações, backtracking
$n \le 18$–22	2^n·n	Bitmask DP, SOS, brute force de subconjuntos
$n \le 40$	2^(n/2)	Meet in the middle
$n \le 100$–500	n^3	DP em intervalos, Floyd, DP com 3 índices
$n \le 2000$–5000	n^2	DP quadrática, todos os pares, $O(n^2)$ com bitset
$n \le 10^5$	$n \cdot \sqrt{n}$, $n \log^2 n$	Mo, sqrt decomposition, D&C, CDQ
$n \le 2 \cdot 10^5$–10^6	n log n, n	Sort, segtree, two pointers, guloso, prefix sums
$n, x \le 10^{18}$	log, polylog	Fórmula, exponenciação de matriz, digit DP, binária na resposta; fatoração com Miller–Rabin + Pollard–Rho ($\sqrt{x}$ só até 10^{12})
$n \le 10^7$	n (linear, sem log)	Crivo linear, prefix sums, two pointers; evite map/set; leitura rápida (fread)
$n \le 10^8$	n com constante minúscula	Só laço simples, crivo em bitset/segmentado; sem memória por elemento
$n \le 10^{10}$–10^{11}	n^(2/3), √n·log	Soma de funções multiplicativas: Dirichlet, Min_25, Lucy; contagem de primos
$n \le 10^{12}$	√n, n^(1/3)	Fatorar por tentativa até 10^6, depois Miller–Rabin; divisão em blocos
n ≤ 60 (ou 64)	operações em uint64, base linear, branching (2^(n/2)/3^(n/3) só até n ≈ 40–50)	Máscara cabe em palavra; base linear de 60 bits; bits de 10^{18}; cliques máximas por Bron–Kerbosch
n ≤ 30	2^(n/2)·n (2^n só n ≲ 20–22)	Subconjuntos com bit hacks; meet in the middle vira 2^15; DP de máscara sem log (só n ≲ 20–22)

Limite	Complexidade-alvo	O que tentar
$n \le 50$	2^(n/2), branching	Meet in the middle em 2^25 com sort; conjunto independente máximo por branching e poda
$n \le 300$	n^3 (n^4 só $n \lesssim 100$)	Floyd, DP em intervalo, Hungarian $O(n^3)$, matriz inversa; n^4 só com $n \lesssim 100$ e operações triviais
$n \le 3 \cdot 10^4$–10^5 com TL 3 s	$n^3/64$ (fecho geral) ou $n \cdot m/64$ (DAG esparso); $n^2/64$ (mochila/LCS)	Bitset em fecho transitivo: geral só $n \lesssim 3000$–5000; em DAG esparso até $3 \cdot 10^4$–10^5; mochila booleana, LCS bit-parallel
$n \le 2000$, TL 1 s	n^2 simples	DP dp[i][j] com poucos acessos; n^2 log só com cuidado
$k \le 10$, n grande	k·n, 2^k·n, 3^k·n	k no estado: Steiner (3^k), TSP parcial, DP sobre cores
$\min(n, m) \le 20$ (com $n \cdot m \le 10^5$) ou $n \le 5$ e m grande	profile DP, matriz	DP em perfil (broken profile) por coluna; transição linear → matriz 2^n × 2^n e exponenciação (só $n \le 5$–8)
Resposta ≤ 20 ou "no máximo log"	iterar a resposta	Para cada candidato, checagem O(n log n); ou BFS em camadas até 20
Operações "no máximo 10^6" e $n \le 10^3$	$O(n^2)$ total; O(1) ou O(log n) por operação	Construtivo; a quantidade permite n^2 trocas simples (bubble sort)
T testes, n pequeno por teste	precompute global	Tabelas por tamanho (factoriais, Grundy, padrões) uma vez; não zere arrays de tamanho máximo a cada teste
Modulo $m \le 100$–1000	dp[i][r] em m·n	Estado = resto; resposta por pigeonhole; ciclo de x^k mod m
Valores de peso ≤ W pequenos ($W \le 10$–100)	BFS com W filas, Dial	Caminho mínimo em O(m + n·W); expandir aresta de peso w em w arestas unitárias só se m·w couber

Sinais escondidos nas constraints:

- Soma de n sobre os testes ≤ $2 \cdot 10^5$ → pode ser O(n) por teste; sem essa garantia, pré-processe tudo antes.
- Valores $a_i \le 10^6$ (bem menores que 10^9) → itere sobre valores, crivo, contagem por valor, divisores de cada valor.
- $k \le 20$ ou "no máximo 3 cores" num problema grande → o k vai no estado ou na máscara.
- $n \cdot m \le 10^6$ com n e m livres → $\min(n, m) \le 1000$; troque linhas e colunas se preciso.
- "a_i distintos" ou "é uma permutação" → posição do valor v, ciclos, inversões (ver também "Todos distintos" abaixo).
- Grafo com $m = n - 1$ ou $m \le n + 20$ → árvore mais poucas arestas extras.
- Soma dos valores ≤ 10^5 (com $a_i \ge 0$ inteiros) → há no máximo $\sqrt{(2 \cdot 10^5)} \approx 450$ valores distintos.
- Queries Q grande e offline permitido → ordene queries (Mo, sweep, DSU offline).
- Resposta "mod 998244353" → NTT pode aparecer; "mod p" → contagem, quase nunca guloso.
- TL alto (4–5 s) e $n = 10^5$ → provavelmente √ ou bitset (ver o bullet de TL abaixo).
- Memória baixa (32–64 MB) → DP com rolling array, sem armazenar tudo.

adicionado em 2026-11-10 por agentes; revisado

- **Módulo pequeno ($m \le 100$–1000)** → estado = resto; ou pigeonhole (entre m+1 prefixos há dois iguais mod m ⇒ existe subarray com soma ≡ 0); ou x → x^k mod m com ciclo curto (ver também a linha "Modulo" da tabela).
- **$k \le 5$ ou 6** → k! permutações cabem no estado; ou k distinguíveis sem custo; ou o problema difícil só aparece em $k \ge 3$.
- **Queries ≤ 10 (ou muito poucas)** → pode fazer O(n log n) ou O(n√n) por query, sem estrutura; às vezes a ideia é rodar o algoritmo de uma query inteira cada vez.
- **Σ tamanhos limitada em vários objetos** ($\Sigma|s|$, Σk, Σ deg) → quase sempre a ideia é pesado/leve ou "tamanhos distintos ≤ √(2S)"; a soma limitada permite O(Σ·log) mas não O(n·Σ).
- **Grid n × m com $n \le 5$–8 e m enorme** → DP em perfil por coluna e, se $m \ge 10^9$, exponenciação de matriz da transição de colunas.
- **Pesos pequenos ($w \le 10$–100)** → Dial / BFS em camadas; "valor pequeno × peso grande" → troque o estado da mochila; grafo com pesos 1..W → distância em (vértice, tempo) só até n·W (ver também a linha "Dial" da tabela).
- **"Todos distintos" / "permutação"** → o conjunto de valores é conhecido: soma, xor, produto são fixos; posição→valor é bijeção e ciclos aparecem; "distinto" também limita multiplicidade a 1 (hash por soma de aleatórios).
- **TL de 3 s ou mais** com $n \le 10^5$ → √ decomposition, bitset $n^2/64$, ou um log a mais que o ótimo; TL de 0,5–1 s com $n = 10^6$ → O(n log n) ainda passa; só O(n) para $n \ge 10^7$ ou constante alta.
- **Resposta mod 2** → paridade: cancele em pares (involução), C(n,k) é ímpar ⇔ $k \subseteq n$ em bits (Lucas), determinante mod 2 por Gauss em bitset; muitos problemas "conte as paridades" caem em xor.

- **"Imprima qualquer"** → existe um critério simples de existência; ache a condição de impossível e depois uma construção que a dispense; teste n pequeno com brute force que lista todas as soluções para ver o padrão.
- **"Se existir"** → a decisão pode ser mais fácil que a construção: faça primeiro o teste de existência (paridade, Hall, matching), depois reconstrua por backtrack do certificado.
- **"Mostre que" / "prove" em problema de olimpíada** → o enunciado dá a resposta; procure o invariante, o extremo (maior, mais à esquerda) ou o pigeonhole que a prova usa; esse é o algoritmo.
- **Entrada muito grande (10^6 números)** → leitura rápida vira parte do limite; saída com 10^6 linhas → '\n' em vez de endl, string buffer.
- **"$n \le 2 \cdot 10^5$ e $a_i \le 2 \cdot 10^5$"** → valores como índices: arrays por valor, crivo, harmônico Σ N/d; sem isso o limite estaria 10^9.
- **Limite com forma $n \cdot k \le 10^6$, $n \cdot m \le 10^5$** → o produto é o tamanho do estado; DP dp[i][j] pode ser O(nk); sempre inverta o menor.
- **Memória 16–64 MB** → rolling array, bitset, hash de estados, evitar vector de vector pequeno (overhead por vetor ≈ 24 B); árvore em lista de adjacência plana.
- **Queries online forçadas (resposta anterior no input)** → sem offline: persistente, sqrt, ou estrutura online (binary lifting, sparse).
- **Entrada com "garantido que resposta existe"** → não precisa tratar impossível; às vezes o enunciado garante um grafo conexo ou árvore, o que dispensa uma busca.
- **"Cada elemento aparece no máximo k vezes"** → somas sobre pares de mesma cor/valor limitadas a n·k; bruta de pares iguais é aceitável.
- **Estrutura "quase árvore" (cacto, cada aresta em ≤ 1 ciclo, série-paralelo)** → treewidth ≤ 2, DP em decomposição; grau ≤ 3 sozinho não limita treewidth.

Reformulações universais

Estas servem para qualquer tipo de problema; teste todas antes de procurar técnica.

- **Fixe algo.** Fixe o máximo, o mínimo, o elemento do meio, a posição da resposta, o primeiro passo, o valor do gcd. Depois conte/otimize o resto.
- **Contribuição.** "Soma sobre todos os subarrays/pares/caminhos de f" → troque a ordem da soma: quanto cada elemento/aresta contribui? (ex.: a_i é máximo em $(i - L) \cdot (R - i)$ subarrays, com L = anterior estritamente maior e R = próximo maior ou igual).
- **Complemento.** Contar "pelo menos um" ou "algum par ruim" → total menos o ruim; ruim com vários defeitos → inclusão-exclusão.
- **Inverta o tempo.** Remoções → adições ao contrário (DSU offline). Processo que destrói → rode de trás pra frente.
- **Inverta a pergunta.** "Mínimo k tal que dá" → binária em k + checagem (confirme que a factibilidade é monótona em k antes). "Qual valor é o maior possível" → "dá pra atingir ≥ x?".
- **Troque papéis.** Itere sobre valores em vez de índices; sobre a resposta em vez da entrada; sobre arestas em vez de vértices.
- **Invariante.** Operação que se repete → o que nunca muda? (paridade, soma mod k, xor total, número de inversões mod 2, gcd).
- **Monovariante.** Algo que sempre cresce/decresce → prova término e limita o número de passos.
- **Caso extremo.** Olhe o maior elemento, a folha, o vértice de grau 1, a primeira/última posição. Quem é forçado?
- **Simplifique.** Resolva com n = 2, com valores só 0/1, sem a query de update, com o array ordenado. Depois generalize.
- **Transforme o array.** Prefix sums, diferenças ($a_i - a_{i-1}$), $a_i - i$, $a_i \cdot i$, troque sinal, tome log.
- **Mude a condição.** "Soma = 0" → prefixos iguais. "Média ≥ k" → subtraia k e veja soma ≥ 0. "Mediana ≥ x" → vire ±1 (testa a mediana superior; para a média dos centrais com n par, ajuste o critério).
- **Modele como grafo.** Restrições entre pares → arestas; estados → vértices; "trocar a_i por a_j" → ciclos da permutação.
- **Separe as dimensões.** Distância Manhattan → x e y independentes; rotacione 45° para Chebyshev ($u = x + y$, $v = x - y$).
- **Encontre o limite pequeno escondido.** "Cada valor dobra" → log passos; "gcd muda" → log mudanças; "divisores" → ≈ 240 para 10^6.

adicionado em 2026-11-10 por agentes; revisado

- **Troque a ordem dos fors (do pensamento).** Escreva o brute force como laços aninhados e pergunte: e se o laço de dentro virasse o de fora? "Para cada i, quais j contam?" vira "para cada j, quantos i contam?". O laço de fora deve ser aquele que deixa o de dentro virar fórmula fechada, prefixo, BIT ou contagem. Ex.: $\Sigma_{i \le n} \Sigma_{d \mid i} f(d) = \Sigma_d f(d) \cdot \lfloor n/d \rfloor$; Σ de max de todos os subarrays = $\Sigma_i a_i \cdot (i - L) \cdot (R - i)$, com L = anterior estritamente maior e R = próximo maior ou igual (sentinelas 0 e n+1).

- **Quais são os laços possíveis?** Índice · valor · bit · divisor/múltiplo · tempo · query · aresta · vértice · tamanho do bloco · resposta candidata. Liste todos os laços que o brute force poderia ter e teste cada um como o mais externo.
- **Fatiar por camadas.** Para a_i inteiro não negativo, Σ a_i = Σ_{t≥1} #{i : a_i ≥ t}. Histogramas, áreas, "soma de mínimos" e E[X] = Σ_{k≥1} P(X ≥ k) (X inteiro não negativo) ficam fáceis ao iterar sobre o nível t em vez do elemento i.
- **Queries × elementos.** Em vez de "para cada query, percorra os elementos", faça "para cada elemento, atualize as queries que ele afeta" (offline, sweep, diferenças). Inverta quando o número de queries ou de elementos for o lado difícil.
- **Pares (i, j): fixe um lado.** Fixe j e conte i com a propriedade (BIT, map, two pointers). Se f(i, j) é separável, Σ_{i,j} f(i)·g(j) = (Σ f) (Σ g); se tem uma condição, ordene e vire prefixo.
- **DP: ordem de preenchimento e eixo externo.** Itere por valor em vez de índice (dp[valor][...]), por comprimento em vez de início, ou por fim em vez de começo. Em knapsack, itens fora/capacidade dentro conta combinações (no 0/1, capacidade decrescente); capacidade fora/itens dentro conta sequências (ordem importa) no caso ilimitado, não no 0/1. Ordem trocada muda o que é contado.
- **Push vs pull.** Se "quem chega em v" é difícil, calcule "para quem u contribui" (e vice-versa). Também vale para BFS/DP em grafo e para somas sobre divisores/múltiplos.
- **Agrupe por valor igual.** Troque "para cada elemento" por "para cada valor distinto × multiplicidade". Com a_i ≥ 0 e soma limitada, o número de valores distintos é O(√Σ), o que limita as iterações a √.
- **Transposta.** Troque linhas por colunas e trabalhe com a menor dimensão (n·m ≤ 10^6 ⇒ min(n, m) ≤ 1000); vale também para matrizes de adjacência e tabelas de DP.
- **∃ e ∀ trocados.** "Existe i tal que para todo j" × "para todo j existe i" são problemas diferentes; ao reescrever a condição, confira qual quantificador está dentro de qual. Min-max e max-min só coincidem sob dualidade (jogos, LP, fluxo).
- **Soma por partes (Abel).** Σ a_i·b_i = A_n·b_n + Σ_{i=1}^{n−1} A_i·(b_i − b_{i+1}), com A_i = Σ_{j≤i} a_j (b não precisa ser monótono). Troca "valor × peso" por "prefixos × saltos", mais o termo de borda.

adicionado em 2026-11-10 por agentes; revisado

- **Dualidade.** Um min vira um max do problema dual quando há teorema de igualdade: König (cobertura mínima = matching máximo), max-flow = min-cut, Dilworth (cadeias = antichain máxima), LP dual. Se o primal é difícil, o dual muitas vezes é um fluxo/greedy.
- **Min-max theorems.** König (bipartido), Hall (bipartido com |L| = |R|: existe matching perfeito ⇔ ∀S ⊆ L, |N(S)| ≥ |S|), Dilworth/Mirsky, Menger (caminhos disjuntos = corte mínimo), Gallai (independente + cobertura = n), Erdős–Szekeres. "Cobrir com o menor número de X" pede o pior caso (antichain) como certificado.
- **Relaxação.** Resolva o problema relaxado (fracionário, sem uma restrição) e prove que o ótimo relaxado é atingível; matrizes totalmente unimodulares (bipartido, intervalos consecutivos, fluxo) garantem que LP = inteiro.
- **Forma paramétrica.** Se há uma restrição de quantidade ("exatamente k"), Lagrange/Aliens; se é "razão", binária em λ; se é "mínimo do máximo", binária na resposta; todas trocam a restrição por penalidade.
- **Lifting de dimensão.** Acrescente uma dimensão (tempo, nível, estado extra) para tornar o problema local: grafo de estados (v, t), duplicar o grafo para "usei k bônus", DP em (posição, valor).
- **Escala log.** Uma operação multiplicativa vira soma em log; ou processe bit a bit de cima para baixo (potências de 2), e binary lifting resume saltos 2^k.
- **Amortização.** Se cada elemento entra e sai um número limitado de vezes (pilha monotônica, two pointers, potencial), o custo é O(n) total; DSU custa O((n+m)α(n)) para n elementos e m operações; procure o potencial que só decresce.
- **Extremal principle.** Considere o objeto extremo (maior elemento, par mais próximo, caminho mais longo, ciclo mais curto) e mostre que ele tem uma propriedade; frequentemente é a ordem do guloso ou o vértice a processar primeiro.
- **Descida infinita / Vieta jumping.** Dada uma solução (a, b) de uma equação quadrática simétrica, (a, b') com b' = (coef) − b é outra; vá descendo até o caso mínimo. Em programação: operações que reduzem o par (Euclides) → o mínimo é caso base.
- **Indução forte.** "Assuma que vale para todo tamanho menor" é um algoritmo recursivo: ache a parte menor (remover folha, maior elemento, par) e verifique que a construção estende; vale como prova e como implementação.
- **Pigeonhole.** Mais objetos que gavetas ⇒ repetição: prefixos mod n, estados repetidos (ciclo ≤ nº de estados), "n + 1 números em [1, 2n] contêm dois onde um divide o outro". Use para provar existência e para limitar passos.
- **Double counting.** Conte um mesmo conjunto de pares de duas formas e iguale: Σ grau = 2m, número de pares (elemento, conjunto), e Σ_i f(i) = Σ_j g(j); frequentemente troca duas somas.
- **Linearização.** Termo quadrático Σ_{i<j} a_i·a_j → ((Σa)² − Σa²)/2; produto de dois indicadores → indicador da interseção (I_A·I_B = I_{A∩B}); esperança de produto de variáveis inteiras não negativas (dependentes) → ainda soma de pares de indicadores; max (inteiro não negativo) → soma de indicadores [max ≥ k].
- **Soma de potências / polinômio em n.** Se uma contagem é polinomial em n de grau d (soma de potências, contagem em grade), calcule d+1 pontos por brute force e interpole por Lagrange.

- **Escolha de coordenadas.** Origem no ponto crítico, eixos rotacionados (x+y, x−y), posições como diferenças, tempo como terceira coordenada; muitas condições viram condições independentes por eixo.
- **Fixe o par.** Fixe dois elementos (extremos, ou o par que realiza a resposta) e conte o resto livre. Quando o par é "o mais distante" (diâmetro), todas as outras posições ficam em uma região fácil.
- **O que acontece no limite?** Faça n → ∞ ou k grande para ver o regime assintótico (a resposta estabiliza, cresce linear, periódico). Ajuda a chutar a forma da fórmula e a ver estabilização (2–3 cópias).
- **Homogeneização.** Igualdade com constante → subtrair (a_i − k) para virar soma 0; desigualdade → divide por escala para fixar o máximo em 1.
- **Normalização.** Reduza a uma forma canônica (ordenar, gcd = 1, rotação mínima, pilha com mod k, árvore enraizada no centro) para comparar objetos como iguais.
- **Parity argument.** Coloque 0/1 e procure o que muda de paridade a cada lance; a configuração final com paridade errada é impossível. Também conta grau ímpar em grafo, número de inversões de permutação, soma de cores num tabuleiro.
- **Coloring argument.** Pinte o tabuleiro (xadrez, 3 cores por (i + j) mod 3, linhas alternadas); cada peça cobre cores num certo número, e a contagem de cada cor é um invariante.
- **Grafo auxiliar.** Crie vértices que representam coisas que o enunciado não chama de vértice (valores, intervalos, restrições, classes mod m) e arestas que representam a operação; resultado em componentes, ciclos, caminho Euler.
- **Reduza a contagem.** "Existe?" → "quantos?" e verifique ≠ 0 mod p (certificado por Schwartz–Zippel, determinante, matrix-tree); "uma solução" → "conte solução" em modelo pequeno mostra se é única.
- **Probabilistic method.** Para provar existência: aleatório tem esperança boa ⇒ existe um bom. Para construir: derandomize por esperança condicional (escolha cada bit mantendo a esperança ≥ alvo).
- **Reflexão (unfolding).** Trajetória que reflete em paredes = reta no espaço espelhado (bolas em tabuleiro, luz em espelhos); bolas idênticas colidindo = atravessam, só os nomes trocam.
- **Cancelamento por involução.** Pareie objetos de sinais opostos (swap de um par) e o que sobra são os pontos fixos: soma alternada, determinantes, contagem mod 2.
- **Limite inferior = construção.** Prove cota fácil (máximo de linhas, de passos) e tente construir exatamente esse valor; se não der, o gap mostra o invariante.
- **Telescopia e soma por partes (Abel).** $\Sigma a_i \cdot (b_i - b_{i-1}) = a_n b_n - a_1 b_0 - \Sigma_{i=1}^{n-1}(a_{i+1} - a_i) b_i$ (terminais incluídos); reduz soma de produtos a diferenças de somas parciais; também transforma $\Sigma \lfloor \cdot \rfloor$ e somas com mudança de limite em bloco.
- **Estrutura algébrica.** Operação associativa → segtree, doubling, matriz; com identidade e inversa → prefix; idempotente (min, gcd, or) → sparse table; comutativa → Mo e ordem livre.
- **Corte na árvore de decisão.** Para limites de queries/comparações use a árvore de decisão: k respostas possíveis exigem $\log_b k$ queries; o enunciado costuma dar exatamente esse limite.

Cadeias de pulos (gatilho → observação → técnica)

Cada linha é um salto que você costuma dar em vários passos; a coluna "Aparece em" diz em quais seções a ideia também está repetida.

Vi no enunciado	Então	Logo tente	Aparece em
Σ tamanhos $\le S$ (strings, listas, pesos)	$\le \sqrt{(2S)}$ tamanhos distintos	Agrupar por tamanho; laço externo sobre tamanhos distintos	Strings, DP, Arrays, Estruturas
Σ tamanhos $\le S$ e itens "grandes"	Itens com tamanho $> \sqrt{S}$ são $\le \sqrt{S}$	Pesado/leve: grandes um a um, pequenos em tabela	Strings, Grafos, Estruturas
Grafo com m arestas	$\le 2\sqrt{m}$ vértices com grau $\ge \sqrt{m}$	Triângulos, queries por vértice em $m\sqrt{m}$	Grafos, Árvores
Valor que dobra / divide por 2	Só log passos	Simule direto, ou binary lifting	Teoria dos números, Árvores
gcd / OR / AND de subarray	Muda $\le \log(a)$ vezes ao estender	Para cada fim, lista de (valor, início)	Teoria dos números, Bits
x mod y repetido	x cai pela metade quando $y \le x$	Pule elementos que não mudam (segtree com max)	Teoria dos números, Estruturas
$a_i \le 10^6$	Divisores ≤ 240, primos distintos ≤ 7	Iterar divisores/múltiplos, crivo, Möbius	Teoria dos números, Combinatória

Vi no enunciado	Então	Logo tente	Aparece em
"Pelo menos metade dos elementos"	Sorteio acerta com prob. ≥ 1/2	Aleatorizar 30–60 vezes	Aleatorização, Teoria dos números
Configuração simétrica num jogo	Segundo jogador pode copiar	Espelho; primeiro ganha se cria a simetria	Teoria dos jogos, Construtivos
Jogo com duração fixa	Só a paridade de lances importa	Contar lances totais	Teoria dos jogos
Operação reversível	Estados formam classes	Forma canônica para os dois lados	Construtivos, Strings, Grafos
Operação em intervalo [l, r]	Vira 2 pontos no array de diferenças	Raciocinar nas diferenças; BIT	Arrays, Estruturas
"Soma = 0" / "balanceado"	Prefixos iguais	Contar prefixos com map	Arrays, Strings, Bits
Mediana ≥ x	Troque por ±1	Binária em x; P[r] − P[l−1] > 0 (mediana inferior) ou ≥ 0 (superior)	Busca binária, Arrays
Resposta não depende da raiz? Depende	Recalcule mudando 1 aresta	Rerooting	Árvores, DP
Caminhos em árvore com condição	Todo caminho passa por um centroide	Centroid decomposition	Árvores, Estruturas
Queries sem updates	Dá para ordenar	Offline + sweep, Mo	Estruturas, Strings, Árvores
s repetida k vezes / infinita	Transiente pode ser O(|p|/|s|), não 2–3 cópias	Calcule ⌈|p|/|s|⌉+1 cópias e extrapole	Strings, DP
n ≤ 20 em problema de conjunto	2^n cabe	Máscara, SOS, meet in the middle	DP, Bits, Combinatória
Restrições "a ou b" em pares	Implicações booleanas	2-SAT	Grafos, Construtivos
Soma dupla / laços aninhados em $O(n^2)$ ou pior	Inverta qual índice é o externo	Contribuição, camadas, ou agrupar por valor/divisor	Arrays, Teoria dos números, Combinatória, DP
Σ sobre divisores ou múltiplos de cada i	Troque para Σ sobre d de (n/d termos)	Harmônica O(n log n), blocos de ⌊n/d⌋, Möbius	Teoria dos números, Combinatória
"Para cada query, percorra tudo"	Troque queries por elementos	Offline + sweep/diferenças/BIT	Estruturas, Arrays
Valor esperado de variável inteira não negativa	E[X] = Σ_k P(X ≥ k)	Calcule P(X ≥ k) para cada k	Combinatória
Tarefas com duração, peso ou prazo e "ordem das tarefas"	A ordem ótima é decidida por uma comparação de pares	Exchange argument: ordene por t/w (Smith) ou prazo (EDF)	Arrays, DP
"Max de k intervalos / k máquinas" ou "salas mínimas"	Máquina livre com maior fim ≤ início	Multiset ordenado por fim	Arrays, Estruturas
"Pelo menos k", "exatamente k itens" com custo convexo	f(k) convexa em k (minimização)	Aliens: penalidade λ, g(λ)=min_k[f(k)+λk], resposta g(λ)−λk; desempate consistente	Busca binária, DP
Razão Σa/Σb ou média máxima	max Σ(a − r·b) é monótona em r	Binária em r ou Dinkelbach	Busca binária, Arrays
n ≤ 40 e subconjuntos	2^20 por metade	Meet in the middle + two pointers/ordenar	Arrays, Bits, Combinatória
Permutação e "operações de troca"	Ciclos = componentes	n − ciclos trocas; inversões para adjacentes; p^k quebra ciclos em gcd(c, k)	Arrays, Construtivos, Combinatória
Escolher itens não adjacentes de soma máxima	Escolha local pode ser desfeita	Heap + lista ligada com nó v_l + v_r − v_i	Arrays, DP

Vi no enunciado	Então	Logo tente	Aparece em
Tornar não decrescente com custo Σ|Δ|	f_i convexa em x	Slope trick com max-heap	Arrays, DP, Busca binária
Query "primeiro índice com prefixo ≥ x"	Soma de prefixo é monótona (exige a_i ≥ 0)	Descida no BIT/segtree em O(log)	Busca binária, Estruturas
"Primeiro instante em que conecta/ocorre" para muitas queries	Resposta monótona no tempo	Binária paralela ou Kruskal reconstruction tree	Busca binária, Grafos, Árvores
Cobrir todas as strings/estados com uma sequência curta	Estados = vértices, símbolo = aresta	Ciclo Euleriano (de Bruijn)	Construtivos, Grafos, Strings
Todo número como soma de poucas peças	Representação em base b / Fibonacci	Construção por dígitos (binária, ternário balanceado)	Construtivos, Bits, Teoria dos números
Interativo com limite ≈ $\log_2$(possibilidades)	Cada query deve dividir ao meio	Binária / centroide / descida na árvore	Construtivos, Árvores
Somar contribuições de pares com empates	Contagem dupla quando valores iguais	Desempate assimétrico (estrito de um lado) na pilha monotônica	Arrays, Estruturas
Colocar itens com distância mínima / dividir recursos em blocos	check(d) é monótono e guloso	Binária na resposta + guloso O(n)	Busca binária, Arrays
dp[i] = min_j (dp[j] + $(S_i - S_j)^2$) ou produto cruzado	Abra o quadrado: termo $S_i \cdot S_j$ é reta em x = S_i	CHT (deque se S monótono) / Li Chao / D&C se só Monge	DP, Estruturas
dp[k][i] com custo de segmento C(j, i) e k ≤ 100	opt monótono se C é Monge	D&C DP (O(kn log n)); Knuth em intervalo; se k grande, Aliens	DP, Busca binária
"Exatamente k" itens/segmentos e n grande	f(k) costuma ser convexa (minimização: fluxo, Monge, matroide)	Aliens/WQS: g(λ)=min_k[f(k)+λk] (penalidade λ) + DP de 1 camada; confira convexidade com brute force	DP, Busca binária
Mochila com Σ pesos ≤ S ou peso ≤ √S	≤ √(2S) pesos distintos	Binary splitting/deque por peso distinto + bitset	DP, Números
Árvore + "escolha k vértices"	merge de filhos por tamanho é $O(n^2)$ total	dp[v][k] com loops limitados por tamanho (ou ordem DFS pegar/pular em O(nk))	DP, Árvores
Árvore e estado é profundidade/altura (dist, nível)	Vetor por altura reaproveitado do filho mais profundo	Long-path decomposition, O(n) total	DP, Árvores
Contagem de permutações com restrição entre adjacentes / soma de diferenças	Insira em ordem de valor, cuide das componentes	Connected components DP, dp[i][c] (+ soma acumulada)	DP, Combinatória
Permutação com sinais < > em posições	Posto relativo do último elemento	Insertion DP com prefix sums $O(n^2)$; ou EGF/ziguezague	DP, Combinatória
Valor pequeno ou resposta limitada (ovos, log passos, valor ≤ 10^5)	Estado e valor se trocam	dp[res] = melhor parâmetro (dp[ops][ovos] = alcance)	DP
Recorrência linear e n ≤ 10^{18}	Ordem k pequena	Matriz / Kitamasa; sem fórmula: 2k termos + Berlekamp-Massey	DP, Combinatória
dp(n) parece polinômio em n	Diferenças finitas de ordem d+1 zeram	d+1 valores brutos + Lagrange	DP, Combinatória
Esperança em estados com ciclos	Sistema linear (I − Q)E = 1	Gauss mod p / double (n ≤ 500); em cadeia: E[i] = a_i·E[0] + b_i	DP, Combinatória
Esperança de um tempo/contagem total	Linearidade: Σ P(evento k ainda ativo)	Fórmula em vez de DP de estados	DP, Combinatória
Soma sobre subconjuntos / pares com AND = 0 / cobertura	Zeta e Möbius sobre máscaras	SOS (2^n n); k-colorações/cobertura por IE sobre SOS	DP, Bits, Combinatória
Partições de conjunto/union disjunta com peso	Primeiro bloco contém o menor bit	3^n submáscaras ou subset convolution 2^n n^2	DP, Bits

Vi no enunciado	Então	Logo tente	Aparece em
Partição de n em partes (distintas, ≤ k, ímpares)	Nº de partes distintas ≤ √(2n); conjugado troca "nº de partes" por "maior parte"	dp[soma][nº de partes] O(n√n); pentagonal para p(n)	DP, Combinatória
Heap/lista ligada resolve k=1 e dp côncava em k (maximização)	Arrependimento dá resposta para todo k	Escolha k não adjacentes por heap + lista, ou Aliens: bônus λ, g(λ)=max_k[f(k)+λk], resposta g(λ)−λk	DP, Arrays, Busca binária
Minimizar Σ|a_i − b_i| com b monótono/regra de vizinhança	f(x) convexa linear por partes	Mesmo slope trick de "tornar não decrescente" acima, com heaps (O(n log n))	DP, Arrays
Grid com m ≤ 12–15 e vizinhança local	Perfil de m bits processado célula a célula	Broken profile (plug DP; conectividade → parênteses em base 3/4)	DP
dp[i] depende de ciclo de volta / caminho com custo ≥ 0	Não é DAG por posição	Dijkstra/0-1 BFS sobre estados	DP, Grafos
Guloso com contraexemplo pequeno, "exatamente k" ou capacidade	Escolha local depende do futuro	DP (mochila/estado extra); se côncava em k, guloso com arrependimento ou Aliens	DP, Arrays
Mochila com itens entrando e saindo no tempo	Cada item vive num intervalo	Segtree no tempo + dp copiada por nível (ou "undo" modular)	DP, Estruturas
Mochila com Σ valor pequeno e peso gigante	dp[v] = menor peso	Troca estado e valor, O(n·Σv)	DP
Sequência cíclica / circular com custo de fechamento	Corte um ponto e feche	Duplicar, ou fixar estado inicial e rodar linear (|S| vezes), ou tr(M^n)	DP, Combinatória
dp[i][c] = custo + min c' ≠ c	Só o melhor e o segundo melhor importam	Top-2 por camada, O(nk)	DP
Contar objetos rotulados conexos/DAGs	Conexos: fixe o vértice 1 e sua componente; DAGs: IE sobre fontes/sorvedouros	Conexos: conn[n] = total − Σ_{k=1}^{n−1} C(n−1,k−1)·conn[k]·total_{n−k}; DAG: recorrência por fontes	DP, Combinatória, Grafos
a^b^c... mod m	Expoente enorme: reduza mod φ(m) somando φ(m)	Recursão pela cadeia φ(φ(·)) (≤ 2 $\log_2$ m níveis) com expoentes truncados	Combinatória
Σ f(⌊n/i⌋) ou Σ sobre pares com gcd	⌊n/i⌋ tem 2√n valores; gcd → φ ou μ	Blocos + prefixos de φ/μ; para n ≥ 10^{10} Dirichlet sieve O(n^{2/3})	Combinatória
Soma de função multiplicativa até n ≈ 10^{10}–10^{12}	Só 2√n valores ⌊n/i⌋ importam	Lucy → Min_25, ou powerful numbers, ou f = g∗h	Estruturas
Σ ⌊(a i + b)/m⌋ ou pontos de rede sob reta	Euclides dobra os parâmetros	Floor sum recursivo O(log)	Geometria
Equação em inteiros "sem solução"	Resíduos limitados (mod 3, 4, 7, 8, 9)	Testar todos os resíduos pequenos; LTE para v_p	Construtivos
Pares com lcm ou gcd = g em array com a_i ≤ 10^6	Zeta sobre múltiplos/divisores	GCD/LCM convolution + Möbius	Estruturas
"Quantos colares/giros/espelhos distintos"	Contagem por órbitas	Só giros: (1/n)Σ_{d|n} φ(d)k^{n/d}; com espelhos (diédrico): (1/(2n))[Σ_{d|n} φ(d)k^{n/d} + R], R = n·k^{(n+1)/2} (n ímpar) ou (n/2)(k^{n/2}+k^{n/2+1}) (n par)	Teoria dos números
"Conte estruturas conexas" (grafos, perms)	Conte todas e tire as com componente de vértice 1 de tamanho k	Recorrência c(n) = total − Σ C(n−1, k−1)c(k)·resto; ou log de EGF	Grafos
"Conte árvores/ligações entre componentes"	Árvores rotuladas = Prüfer	Fórmula n^{k−2}∏ s_i, ou Matrix-Tree se pesos	Grafos, Árvores
Contar circuitos eulerianos	Cada vértice ordena suas saídas	BEST: t_w·∏(deg−1)!, t_w por Matrix-Tree	Grafos
Caminhos não-cruzantes em grid	Troca de rotas nos cruzamentos	LGV: det[C(·)]	Grafos
Sequência (ou DP) com n ≤ 10^{18} e ordem pequena	GF racional	Mesma receita de "recorrência linear" acima: Berlekamp–Massey + Kitamasa	DP, Estruturas

Vi no enunciado	Então	Logo tente	Aparece em
Valor é polinômio em n	Diferenças finitas constantes	Mesma interpolação de "dp(n) polinômio" acima: Lagrange com d+1 pontos	DP
E[tempo] em passeio ou processo com ganho linear	Existe potencial Φ com E[ΔΦ] = −1 (ou martingale)	Optional stopping: E[T] = $\Phi_0 - \Phi_{fim}$	Teoria dos jogos, Grafos
Probabilidade de "n eventos aleatórios sem coincidência"	Aniversário	$e^{-n^2/(2N)}$ ("sem coincidência"); $1 - e^{-n^2/(2N)}$ = colisão; 64 bits não basta para n ≈ 10^9	Aleatorização, Strings
"Existe construção com ≥ metade"	Média acima do limiar	Método probabilístico + esperanças condicionais	Construtivos, Aleatorização
C(n, k) mod m composto ou p^e	Fatore m; separe $v_p(n!)$ e o fatorial sem p	$n!_p$ por blocos de p^e (Granville); o sinal $(-1)^{\lfloor n/p \rfloor}$ só vale mod p; junte por CRT	Teoria dos números
Perm/estrutura com "pai < filho"	Cada subárvore tem mínimo na raiz com prob. 1/sz	$n!/\prod sz(v)$	Árvores
Soma com $\lvert a_i - a_j \rvert$ ou $(a_i - a_j)^2$	Ordenar / completar quadrado	$\Sigma a_{(i)}(2i-n-1)$ ou $n\Sigma a^2 - (\Sigma a)^2$	Arrays
n+1 prefixos em 2^k valores, com n+1 > 2^k (não "n > nº de bits")	Pigeonhole: dois prefixos xor iguais	Existe subarray/subconjunto de xor 0; limita o brute force	Bits, Combinatória
Dado s = a+b e x = a⊕b	$s - x = 2(a\&b)$	Checar paridade/subconjunto de bits; 2^popcount(x) soluções	Bits, Teoria dos números
Max/min xor de passeio em grafo	Passeio = caminho da árvore + ciclos fundamentais	Base linear dos ciclos + px	Bits, Grafos
Max xor em [l, r] / com versões	Prefixos são versões	Trie persistente ou base com índice mais recente	Bits, Estruturas
Pares com a_i & a_j = 0 / "contém"	Complemento é submáscara	SOS sobre cnt e consulta em ~a	Bits, DP
"Soma sobre k passos com xor total x" / xor convolution	Caracteres do grupo Z_2^n	Walsh–Hadamard, potência pontual, inversa	Bits, Combinatória
a_i + a_j com bit k: contar/xor das somas	Bit k só depende de a mod 2^(k+1)	Ordenar mod 2^(k+1) + two pointers por bit	Bits, Arrays
Distância de Hamming mínima/≤ d em 2^k máscaras	Hipercubo é grafo	BFS multi-fonte por flip de bit	Bits, Grafos
Operação a_i ← a_i⊕a_j	Span é invariante	Base linear, rank; resposta em função de n − rank	Bits, Construtivos
Contar ocorrências de vários padrões	Nó do AC sobe pelos links	Árvore de links + soma de subárvore (BIT/Euler)	Strings, Árvores, Estruturas
Ocorrências de p em texto de tamanho ≤ 2|p|	Se duas estão a distância < |p|, d é período	Ocorrências formam PA; guardar (início, razão, qtd)	Strings
Bordas de s / períodos	Borda b ⇔ período n − b; Fine–Wilf	Bordas ≥ n/2 em PA; O(log n) PAs; série link	Strings
Problema pede "ocorrências de t" e "substring t de s"	Estado do SAM = classe de endpos	cnt na subárvore da link tree; binary lifting por len	Strings, Árvores
Suffix tree de s	Link tree do SAM de rev(s)	Construa SAM de rev(s) em vez de Ukkonen	Strings
SA + LCP, "todos os pares de sufixos"	LCP(i, j) = min no intervalo do array lcp	Soma de mínimos de subarrays por pilha monotônica	Strings, Arrays
Palíndromos terminando/começando em cada posição	Manacher dá um raio por centro	Adicionar PAs por segunda diferença	Strings, Arrays
Partição em palíndromos / fatores	Palíndromos-sufixo em O(log n) PAs	Série link na eertree, dp em O(n log n)	Strings, DP
String definida recursivamente (Fibonacci, SLP)	Só |p|−1 caracteres de borda importam	Recorrência cnt_n = cnt_{n−1} + cnt_{n−2} + cruzamento	Strings, DP
Rotações / "é potência"	Rotação de s é substring de s+s	A menor ocorrência de s em s+s (p > 0) dá a raiz primitiva; qualquer p dá uma raiz de comprimento gcd(p, n)	Strings

Vi no enunciado	Então	Logo tente	Aparece em
Árvore de parênteses / árvore ↔ sequência	DFS escreve (ao entrar e) ao sair	Subárvore = intervalo, LCA = RMQ do balanço	Strings, Árvores
Substring válida de parênteses com updates	Válida ⇔ soma 0 e min prefixo ≥ 0	Segtree com (soma, min prefixo)	Strings, Estruturas
LCS/LIS ou LCS com n·m/64 viável	Linha da DP é bitset monotônico	Fórmula V = (V + (V & M)) | (V & ~M)	Strings, Bits, DP
Mismatch ≤ k entre padrão e texto	Cada erro é um salto de LCE	Kangaroo O(k) por posição; ou FFT por letra	Strings
Operação "desfazer último caractere"	Amortização de KMP/Z quebra	Autômato completo ou pilha de estados	Strings
m ≤ n + k pequeno, queries de distância	O grafo é uma árvore + poucas arestas	Árvore geradora + Dijkstra/BFS dos ≤ 2k + 2 extremos, distância em árvore por LCA	Grafos, Árvores
Distância máxima de cada vértice / "qual é o mais distante"	Sempre termina numa ponta do diâmetro	Duas BFS + max(d(v, a), d(v, b)); centro na metade	Árvores, Grafos
"Menor valor tal que u e v se conectam" / minimax	Caminho na MST	MST + LCA (max) ou Kruskal reconstruction tree	Grafos, Árvores
Maximizar emparelhamento com conflitos/coberturas	Bipartido + König	Matching ⇒ cobertura mínima ⇒ conjunto independente máximo / antichain (Dilworth)	Grafos
Escolher conjunto com lucros e dependências ou custos de pares	Corte mínimo / closure	S/T com ∞ nas dependências; soma de lucros − corte	Grafos, Estruturas
Orientar arestas com grau ou paridade pedidos	Euler (vértice auxiliar para ímpares) ou fluxo	Circuito euleriano e orientação ao longo dele; senão Hall/fluxo	Grafos, Construtivos
Remover vértices/arestas em ordem e pedir conectividade/soma	Inverta o tempo	Adicionar na ordem reversa com DSU (Floyd: adicionar vértice em $O(n^2)$)	Grafos, Estruturas
Soma de caminhos / contribuição por aresta	Aresta contribui sz·(n − sz)	Subárvore sizes; soma Σ w·sz·(n − sz)	Árvores, Combinatória
Vértices marcados com distância ao mais próximo	Centroid tree de altura log n	best[c] por centroide + dist(v, c) por ancestral	Árvores, Estruturas
Update em caminho, query pontual/total	Diferenças em árvore	diff[u]++, diff[v]++, diff[lca]--, diff[pai(lca)]--; soma de subárvore	Árvores, Estruturas
Árvores a comparar sem raiz fixa	Hash canônico (AHU) nos centros	Id por multiconjunto ordenado de filhos; compare nos 1–2 centros	Árvores
Nota/sequência de graus pedida	Erdős–Gallai / Havel–Hakimi ou Landau (torneio)	Construção gulosa pelo maior grau	Grafos, Construtivos
Dependência por grupos de ancestral (primeiro ancestral com propriedade)	Monotonia ao subir	Binary lifting com busca em O(log); ou grafo de saltos	Árvores, Grafos
Remover vértice/aresta, "e se esta aresta sumisse"	Árvore de caminhos mínimos / DFS tree	Aresta fora da árvore cobre caminho (replacement path com DSU de pulo)	Grafos, Árvores
"Cada elemento tem 2 opções" com conflitos pareados	2-SAT	Cláusulas + SCC; cardinalidade ≤ 1 por variáveis de prefixo	Grafos
Soma de f(i) em intervalo com histórico de versões	Tempo como dimensão	Segtree com soma histórica (acumulado ao longo das versões) ≠ persistente por versão (consulta por versão)	Estruturas
Pontos sob reta, "acima/abaixo" em muitas queries	Dualidade (a, b) ↔ y = ax − b	CHT / Li Chao / hull	Geometria, Estruturas, DP
Retângulo/união/cobertura em plano	Sweep em x, eventos de abertura/fechamento	Segtree em y com (cobertura, comprimento)	Geometria, Estruturas
Ordenar pontos por ângulo	Semiplano + cross (não atan2)	Sweep angular com dois ponteiros	Geometria
Triângulos/conjuntos "contêm a origem"	Complemento: todos no mesmo semiplano	Ordenar por ângulo + C(k, 2)	Geometria, Combinatória
Manhattan com muitas queries	Rotacionar (x + y, x − y) ou 4 octantes	BIT/segtree por octante, MST Manhattan	Geometria, Estruturas, Grafos

Vi no enunciado	Então	Logo tente	Aparece em
Query em intervalo só com add (sem remoção)	Mo com rollback / duas pilhas	Mo rollback, DSU rollback	Estruturas
Fila/janela com operação sem inverso (gcd, matriz)	Duas pilhas com agregados	Janela deslizante em O(1) amortizado	Estruturas, Arrays
Vetor xor máximo em subarray	Prefix linear basis com timestamp	Base linear persistente	Estruturas, Bits
"Primeiro i com condição" em estrutura de intervalo	Monotonia + agregado no nó	Descida na segtree/BIT em vez de binária+query	Estruturas, Busca binária
Conectividade com arestas ativas em intervalo de tempo	Insira aresta nos O(log) nós do tempo	Segtree no tempo + DSU rollback	Estruturas, Grafos
Problema geométrico com n pequeno e n^2 eventos	Rotação de direção; ordem muda em pares	Rotating sweep em $O(n^2 \log n)$	Geometria
Elemento/reta/ponto "comum a ≥ fração δ"	Amostrar e verificar	$1/\delta \cdot \log(1/\varepsilon)$ tentativas	Aleatorização, Geometria
Corte/conectividade em grafo com remoções	Valor aleatório em cada aresta, xor	Hash de corte por xor	Aleatorização, Grafos
Verificar identidade algébrica (matriz, polinômio)	Avaliar em ponto aleatório	Freivalds / Schwartz–Zippel	Aleatorização
Otimização sem estrutura clara, TL longo	Vizinhança + delta O(1)	Simulated annealing / hill climbing	Aleatorização
Fila de n+1 itens e módulo n; "existe subarray com soma divisível"	n + 1 prefixos em n restos ⇒ dois iguais	Pigeonhole direto; construção: guarde o primeiro índice de cada resto	Combinatória, Arrays, Construtivos
Sequência de n números distintos; pede sequência crescente ou decrescente longa	Erdős–Szekeres: LIS·LDS ≥ n	Um dos dois ≥ √n; use o rótulo (LIS até i, LDS até i) como par único	Arrays, Combinatória
2n−1 inteiros (ou mais) e "n com soma divisível por n"	Erdős–Ginzburg–Ziv	Existência garantida; construção por indução em fatores primos / brute	Combinatória, Construtivos
Ordem de aplicar operações em árvore/floresta com precedência pai → filho	Ordens topológicas = n!/Π tamanho(v)	Multiplique inversos modulares dos tamanhos de subárvore	Árvores, Combinatória
Contar árvores com graus dados	Código de Prüfer	(n−2)!/Π(d_i−1)!; Cayley n^(n−2)	Árvores, Combinatória
Contar spanning trees de um grafo	Matrix-tree	Determinante do Laplaciano (menor) por Gauss mod p primo (pivôs invertíveis); p composto → CRT/Bareiss; n ≤ 400	Grafos, Combinatória
Caminhos disjuntos em vértices, k pares (s_i, t_i) em DAG	Lema LGV	Det da matriz de contagem de caminhos; exige fontes/sumidouros compatíveis (ordem não cruzada), senão o determinante é assinado e pode zerar	Combinatória, Grafos
Domino/ladrilhos em tabuleiro n × m com n ≤ 10	Estado = perfil de bordas	DP broken profile O(n·m·2^n); m enorme → matriz	DP, Combinatória
Soma Σ_i Σ_j f(gcd(i, j)) ou Σ gcd(i, n)	Agrupe por d = gcd	Σ_d f(d)·(contagem de pares com gcd d) via φ ou Möbius	Teoria dos números, Combinatória
C(n, k) mod 2, resposta mod 2	Lucas p = 2	Ímpar ⇔ k & (n − k) = 0 ⇔ k ⊆ n; paridade de soma de binomiais por bits	Combinatória, Bits
Xor de todos os subarrays/pares/subconjuntos	Cada valor aparece uma quantidade c de vezes	Só importa c mod 2: conte quais têm contagem ímpar (subarrays: i·(n − i + 1); pares não ordenados: n − 1; ordenados: 2(n − 1); subconjuntos: 2^{n − 1})	Bits, Arrays
"Maximizar xor de caminho" em grafo geral	Qualquer caminho = caminho da árvore + combinação de ciclos	Base linear dos ciclos fundamentais; guloso do bit alto	Bits, Grafos

Vi no enunciado	Então	Logo tente	Aparece em
Operação "a_i ^= a_j" repetida	Espaço gerado é invariante	Gauss em GF(2) com bitset; posto, base canônica	Bits, Construtivos
Contagem polinomial em n de grau d pequeno	Resposta é polinômio de grau ≤ d	Calcular d+1 valores por brute e interpolar (Lagrange); "n até 10^{18}"	Combinatória, DP
Recorrência linear desconhecida, termos pequenos fáceis	Se for linear de ordem d, 2d termos bastam	Berlekamp–Massey + Kitamasa/matriz para n enorme	DP, Combinatória, Teoria dos números
Σ_{i ≤ n} i^k, k até 10^3 e n até 10^{18}	Polinômio de grau k + 1 em n	Interpolação de Lagrange com k + 2 pontos, ou Bernoulli	Combinatória, Teoria dos números
Grau-sequência dada, "existe grafo simples"	Condição de realização	Erdős–Gallai / Havel–Hakimi; torneio: Landau; matriz 0/1: Gale–Ryser	Construtivos, Grafos
Multigrafo bipartido k-regular	Hall ⇒ matching perfeito	Decomponha em k matchings perfeitos; Δ-coloração de arestas (König)	Grafos, Construtivos
Torneio (um sentido por par) e "caminho passando por todos"	Rédei: todo torneio tem caminho hamiltoniano	Inserção binária ou merge sort com comparador "u vence v"	Grafos, Construtivos
Grafo conexo com m par, "decomponha em caminhos de 2 arestas"	DFS pós-ordem	Pareie arestas incidentes no filho; sobra a aresta ao pai	Grafos, Construtivos
Orientar arestas para ficar fortemente conexo	Sem pontes (2-aresta-conexo) é condição necessária e suficiente	DFS: árvore para baixo, retorno para cima	Grafos
k vértices de grau ímpar em um grafo conexo	Cobertura por max(1, k/2) trilhas (k = 0: 1, grafo euleriano)	Ligar pares de ímpares com arestas virtuais e achar Euler	Grafos, Construtivos
Cobrir caminhos/DAG com menos caminhos	V − matching no grafo split	Disjunto em vértices; se podem se sobrepor, fecho transitivo antes	Grafos, Combinatória
Maior antichain / maior conjunto sem relação em poset	Dilworth	Tamanho = n − matching no fecho transitivo (ou min cadeias)	Grafos, Combinatória
Maior conjunto independente em grafo bipartido	König	n − matching máximo; reconstrua pelo corte (alcançáveis no residual)	Grafos
Escolher intervalos com cada ponto coberto ≤ k vezes, max peso	Matriz de intervalos consecutivos é TU	Fluxo de custo mínimo numa linha de pontos, capacidade k	Grafos, Arrays
Atribuir tarefas com janela [l_i, r_i] e capacidade 1 por slot	Hall só para intervalos contíguos	Checar #itens com janela ⊆ [L, R] ≤ R − L + 1; ou EDF com heap	Arrays, Grafos
Tarefas com prazo e lucro, 1 unidade de tempo	Matroide de agendamento	Ordene por lucro e use DSU para o último slot livre	Arrays, Estruturas
Rotular cada vértice com A/B, custo unário e custo se vizinhos diferem	Energia submodular	Corte mínimo s-t (segmentação, "projeto" generalizado)	Grafos
Custo convexo por unidade numa aresta	Marginais crescentes	Duplique a aresta em cópias de custo marginal, min cost flow	Grafos, DP
Restrição de razão (min Σa/Σb) num ciclo	Binária em λ	Karp só para média por aresta (Σa / número de arestas); com b arbitrário, binária em λ + Bellman-Ford com custo a − λb	Grafos, Busca binária
Vários agentes se espalham ao mesmo tempo, "quem chega antes"	Frente simultânea	BFS/Dijkstra multi-fonte guardando a origem (Voronoi em grafo)	Grafos
"Se removermos o vértice v, u ainda é alcançável?"	Pertinência na árvore dominadora	Árvore dominadora (Lengauer–Tarjan) em grafo direcionado/DAG	Grafos
Restrições "maior aresta no caminho" / "caminho de gargalo"	MST contém o gargalo	Árvore de reconstrução de Kruskal: LCA = peso do gargalo	Grafos, Árvores
Grafo com pesos de aresta pequenos W, distâncias grandes	Dial	Bucket queue por distância em O(m + nW); estados (v, tempo) até n·W são outra técnica (DP em camadas)	Grafos

Vi no enunciado	Então	Logo tente	Aparece em
Passos grande k em grafo, "caminho com exatamente k arestas"	Potência da matriz de adjacência	Min-plus ou contagem, exponenciação por doubling	Grafos, DP
Terminais k ≤ 10 e custo mínimo para conectá-los	Árvore de Steiner	dp[mask][v] com 3^k·n + 2^k·m log	Grafos, DP
Grafo com m arestas e "triângulos / pequenos subgrafos"	Oriente de menor para maior grau	Grau de saída ≤ √(2m); triângulos em O(m√m)	Grafos, Estruturas
Árvore e random walk com esperança de passos	Tempo de atingimento por aresta	E[T de u a v adjacentes] = 2·(arestas no lado de u) + 1; soma pelo caminho	Árvores, Combinatória
Formigas/bolas idênticas que colidem e voltam	Atravessam sem interagir	Trajetória livre; a ordem relativa fica, só os rótulos trocam	Arrays, Construtivos
Jogador A e B alternam, últimos a mover perde	Misère	Nim: igual ao normal exceto todas as pilhas ≤ 1; outros: brute força a diferença	Teoria dos jogos
Tokens independentes em DAG ou pilhas somadas	Sprague-Grundy	xor dos Grundy; token único sem soma → só vitória/derrota	Teoria dos jogos, Grafos
Jogo com lance que o oponente pode desfazer	Lance reversível exige definição precisa (posição ≤ original, jogo finito)	Não é simplesmente apagar o lance: substitua-o pelas opções da resposta	Teoria dos jogos
Jogo em tabuleiro retangular de alinhamento/conexão	Roubo de estratégia	1º nunca perde; Chomp, Hex: 1º ganha, a questão é construtivo ou espelho	Teoria dos jogos, Construtivos
Jogo "token em grafo não-direcionado, vértices não repetem"	Geography bipartido	Ganha quem move de v ⇔ v está em todo matching máximo	Teoria dos jogos, Grafos
Jogo de Euclides ou subtração com números grandes	Quociente ≥ 2 dá escolha	Simule Euclides e conte lances forçados por paridade	Teoria dos jogos, Teoria dos números
Jogo de moedas (virar moedas)	Cada cara é jogo independente	Soma: xor de g(i) por posição de cara (Turtles: g(i) = i; Ruler: maior potência de 2 em i)	Teoria dos jogos
Jogo de palavras (prefixo de dicionário) com k rodadas	Trie + (pode forçar ganhar, pode forçar perder)	Se ambos → 1º ganha; só ganhar → paridade de k; senão → 2º	Teoria dos jogos, Strings
Jogo com itens usados de 1..n e soma-alvo	Estado = máscara	popcount dá a vez; dp[mask] com n ≤ 20	Teoria dos jogos, DP
Jogo infinito/ciclos, resultado "empate"	Retrógrada com contador de saídas	Vitória se algum sucessor é derrota; derrota se todos são vitória; resto empate	Teoria dos jogos, Grafos
Grundy grande demais ou n 10^{18}	Eventual periodicidade (só garantida com conjunto de subtração finito)	Brute 10^4 valores é heurístico: ache período e pré-período, confirme e depois consulte por mod	Teoria dos jogos, Teoria dos números
Operação associativa sem inversa em janela deslizante	Duas pilhas	Fila a partir de duas pilhas guardando agregado; O(1) amortizado	Estruturas, Arrays
Operação associativa sem inversa, queries estáticas, n $5 \cdot 10^5$	Disjoint sparse table / sqrt tree	Query O(1) com 2 acessos	Estruturas
Estrutura só suporta insert e rollback, mas dados têm remoção arbitrária	Segtree no tempo	Offline: cada item vive num intervalo; DFS na segtree com rollback	Estruturas, Grafos
Elemento majoritário em subarray, sem estrutura pesada	Boyer–Moore não é associativo (o merge preserva só a maioria estrita)	Segtree de (candidato, contagem); verificação final por posições + binária	Estruturas, Aleatorização
Transformações compostas x → a·x + b sobre intervalos	Monoide não comutativo	Segtree que guarda (a, b) respeitando a ordem de composição	Estruturas, Combinatória
Computar sobre pares com a_i = a_j em vez de todos	Limite de multiplicidade	$\Sigma c_v^2 \le n \cdot \max c$; agrupa por valor; pesado/leve em c_v	Estruturas, Arrays
Pontos e retas em y = ax + b, "melhor reta num x"	Dualidade ponto-reta	(a, b) viram pontos; envelope = fecho convexo ⇒ CHT / Li Chao	Geometria, DP

Vi no enunciado	Então	Logo tente	Aparece em
Restrições lineares em 2 variáveis, maximizar linear	Programação linear 2D	Interseção de semiplanos; ótimo em um vértice	Geometria
Minimize Σ |x_i − t_i| com restrições x_i ≤ x_{i+1}	Slope trick	Heap com pontos de quebra; ótimo O(n log n)	DP, Arrays
Custo convexo em cadeia de variáveis (Lipschitz, monotônico)	f convexa por partes	Slope trick: guarde as inclinações em heaps	DP, Estruturas
Mochila com Σ pesos pequena e muitos itens iguais	Pesos distintos ≤ √(2S)	Binary splitting ou deque por resto; bitset para viabilidade	DP
Partição em somas com n 10^5 e soma ≤ 10^5	Subset sum com multiplicidades	bitset com divisão binária (n√n/64)	DP, Estruturas
Strings: bordas/períodos de prefixo	Lema de periodicidade	Bordas formam O(log n) progressões aritméticas; prefixo-função em O(n)	Strings
Contar padrões que casam com curinga	Vira convolução	FFT/NTT por caractere ou por soma de quadrados	Strings, Combinatória
Reflexões e espelhos (bolas, luz, bilhar)	Espelhe o mundo	Trajetória reta em grade replicada; resultado por gcd/mod	Geometria, Teoria dos números
Problema de olimpíada "mostre que sempre há..."	Extremo/invariante/pigeonhole provam	Transforme a prova em construção (indução forte + reduzir caso)	Construtivos, Combinatória
Soma de produtos A·B sobre pares de blocos	Divisão e conquista + FFT	Convolução com D&C para recorrência tipo online (CDQ + FFT)	Combinatória, Estruturas
Árvore de decisão limitada (k perguntas, N respostas)	N ≤ b^k	Limite inferior de queries é ⌈log_b N⌉; ache a pergunta que divide em b partes	Construtivos, Teoria dos jogos
Soma/convolução sobre pares com índices pequenos (≤ 10^6)	Vira produto de polinômios	NTT/FFT; mod arbitrário → 3 NTTs + CRT	Álgebra, Strings, Combinatória
Contagem de estruturas rotuladas conexas	Total = exp(conexas) em EGF	ln/exp de série formal (O(n log n))	Álgebra, Combinatória, Grafos
n até 10^{18} e recorrência linear de ordem k	Termo n = x^n mod polinômio característico	Kitamasa/Bostan–Mori; BM para achar recorrência	Álgebra, DP, Combinatória
Resposta é polinomial em n (parece)	grau d ⇒ d+1 pontos bastam	Brute força d+2 valores, depois Lagrange	Álgebra, Combinatória, Olimpíada
Σ_{k≤n} k^d com n enorme	Polinômio de grau d + 1 em n	Lagrange O(d) ou Bernoulli	Álgebra, Teoria dos números
Contar árvores geradoras / ciclos eulerianos	Determinante do Laplaciano	Matrix-tree, BEST	Álgebra, Grafos, Combinatória
k caminhos disjuntos em DAG/grid	Sinal de permutação cancela cruzamentos	LGV (determinante) com fontes/sumidouros compatíveis; senão o determinante é assinado e pode zerar	Álgebra, Grafos
Sistema de equações xor	Linear sobre GF(2)	Gauss com bitset, 2^{n − posto}	Álgebra, Bits
"Todo subconjunto" com n ≤ 20 e combinação OR/AND/XOR	Transformadas de conjuntos	zeta/Möbius, subset convolution, Walsh	Álgebra, DP, Bits
Existência garantida por pombal / Erdős–Szekeres	Só precisa construir	Prefixos mod n, LIS/LDS, EGZ	Olimpíada, Construtivos
Operação que sempre reduz uma grandeza inteira	Monovariante limita passos	Simule até estabilizar ou conte inversões	Olimpíada, Construtivos, Arrays
Grafo/instância sem K_{2,2} ou sem triângulo (separar)	Sem K_{2,2}: m = O(n^{3/2}) e Σ C(d_v,2) = O(n^2); sem triângulo: m ≤ $n^2/4$	Pares de vizinhos em O(n^2) só vale para K_{2,2}-free; triangle-free pode ter Σ C(d_v,2) = Θ(n^3) (K_{n/2,n/2})	Olimpíada, Grafos

Vi no enunciado	Então	Logo tente	Aparece em
Grafo planar com n grande	$m \le 3n - 6$, degenerescência ≤ 5	Orientação grau ≤ 5; dual para corte/fluxo	Olimpíada, Grafos
"Maximize/minimize" com certificado	Dual do LP	Fluxo = corte, Hall/König, potenciais da Hungarian	Otimização, Grafos
Escolhas sem ciclo / $\le c$ por cor / lin. independentes	Conjuntos formam matroide	Guloso por peso (1 matroide) ou interseção de 2 matroides (polinomial; 3+ é NP-difícil)	Otimização, Grafos, Bits
f(k) com k itens parece convexa	Penalize cada item com λ	Aliens/WQS + busca binária em λ	Otimização, DP, Busca binária
Custo convexo por unidade de fluxo	Divida em arestas unitárias crescentes	Min-cost flow com custos marginais	Otimização, Grafos
Mover unidades em linha ou círculo	Custo = $\Sigma \lvert\text{prefixo}\rvert$	Mediana dos prefixos (círculo)	Otimização, Arrays
Piso/teto com $a \cdot i + b$ e $n \approx 10^9$	Reta com pontos de rede	Floor sum (Euclides)	Olimpíada, Teoria dos números
Sequência $\{k\alpha\}$, "girar por α"	Equidistribuição / três intervalos	Frações contínuas, Stern–Brocot	Olimpíada, Teoria dos números
Esperança de tempo em passeio linear	Existe potencial com $\Delta = -1$	Martingal / optional stopping	Olimpíada, Combinatória
Grafo denso com Dirac/Ore	Hamiltoniano garantido	Pósa/rotações para construir	Olimpíada, Grafos
Problema min-max com atribuição	Verificação monótona	Binária + fluxo; checar capacidades	ICPC, Busca binária, Grafos
Alcançabilidade em DAG com $n \sim 5 \cdot 10^4$	Fechamento transitivo	SCC + bitset em blocos	ICPC, Grafos, Bits
Path em árvore, contagem por distância	Todo caminho passa por um centroide	Centroide + FFT ($A^2 - \Sigma \text{filhos}^2$)	ICPC, Árvores, Álgebra
DP com min sobre ancestrais e termo bilinear	Hull por caminho da raiz	CHT/Li Chao persistente ou com rollback	ICPC, DP, Árvores
Queries sobre intervalo de arestas/tempo	Estrutura só com adição + desfazer	Mo com rollback ou segtree no tempo + DSU	ICPC, Estruturas, Grafos
Substring/automaton em DP	DAG do SAM / AC	DP no SAM, DP em AC + matriz/digit DP	ICPC, Strings, DP

Arrays e guloso

Primeira pergunta: a ordem dos elementos importa? Se não, ordene.

- **Ordenar** por valor, por fim de intervalo, por razão a/b, ou pela comparação de troca (a antes de b se $a+b < b+a$). Prove com exchange argument.
- **De trás pra frente.** O último elemento costuma ser forçado; decisões do fim não dependem do começo.
- **Two pointers / janela.** "Subarray com soma $\le S$", "no máximo k distintos" → se encolher a janela nunca piora, é janela deslizante.
- **Prefix sums + map.** Contar subarrays com soma/xor = k → para soma, conte prefixos iguais a pref − k; para xor, prefixos iguais a pref $\oplus$ k.
- **Pilha monotônica.** Próximo maior/menor, maior retângulo, contribuição de cada elemento como máximo/mínimo.
- **Deque monotônico.** Máximo em janela deslizante; DP com transição de janela.
- **Heap guloso.** Escolher o melhor disponível a cada passo; "arrependimento" (tire do heap e troque se vale a pena).
- **Intervalos.** Ordene por fim para máximo de disjuntos; por início para cobrir; sweep de eventos (+1 no início, −1 no fim) para sobreposição.
- **Operação em subarray/prefixo.** Olhe o array de diferenças: operação em intervalo vira 2 mudanças pontuais.
- **Mediana.** Minimizar $\Sigma|a_i - x|$ → x = mediana. Minimizar $\Sigma(a_i - x)^2$ → média.
- **Elementos ruins são poucos.** Se só importam os k maiores/menores, mantenha só eles.
- **Paridade/posição.** Swaps adjacentes → número de inversões; elementos em posições pares/ímpares separados.

- **Contar pares i < j com condição.** Ordene e use binária, BIT por valor, ou merge sort (inversões).
- **Guloso quebrou?** Gere um contraexemplo com brute force; quase sempre vira DP.

adicionado em 2026-11-10 por agentes; revisado

- **Scheduling: minimizar atraso máximo.** Jobs com duração e prazo, minimizar max(C_i − d_i) → ordene por prazo (EDF); exchange argument: trocar um par invertido nunca piora.
- **Scheduling: minimizar Σ w_i·C_i.** Ordene por t_i/w_i crescente (regra de Smith); compare t_i·w_j < t_j·w_i para evitar divisão e double.
- **Jobs com prazo e lucro, 1 unidade cada.** Ordene por prazo, empurre lucros num min-heap e, se o tamanho passar do prazo atual, remova o menor. O heap final é a resposta.
- **Intervalos: mínimo de pontos para furar todos.** Ordene por fim e coloque o ponto no fim do primeiro intervalo não furado. Mesmo problema que "máximo de disjuntos", em dupla.
- **Intervalos: mínimo de salas = máxima sobreposição.** Para atribuir de fato, ordene por início e use heap com os fins; reaproveite a sala cujo fim ≤ início.
- **Intervalos com k máquinas, maximizar quantos atender.** Ordene por fim; para cada intervalo escolha a máquina livre com maior fim < início (intervalos fechados; use multiset + lower_bound e recue um; semiabertos usam ≤ e upper_bound); se não há, descarte.
- **Cobrir [0, L] com o mínimo de intervalos.** Ordene por início; entre os que começam ≤ cobertura atual, pegue o que alcança mais longe; se nenhum avança, impossível.
- **Empates em sweep de eventos.** Intervalos fechados [l, r]: processe abertura antes de fechamento no mesmo ponto; semiabertos [l, r): fechamento antes. Errar isso muda a sobreposição máxima por 1.
- **Pareamento na reta (pontos com pontos).** Minimizar Σ|a_i − b_σ(i)| ou o max |a_i − b_σ(i)| → ordene os dois lados e case em ordem; vale pela desigualdade do rearranjo.
- **Desigualdade do rearranjo.** Σ a_i·b_σ(i) é máximo com ambos ordenados do mesmo jeito e mínimo com ordens opostas. Serve para "atribuir pesos a posições/arestas por frequência de uso".
- **Pontos × segmentos (atribuir cada ponto a um segmento que o contém).** Varra por x; heap dos segmentos abertos ordenado por fim; dê o ponto ao segmento que expira primeiro.
- **Heap com arrependimento: comprar/vender ações.** O push duplo (vender e empurrar p duas vezes) superconta vendas sobrepostas: [0,1,4,7] retorna 10, mas o máximo é 7. Para o modelo "uma ação em mãos por vez", some as diferenças positivas adjacentes: Σ max(0, p_i − p_{i−1}).
- **Escolher k elementos não adjacentes de soma máxima.** Lista ligada + heap; ao escolher i, substitua os vizinhos e i por um nó de valor v_l + v_r − v_i (arrependimento). Repita k vezes.
- **Cortar em k segmentos minimizando/maximizando soma de custos que dependem dos cortes.** Se o custo é aditivo nos cortes, escolha os k − 1 melhores cortes com sort; vale para "soma de (último − primeiro) por bloco" → soma total menos os k − 1 maiores buracos.
- **Tornar array não decrescente com mínimo Σ|a_i − b_i| (slope trick).** Max-heap: empurre a_i; se top > a_i, ans += top − a_i, pop e empurre a_i de novo. Estritamente crescente: troque a_i por a_i − i.
- **a_i − i.** Subsequência "a_j − a_i ≥ j − i" vira não decrescente em b_i = a_i − i; "|a_i − a_j| = |i − j|" vira iguais em a_i − i ou a_i + i (diagonais).
- **Exatamente k distintos / soma exatamente k.** exato(k) = noMáximo(k) − noMáximo(k − 1) com janela deslizante; para soma com valores ≥ 0 vale o mesmo.
- **Janela com max − min ≤ k.** Dois deques (um de max, um de min); avance o início até a condição voltar. Cada elemento entra e sai uma vez.
- **Contribuição com empates na pilha monotônica.** Para Σ max de todos os subarrays, um lado usa ">" estrito e o outro ">=", senão subarrays com valores iguais são contados duas vezes ou nenhuma.
- **Σ (max − min) de todos subarrays** = Σ max − Σ min, cada um com pilha monotônica; evite fazer os dois juntos numa estrutura só.
- **Maior submatriz de 1s** → histograma por linha (altura acumulada) + maior retângulo com pilha; O(nm). A contagem de submatrizes de 1s usa a mesma pilha com DP de "largura × altura".
- **Árvore cartesiana** em O(n) com pilha: max de [l, r] é o LCA; recursão "divida no máximo" tem profundidade n mas pode ser iterada; serve para "subarrays em que o máximo é k-ésimo", small-to-large pelo lado menor.
- **Menor subsequência lexicográfica de tamanho k / remover k dígitos.** Pilha: enquanto topo > novo e ainda sobram remoções (ou elementos suficientes), desempilhe. Com "cada caractere distinto uma vez": guarde última ocorrência e "já na pilha".
- **Circular com guloso (posto de gasolina).** Resposta = posição após o mínimo do prefixo de saldos; se a soma total < 0, impossível. Substitui busca binária ou simulação 2n.

- **Mediana dinâmica** (janela deslizante) → dois multisets balanceados ou dois heaps com remoção preguiçosa; custo $\Sigma|a_i - \text{med}|$ sai das somas de cada metade.
- **Tornar elementos consecutivos / formar progressão com +1.** Minimizar $\Sigma|a_i - (x + i)|$ → mediana de $b_i = a_i - i$.
- **Elemento > n/2 em O(1) de memória** → Boyer–Moore (candidato + contador); para > n/k mantenha k − 1 candidatos. Sempre confirme o candidato com uma segunda passada.
- **Mex de prefixos é não decrescente; mex de todos os subarrays** → dois ponteiros sobre contagens, ou segtree sobre o último índice de cada valor.
- **Subarray com maioria / "mais x que y".** Troque por ±1 e use prefixos (mínimo de prefixo à esquerda, BIT se for contar). Maioria estrita → soma > 0.
- **Subarrays com soma ≥ 0 com negativos.** Two pointers não vale; use prefixos + BIT/merge sort. Two pointers só serve quando estender monotonicamente muda a condição (valores ≥ 0).
- **Menor operações +1/−1 em subarray para zerar/igualar.** No array de diferenças d, cada operação muda dois pontos em ±1; resposta ≈ $\Sigma \max(0, d_i) = \Sigma|d_i|/2$ (cuidado com as bordas de d_0 e d_{n+1}).
- **Prefixo 2D e diferenças 2D.** Soma de retângulo = P[r2][c2] − P[r1−1][c2] − P[r2][c1−1] + P[r1−1][c1−1]; adicionar retângulo = 4 pontos em D e depois prefixo 2D. Muitas adições + 1 leitura final → offline barato.
- **Maior soma de submatriz.** Fixe duas linhas, comprima colunas por prefixo e rode Kadane: $O(\min(n, m)^2 \cdot \max(n, m))$.
- **Transponha** quando $n \cdot m \le 10^6$ mas $n \gg m$: itere sobre o lado pequeno e deixe o loop interno no grande (cache e complexidade).
- **Rotações/reflexões de grid**: 8 simetrias em quadrado; para "mesmo padrão a menos de rotação" tome a forma canônica (mínima entre as 4/8).
- **Meet in the middle (n ≤ 40 com 2 escolhas; n ≈ 24–26 com 3).** Gere as somas das duas metades, ordene e use dois ponteiros; contagem de subconjuntos com soma ≤ S, ou xor/max mod M, cabem aqui. 3^(n/2) quando cada item tem 3 escolhas.
- **4-sum / soma de quatro (n ≤ 2000).** Armazene todas as somas de pares em hash/sorted e procure o complemento: $O(n^2)$; com três listas A + B + C ≥ x faça n^2 + two pointers.
- **Permutação é um conjunto de ciclos.** Trocas arbitrárias mínimas = n − #ciclos; trocas adjacentes mínimas = #inversões; paridade da permutação = (n − #ciclos) mod 2.
- **Potência de permutação.** p^k quebra um ciclo de tamanho c em gcd(c, k) ciclos de tamanho c/gcd(c, k); ciclo de c e gcd(c,k)=1 vira um ciclo só. Raiz quadrada: ciclos ímpares podem ficar sozinhos, ciclos pares só existem aos pares do mesmo tamanho.
- **Mínimo de trocas adjacentes para transformar string/array A em B.** Mapeie a i-ésima ocorrência de cada valor de A para a i-ésima de B (preserva ordem relativa) e conte inversões da permutação resultante.
- **"Mova elemento para o início/fim" para ordenar.** Os que ficam parados são os maiores valores consecutivos já em ordem de posição (só início), ou um bloco contíguo de valores em ordem (início e fim); resposta = n − tamanho do maior bloco.
- **LIS.** O(n log n) com vetor de caudas: lower_bound para estritamente crescente, upper_bound para não decrescente (trocar errado é bug clássico). Reconstrução precisa de pais por posição.
- **Dilworth em arrays.** Mínimo de subsequências estritamente crescentes que cobrem o array = tamanho da maior subsequência não crescente; o guloso "anexe x à subsequência cujo último elemento é o maior < x; se não houver, abra uma nova" (multiset) constrói a partição.
- **LIS com restrições extras (k-LIS, LIS em subárvore, LIS pesada)** → dp[v] = max sobre valores < v indexado por valor comprimido num BIT de max; peso entra como soma.
- **Permutação com k inversões exatas.** Insira os valores do menor ao maior; ao inserir o i-ésimo você escolhe quantas inversões novas (0..i−1), então qualquer k ≤ n(n−1)/2 é construível por guloso.
- **Inversões de a + b ou "inversões após operação"**: separe em pares que mudam e pares que não; swap de i, j com valores distintos muda inversões por um número ímpar (paridade sempre inverte); trocar iguais não muda nada.
- **Cuidado com overflow** em contagens de pares (n^2 com $n = 2 \cdot 10^5 = 4 \cdot 10^{10}$), somas de prefixos ($2 \cdot 10^5 \times 10^9$) e produtos cruzados ($a \cdot d$ vs $b \cdot c$ com valores 10^9 → __int128).
- **Comparador do sort deve ser ordem fraca estrita.** `<=` ou comparador não transitivo (ex.: com razão em double) causa UB/crash; use cross-multiplication em inteiros e desempate por índice.
- **Off-by-one em intervalos.** Padronize [l, r) em código e converta na entrada; prefixos P[0] = 0 com tamanho n + 1; diferenças com tamanho n + 2.
- **Sentinelas** (±∞, ou um elemento extra no fim) removem casos de borda em pilha monotônica, sweep e merge de intervalos.
- **Guloso com duas chaves (vantagem $a_i - b_i$).** Escolher para cada item o lado A ou B com cotas → comece com todos em B e escolha os k com maior ganho $a_i - b_i$; com cotas mínimas e máximas, ordene por ganho e corte nas cotas.

- **Dois recursos por item e orçamento (comprar com dinheiro e cupons).** Guloso por custo marginal com dois heaps, ou DP com troca de estado; teste o guloso contra brute force antes de confiar.
- **Ordenar por comparação de concatenação.** Menor/maior número formado por concatenar strings: ordene por a+b < b+a; prova por transitividade da comparação numérica.
- **Pilha para colisões e cancelamentos** (robôs que se anulam, parênteses reduzidos, apagar pares adjacentes): o resultado final é canônico e independe da ordem.
- **Sparse table / pré-cômputo antes do guloso**: se o guloso consulta max/min de intervalo, não reordene o array, use RMQ O(1) para manter n log n.

Repetidas de outras seções:

- **Σ tamanhos ≤ S** → ≤ √(2S) tamanhos distintos; agrupe por tamanho.
- **Soma/balanço = 0** → prefixos iguais (também em strings de parênteses e xor).
- **gcd/OR/AND de subarray** → ≤ log mudanças por fim fixo.
- **Jogo em array** → pontas, paridade de posições, DP em intervalo (ver Teoria dos jogos).

Busca binária e otimização

Gatilho principal: "minimizar o máximo" ou "maximizar o mínimo" → binária na resposta.

- **Binária na resposta** sempre que a checagem "dá com x?" é monótona e mais fácil que achar o ótimo.
- **k-ésimo menor** de um conjunto implícito (somas de pares, frações, produtos de tabela) → binária em x e conte quantos ≤ x.
- **Média/razão máxima** (Σa/Σb) → binária em r e verifique se max Σ(a − r·b) ≥ 0.
- **Binária paralela.** Várias queries cada uma com sua binária e uma estrutura cara para simular → processe todas juntas por nível.
- **Binária na segtree** (descer na árvore) em vez de binária com query: tira um log.
- **Função unimodal** → ternária; inteiros → binária na diferença f(x+1) − f(x).
- **Função convexa na quantidade** (custo com exatamente k itens) → Aliens trick / lambda optimization.
- **Binária + guloso**, **binária + DP**, **binária + fluxo/matching** são combinações comuns: a checagem quase nunca é o próprio problema.

---

adicionado em 2026-11-10 por agentes; revisado

- **Binária em reais.** Use um número fixo de iterações (100) em vez de `hi - lo > eps`, que pode não terminar com doubles grandes; imprima com 9+ casas e confira se o erro exigido é relativo ou absoluto.
- **Binária inteira sem loop infinito.** Defina o invariante (lo = ok, hi = ruim) e use mid = lo + (hi − lo)/2 com a atualização consistente; para "maior x ok" use mid = (lo + hi + 1)/2. Negativos: `/2` arredonda para 0, não para baixo.
- **Limite superior desconhecido.** Busca exponencial (1, 2, 4, ...) até o check falhar e depois binária no último intervalo; útil em interativos com n grande e valores 10^{18}.
- **mid*mid ou mid*k em 10^{18}.** Faça o check em __int128 ou compare com divisão (mid ≤ lim / k) para não estourar.
- **Binária em array bitônico ou rotacionado.** Compare a[m] com a[m+1] (pico) ou com a[0] (rotação); platôs de valores iguais quebram a monotonicidade e forçam O(n) no pior caso.
- **Dinkelbach para razão máxima Σa/Σb.** Itere r ← (Σa)/(Σb) do conjunto ótimo para o r atual; converge superlinearmente em poucas iterações e dá o valor exato, sem binária em real.
- **k-ésimo de A[i] + B[j].** Binária em x, conte pares ≤ x com dois ponteiros nos dois arrays ordenados; para k-ésimo de A[i]·B[j] com sinais, divida em quatro casos.
- **k-ésimo em matriz com linhas e colunas ordenadas / tabela de multiplicação n × m.** Binária em valor + "escada" O(n + m) ou soma min(m, ⌊x / i⌋) por linha.
- **k-ésima menor fração p/q com q ≤ n.** Binária em real x, conte frações ≤ x com dois ponteiros e guarde a maior fração ≤ x vista; se as frações são reduzidas/distintas, a contagem por piso repete representações — use Farey/Stern–Brocot (ou Möbius) para o rank exato, pois só a maior ≤ x não basta.
- **Mediana/k-ésimo de dois arrays ordenados** → binária na partição (quantos tirar do primeiro) em O(log min(n, m)), comparando as bordas cruzadas.
- **Binária em estruturas.** BIT (só com frequências não negativas): descida por potências de 2 acha o menor prefixo com soma ≥ x em O(log); segtree: descida pelo nó em vez de binária + query; sparse table: saltos 2^k decrescentes; árvore: binary lifting para "primeiro ancestral que viola".
- **Binária sobre o tempo com DSU** ("primeiro instante em que u e v conectam"): Kruskal reconstruction tree, ou binária paralela com um DSU percorrido uma vez por nível.

- **Binária paralela (variante D&C).** Em vez de rodar a estrutura log vezes, resolva(lo, hi, queries): aplique os eventos de [lo, mid], decida cada query para esquerda/direita, desfaça (rollback) e recurse; $O((n + q) \log^2 n)$, pois o DSU rollback sem compressão de caminho paga O(log n) por operação.
- **Monotonicidade precisa ser provada.** Se check(x) = "existe solução com valor x" e esticar x pode piorar (ex.: tamanho exato em vez de ≤), a binária falha; reformule como ≥ ou ≤.
- **Check mais barato que n log.** Pré-ordene fora da binária; um check O(n) + binária bate sort dentro do check. Dois ponteiros costuma tirar a binária interna ("menor janela" não precisa de binária + prefixo).
- **Maximizar a menor distância** (colocar k itens numa reta, "vacas agressivas") → binária em d + guloso que coloca o próximo ponto o mais cedo possível.
- **Partição em k blocos minimizando o maior soma** → binária em S + guloso que fecha o bloco quando estoura; é o padrão de "minimizar o máximo" com 2 níveis.
- **Gargalo (bottleneck).** Caminho que minimiza a maior aresta = caminho na MST; binária + DSU/BFS funciona mas é mais lenta; "min cap de fluxo ≥ F" → binária na capacidade + max-flow.
- **Aliens com empates.** Se há vários k ótimos para o mesmo λ, escolha sistematicamente o menor (ou o maior) número de itens e compare com K; fixe a convenção: com $val(\lambda) = \max_k [f(k) - \lambda \cdot k]$ a resposta é $val(\lambda) + \lambda \cdot K$, e com $val(\lambda) = \min_k [f(k) + \lambda \cdot k]$ é $val(\lambda) - \lambda \cdot K$. λ inteiro basta quando custos são inteiros.
- **Aliens clássicos.** Escolher exatamente k subarrays disjuntos de soma máxima; MST com exatamente k arestas vermelhas (somar λ às vermelhas); fatiar o array em k blocos com custo quadrangular; "exatamente k trocas/compras" em ações.
- **Teste de convexidade para Aliens.** Calcule f(k) por brute force para k pequeno e verifique $f(k+1) - f(k)$ monótona; se for côncava/convexa só por trechos, Aliens falha.
- **Ternária em inteiros.** Platôs que não estão no ótimo quebram a ternária; use binária em f(m) vs f(m+1) se estritamente unimodal, ou golden section com 1 avaliação por passo.
- **Convexidade fecha por soma e max.** Soma de convexas é convexa ($\Sigma|x - a_i|$ convexa), max de convexas é convexa, composição afim também; isso justifica ternária em "minimizar a maior distância" e em "ponto ótimo de custo agregado".
- **Ternária aninhada** em duas variáveis (f conjuntamente convexa — Hessiana PSD; convexidade separada em x e em y não garante que $g(x) = \min_y f(x, y)$ seja convexa/unimodal): $O(\log^2 \cdot \text{custo})$, ou resolva um eixo analiticamente.
- **Convex hull trick por binária.** Consultas arbitrárias numa envoltória de retas em ordem de inclinação → lower_bound por interseção; consultas monótonas → ponteiro; sem ordem → Li Chao.
- **Sqrt inteiro e raiz k-ésima** → calcule com sqrtl e corrija ±1 em um while, ou binária inteira; double perde precisão acima de 2^{53}.
- **Binária em ângulo / raio / tempo contínuo** quando o check é "círculo de raio r cobre tudo" ou "distância no tempo t": o check geométrico costuma ser sweep angular ou interseção de intervalos.
- **Slope trick** (função convexa linear por partes mantida em dois heaps) para dp[i][x] = min convexa em x com transições "min de janela" e "+ |x − a_i|".
- **Busca em f contínua sem monotonicidade (Bolzano).** Se f(l) e f(r) têm sinais opostos existe raiz; binária acha uma raiz mesmo sem ser monótona.

DP

Pergunta-chave: o que eu preciso lembrar do passado para decidir o futuro? Isso é o estado.

Escolhendo o estado:

- **Prefixo** dp[i]; **intervalo** dp[l][r] quando juntar/remover pedaços; **subárvore** dp[v]; **máscara** dp[mask] com n ≤ 20; **dígitos** dp[pos][tight][...] para contar até 10^{18}.
- **Troque estado e valor.** dp[i][valor grande] = possível? → dp[i][resposta pequena] = menor valor. (Mochila com peso 10^9 e valor 10^3: dp[valor] = menor peso.)
- **Modele pela última operação**, não pela primeira (você tende a usar a primeira: vale testar a outra).
- **Estado com "o que ainda está aberto".** Parênteses abertos, grupos não fechados, componentes conectadas (connected components DP para permutações).
- **Conte objetos ruins com DP** quando inclusão-exclusão tem defeitos que conflitam.
- **Probabilidade/esperança** → linearidade; dp de trás pra frente a partir do estado final.

Otimizando a transição:

- $dp[i] = \min_j (dp[j] + b_j \cdot a_i)$ → **CHT / Li Chao**.
- $opt(i)$ monótono em $dp[k][i] = \min_j dp[k-1][j] + cost(j, i)$ → **D&C DP**; em intervalos, com quadrangle inequality e w monótono → **Knuth** (não misture com o D&C).

- Transição sobre um intervalo de j → **prefix min / deque / segtree** sobre dp.
- Transição linear com n até 10^{18} → **exponenciação de matriz** (também em semi-anel min-plus).
- Somas sobre subconjuntos/supermáscaras → **SOS DP**.
- Mochila só de booleanos → **bitset** (÷64). Itens com soma total S → só √(2S) pesos distintos.
- Estados pouco alcançáveis → **map/memo** em vez de array.
- "Exatamente k" com custo convexo em k → **Aliens trick**.
- DP em árvore que você precisa para toda raiz → **rerooting**.
- Merge de DPs de filhos com tamanho → **small-to-large** ou soma $O(n^2)$ total pelo argumento de tamanhos.

adicionado em 2026-11-10 por agentes; revisado

- **Digit DP, forma geral.** Conte f(N) = #{x ≤ N com propriedade} com estado (pos, tight, started, resto da propriedade); responda [L, R] como f(R) − f(L−1). "started" evita contar zeros à esquerda quando o dígito 0 importa (ex.: dígitos distintos, soma de dígitos com 0 proibido).
- **Digit DP com soma/produto sobre os números.** Guarde no estado a dupla (quantidade, soma dos valores) e combine: soma' = soma_filho + dígito·10^pos·qtd_filho. Serve para "soma de todos os x ≤ N com propriedade".
- **Digit DP em duas ou mais cotas (a ≤ A, b ≤ B, a op b).** Um flag tight por número; para AND/XOR/OR de pares, estado (bit, tightA, tightB) e 3 flags de 2 estados cada; "a xor b ≤ K" tem terceiro flag.
- **Digit DP em base b com restrição de gcd/mod grande.** Se mod é grande (10^9) não cabe no estado; troque por contagem de múltiplos por fórmula ou use meet in the middle nos dígitos (metade alta × metade baixa).
- **Digit DP + autômato.** "Números ≤ N sem substring proibida / com padrão" → estado do Aho-Corasick/KMP no lugar do resto; mesmo molde de contar strings.
- **K-ésimo número com propriedade** → faça a digit DP de contagem e desça dígito a dígito escolhendo o menor prefixo cuja contagem acumulada passa de k (contagem + construção).
- **Broken profile (plug DP simples).** Grid n × m com m ≤ ~12–15: dp[célula][máscara de m bits] processando célula a célula, em vez de linha a linha; transição O(1) por célula em vez de O(2^m) por linha. Ladrilhos de dominó/L, independentes em grade, colorir sem vizinhos iguais.
- **Plug DP com conectividade.** Para caminho/ciclo hamiltoniano em grid ou "um único ciclo": estado = emparelhamento de plugs (parênteses) na fronteira, codificado em base 3 ou 4 (0 = sem plug, 1 = abre, 2 = fecha). Estados alcançáveis são poucos (Motzkin): guarde em hash map.
- **Máscara por linha com compatibilidade.** dp[linha][máscara] com máscaras válidas pré-filtradas (sem bits vizinhos): só Fib(m) máscaras; transição vira soma sobre máscaras compatíveis (pré-calcule listas ou use SOS no complemento).
- **Subconjuntos de máscara, 3^n.** dp[mask] = melhor partição de mask em grupos: itere submáscaras (3^n). Para n até 16–18 cabe; se o grupo contém sempre o menor bit de mask, corta um fator constante (fixa o menor elemento).
- **TSP / hamiltoniano contando.** dp[mask][último] em 2^n·n^2; para contar ciclos hamiltonianos com memória polinomial, IE sobre subconjuntos de vértices com contagem de passeios fechados em 2^n·n^3 — inviável em n = 25; use n ≲ 20 ou variante O(2^n·n^2).
- **Bitmask DP sobre posição + máscara de últimos k elementos.** Quando a restrição envolve só vizinhança de comprimento k (ex.: sem k iguais seguidos, k ≤ 10–20) → a máscara é a janela, deslizante.
- **Permutações como máscara de "já usados" com custo por posição** (alocação, atribuição com n ≤ 20): dp[mask] = custo mínimo para colocar os |mask| primeiros itens; a posição é popcount(mask), não precisa ir no estado.
- **Árvore knapsack $O(n^2)$.** dp[v][k] = melhor com k vértices da subárvore; ao fundir filho c, limite os loops por min(k, sz): a soma total é $O(n^2)$ (cada par de vértices é "visto" uma vez no LCA). Vale também para "k folhas / k arestas / k componentes".
- **Árvore knapsack com limite K.** Cortar os índices em min(sz, K) dá O(nK) total. Truque de ordem DFS (dp sobre a ordem dfs: pegar v implica ir para tin+1, pular implica ir para tout) deixa a DP linear em n·K sem merge.
- **DP sobre ordem DFS (pegar/pular subárvore).** Para "escolher conjunto fechado para cima / dependência pai→filho com orçamento": dp[i][j] sobre a posição i na ordem de Euler; escolher i desce para i+1, não escolher salta para tout[i]. Vira knapsack linear.
- **DP em árvore com estado "o que sobe".** O que a subárvore precisa comunicar ao pai (cor da raiz, se a raiz está coberta, comprimento da cadeia aberta). Cobertura de vértices/dominating set: 3 estados (escolhido, dominado por filho, ainda precisa de pai).
- **Rerooting com operação não invertível.** Calcule para cada vértice prefix e suffix dos resultados dos filhos; up[c] = combine(up[v], prefix[i−1], suffix[i+1]). Com max/min/gcd é o único jeito sem divisão.
- **Rerooting com contribuição.** Resposta para toda raiz de Σ dist, Σ f(dist): mantenha cnt e soma por subárvore e mova a raiz da aresta (v→c) atualizando ±sz[c]; quando f não é linear, guarde os momentos de f (centroide é alternativa, não obrigação).
- **DP em DAG de estados implícito.** Se o grafo de estados é um DAG (tempo/posição cresce), dp com memo recursiva evita ordenar; verifique profundidade da recursão e converta para iterativa se > 10^5.

- **DP em intervalo clássica $O(n^3)$.** dp[l][r] = min_k dp[l][k] + dp[k+1][r] + custo(l, r,): multiplicação de matrizes, estourar balões (escolha o ÚLTIMO a estourar, não o primeiro), triangulação de polígono; remover caixas precisa de dp[l][r][k] (tipicamente $O(n^4)$), não cabe em 2D.
- **Intervalo: escolha o último elemento a ser tratado.** Quando remover no meio muda vizinhos, pense em qual elemento é removido por último no intervalo: então os dois lados ficam independentes. Balões, cortar vara, merge de pilhas.
- **Intervalo com string: dp[l][r] para palíndromos, edição, "strange printer".** Se s[l] = s[r] une os extremos (dp[l+1][r−1] + c); impressora estranha: dp[i][i] = 1, dp[l][r] = dp[l][r−1] + 1 e, para k com s[k] = s[r], dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r−1]).
- **Intervalo com n grande mas estrutura de pilha.** "Remover pares adjacentes casáveis" pode virar DP $O(n^2)$ se a forma canônica é única (parênteses), ou O(n) guloso com pilha. Teste se algum guloso funciona antes.
- **Intervalo circular.** Duplique o array (tamanho 2n) e calcule dp[l][r] com r − l < n; a resposta é o melhor dp[i][i+n−1]. Para otimização em círculos de remoção ("matar vizinhos"), fixe um elemento como ponto de corte.
- **Knuth (intervalo).** opt[l][r−1] ≤ opt[l][r] ≤ opt[l+1][r] dá $O(n^2)$ total para dp[l][r] = min_k dp[l][k] + dp[k+1][r] + w(l, r) com w satisfazendo desigualdade do quadrângulo e monotonia. Árvore de busca binária ótima, merge ótimo de arquivos adjacentes.
- **Hu-Tucker / Garsia-Wachs** quando o problema é "juntar adjacentes com custo = soma" e n grande: O(n log n), não precisa Knuth. Se pode juntar qualquer par, é Huffman (heap).
- **D&C DP.** dp[k][i] = min_{j<i} dp[k−1][j] + C(j, i) com opt monótono: resolva a camada recursivamente (mid → opt, divida) em O(n log n) por camada; custo C(j, i) precisa ser O(1) via prefix sums (ou Mo-like movendo ponteiros em O(n log n) total).
- **Custo C(j, i) = soma de pares dentro do segmento** (ex.: nº de pares iguais, inversões no bloco) → D&C DP + ponteiros que se movem como Mo (adicionar/remover nas pontas); complexidade O(k·n log n) com ponteiros amortizados.
- **Desigualdade do quadrângulo (Monge) como teste.** C(a, c) + C(b, d) ≤ C(a, d) + C(b, c) para a ≤ b ≤ c ≤ d → opt monótono (D&C, Knuth) e função convexa em k (Aliens). Verifique com brute force em valores pequenos antes de apostar.
- **SMAWK / matriz totalmente monótona.** Mínimo por linha de matriz Monge em O(n) (sem log do D&C); raro, mas é a saída quando D&C dá TLE por log extra ou em min-plus de convexa × Monge.
- **Otimização 1D/1D (dp[i] = min_{j<i} dp[j] + C(j, i)) com Monge.** Quando dp[j] depende de dp anteriores da mesma camada, D&C simples não vale: use pilha de candidatos + busca binária (Knuth online) ou CDQ + D&C em $O(n \log^2 n)$.
- **CHT clássico.** dp[i] = min_j (m_j·x_i + b_j): inclinações monótonas e consultas monótonas → deque O(n); caso contrário Li Chao (O(n log n)) ou CHT dinâmico com set. Aparece em "custo = (soma)² + C" após expandir $(S_i - S_j)^2$.
- **Reconhecer CHT por expansão.** Custo de segmento $(S_i - S_j)^2$ ou (a_i − a_j)·b_i etc.: abra o produto; termos só de i saem do min, termos só de j viram intercepto, o termo cruzado S_i·S_j vira reta de inclinação S_j avaliada em S_i.
- **Li Chao em árvore / persistente.** DP em árvore com dp[v] = min_{u ancestral} (dp[u] + b_u·a_v): Li Chao com rollback (desfaz inserção ao sair do vértice) em O(n log n); ou small-to-large mesclando Li Chaos.
- **Divide e conquista CDQ para DP online.** dp[i] depende de dp[j] com j < i por uma estrutura que só aceita ordem offline: resolve(l, mid), aplique contribuição de [l, mid) em [mid, r), resolve(mid, r). Convolução online, CHT com consultas fora de ordem.
- **Convolução online (relaxada).** dp[i] = Σ_{j<i} dp[j]·f[i−j] (f conhecida): CDQ com NTT, $O(n \log^2 n)$. Contagem de partições ordenadas, sequências com blocos, "composições com pesos".
- **Sequência P-recursiva / equação diferencial do GF.** Se dp tem recorrência de coeficientes polinomiais, calcule n termos em O(n) mesmo sem fechado (ex.: involuções, números de Motzkin, Bell via outra recorrência); ache a recorrência por chute e ajuste (guess P-recursive) com brute force pequeno.
- **Recorrência linear de ordem pequena e n enorme.** Exponencie matriz $O(k^3 \log n)$; com k ~ 100–2000 e módulo primo use Kitamasa/Bostan–Mori em O(k log k log n) ou $O(k^2 \log n)$. Se só tem a sequência, Berlekamp–Massey recupera a recorrência com 2k termos.
- **Berlekamp–Massey como detetive.** Calcule 2k termos de uma DP por brute force e rode BM mod p: se a ordem achada é pequena (≤ ~50), a sequência é linear recorrente e n grande sai por Kitamasa. Funciona para contagem em grid de largura fixa (transfer matrix).
- **Contagem de caminhos de comprimento L em grafo pequeno, L enorme** → potência de matriz de adjacência; para resposta "soma sobre todos comprimentos até L" aumente a matriz com estado acumulador (n+1) ou use bloco [[A, I],[0, I]].
- **Min-plus matrix power.** Menor custo de exatamente L arestas em grafo com n ≤ ~100 e L grande → exponenciação em semi-anel (min, +); matriz com custos 0..c pequenos pode ser compactada expandindo vértices em cadeias.
- **Distribuição estacionária / absorvente.** Probabilidade de terminar em cada estado absorvente: sistema (I − Q)x = r por Gauss mod p ou em double; n ≤ ~500. Se o grafo é quase DAG (poucos ciclos), elimine variáveis só nos ciclos.
- **Esperança com ciclos nos estados.** E[v] = 1 + Σ p·E[w] com possibilidade de voltar: se cada estado só volta para si mesmo, E[v] = (1 + Σ_{w≠v} p E[w])/(1 − p_vv). Se volta para estados anteriores numa cadeia, escreva E[i] = a_i·E[0] + b_i e feche E[0] por equação linear (variável simbólica).

- **Esperança por linearidade em vez de DP.** Antes de montar DP de esperança, tente somar probabilidades de indicadores (E[#inversões], E[#componentes], E[tempo] = Σ P(passo k ainda ativo)). Muitos problemas que parecem DP de estados viram fórmula O(n).
- **Passeio aleatório em linha/árvore.** Tempo esperado de atingir vértice em árvore: E(v→pai) = 2·sz − 1 (arestas na subárvore + 1); equações de diferença em cadeia resolvem com telescópio sem Gauss. Cover time/hitting time: use resistência elétrica (hitting em grafo: E_ij + E_ji = 2m·R_ij).
- **Probabilidade de vitória com estado pequeno + soma de 1 − p.** Para esperança de "repita até sucesso" com custo por tentativa, E = custo/p_sucesso; com várias etapas independentes, soma de E por etapa (linearidade, não multiplique).
- **Coupon collector generalizado.** E[passos para ver todos de n] = n·H_n; para completar o conjunto (tipos distintos) ou um subconjunto fixo: dp[j] = esperança dado j já coletados, E[j] = (n/(n−j)) + E[j+1], de trás pra frente; para k cópias de cada tipo o estado é o vetor de contagens, não só j.
- **Contar sem esperança: dp[estado] como distribuição, não valor.** "Probabilidade de terminar com valor = x" → dp[i][x] com transições por probabilidade (mod p usa inverso); só vira esperança se só o valor médio interessa (reduz uma dimensão).
- **DP em autômato.** Contar/otimizar strings de comprimento L que evitam/contêm padrões: dp[pos][estado do Aho-Corasick] com L ≤ 10^3 e Σ|padrões| ≤ 10^3; para L enorme use matriz (estados) com potência.
- **DP no autômato do KMP (um padrão).** dp[i][j] = nº de strings de comprimento i com borda j; ou "mínimo de trocas para não conter p": construa tabela de transições aut[j][c] em O(|p|·26) e use dp[i][j].
- **LCS entre uma string e conjunto de padrões / subsequência com Aho** → estado duplo (posição, nó do autômato) quando padrões são pequenos; Σ pequeno do alfabeto mantém a tabela manejável.
- **Contar ocorrências de t como subsequência de s** (ou número de pares (i < j) de padrão fixo) → para cada caractere de s, percorra j de |t| até 1 e faça dp[j] += dp[j−1] se s_i = t_j (j decrescente evita reusar o mesmo caractere); O(|s|·|t|) com 1 array.
- **Menor supersequência comum / LCS reconstruída** → tabela dp[i][j] completa (Hirschberg se memória O(min) com reconstrução).
- **LCS com permutações → LIS.** Se as duas sequências são permutações (ou cada valor aparece uma vez), LCS = LIS da sequência de posições, O(n log n). Valores repetidos: mapeie ocorrências em ordem decrescente de posição para manter a corretude.
- **LCS bitset.** Para |s|, |t| até ~10^5 com alfabeto limitado: dp por linha em bitset, atualização x = row | match; row = x & (x ^ (x − ((row << 1) | 1))); O(n^2/64). Também edit distance bitset (Myers) para distância pequena.
- **LCS/edit distance com distância ≤ k pequena.** Só a faixa diagonal |i − j| ≤ k importa → O(nk); ou Landau–Vishkin com LCP via hash em O(n + k^2).
- **LIS com contagem.** Para contar LIS use BIT sobre valores guardando (comprimento, quantidade) com merge por max e soma nos empates; ou segtree com o mesmo par.
- **LIS em O(n log n) com reconstrução.** Guarde para cada posição o índice anterior e o índice da menor cauda de cada comprimento. "Mínimo de remoções para ficar ordenado" = n − LIS (não-decrescente: use upper_bound).
- **Cobertura mínima por cadeias (Dilworth).** Mínimo de subsequências crescentes disjuntas cobrindo tudo = tamanho da maior decrescente (não-estrita); mesmo em O(n log n) com patience sorting. Aparece como "quantas pilhas/trilhos".
- **LIS em k trocas / com updates.** Valores pequenos (≤ 1000) → dp[i][último] com BIT; "LIS com alterações pontuais" é difícil; pense em offline com CDQ (LIS 2D) em vez de segtree dinâmica.
- **Maior subsequência com soma de pesos (LIS ponderada).** dp[i] = w_i + max_{j<i, a_j<a_i} dp[j] → BIT de max por valor; mesma ideia para "subsequência com |a_i − a_j| ≤ d" (segtree de max sobre valores no intervalo [a_i − d, a_i + d]).
- **Subsequência com restrição de vizinhos (|a_i − a_j| ≤ d, j anterior)** → dp sobre valores com segtree de max: dp[a_i] = max_{[a_i−d, a_i+d]} + 1. O(n log V).
- **Mochila 0/1 O(nW) com 1 array.** Itere W de cima para baixo para 0/1; de baixo para cima para ilimitada. Quase todo bug de mochila é a ordem.
- **Mochila limitada por binary splitting.** Quantidade c_i → itens de 1, 2, 4, ..., resto: O(nW log c). Com Σ c_i·w_i ≤ S ainda dá √S pesos distintos e cada um pode ser dividido ao mesmo tempo.
- **Mochila limitada com fila monótona.** Para cada resíduo r mod w_i, dp'[r + t·w] = max_{t−c ≤ s ≤ t} dp[r + s·w] + (t − s)·v: deque sobre dp[s] − s·v; O(nW) sem log.
- **Mochila limitada só de viabilidade (subset sum com multiplicidade).** Para cada peso w com cnt c: dp'[x] = true se dp[x] ou dp'[x − w] com usos < c; guarde used[x] mínimo usado do peso atual. O(nW) simples sem deque.
- **Subset sum com bitset.** dp |= dp << w; O(nW/64). Com Σ pesos ≤ S e pesos distintos ≤ √(2S): agrupe e use binary splitting + bitset (S√S log S/64).
- **Subset sum com pesos pequenos e soma grande (Pisinger / balanced).** Soma de n itens com pesos ≤ C: existe solução com complexidade O(nC) independente de W (algoritmo de Pisinger: comece de uma solução gulosa e vá e volta no máximo ±C).

- **Subset sum modular / pigeonhole.** n itens com n ≥ m: algum subconjunto soma 0 mod m (prefixos); existência garantida → o problema é construir, não decidir (Erdős–Ginzburg–Ziv: entre 2m − 1 inteiros existem m com soma ≡ 0 mod m).
- **Mochila com valor pequeno e peso gigante.** dp[v] = menor peso para obter valor v; se Σ v ≤ 10^5 roda em n·Σv mesmo com W = 10^9. Já está na seção base, mas vale também para "contar # de itens" como valor.
- **Mochila onde itens têm peso 1, 2, 3 só (ou poucos tipos).** Ordene cada tipo por valor decrescente, prefix sums por tipo, enumere quantidades de dois tipos e ache o terceiro por busca (função côncava em quantidade) → O(W^2 log W): enumerar dois tipos já é Θ(W^2).
- **Valores côncavos por grupo → merge guloso.** Mochila onde cada grupo tem ganho marginal decrescente (função côncava de qtd) → heap/ordenar todos os incrementos marginais; custo unitário igual vira guloso.
- **Mochila min-plus convexa.** Mesclar duas funções convexas (min-plus convolução) = mesclar suas diferenças ordenadas em O(n); arrays convexos em cada filho de árvore (ex.: knapsack com itens de custo crescente) somam por merge de listas de inclinações.
- **Mochila de "mochila de √" (pesos distintos).** Σ pesos ≤ S → ≤ √(2S) pesos distintos; combinado com cnt_i·w_i ≤ S dá total O(S√S) em vez de nS. Também: itens com w ≤ √ resolvidos por DP e itens com w > √ são ≤ √ no total usados.
- **Mochila com contagem (número de modos)** → dp[x] += dp[x − w]; para "tamanho da lista com restrição de ordem de escolha" cuidado: ordem importa → itere W por fora (composições) vs itens por fora (combinações).
- **Moedas: ordem importa ou não?** Combinações (multiconjuntos) → itens por fora, valor por dentro. Sequências ordenadas (composições) → valor por fora, itens por dentro. A troca de laços é o erro mais comum e muda a resposta.
- **DP em partições de n (√n).** Partição de n em partes distintas tem ≤ √(2n) partes; partição com partes ≤ k: dp[i][j] = dp[i][j−1] + dp[i−j][j]; O(n√n) via "partes ≤ √n (mochila) + número de partes ≤ √n (conjugada)": p(n) = DP de partes pequenas + DP de poucas partes grandes.
- **Pentagonal number theorem.** Contar partições p(n) para n até 10^5–10^6 em O(n√n) via p(n) = Σ ±p(n − k(3k∓1)/2); para só p(n) de um n isolado use Hardy–Ramanujan–Rademacher se n gigantesco (raro).
- **Contar partições com partes distintas = partes ímpares** (Euler) — troca de modelo pode dar DP mais simples; também "diagrama de Young transposto" troca "k partes" por "maior parte = k".
- **Insertion DP (permutações).** Construa a permutação inserindo n elementos em ordem de valor/tempo; dp[i][j] = nº de arranjos dos i primeiros elementos em que o i-ésimo elemento ficou na posição relativa j. Para contar permutações com restrição tipo "a_i < a_{i+1}" (sinais <,>) use prefix sums em j (O(n^2)).
- **Connected components DP (permutações / sequências com condição entre vizinhos).** Insira elementos em ordem crescente; estado dp[i][c] = nº de componentes (blocos) após i elementos; cada novo elemento cria componente nova (c+1−gap), cola em uma ponta (2c) ou une duas (c−1). Contagem de permutações com |p_i − p_{i+1}| restrito, soma de diferenças adjacentes fixa (adicionar ao estado a soma acumulada = 2·c·δ).
- **Soma de |p_i − p_{i+1}| por componentes.** Ordene valores crescentes, a cada passo o custo acumulado é (nº de componentes abertas·2)·(a_{i+1} − a_i); dp[i][c][soma] com soma ≤ K pequeno. Problemas "contar arranjos com soma total de diferenças ≤ K".
- **Contagem de permutações por sinais (zigzag/up-down).** dp[i][j] = nº de permutações de i com último elemento ocupando posto j; transição por prefix sum. Para n 10^5 use EGF: sec + tan (números de Euler ziguezague) ou divisão de polinômios.
- **Contar permutações com k inversões.** dp[i][k] = Σ_{t=0}^{i−1} dp[i−1][k−t] → prefix sum, O(n·K); para K ≤ n(n−1)/2 grande e n pequeno é o produto Π(1 + x + … + x^{i−1}) — use polinômio.
- **Mínimo de adjacentes trocados / swap distance** = inversões; para "transforme a em b com swaps" mapeie a para posições em b e conte inversões (BIT). DP só se houver restrições extras.
- **DP em permutação circular.** Fixe o elemento 1 (ou o menor) numa posição para quebrar a simetria; contar n arranjos circulares = (n−1)! × linear; para DP circular sobre a escolha de itens (nenhum vizinho adjacente), rode duas vezes: primeiro e último excluídos de um deles.
- **Escolher k itens sem dois adjacentes (círculo/linha).** dp[i][j][pegou i-1?] em O(nk); com n grande e k pequeno há também o guloso heap com lista ligada (escolha o melhor, substitua por vizinhos − atual) — o "arrependimento" certo para max soma sem adjacentes com k.
- **Heap + lista ligada vs DP.** "Escolha k não-adjacentes" onde dp é côncava em k → o guloso com arrependimento dá a resposta exata para cada k; use também como alternativa ao Aliens.
- **Aliens / WQS (lambda).** "Exatamente k" com f(k) convexa: busque λ tal que o ótimo da DP com penalidade λ por item use k itens; resposta = g(λ) − λ·k, com g(λ) = min_k [f(k) + λ·k]. Cuidado com empates: guarde o MENOR (ou maior) número de itens entre ótimos para a binária ser coerente; λ inteiro basta quando f é inteira.

- **Aliens: como reconhecer convexidade.** Se existe fluxo de custo mínimo de fundo (k unidades de fluxo), ou matroide/guloso com arrependimento para custo mínimo (máximo em matroide é côncavo, não convexo), f é convexa; se k é "número de segmentos" com custo Monge, também. Sem prova, valide com brute force imprimindo f(k) para k pequeno e checando segundas diferenças.
- **Aliens aninhado (duas restrições k_1, k_2).** Dois λ em binária dupla, $O(\log^2 \cdot DP)$; só funciona se f é convexa em ambos jointly (ex.: certos problemas de emparelhamento); desconfie e teste.
- **Slope trick: reconhecer.** dp[i][x] = min_y dp[i−1][y] + |x − a_i| com restrição em y−x (monotonia, |x−y| ≤ d) e dp[0] convexa → dp[i] continua convexa linear por partes; guarde só os pontos de quebra e um valor mínimo.
- **Slope trick com translação.** "Janela deslizante" desloca os heaps esquerdo e direito por offsets lazy diferentes (addL, addR); inserção de |x − a| move um ponto entre heaps. Custo total O(n log n).
- **SOS DP.** f[mask] = Σ_{sub ⊆ mask} a[sub] em O(2^n·n); também superset e "subconjuntos com um bit por vez" (zeta, e Möbius para inverter). Para contar pares (i, j) com a_i & a_j = 0: SOS no complemento.
- **SOS com min/max e "dois melhores".** Para "melhor par dentro do submask" guarde os 2 melhores com índices distintos por máscara; mescla de top-2 em cada passo.
- **Convolução OR/AND/XOR por SOS e Walsh-Hadamard.** (a ⊛_OR b)[m] = Möbius(zeta(a)·zeta(b)); XOR → WHT (2^n n). Contagem de subconjuntos com OR/xor fixo = potência (exponencial por FWHT).
- **Subset convolution** (união disjunta) em $O(2^n n^2)$: ranked zeta — um polinômio por popcount. Aparece em "particione conjunto em k partes com função de peso multiplicativa" (exp de conjunto).
- **DP de contagem de partições de conjunto em máscara.** dp[mask] = Σ_{sub ∋ menor bit de mask} g[sub]·dp[mask \ sub], em 3^n; com subset convolution + exp de GF por conjuntos em $2^n n^2$.
- **Meet in the middle em DP/knapsack.** n ≤ 40 com valores grandes: divida em metades, enumere 2^{n/2} somas, ordene e use two pointers; para "soma ≤ W maximizando valor" com valor também, mantenha pareto (valor crescente com peso crescente) e busque binário.
- **Reduza o estado por monotonia ou dominância.** Se dp[i][j] é monótono em j (mais j nunca pior), guarde só a fronteira (j mínimo por valor / estados Pareto); a DP vira "lista de pares não dominados" e para de estourar.
- **Troca de dimensão por valor da DP (generalizada).** Se n é pequeno mas as coordenadas enormes, ou se o valor da resposta é pequeno (≤ log ou ≤ √): indexe dp[resposta] = melhor parâmetro. Ex.: dp[k] = menor posição final com k escolhidos (LIS é isso).
- **Resposta limitada por log ou √.** Se o valor ótimo é provadamente ≤ O(log n) ou ≤ O(√n) (ex.: ovos/andares — com 2 ovos é Θ(√n), não log, e com e ovos é Θ(n^{1/e}) — e menor número de operações de dobra), faça dp[j][ops] = alcance máximo com ops tentativas — troca estado e valor; ovos: f(t, e) = f(t−1, e−1) + f(t−1, e) + 1.
- **DP de pior caso (minimax) com resposta log.** "Mínimo de tentativas no pior caso" → inverta: quantos casos distinguíveis com t tentativas; O(t·k) em vez de $O(n^2 k)$.
- **Meio-DP: balanceie dimensões.** Se dp[i][j] tem n ≤ 10^5 e k ≤ 20, a transição k·n·log pode entrar; se k ≤ ~√n e n grande, trocar quem vai no estado (segmentos × posição) por dp[k][i] ou o contrário muda log.
- **DP sobre pequena dimensão: n·m ≤ 10^6.** Itere pela menor dimensão no estado de máscara/profundidade e pela maior no laço; se m ≤ 20, máscara na menor; se ambos ≤ 10^3, DP O(nm) rolando linha.
- **Redução de memória: rolling array.** dp[i] só olha dp[i−1] → 2 linhas; só olha dp[i−1][j] e dp[i−1][j−1] → 1 linha iterando j decrescente; Hirschberg para reconstruir caminho em O(min) memória (divide pela linha do meio e guarda onde cruza).
- **Reconstrução do caminho com memória apertada.** Guarde só um bit "de onde veio" por estado (bitset), ou refaça a DP para trás comparando valores (re-derive a escolha), ou use D&C de Hirschberg; evite guardar ponteiros de 32 bits em dp $10^4 \times 10^4$.
- **Memória de DP em árvore.** dp[v][·] pode ser liberado após o merge ao pai (vector move); com knapsack $O(n^2)$ e n=5000 a memória é O(n) simultâneo se processar filhos grandes primeiro (heavy-first) — ou use ordem DFS com alocação por tamanho.
- **DP em DAG vira caminho mínimo/longo.** Se estados são vértices e transições arestas com custo, dp em ordem topológica é caminho mais curto/longo; com ciclos de custo ≥ 0 use Dijkstra sobre estados em vez de DP por camada.
- **DP que vira caminho mínimo (e vice-versa).** dp[i] = min_{j: condição} dp[j] + w e as dependências formam ciclos (ex.: pode ir "para trás") → DP em ordem não funciona, use Dijkstra/0-1 BFS sobre i; teste: a transição tem custo ≥ 0 e o grafo não é DAG por posição?
- **Caminho mínimo em grafo de estados pequenos com k grande.** dist[v][estado] com estado = (mod k, paridade, nº usos); "k-ésima operação" muda o custo ciclicamente → Dijkstra em v × (tempo mod T) com T pequeno (ex.: semáforos).
- **Guloso vira DP: sinais.** (1) exchange argument falha em empate de custos, (2) o melhor local depende de escolhas futuras (mochila), (3) restrição de capacidade ou contagem exata, (4) pequenos casos mostram escolhas diferentes com mesma entrada local. Sinal forte: n ≤ 5000 ou "exatamente k".

- **DP vira guloso/estrutura: sinais.** dp[i] = melhor sobre prefixo e a escolha ótima parece sempre "a mais recente barata" → pilha monotônica; dp côncava/convexa em k → heap de arrependimento ou Aliens; dp[i] = max_{j<i} f(j) + g(i) → prefixo.
- **Inverta quem é o índice (dp sobre valores em vez de posições).** Se a_i ≤ $2 \cdot 10^5$ e n grande, dp[v] = melhor sequência terminando no valor v (com BIT/segtree) evita estado de posição; "transição por divisores/múltiplos" → harmônico em v.
- **DP com transição por divisores.** dp[x] = Σ_{d | x, d < x} dp[d] ou max sobre múltiplos: itere crivo em N log N; para subsequência com "cada elemento divide o seguinte" (maior cadeia de divisibilidade), use o crivo de múltiplos em O(V log V) (ou enumere divisores em O(n√V)).
- **DP por intervalo de valores (compactada).** Coordenadas enormes mas poucos pontos relevantes: comprima para os pontos de quebra (os próprios a_i; se a restrição envolve distância d, ótimos podem ser a_i ± k·d, até $O(n^2)$ candidatos — use slope trick se necessário) e rode dp[i][j] só sobre eles.
- **DP com restrição de distância em ordem (dist-k).** "Escolha subconjunto com índices a mais de k de distância" → dp[i] = max(dp[i−1], dp[i−k−1] + v_i) com prefix max, não precisa de estrutura.
- **DP com troca de estado: dp[i] relativo a "melhor até aqui".** Guarde dp[i] = melhor terminando exatamente em i e g[i] = max_{j ≤ i} dp[j]; separar "terminando em i" de "acumulado" simplifica transições com janelas.
- **DP em jogos.** Vitória/derrota: dp[estado] = existe movimento para estado perdedor; com pontuação: dp[l][r] = melhor diferença = max(a_l − dp[l+1][r], a_r − dp[l][r−1]); jogos que mudam quem joga → dp[estado][vez] explícito, ou negamax.
- **Jogo com pilha de moedas / passar vez / limite de lances por turno.** dp[i][m] = melhor diferença com m = máximo que pode pegar agora (clássico "pegar até 2M pedras"); estado = (restante, limite).
- **Jogo com estratégias probabilísticas / mistas.** Quando os dois escolhem simultaneamente, resolva o jogo de soma zero por LP/minimax em matriz pequena (2×2 fórmula fechada); raro, mas é onde DP "normal" falha.
- **DP em ciclo / circular com custo de fechamento.** "Costurar" início e fim: fixe o valor/estado do primeiro elemento e rode a DP linear exigindo estado final compatível (|S| execuções, ou matriz de transição e traço: Σ diagonal de produto de matrizes).
- **Traço de produto de matrizes para contagem circular.** Número de colorações de ciclo de n vértices com regra de vizinhança M (matriz k×k) = tr(M^n); para problemas "string cíclica sem padrão" calcule traço do autômato.
- **DP com bitset (otimização).** dp[i] |= dp[j] para j ∈ janela: troque laços internos por operações de 64 bits; para alcançabilidade em DAG/transitivo (n ≤ $5 \cdot 10^4$ em ~n·m/64) mantenha bitset de alcançáveis por vértice em ordem reversa topológica.
- **Bitset em DP de caminhos em grade com "alcançável?"** b = reach[i−1] & livre[i]; repita b |= (b << 1) & livre[i] até estabilizar; reach[i] = b, cada passada em O(m/64); problemas de "existe caminho/escada" com m ≤ 10^5.
- **Contagem de caminhos em grade com obstáculos (n pequeno, obstáculos poucos).** Ordene obstáculos e dp[k] = caminhos até k sem passar por outros = C(até k) − Σ dp[j]·C(j → k): $O(K^2)$ em vez de O(nm) quando grade enorme.
- **Lagrange / interpolação em DP.** Se dp(n) é polinômio em n de grau d (ex.: soma de potências, contagem de caminhos com passos limitados, soma por camada de uma DP polinomial), calcule d+1 valores por brute force e interpole (Lagrange O(d) com fatoriais para pontos consecutivos).
- **Detectar polinômio em n.** Diferenças finitas da sequência: se as diferenças de ordem d+1 zeram, é polinômio de grau d → Lagrange. Aparece em "número de formas de colorir/distribuir com k cores" (polinômio cromático) e somas Σ i^k.
- **Estado cresce linearmente mas n é enorme.** Se dp[n][j] for polinomial em n para cada j fixo, ou a sequência em n se comporta como quasi-polinômio de período fixo, calcule primeiros termos e extrapole (BM, Lagrange ou período).
- **Partições ordenadas/composições com restrição "partes em [a,b]".** dp[n] = Σ_{k=a}^{b} dp[n−k] → prefix sums O(n); com restrição em quantas partes também use GF 1D (coeficiente por polinômio com potência) e binomial.
- **Contagem de sequências com soma de elementos fixa e limites superiores** → IE sobre limites estourados (estrelas e barras com C(n − Σ(b_i+1) + k − 1, k−1)), em O(2^k) ou DP sobre IE em O(kn) se limites iguais.
- **DP + IE: contar objetos sem padrão proibido.** Calcule f[S] = # que contêm pelo menos esses defeitos e combine com sinais; se defeitos formam estrutura de segmentos, a IE vira DP linear dp[i] = soma com sinal sobre último defeito (muito comum em "sem k iguais consecutivos").
- **Inclusão-exclusão por segmentos vs. DP de complemento.** Para "nenhum bloco de k seguidos iguais": dp[i] = (m−1)·(dp[i−1] + dp[i−2] ...) ou conte o complemento com defeito marcado por IE linear; escolha o que dá O(n).
- **DP sobre soma de dígitos/valores pequenos com truque de balanço.** Sequências com soma prefixos limitada (caminhos que não cruzam): use reflexão para fechada; se a barreira é móvel ou várias, DP O(n·alt) com limite de altura (altura ≤ √n ou ≤ K).
- **Altura máxima do passeio limitada por √ / estados "|x| ≤ √n".** Em problemas de soma 1+2+...+k ≤ n o nº de passos é ≤ √(2n): dp[i][j] com i = soma e j ≤ √n = último valor. Também em partições em partes distintas.
- **DP com somas de 1+...+k ≤ n (estado "último valor")** → dp[soma][último] com último ≤ √(2n); transição por prefix sum sobre último, O(n√n).
- **Pareto / "listas não dominadas" para DP bicritério.** dp[v] = conjunto de pares (custo, tempo) não dominados; se o tamanho ficar ≤ ~√ ou log, é viável; caso contrário fixe um critério por binária (resposta) e DP no outro.

- **Binária na resposta + DP monótona.** "Menor X tal que existe divisão em ≤ k partes com custo ≤ X" → checagem greedy O(n) ou DP em O(n log n); costuma ser mais rápida e simples que DP de camada k.
- **Dividir em k segmentos minimizando o maior (ou soma).** Max: binária + guloso (exige a_i ≥ 0; com negativos use DP); soma de custos convexos: Aliens ou D&C; "partições de array em k blocos custo soma-quadrado" → CHT/D&C/Knuth conforme formato.
- **DP com técnica "pular estados inúteis" (map/memo).** Se n é 10^{18} e a DP só visita O($\log^2$) estados distintos (por divisões ⌊n/2⌋, ⌊n/3⌋), memoize em map: a recursão "min operações para reduzir n a 0 por /2, /3, −1" visita Θ(n) estados com a transição −1 literal; subtraia de uma vez até o múltiplo (n%2, n%3) para ficar O($\log^2$).
- **Floor-DP: estados ⌊n/k⌋.** Somas multiplicativas (Dirichlet) dp[⌊n/k⌋] com 2√n estados (Min_25, Du Jiao): tabelas indexadas por valores de ⌊n/i⌋ em O(n^{3/4}) ou O(n^{2/3}). Problemas "soma de f sobre 1..n" com f multiplicativa e n até 10^{10}–10^{12}.
- **Crivo Dirichlet / DP sobre primos.** dp sobre primos em ordem (soma de f(p^k) com mínimo p do fator): Min_25 em duas fases; "contar números ≤ N cujo menor fator primo é ≥ p" é o estado S(n, j).
- **DP sobre fatoração (números altamente compostos).** Número com ≤ 10^18 tem ≤ 15 primos distintos e expoentes decrescentes: DFS gerando "números altamente compostos" com expoentes não-crescentes é muito pequeno (~10^5) e resolve "maior número de divisores ≤ N".
- **Divisão em blocos (√) para DP de transição lenta.** dp[i] depende de dp[i − k·b] para b na lista: para saltos pequenos (≤ √n) mantenha tabela por resíduo; para grandes percorra diretamente. Mochila "a cada segundo você anda p_i": heavy/light por p_i.
- **Dp[i] += dp[i − b_i·k] (salto b_i, qualquer múltiplo).** b_i ≥ √n → pule direto (≤ √n saltos); b_i < √n → arrays extras por (b, i mod b) com atualização em O(1) amortizada; total O(n√n).
- **Small-to-large em DP de árvore.** Quando dp[v] é uma estrutura (map/set) e o merge é inserir elementos da menor na maior: O(n log n) (ou $\log^2$); para dp em que índice é profundidade, use "long-path decomposition" (DP de profundidade em O(n) com array deslocado: o filho mais profundo reaproveita o array do pai).
- **Long-path decomposition (DP por profundidade).** dp[v][d] = f(subárvore, profundidade d): o filho de maior altura herda o vetor (ponteiro + 1) e os outros são mesclados elemento a elemento; custo total O(n), em vez de O(n log n) do small-to-large. Para "contar vértices a distância d" ou "pares com dist = k".
- **DP de profundidade com soma ≤ ... na árvore.** Quando a restrição é sobre profundidade/distância em subárvore, não use knapsack em tamanho (n^2) — use long-path (altura) onde o estado tem tamanho = altura, não = nº de vértices.
- **Contagem de matchings / independent sets em árvore com polinômio.** dp[v] = polinômio em x (tamanho do conjunto); merge = multiplicação de polinômios por NTT com D&C por HLD (O(n $\log^2$ n)): "contar conjuntos independentes de tamanho k".
- **DP em árvore com polinômio e HLD + D&C (matriz).** Updates pontuais em dp de árvore (ex.: conjunto independente máximo com mudanças de peso): HLD + segtree de matrizes 2×2 (semi-anel max-plus) → O($\log^2$) por update.
- **Segtree de matrizes para DP dinâmica.** dp[i] = M_i · dp[i−1]; update pontual em M_i e consulta de intervalo = produto de matrizes na segtree (max-plus ou mod p). Resolve "DP com alteração de parâmetros" em O(k^3 log n) por update.
- **DP com remoção de elemento (undo).** DP "sem o elemento x" para todo x: prefixo × sufixo (combina dp do prefixo e do sufixo) em vez de refazer; para mochila contável com remoção use a operação inversa (dp[x] −= dp[x−w], válida na contagem polinomial/modular).
- **Mochila com "remover item" (undo) / adicionar online.** Contagem modular: remover = inverso da multiplicação por (1 + x^w): dp'[x] = dp[x] − dp'[x−w], O(W). Para max-plus sem inverso use D&C no tempo (segtree offline: adiciona nos nós, desfaz ao subir) em O(nW log).
- **Mochila offline com itens com intervalo de vida.** Cada item vive em [l, r] de tempo → segtree sobre o tempo, cada item entra em O(log) nós; DFS copiando a dp (memória log·W); resposta por tempo na folha.
- **Tempo como dimensão: itens com validade (D&C offline).** "Em cada instante qual a melhor mochila dos itens ativos" → a mesma segtree-no-tempo (DSU rollback também usa isso); sem rollback seja explícito com cópia da dp por nível.
- **DP em subsequências com dois índices crescentes (i < j, a_i < a_j, b_i < b_j).** LIS 2D → CDQ + BIT em O(n $\log^2$ n); com LIS ponderada, dp precisa ser calculada em ordem (CDQ com ordem de dp).
- **Conta de caminhos em DAG modulo p com n grande e k pequeno.** Contar caminhos com ≤ k arestas "de cor especial": dp[v][j] em O((n+m)k) ou matriz; se k ≤ 20 e cores, máscara.
- **Número de caminhos / maior caminho em DAG de intervalos (arestas para intervalo).** dp ordenada topológica com segtree/prefix sum sobre o intervalo de destinos; se destinos formam intervalo contíguo, dp[i] += prefix.
- **DP em string com "transição por todos os palíndromos/períodos".** Menor partição em palíndromos: eertree com séries de links (diff, slink) mantém dp em O(n log n) (já citado em strings); partição em periódicos: usa Z/KMP por início.
- **Soma sobre subsets com restrição de popcount ou peso (DP em bits).** dp[bit][carry] para somas de pares com xor/and/or: processe bit a bit com carry (0/1) e contagem; "pares (i, j) com a_i + b_j ≥ K" em bits do mais alto ao mais baixo.

- **DP sobre bits com carry.** Contar (x, y) com x + y = z, x & y = 0, ou x ≤ A, y ≤ B: dp[bit][carry][borrowA][borrowB] do bit menos significativo (limites via borrow), ou dp[bit][tightA][tightB] do mais significativo (sem carry) — não misture carry (LSB) com tight (MSB); mais simples que fórmulas.
- **Árvore de recursão de "estados equivalentes" (memo sobre estruturas).** Em jogos/contagem de configurações simétricas, normalize o estado (ordene multiconjunto, rotacione) antes de memoizar: reduz estados exponencialmente (partições, necklaces).
- **Estado = multiconjunto ordenado (partições).** n ≤ 40–60 com configuração "pilhas de pedras" ou "grupos": estados são partições (≤ 10^6 para n ~ 60); memo em map com vetor ordenado.
- **DP "dos dois lados" (prefixo e sufixo) para escolher um ponto de corte.** Resposta = max sobre i de pre[i] + suf[i+1]; com 3 cortes, mantenha estado "quantos cortes já fiz" (0..3) numa única passada.
- **Máquina de estados de poucas fases.** Problemas "no máximo k transações / 3 fases (subir-descer-subir)" → dp[i][fase] com fase ≤ k; compre-venda de ações com cooldown/taxa são autômatos pequenos, escreva o diagrama de estados antes de codar.
- **Programação dinâmica sobre "tempo" quando custo é por tempo de espera.** Fila/escalonamento com custo Σ w_i·C_i: ordene por razão (exchange argument); se há prazos e lucros → dp[t] com mochila sobre tempo ou guloso com heap (job sequencing) — diferencie.
- **Escalonamento com prazos e penalidade (1 || Σ w_j U_j).** Guloso com heap: ordene por prazo, insira, se estourou remova o de maior p_j — Moore–Hodgson só vale sem pesos (minimizar nº de atrasados); com pesos é NP-hard fraco e vira mochila (DP em tempo).
- **Maximizar nº de intervalos disjuntos com pesos.** Ordene por fim; dp[i] = max(dp[i−1], w_i + dp[último j com fim ≤ início_i]) com binária; para k máquinas/intervalos sobrepostos até k → fluxo de custo mínimo (não é matroide).
- **Cobertura por k máquinas (intervalo, "no máximo k simultâneos").** Fluxo de custo mínimo em linha do tempo com arestas de capacidade k e arestas de intervalo; é DP "não óbvia" que vira fluxo; guloso com heap vale se pesos iguais.
- **Min-cost flow como DP.** Problemas de escolher conjuntos com restrição "no máximo k cobrindo cada ponto" ou "caminhos disjuntos em DAG com k": DP em n·k não funciona, mas fluxo de custo mínimo em DAG linear resolve; reconhecer: pesos de intervalos e restrição de cobertura.
- **Estados "último que mudou" para custo de troca.** Custo de trocar de tipo (ex.: máquinas/cores) → dp[i][último] com último podendo ser comprimido: guarde só o melhor e o segundo melhor de cada grupo (top-2 em dp[i−1]) para transição O(1) por tipo.
- **Top-2 trick em transição "diferente do anterior".** dp[i][c] = custo(i, c) + min_{c' ≠ c} dp[i−1][c']: mantenha mínimo e segundo mínimo de dp[i−1] (com cor), O(n·k) em vez de O(n·k^2). Pintar casas, escalonar tarefas sem repetir.
- **DP em "duas pessoas andando" (caminhos ida-e-volta).** Dois caminhantes simultâneos com passo t: dp[t][x1][x2] com y = t − x; 2n × n × n em vez de caminhos 4D. Cereja/ida e volta em grid; se se cruzam conta só uma vez.
- **DP com dois ponteiros simultâneos em duas strings/arrays (intercalar).** Intercalação de duas sequências com restrição: dp[i][j] = pode formar o prefixo i+j da terceira com i da primeira e j da segunda; O(nm); troque por bitset se n,m ~ 10^4.
- **Contagem de caminhos Dyck com altura limitada** (parênteses balanceados com profundidade ≤ h) → dp[i][altura]; para n enorme use matriz h×h ou transfer; também fórmula por reflexão alternada (IE sobre reflexões múltiplas).
- **Contar sequências que evitam padrão consecutivo por GF.** "Sem k iguais seguidos" para n até 10^{18} → recorrência linear de ordem k−1 (ou k·alfabeto) → Kitamasa/matriz.
- **Contar árvores, florestas ou estruturas rotuladas por DP em tamanho.** dp[n] = Σ binomial·dp[i]·dp[n−1−i] (EGF); sequências rotuladas por tamanho do primeiro bloco: dp[n] = Σ C(n−1, k−1)·g[k]·dp[n−k] — a escolha do menor rótulo no primeiro bloco evita contar ordem duas vezes.
- **Estruturas rotuladas: fixe o menor rótulo.** Para contar partições/componentes de conjunto, sempre inclua o elemento 1 no primeiro bloco (C(n−1, k−1)); é o segredo da recorrência de Bell, grafos conexos rotulados (conexos = total − não conexos fixando o componente do 1).
- **Grafos conexos rotulados.** conn[n] = 2^{C(n,2)} − Σ_{k<n} C(n−1, k−1)·conn[k]·2^{C(n−k,2)}; mesmo molde (total − componente do vértice 1) para torneios fortemente conexos, DAGs rotulados (IE com sinais alternados sobre fontes).
- **DAGs rotulados (contagem).** dag[n] = Σ_{k≥1} (−1)^{k+1} C(n, k)·2^{k(n−k)}·dag[n−k] (IE sobre o conjunto de fontes); n até 5000 com NTT online.
- **DP de contagem de matrizes/tabelas 0/1 com somas de linha e coluna.** Ordene colunas por soma e processe linha a linha com estado = nº de colunas ainda precisando de 1,2,... (quando as somas ≤ 2): dp[i][c1][c2] com contagem por escolhas.
- **Gale–Ryser / contagem por graus.** Existência de matriz 0/1 com somas dadas → conjugada majora (verificação O(n)); contagem exata é difícil, mas com somas ≤ 2 o DP em (c1, c2) funciona.

- **Coloração de grafo / cobertura de conjuntos por DP em subconjuntos.** Número cromático: dp[S] = 1 + min sobre independentes I ⊆ S contendo menor bit; ou contagem de k-colorações por IE sobre arestas: Σ_{F⊆E} (−1)^{|F|} k^{c(F)} com c(F) = nº de componentes de (V, F), O(2^m); a soma Σ_{S} (−1)^{n−|S|} I(S)^k conta coberturas com sobreposição, não colorações.
- **Contagem de subconjuntos independentes por SOS.** I(S) = nº de independentes ⊆ S: calcule ind[S] e zeta; cobre problemas de "k-coloração", "cobertura por k conjuntos", "partição em k independentes".
- **DP de "nº de cobertura por k conjuntos" via IE + SOS.** Cobrir U com k conjuntos de uma família F: Σ_{T ⊆ U} (−1)^{|U\T|} (|{f ∈ F: f ⊆ T}|)^k; O(2^n n) após zeta de F. Dá cobertura exata contável e, com cuidado, existência (módulo primo grande).
- **Steiner tree DP.** dp[mask][v] = menor árvore conectando terminais de mask com raiz v: merge dp[sub][v] + dp[mask\sub][v] e relaxamento por Dijkstra em cada mask; O(3^k n + 2^k m log) com k ≤ 10 terminais.
- **Caminho simples longo por DP sobre subconjuntos de vértices.** n ≤ 20 hamilton; com n maior e caminho curto (k ≤ 10): color coding + dp[mask][v] em 2^k·m·e^k.
- **Cobertura/dominação em grade larga.** Dominating set / vertex cover / independent set em grafo de treewidth pequena: DP sobre decomposição em árvore (largura ≤ ~10–15) com estados 3^w para dominating set e 2^w para vertex cover/independent set; em grade n × m com n ≤ 12 vira broken profile.
- **Grafo com treewidth/pathwidth pequena.** Se o enunciado diz "grafo de grade com 5 linhas", "série-paralelo", "cacto": cacto → DP em ciclos (ciclo como nó); série-paralelo → DP por redução (série/paralelo) com tabela por (estado das duas pontas).
- **Cacto / grafo com ciclos que não compartilham aresta.** DP na árvore dfs: cada ciclo é tratado ao sair do vértice mais alto; estados (usado, não usado) por vértice do ciclo, depois mini-DP circular por ciclo (cut no ponto de articulação).
- **DP sobre "decomposição em cadeias" (Dilworth + largura pequena).** Se a largura de um poset é pequena (k ≤ 10), DP com estado = antichain / frente de cadeias (índices por cadeia) tem O(n^k) estados; usa-se para "scheduling com precedência e poucas máquinas".
- **Número de ideais (down-sets) pequeno.** Em poset/árvore, estados "conjunto fechado para baixo" crescem, mas para cadeias paralelas k são n^k: DP sobre vetores de progresso por cadeia.
- **DP monotônica em ordem topológica + corte mínimo.** Escolher subconjunto fechado de máximo peso (closure) é fluxo, não DP; se o fecho é em árvore (pai exige filho) é DP de subárvore com knapsack — reconheça pela estrutura do poset (árvore = DP, DAG geral = corte).

Repetidas de outras seções:

- **Mochila com Σ pesos ≤ S** → ≤ √(2S) pesos distintos; agrupe e use mochila limitada (binário ou deque).
- **DP sobre string com padrões** → DP no autômato de Aho-Corasick ou de KMP.
- **Jogo com pontuação** → DP minimax (melhor diferença para quem joga).
- **s repetida k vezes** → DP em 2–3 cópias e extrapole, ou matriz.
- **Aliens** para "exatamente k" com f convexa; confirme convexidade por brute force.
- **LIS por BIT** quando o DP indexa por valor e precisa de peso ou restrição extra.

Teoria dos números e divisores

Os limites pequenos escondidos resolvem metade dos problemas: até 10^9 há no máximo 1344 divisores e 9 primos distintos; até 10^{18}, 15 primos distintos.

- **Divisores:** itere até √x; para todos os x ≤ N, itere múltiplos (N log N harmônico).
- **gcd de prefixo/subarray** muda no máximo log(a) vezes → para cada fim, só log valores distintos de gcd.
- **gcd(a, b) = gcd(a, b − a)** → gcd de um array somado com x vira gcd(a_1 + x, diferenças).
- **lcm cresce rápido**: passa de 10^{18} em poucos passos; corte quando estourar.
- **Fatore em primos** e trate cada primo independente (expoentes como vetor; gcd = min, lcm = max, produto = soma).
- **Quadrado perfeito / produto quadrado** → vetor de paridades dos expoentes, xor sobre GF(2) (base linear).
- **Contar pares com gcd = g** → conte "múltiplos de d" e faça Möbius/inclusão-exclusão de cima para baixo.
- **Σ ⌊n/i⌋** → só 2√n valores distintos (divisão em blocos).
- **x mod y** com x/2 < y ≤ x é x − y; mod reduz x pelo menos à metade quando y ≤ x.
- **a^b mod m** com b enorme → Euler: reduza o expoente mod φ(m) (cuidado com gcd ≠ 1).
- **Equação linear ax + by = c** → Euclides estendido; sistema de congruências → CRT.
- **Primos até 10^7** → crivo linear guardando o menor fator primo.
- **Paridade de números**, **resíduos mod 2, 3, 4** costumam bastar em construtivos.
- **Fatorar até 10^{18}** → Miller–Rabin + Pollard rho.

adicionado em 2026-11-10 por agentes; revisado

- **Crivo linear para funções multiplicativas.** Guarde o menor fator lp[i]; ao gerar i·p: φ(ip) = φ(i)·p se p | i, senão φ(i)·(p−1); μ(ip) = 0 se p | i, senão −μ(i). Para σ, τ guarde também a parte da maior potência de p em i.
- **Crivo segmentado.** Primos de [L, R] com R ≤ 10^{12} e R−L ≤ 10^{7}: crive com primos ≤ √R; para fatorar/φ de todo o intervalo, divida cada rem[x] pelos primos e, se sobrar > 1, o resto é um primo.
- **Miller–Rabin determinístico.** Com as 12 primeiras bases primas (2…37) vale para n < 3,3·10^{24}, logo para todo 64 bits; para n < 2^{32} bastam 2, 7, 61. Precisa de mulmod com __int128.
- **Pollard rho (Brent).** Fatora n ≤ 10^{18} em ~n^{1/4} passos; tire primos pequenos por divisão, teste primalidade antes de cada recursão, acumule produtos para gcd em lote. Recursão: se achou d, fatore d e n/d.
- **Fatorar muitos números ≤ 10^7.** Crivo de menor fator primo + loop de divisões = O(log) por número; n ≤ 10^{12} → divisão por primos ≤ 10^6 (78498 primos) e o resto é 1 ou primo (para n ≤ 10^{18} o resto é 1, primo, p^2 ou p·q).
- **Soma sobre divisores via crivo de primos (Dirichlet prefix sum).** Para cada primo p, para i = 1..N/p crescente: F[i·p] += F[i] dá F'[n] = Σ_{d|n} f(d) em O(N log log N). Sobre múltiplos: i decrescente, F[i] += F[i·p]. Inverter = subtrair na ordem reversa (é o Möbius).
- **φ e somas com φ.** Σ_{d|n} φ(d) = n; Σ_{i≤n, gcd(i,n)=1} i = n·φ(n)/2 (n > 1); Σ_{i=1}^n gcd(i, n) = Σ_{d|n} d·φ(n/d); φ(n) ≥ √(n/2) e φ(n) é par para n ≥ 3.
- **Σ_{i,j≤n} gcd(i, j) = Σ_d φ(d)·⌊n/d⌋².** Vem de gcd = Σ_{d|gcd} φ(d). Com Möbius: pares coprimos = Σ μ(d)⌊n/d⌋² = 2Φ(n) − 1 (Φ = soma de φ).
- **Möbius com dois limites.** #{(i≤n, j≤m): gcd = 1} = Σ μ(d)⌊n/d⌋⌊m/d⌋; agrupe d em blocos onde ambos ⌊n/d⌋ e ⌊m/d⌋ são constantes: O(√n + √m) com prefixos de μ.
- **Pares coprimos ponderados.** Σ_{i,j≤n, gcd=1} i·j = Σ_d μ(d)d²·S(⌊n/d⌋)², S(m) = m(m+1)/2; e Σ_{i,j≤n} lcm(i, j) = Σ_g g·C(⌊n/g⌋), C(m) = Σ_d μ(d)d²S(⌊m/d⌋)².
- **k-tuplas com gcd 1.** #{tuplas em [1, n]^k com gcd = 1} = Σ_d μ(d)⌊n/d⌋^k. Com gcd exatamente g: use n/g.
- **Inversão de Möbius (as duas formas).** g(n) = Σ_{d|n} f(d) ⇔ f(n) = Σ_{d|n} μ(n/d)g(d); G(n) = Σ_{n|m} F(m) ⇔ F(n) = Σ_{n|m} μ(m/n)G(m). "Soma sobre múltiplos" é o caso que aparece em contagem de gcd.
- **Convolução de Dirichlet: identidades úteis.** 1∗μ = ε, 1∗φ = id, μ∗id = φ, 1∗1 = τ, 1∗id = σ, id_k∗μ = J_k. Se f = g∗h e você sabe os prefixos de g e h, o prefixo de f sai por Σ_{d≤n} h(d)·G(⌊n/d⌋).
- **Hipérbole de Dirichlet.** Σ_{ab≤n} F(a)G(b) em O(√n): Σ_{a≤A} f(a)G(n/a) + Σ_{b≤B} g(b)F(n/b) − F(A)G(B) com A·B ≤ n < (A+1)(B+1). Dá Σ τ(i) = 2Σ_{i≤√n}⌊n/i⌋ − ⌊√n⌋².
- **Prefixo de φ ou μ até 10^{10}–10^{11} (sieve de Dirichlet).** M(n) = 1 − Σ_{d≥2} M(⌊n/d⌋) e Φ(n) = n(n+1)/2 − Σ_{d≥2} Φ(⌊n/d⌋); crive até n^{2/3} e memorize o resto por ⌊n/i⌋: O(n^{2/3}).
- **Truque geral do prefixo.** Para f difícil, ache g com prefixo fácil e f∗g fácil: (id^k·φ)∗id^k = id^{k+1}; (id·μ)∗id = ε. Então S_f(n) = [S_{f∗g}(n) − Σ_{d≥2} g(d)S_f(⌊n/d⌋)]/g(1).
- **Lucy_Hedgehog.** Contagem de primos ≤ n e Σp, Σp^2 em O(n^{3/4}) sobre os 2√n valores ⌊n/i⌋: S(v, p) = S(v, p−1) − f(p) [S(⌊v/p⌋, p−1) − S(p−1, p−1)] para v ≥ p^2; f completamente multiplicativa (p^k).
- **Min_25.** Fase 1 = Lucy, exige f(p) polinomial em p (soma de f sobre primos); fase 2 soma qualquer f multiplicativa com f(p^e) calculável, por DFS/DP sobre ⌊n/i⌋; n até 10^{10}–10^{12}.
- **Powerful numbers.** Se f multiplicativa e existe g multiplicativa com f(p) = g(p) e prefixo de g fácil, então f = g∗h com h só em números "powerful" (todo expoente ≥ 2, são O(√n)): Σf = Σ_{m powerful} h(m)·G(⌊n/m⌋).
- **Floor sum (Euclid-like).** Σ_{i=0}^{n−1} ⌊(a·i + b)/m⌋ em O(log m); trocando papéis (a ≥ m → reduza; senão troque i e o valor). Variantes com Σ i·⌊·⌋ e Σ ⌊·⌋² também são recursivas.
- **Onde floor sum aparece.** Pontos de rede sob uma reta; Σ (a·i + b) mod m = Σ(a·i + b) − m·floorsum; contar (x, y) com a·x + b·y ≤ n; mínimo de (a·i + b) mod m para i ≤ n (versão Euclid-like).
- **Blocos de ⌊n/i⌋: propriedades.** ⌊⌊n/a⌋/b⌋ = ⌊n/(ab)⌋, então o conjunto de valores é fechado; ⌈n/i⌉ = ⌊(n−1)/i⌋ + 1. Indexe DPs por valor v ≤ √n em lo[v] e por n/v em hi[n/v].
- **Somas clássicas por blocos.** Σ_{i≤n} σ(i) = Σ d⌊n/d⌋; Σ_{k≤n} (n mod k) = n^2 − Σ k⌊n/k⌋; Σ_{i≤n} τ(i) ≈ n ln n + (2γ−1)n.
- **CRT com módulos não coprimos.** x ≡ a_1 (m_1), x ≡ a_2 (m_2) tem solução sse g = gcd | a_2 − a_1; então x = a_1 + m_1·[(a_2−a_1)/g · inv(m_1/g mod m_2/g) mod (m_2/g)], novo módulo lcm. Detecte lcm > 10^{18} (use __int128).
- **Garner.** Reconstrói um número grande (ou mod M qualquer) a partir de resíduos em módulos primos entre si por base mista, sem bigint: usado em NTT com 3 primos para resposta mod arbitrário.
- **Euclides estendido: todas as soluções.** ax + by = c tem solução sse g | c; x = x_0 + k·b/g, y = y_0 − k·a/g. Contar soluções com x ∈ [x_1, x_2], y ∈ [y_1, y_2] = interseção de dois intervalos de k (cuidado com floor/ceil de negativos).
- **Inverso em lote.** Produtos de prefixo, um único inverso do total, volta multiplicando: O(n + log). Para 1..n: inv[i] = −(p/i)·inv[p mod i]; para fatoriais: invfact[n] = fact[n]^{−1} e desce multiplicando por i.

- **Ordem multiplicativa.** ord_n(a) só existe com gcd(a, n) = 1 (senão há pré-período); ord_n(a) | φ(n) (ou λ(n)); calcule começando em φ e, para cada primo q de φ, divida por q enquanto a^{x/q} ≡ 1. Com gcd(a, n) = 1, todo ciclo de a^k mod n tem período ord.
- **Raiz primitiva.** Existe sse n = 1, 2, 4, p^k, 2p^k com p primo ímpar; g é raiz mod p sse g^{(p−1)/q} ≢ 1 para todo primo q | p−1; a menor costuma ser pequena (tente 2, 3, 5…); há φ(p−1) raízes.
- **Logaritmo/índice via raiz primitiva.** Mult. mod p vira soma mod p−1: x^k ≡ a ⇔ k·ind(x) ≡ ind(a); para a ≢ 0, a é potência k-ésima sse a^{(p−1)/gcd(k, p−1)} ≡ 1, e então há gcd(k, p−1) raízes (a ≡ 0 tem só x ≡ 0).
- **BSGS.** a^x ≡ b (mod m) em O(√m) com hash: x = i·s − j, s ≈ √m. Para gcd(a, m) ≠ 1 (exBSGS): enquanto g = gcd(a, m) > 1, se g ∤ b não há solução; faça b ← (b/g)·inv(a/g mod m/g) (mod m/g), m ← m/g e conte o expoente deslocado, até ficar coprimo.
- **Raiz quadrada mod p.** Critério de Euler a^{(p−1)/2}; p ≡ 3 (mod 4) → a^{(p+1)/4}; geral → Tonelli–Shanks ou Cipolla. Mod p^k: Hensel; mod composto: CRT; com n ímpar, gcd(a, n) = 1 e a QR em cada p | n, 2^ω soluções (casos 2^k e p | a têm contagem especial).
- **Legendre/Jacobi.** Para a ímpar, (a/p)(p/a) = (−1)^{(p−1)(a−1)/4}; (2/p) = (−1)^{(p²−1)/8}; Jacobi em O(log) estilo Euclides, sem fatorar. $x^2 \equiv -1 \pmod p$ tem solução sse p = 2 ou p ≡ 1 (mod 4).
- **Soma de dois/três/quatro quadrados.** n é soma de 2 quadrados sse todo primo ≡ 3 (mod 4) tem expoente par ($r_2(n) = 4(d_1 - d_3)$); soma de 3 quadrados sse n ≠ 4^a(8b+7); 4 quadrados sempre (Lagrange).
- **Resíduos que provam impossibilidade.** Quadrados mod 8 ∈ {0, 1, 4}; mod 3 ∈ {0, 1}; cubos mod 9 ∈ {0, 1, 8}; quartas potências mod 16 ∈ {0, 1}. Também: $x^2 \equiv 1 \pmod n$ tem 2^ω soluções para n ímpar e 4 para 2^k, k ≥ 3.
- **LTE (Lifting the Exponent).** p ímpar, p | x−y, p ∤ xy: $v_p(x^n - y^n) = v_p(x-y) + v_p(n)$; n ímpar, p | x+y, p ∤ xy: $v_p(x^n + y^n) = v_p(x+y) + v_p(n)$; p = 2, n par, x, y ímpares: $v_2(x^n - y^n) = v_2(x-y) + v_2(x+y) + v_2(n) - 1$.
- **Legendre/Kummer em p-ádico.** $v_p(n!) = \Sigma \lfloor n/p^i \rfloor = (n - s_p(n))/(p-1)$; $v_p(C(n, k)) = (s_p(k) + s_p(n-k) - s_p(n))/(p-1)$ = número de vai-uns de k + (n−k) na base p. Zeros finais de n! na base b = min_p ⌊v_p(n!)/e_p(b)⌋.
- **Wilson e consequências.** (p−1)! ≡ −1; (n−1)! ≡ 0 (mod n) para composto n > 4; (p−1−k)!·k! ≡ (−1)^{k+1} (mod p), útil para n! mod p com n perto de p.
- **n! mod p com parte livre de p.** n! = p^{⌊n/p⌋}·⌊n/p⌋!·∏_{p∤i≤n} i, e o último produto ≡ (−1)^{⌊n/p⌋}(n mod p)! (mod p): recursão em log_p n com tabela de fatoriais até p. Base de Lucas generalizado e de C(n, k) mod p^e.
- **Torre a^b^c… mod m.** a^e ≡ a^{(e mod φ(m)) + φ(m)} (mod m) quando e ≥ φ(m) (vale para qualquer gcd); recurse com φ(m), φ(φ(m))… (cadeia chega a 1 em ≤ 2·log₂ m passos); compare expoentes truncados (cap em 2^{60}) para saber se e ≥ φ.
- **λ de Carmichael.** Para gcd(a, n) = 1, a^k mod n só depende de k mod λ(n), e λ(n) | φ(n) pode ser bem menor (λ(2^k) = 2^{k−2} para k ≥ 3).
- **Zeckendorf.** Todo inteiro positivo é soma única de Fibonacci não consecutivos (guloso do maior); há ~86 Fibonacci ≤ 10^{18}. Strings binárias de comprimento n sem dois 1 adjacentes = F_{n+2}.
- **Fibonacci: identidades.** gcd(F_a, F_b) = F_{gcd(a, b)}; doubling F_{2k} = F_k(2F_{k+1} − F_k), F_{2k+1} = F_k^2 + F_{k+1}^2; período de Pisano π(m) ≤ 6m; Cassini $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$.
- **gcd(x^a − 1, x^b − 1) = x^{gcd(a, b)} − 1** (vale para 2^a-1). Análogo: gcd($a^n - b^n$, $a^m - b^m$) = a^{gcd}− b^{gcd} se gcd(a, b) = 1.
- **Frações contínuas.** [a_0; a_1, …] sai do algoritmo de Euclides em O(log); convergentes p_k = a_k p_{k−1} + p_{k−2} (idem q_k) são as melhores aproximações; com semi-convergentes acha-se a melhor fração com denominador ≤ N.
- **Fração mais simples num intervalo.** A menor fração (menor denominador) em [l, r] (ou em (l, r)) é achada descendo a árvore de Stern–Brocot em paralelo para l e r até os caminhos divergirem: O(log) com saltos em blocos.
- **Stern–Brocot.** Cada racional positivo aparece uma vez; o caminho L/R em run-length é a fração contínua. Busca binária sobre racionais (achar a/b com predicado monótono) = descer com saltos exponenciais: O($\log^2$) ou O(log).
- **Farey.** Consecutivos a/b < c/d em F_n satisfazem bc − ad = 1; próximo termo: (kc − a)/(kd − b) com k = ⌊(n + b)/d⌋; |F_n| = 1 + Σ_{i≤n} φ(i). Contar frações irredutíveis de denominador ≤ n = Σφ.
- **Moeda / Frobenius.** Para a, b coprimos o maior não representável é ab − a − b e há (a−1)(b−1)/2 não representáveis. Com muitas moedas e a menor pequena: Dijkstra nos resíduos mod a_1 (dist[r] = menor soma ≡ r).
- **Pell $x^2 - Dy^2 = 1$.** Solução fundamental vem do período da fração contínua de √D; as demais são potências de ($x_1 + y_1\sqrt{D}$), logo matriz 2×2 elevada ao k. Soluções fundamentais podem ter ~√D dígitos: use bigint ou só mod.
- **Triplas pitagóricas.** Primitivas: (m^2-n^2, 2mn, m^2+n^2) com m > n, gcd = 1, paridades opostas; ao menos um cateto é múltiplo de 3, de 4, e um dos lados de 5.
- **Base fatorial (Lehmer).** Permutação ↔ número < n! com dígitos d_i ∈ [0, i−1] (número de menores à direita); k-ésima permutação e posto com BIT em O(n log n); n! passa de 2^{63} em n = 21.
- **σ(n) mod p e séries geométricas.** σ(n) = ∏ (p^{e+1} − 1)/(p − 1); se p−1 ≡ 0 mod o módulo, use Σ_{i≤e} p^i por divisão e conquista (dobrar) sem inverso.

- **Funções multiplicativas.** f fica determinada pelos valores em potências de primos; a soma $\Sigma_{d|n} f(d)$ de multiplicativa é multiplicativa; para provar multiplicatividade verifique $f(mn) = f(m)f(n)$ com $\gcd = 1$.
- **Divisores: fórmulas prontas.** $\Sigma_{d|n} d = \prod(p^{e+1} - 1)/(p - 1)$; produto dos divisores $= n^{\tau(n)/2}$; $\tau(n)$ é ímpar sse n é quadrado (luzes/portas alternadas); #pares ordenados com $\text{lcm}(a, b) = n$ é $\tau(n^2)$; com $\gcd = 1$ e $ab = n$ é 2^{ω}.
- **Tamanho de divisores (máximos).** $n \le 10^3$: 32; 10^4: 64; 10^5: 128; 10^6: 240; 10^9: 1344; 10^{12}: 6720; 10^{15}: 26880; 10^{18}: 103680. $\Sigma_{i \le N} \tau(i) \approx N \ln N$.
- **Números altamente compostos.** Têm expoentes não crescentes nos primos 2, 3, 5, ...; gere por DFS (primo $k \le 15$ primos para 10^{18}) para achar o maior $\tau \le N$ ou o menor n com $\tau(n) \ge k$.
- **Divisibilidade encadeada é curta.** Cadeia $a_1 \mid a_2 \mid \ldots$ com $a_i \ne a_{i+1}$ em [1, N] tem $\le \log_2 N + 1$ elementos; DP sobre cadeias de divisores é $O(N \log N \log N)$.
- **Antichain de divisibilidade.** Entre 1..2n, qualquer n+1 números contêm um par a | b (agrupe pela parte ímpar); então a maior antichain é n.
- **lcm de 1..n.** $= \prod p^{\lfloor \log_p n \rfloor}$, $\ln \text{lcm}(1..n) \approx n$; $\text{lcm}(1..41) \approx 2{,}2 \cdot 10^{17}$ e $\text{lcm}(1..43) \approx 9{,}4 \cdot 10^{18}$ já estoura 2^{63}.
- **Primos: densidade e gaps.** $\pi(10^6) = 78498$, $\pi(10^7) = 664579$, $\pi(10^9) = 50847534$; maior gap $< 10^9$ é 282, $< 10^{18}$ é < 1500. $\Sigma_{p \le n} 1/p \approx \ln \ln n$, então iterar múltiplos só de primos custa $n \ln \ln n$.
- **Contar coprimos com m.** $\#\{x \le n : \gcd(x, m) = 1\} = \Sigma_{d \mid \text{rad}(m)} \mu(d) \lfloor n/d \rfloor$ (2^{ω} termos, ≤ 512 para $m \le 10^9$); o k-ésimo coprimo = busca binária + essa contagem.
- **Livres de quadrado e potências perfeitas.** #squarefree $\le n = \Sigma_{d \le \sqrt{n}} \mu(d) \lfloor n/d^2 \rfloor$; #potências perfeitas em [2, n] $= -\Sigma_{k \ge 2} \mu(k)(\lfloor n^{1/k} \rfloor - 1)$. Raiz inteira: use sqrtl e ajuste ±1.
- **Ponto de rede no círculo (Gauss).** $\#\{(x, y) : x^2 + y^2 \le r^2\} = \Sigma_x (2\lfloor \sqrt{r^2 - x^2} \rfloor + 1)$ em $O(r)$; segmento de (0,0) a (a, b) contém $\gcd(|a|, |b|) + 1$ pontos de rede.
- **Inversa de φ.** $\varphi(n) = k$: enumere divisores d de k com d+1 primo e faça DFS multiplicando; $\varphi(n) = k$ tem poucas soluções e $n \le k \cdot \prod p/(p-1)$.
- **Dízima e expansões.** Período de $1/n = \text{ord}_{n'}(10)$ com $n' = n$ sem fatores 2 e 5; pré-período $= \max(v_2, v_5)$; primeiros dígitos de a^b: $\text{frac}(b \cdot \log_{10} a)$; últimos k dígitos: $\text{mod } 10^k$.
- **Convolução de gcd/lcm.** $h[g] = \Sigma_{\gcd(i, j) = g} f[i] \cdot f'[j]$: zeta sobre múltiplos $F[d] = \Sigma_{d|i} f[i]$, multiplique ponto a ponto, Möbius de volta, tudo em $O(N \log \log N)$. lcm: zeta sobre divisores.
- **Sequências que recorrem mod m.** $x_{n+1} = F(x_n) \bmod m$ tem período $\le m$ (Floyd/Brent acham ciclo); recorrências lineares de ordem k mod m têm período $\le m^k$ (m^2 é o caso $k = 2$).
- **Reduza expoentes por p-ádico.** Para "n mod p^k" de produtos, separe a parte p^v e a unidade; para x^n com n gigante e $\gcd \ne 1$ trate $v_p(x) \cdot n$ separado do resto coprimo.
- **mulmod 64 bits.** __int128, ou mod $2^{61}-1$ com shifts, ou Montgomery; long double só com cuidado. Para módulo fixo primo $\sim 10^9$ evite % em laço quente (Barrett).

Repetidas de outras seções:

- **"gcd de pelo menos metade"** → sorteie um elemento e teste seus divisores (aleatorização).
- **Jogo com números** → divisores, potências de 2, n mod (k+1); calcule pequenos e ache o padrão.
- **$a_i \le 10^6$** → itere valores/múltiplos em vez de índices (também em arrays).
- **Contar árvores com restrição de grau (Prüfer)** → multinomial; **Cayley** n^{n-2}; e Matrix-tree para contagem geral (determinante mod p).

Combinatória, contagem e probabilidade

Antes de qualquer fórmula: conte à mão n = 1..6 e jogue a sequência na OEIS.

- **Inclusão-exclusão:** escreva as duas linhas (quais são os defeitos; quanto vale N(S)). Se N(S) só depende de |S|, a soma vira $\Sigma (-1)^k C(m, k) f(k)$.
- **Condição boa se separa por elemento** → é só multiplicar, sem IE.
- **Bijeção:** caminhos em grid, parênteses, sequências → Catalan, $C(n+m, n)$, estrelas e barras $C(n+k-1, k-1)$.
- **Reflexão:** caminhos que não cruzam uma linha → total menos os refletidos.
- **Contagem por contribuição:** Σ sobre subconjuntos de max → ordene e conte $2^{(i)}$ para cada a_i ser o máximo.
- **Esperança = soma de probabilidades** de eventos indicadores (linearidade); funciona mesmo com dependência.
- **E[max]** → $\Sigma P(\text{max} \ge k)$. **E[número de passos]** → $\Sigma P(\text{ainda não terminou após } k)$.
- **Simetria:** "probabilidade de i vir antes de j" = 1/2; "i é o máximo entre k" = 1/k.
- **Burnside** quando conta "a menos de rotação/simetria".
- **Funções geradoras** quando somas de partes têm restrições; produto de polinômios → NTT.

- **Lucas** para C(n, k) mod p pequeno; **Kummer** para a potência de p em C(n, k).
- **Fatorial até 10^6** pré-computado com inverso; C(n, k) com n até 10^{18} e k pequeno → produto direto.
- **Matriz de transição + exponenciação** para contar caminhos de comprimento k.
- **Stirling, partições, desarranjos** aparecem disfarçados: teste a sequência pequena.

adicionado em 2026-11-10 por agentes; revisado

- **"Exatamente k" a partir de "pelo menos k".** Se g(k) = Σ_{j≥k} C(j, k)f(j) então f(k) = Σ_{j≥k} (−1)^{j−k}C(j, k)g(j) (inversão binomial): conte quantos têm ≥ k defeitos marcados e inverta.
- **IE de surjeções.** #sobrejeções [n] → [k] = Σ_j (−1)^j C(k, j)(k−j)^n = k!·S(n, k); desarranjos = Σ (−1)^j n!/j!.
- **IE com defeitos estruturados.** Se N(S) só depende da estrutura (blocos, intervalos), faça DP que soma sinais ao decidir se cada defeito entra; vira O(n^2) em vez de 2^n. n ≤ 20 → máscara.
- **Min–max (IE de máximos).** max(S) = Σ_{∅≠T⊆S} (−1)^{|T|+1} min(T); vale para esperança também: E[max] = Σ (−1)^{|T|+1}E[min T]. Útil quando min é fácil (mínimo de exponenciais/prob. por estado é soma).
- **Reflexão e ballot.** Caminhos de (0,0) a (n, m) tocando y = x + c (c > 0) = C(n+m, n+c) (reflita o destino), válida para m ≤ n+c; se m > n+c todos os caminhos tocam a reta (total C(n+m, n)) e a fórmula superconta. Ballot: A sempre estritamente à frente com a > b votos = (a−b)/(a+b)·C(a+b, a); permitindo empate (exige a ≥ b) = (a−b+1)/(a+1)·C(a+b, b).
- **Faixa 0 ≤ y ≤ h.** Número de caminhos que não saem da faixa = soma alternada de reflexões Σ_k [C(n, ·) − C(n, ·)] com O(n/h) termos; para h pequeno use DP O(n·h).
- **Cycle lemma (Raney).** Entre as n rotações de uma sequência de passos com soma +1, exatamente uma tem todos os prefixos positivos. Dá Catalan, ballot e árvores k-árias: 1/((m−1)n+1)·C(mn, n) (Fuss–Catalan).
- **Catalan: outros disfarces.** Triangulações de (n+2)-gono, árvores binárias de n nós, partições não-cruzadas, emparelhamentos sem cruzar de 2n pontos, permutações que evitam 231 (ou qualquer padrão de tamanho 3), SYT 2×n. Narayana N(n, k) = (1/n)C(n, k)C(n, k−1) conta Dyck com k picos.
- **Catalan: variantes de passo.** Com passo plano: Motzkin (1, 1, 2, 4, 9, 21, …); com diagonais em grid: Delannoy D(m, n) = Σ_k C(m, k)C(n, k)2^k; Chung–Feller: o número de passos acima do eixo em caminhos ±1 de 2n passos que voltam a 0 é uniforme (cada valor tem C_n caminhos).
- **Identidades de binomiais.** Vandermonde Σ_k C(m, k)C(n, r−k) = C(m+n, r); Σ C(n, k)^2 = C(2n, n); hockey stick Σ_{i=r}^{n} C(i, r) = C(n+1, r+1); Σ_{k=0}^{n} C(k, a)C(n−k, b) = C(n+1, a+b+1).
- **Binomiais: somas ponderadas.** Σ k·C(n, k) = n·2^{n−1}; Σ k^2C(n, k) = n(n+1)2^{n−2}; Σ_k C(n, k)C(k, m) = C(n, m)2^{n−m}; k·C(n, k) = n·C(n−1, k−1); Σ_{k≤m}(−1)^k C(n, k) = (−1)^m C(n−1, m).
- **Somas parciais de binomiais em O(1) por passo.** S(n, m) = Σ_{i≤m} C(n, i) satisfaz S(n+1, m) = 2S(n, m) − C(n, m): ao deslizar (n, m) numa escada mantenha S sem recomputar.
- **Caminhos em grid com obstáculos.** Ordene os k pontos proibidos; f(i) = caminhos(start, i) − Σ_{j antes de i} f(j)·caminhos(j, i): O(k^2) com fatoriais; boundary em escada → Σ de C(x+y, x) por hockey stick.
- **Estrelas e barras com limites.** $x_i \le u$: IE Σ_j (−1)^j C(k, j)·C(n − j(u+1) + k − 1, k − 1); $x_i \ge l_i$: desloque; $x_1 < x_2 < \dots < x_k$: subtraia i de x_i; multiconjuntos de tamanho k de n tipos = C(n+k−1, k).
- **Escolher sem vizinhos.** k posições de n em linha sem duas adjacentes = C(n−k+1, k); em círculo = n/(n−k)·C(n−k, k); com distância ≥ d entre escolhidos: C(n − (k−1)(d−1), k).
- **Composições e partições.** Composições de n = 2^{n−1}; com partes em S → GF 1/(1 − Σ_{s∈S} x^s) (recorrência linear). Partições p(n) por pentágonos de Euler: p(n) = Σ_{k≥1} (−1)^{k+1}[p(n − k(3k−1)/2) + p(n − k(3k+1)/2)], O(n√n).
- **Partições: dualidades e DP √.** Conjugação: partições com maior parte k ↔ exatamente k partes; distintas = ímpares (Euler). Partições de n em k partes distintas: q(n, k) = q(n−k, k) + q(n−k, k−1), só k ≤ √(2n) → O(n√n). Partes ≤ √n por DP, as partes > √n são em número < √n e se contam por conjugação.
- **Stirling 2ª espécie.** S(n, k) = (1/k!)·Σ_j (−1)^{k−j}C(k, j)j^n (um valor em O(k log n)); linha inteira por NTT; x^n = Σ_k S(n, k)·x^{\underline{k}}; Σ_i C(m, i)i^n = Σ_k S(n, k)m^{\underline{k}}2^{m−k}.
- **Momentos via Stirling.** X^n = Σ_k S(n, k)X^{\underline{k}}, e E[X^{\underline{k}}] = E[# k-tuplas ordenadas de indicadores]: calcula E[X^n] (n pequeno) contando k-tuplas.
- **Bell.** B_{n+1} = Σ_k C(n, k)B_k; B_n = Σ_k S(n, k); triângulo de Bell em O(n^2); Dobinski B_n = (1/e)Σ k^n/k!. Partições de conjunto com restrição de blocos → EGF exp(Σ x^s/s!).
- **Stirling 1ª espécie.** c(n, k) = # permutações com k ciclos; x(x+1)…(x+n−1) = Σ c(n, k)x^k; linha por divisão e conquista (ou duplicar com x → x + n/2) + NTT; c(n, k) = (n−1)c(n−1, k) + c(n−1, k−1); E[#ciclos] = H_n.
- **Desarranjos e permutações especiais.** D_n = (n−1)(D_{n−1} + D_{n−2}) = nD_{n−1} + (−1)^n ≈ n!/e; com exatamente k pontos fixos: C(n, k)D_{n−k}; involuções I_n = I_{n−1} + (n−1)I_{n−2}; perms com x^k = id: EGF exp(Σ_{d|k} x^d/d).
- **Ciclo do elemento é uniforme.** Em permutação aleatória, o comprimento do ciclo que contém um elemento fixo é uniforme em 1..n; E[pontos fixos] = 1; P(1 e 2 no mesmo ciclo) = 1/2; E[# recordes] = E[# ciclos] = H_n.

- **Inversões.** Mahoniano: GF $\prod_{i=1}^{n} (1 + x + \dots + x^{i-1})$; perms com k inversões em O(n·k) por prefix sum no DP (adicione 0..i−1 inversões ao inserir i); a tabela de inversões ($d_i \in [0, i-1]$) é livre, então restrições por elemento viram produtos.
- **Descidas e Eulerianos.** $A(n, k) = (k+1)A(n-1, k) + (n-k)A(n-1, k-1) = \sum_j (-1)^j C(n+1, j)(k+1-j)^n$; Worpitzky $x^n = \sum_k A(n, k)C(x + k, n)$. Perm com assinatura <><… dada: DP em $O(n^2)$ pelo posto relativo do último elemento + prefix sums.
- **Zigue-zague.** Alternantes: EGF sec x + tan x; triângulo de Entringer/Seidel $O(n^2)$.
- **Permutações indecomponíveis / conexas.** $n! = \sum_j I(j)(n-j)! \to I(n) = n! - \sum_{j<n} I(j)(n-j)!$; (1, 1, 3, 13, 71…). Mesma ideia: contar a estrutura "de um bloco" tirando de todas a decomposição.
- **Ordens lineares em floresta.** # rotulações com pai < filho (ou extensões lineares de floresta) = $n!/\prod sz(v)$; equivale a "v é o mínimo da sua subárvore" com prob. 1/sz(v), independentes. Para DAG geral n ≤ 20: DP em máscara.
- **Árvore cartesiana / recorrência pelo máximo.** $f(n) = \sum_i C(n-1, i-1)f(i-1)f(n-i)$ (convolução binomial) → EGF com EDO; árvores binárias crescentes = n!.
- **Intercalações.** Mesclar sequências preservando ordem interna: multinomial $n!/\prod n_i!$; k-ésima string lexicográfica com multiconjunto: conte completamentos por multinomial a cada escolha.
- **Hook length e RSK.** # SYT de forma $\lambda = n!/\prod$ hooks; RSK dá perm ↔ (P, Q) mesma forma, então $n! = \sum f_\lambda^2$; LIS = λ_1, LDS = # linhas; # involuções = $\sum f_\lambda$. Perms com LIS ≤ 2 = Catalan.
- **Burnside em grid.** Rotações 90°/180° de n×n: ciclos = $n^2/4$ (ou $(n^2-1)/4 + 1$) e $n^2/2$ (ou $(n^2+1)/2$); Fix = $k^{\#ciclos}$. Para cores com contagem fixa use índice de ciclos (Pólya) com GF.
- **Contagem por simetria/estabilizador.** #classes de estruturas rotuladas = #rotuladas·ā/n!, com ā = média de |Aut| sobre as estruturas rotuladas (Burnside: $(1/n!)\sum_\pi |Fix(\pi)|$); orbit–stabilizer: |órbita| = |G|/|Stab|. Isomorfismo de árvores com raiz: AHU (hash do multiconjunto ordenado dos filhos); sem raiz: centroides.
- **Strings primitivas e Lyndon.** #strings de comprimento n sem período próprio = $\sum_{d|n} \mu(n/d)k^d$; #colares primitivos (Lyndon) = (1/n)·esse valor; mínimo período exato d: Möbius sobre divisores.
- **Prüfer.** Sequência de n−2 ↔ árvore rotulada; vértice v aparece deg(v)−1 vezes. # árvores com graus $d_i = (n-2)!/\prod(d_i-1)!$; com vértice v de grau d: $C(n-2, d-1)(n-1)^{n-d-1}$; Cayley n^{n-2}.
- **Contar árvores que conectam componentes.** Com k componentes de tamanhos s_i, o # de formas de ligá-las em árvore é $n^{k-2}\prod s_i$; florestas com k raízes especificadas: $k \cdot n^{n-k-1}$; $K_{a,b}$ tem $a^{b-1}b^{a-1}$ geradoras.
- **Matrix-Tree.** # árvores geradoras = qualquer cofator de L = D − A (multigrafo ok); com pesos multiplicativos some pesos; dirigido: Laplaciano de grau de entrada → arborescências com raiz escolhida. det mod p em $O(n^3)$, n ≤ ~400; módulo composto: eliminação tipo Euclides.
- **BEST theorem.** Em grafo dirigido euleriano (conexo nos vértices de grau > 0), # circuitos eulerianos = $t_w(G)\cdot\prod_{v: deg(v)>0} (deg(v) - 1)!$ (t_w = # arborescências com raiz w, qualquer w), contando circuitos como ciclos sem ponto inicial; fixando a aresta inicial o valor é o mesmo.
- **Grafos conexos e DAGs rotulados.** Conexos: $c(n) = 2^{C(n,2)} - \sum_{k<n} C(n-1, k-1)c(k)2^{C(n-k,2)}$ (fixe o vértice 1); DAGs: $a(n) = \sum_{k\geq 1} (-1)^{k+1}C(n, k)2^{k(n-k)}a(n-k)$ (IE sobre fontes). Em geral: exp/log de EGF.
- **Contagens clássicas rotuladas.** Árvores com raiz n^{n-1}; florestas com raiz $(n+1)^{n-1}$; funções n→n com ciclos = n^n; emparelhamentos perfeitos $(2n-1)!!$; grafos $2^{C(n,2)}$; injeções $n^{\underline{k}}$.
- **Funções geradoras: dicionário.** OGF: produto = escolhas independentes; EGF: produto = convolução binomial (rotulado); 1/(1−f) = sequências, exp(f) = conjuntos de componentes, ln(1/(1−x)) = ciclos; árvores com raiz $T = x\cdot e^T \to [x^n]T = n^{n-1}/n!$.
- **Lagrange (inversão).** Se $F = x\cdot\varphi(F)$ então $[x^n]F = (1/n)[x^{n-1}]\varphi(x)^n$; $[x^n]F^k = (k/n)[x^{n-k}]\varphi^n$. Dá Catalan, árvores k-árias, contagem de árvores por grau.
- **Strings sem padrão (GF por autocorrelação).** Alfabeto k, padrão de tamanho m com $c(x) = \sum x^j$ sobre os períodos j (j = 0 e os j com borda m−j): GF = $c(x)/(x^m + (1-kx)c(x))$; ex.: "11" binário → Fibonacci.
- **Tempo esperado até um padrão.** Moedas justas: $E = \sum_{k: \text{prefixo k = sufixo k}} 2^k$ (HH: 6, HT: 4); alfabeto q: $\sum q^k$ sobre as bordas (incluindo a string toda).
- **Recorrência linear gigante.** Sequência de ordem d: Kitamasa/Bostan–Mori O(d log d log n); não conhece a recorrência → Berlekamp–Massey com 2d termos (DP com matriz de transição ≤ 50 estados).
- **Sequências P-recursivas.** Catalan, Motzkin, Delannoy satisfazem recorrências com coeficientes polinomiais: ache por ajuste, calcule n até 10^7 em O(n) com inversos pré-calculados.
- **Polinômio em n.** Se f(n) é polinômio de grau d (confira diferenças finitas), f(N) para N gigante = Lagrange com d+1 pontos em O(d); $\sum_{i<n} i^k$ é grau k+1; $\sum_{i<n} i^k r^i = r^nP(n) - P(0)$, grau P = k, para r ≠ 1 (em r = 1, Faulhaber de grau k+1).
- **Filtro de raízes da unidade.** $\sum_{k \equiv r (m)} C(n, k) = (1/m)\sum_j \omega^{-jr}(1+\omega^j)^n$; versão discreta: $(1+x)^n \bmod (x^m - 1)$ por exponenciação rápida de polinômio cíclico (subconjuntos com soma ≡ r mod m).
- **Strings com contagem par de cada letra.** Alfabeto k, comprimento n: $2^{-k}\sum_j C(k, j)(k - 2j)^n$ (EGF cosh^k).
- **Possibilidade de grau.** Sequência é graphical (Erdős–Gallai): soma par e $\sum_{i\leq k} d_i \leq k(k-1) + \sum_{i>k} \min(d_i, k)$ para cada k (ordenada decrescente); constrói por Havel–Hakimi. Bipartido: Gale–Ryser. Torneio: Landau.

- **Torneios: triângulos.** # 3-ciclos = C(n, 3) − Σ_v C(out_v, 2); em grafo, # caminhos de comprimento 2 = Σ C(deg, 2) e # 4-ciclos = (1/2)Σ_{u<w} C(cnt(u, w), 2) com cnt = vizinhos comuns.
- **Torres e posições proibidas.** Permutações evitando posições proibidas = Σ_k (−1)^k r_k (n−k)! (r_k = # modos de k torres não atacantes nas proibidas); em tabuleiro de Young com colunas ordenadas crescente: dp[i][j] = dp[i−1][j] + dp[i−1][j−1]·(h_i − (j−1)).
- **Permanente e determinante.** Permanente 0/1 (= # emparelhamentos perfeitos bipartidos) por Ryser $O(2^n \cdot n)$ com Gray code, n ≤ 25; permanente mod 2 = det mod 2; planar → Kasteleyn (Pfaffian/det). Rank da matriz de Tutte com valores aleatórios = 2·(emparelhamento máximo) em grafo geral.
- **Determinantes úteis.** Lema do determinante de matriz: $\det(A + uv^T) = \det(A)(1 + v^T A^{-1} u)$; Vandermonde ∏(x_j − x_i); polinômio característico por Hessenberg $O(n^3)$; tridiagonal → recorrência de 3 termos.
- **LGV.** Famílias de caminhos disjuntos em DAG (planaridade não é necessária) com fontes/destinos compatíveis = det[e(a_i, b_j)]; com e = C(·) em grid; permutações não-identidade entram com sinal, então precisa garantir que só a identidade é possível sem interseção. Aplica a ladrilhos de losangos e partições planas.
- **Ladrilhos de dominó.** Largura pequena → DP de perfil (broken profile) ou matriz de transição + exponenciação; planar grande → Kasteleyn.
- **Zeta/Möbius em subconjuntos.** $\hat{f}(S) = \Sigma_{T \subseteq S} f(T)$ em $O(2^n \cdot n)$, inversa por subtração; convolução de subconjuntos $O(2^n n^2)$ com posto (popcount); OR/AND por zeta; XOR por FWHT. Posets gerais: μ de cadeia = diferença, de divisores = μ(n/d), de partições = (−1)^{n−1}(n−1)!.
- **Contagem dupla clássica.** Σ_v deg(v) = 2m; Σ_{S⊆[n]} |S| = n·2^{n−1}; Σ de max (ou min) sobre subconjuntos: ordene crescente; max usa a_{(i)}·2^{i−1}, min usa a_{(i)}·2^{n−i}; "contar pares (elemento, conjunto)" pelos dois lados.
- **Somatórios: Abel e inversão de ordem.** Σ a_i b_i = A_n b_n − Σ A_i(b_{i+1} − b_i); Σ_{i≤n}Σ_{j≤i} = Σ_{j≤n}Σ_{i=j}^n; condição i·j ≤ n troca para somar sobre ⌊n/i⌋ valores.
- **Completar o quadrado em somas.** $\Sigma_{i<j}(a_i - a_j)^2 = n\Sigma a^2 - (\Sigma a)^2$; Σ_{i<j} a_i a_j = ((Σa)² − Σa²)/2; Σ_{i<j}|a_i − a_j| = Σ a_{(i)}(2i − n − 1) com a ordenado. Soma sobre pares vira O(n) com prefixos.
- **Telescópica.** 1/(i(i+1)) = 1/i − 1/(i+1); 1/(i(i+k)) = (1/k)(1/i − 1/(i+k)); Σ i·i! = (n+1)! − 1; Σ C(i, k) = hockey; produtos ∏(1 + 1/i) = n+1. Se uma soma tem termo "f(i+1) − f(i)", ache f.
- **Pombos.** Prefixos mod n: qualquer n inteiros têm bloco consecutivo com soma múltipla de n; Erdős–Ginzburg–Ziv: 2n−1 inteiros têm n com soma ≡ 0 (mod n); Erdős–Szekeres: n^2+1 distintos têm monótona de tamanho n+1 (LIS·LDS ≥ n^2+1); Dirichlet $|\alpha - p/q| < 1/(qN)$.
- **Dilworth e Mirsky.** Menor cobertura por cadeias = maior antichain (via matching bipartido); menor cobertura por antichains = maior cadeia; Greene–Kleitman para uniões de k cadeias.
- **Linearidade: truques de indicadores.** E[# valores distintos em n sorteios de N] = $N(1 - (1 - 1/N)^n)$; E[comparações do quicksort] = Σ_{i<j} 2/(j−i+1); E[# operações em "remova subárvore de vértice aleatório"] = Σ_v 1/depth(v) (v é escolhido antes dos ancestrais com prob. 1/depth).
- **Troca de ordem/tail sum.** E[X] = Σ_{k≥1} P(X ≥ k); $E[X^2] = \Sigma_{i,j} E[I_i I_j]$; E[máx de k dados de m faces] = Σ_{j=1}^{m} (1 − ((j−1)/m)^k); máx de n uniformes em [0, 1] tem média n/(n+1) e o j-ésimo menor j/(n+1).
- **Exchangeabilidade.** Sem reposição, P(k-ésima retirada ser vermelha) = r/N independente de k; recordes em permutação aleatória: P(i é recorde) = 1/i, independentes; gaps de n pontos em [0, 1] são intercambiáveis (cada um E = 1/(n+1)).
- **Coupon collector.** E = n·H_n; para colecionar k de n: Σ_{i<k} n/(n−i); probabilidades desiguais p_i: $\Sigma_{\emptyset \neq S}(-1)^{|S|+1}/p(S)$ (min–max); variância = $E[T^2] - E[T]^2$ com $E[T^2] = \Sigma_{\emptyset \neq S}(-1)^{|S|+1}(2 - p(S))/p(S)^2$.
- **Geométrica.** E[tempo até k sucessos] = k/p; P(primeiro sucesso em passo par) = (1−p)/(2−p); E[min de exponenciais] = 1/Σλ_i e quem vence é i com prob. λ_i/Σλ.
- **Cadeia de Markov absorvente.** E[tempos] = $(I - Q)^{-1} \cdot 1$ (Gauss, n ≤ ~500, em mod p com inversos). Estrutura linear/árvore: escreva E_v = α_v·E_{pai} + β_v e resolva sem Gauss; ciclo: duas incógnitas.
- **Ruína do jogador e martingais.** Passeio justo ±1 de i até {0, N}: P(chegar a N) = i/N, E[T] = i(N−i); com p ≠ q: ((q/p)^i − 1)/((q/p)^N − 1). Optional stopping: ache Φ com E[Φ_{t+1} | Φ_t] = Φ_t (ex. $S_t^2 - t$) ou E[ΔΦ] = −1 (então E[T] = Φ_0 − Φ_fim).
- **Passeio aleatório em grafo.** Distribuição estacionária π_v = deg(v)/2m, retorno esperado 2m/deg(v); em árvore, E[passos u → v adjacente] = 2·|E(componente de u após cortar uv)| + 1; tempo de ida e volta = 2m·R_eff.
- **Wald.** E[S_T] = E[T]·E[X] para T tempo de parada (soma de iid); P(vencer duelos) ou "primeiro a atingir" costuma sair de um argumento de simetria.
- **Método probabilístico e derandomização.** Atribuição aleatória garante ≥ esperança: MAX-CUT ≥ m/2, 7/8 das cláusulas 3-SAT; construtivo: método das esperanças condicionais (decida cada variável para não diminuir E). Lovász Local Lemma: e·p·(d+1) ≤ 1.
- **Aniversário.** P(colisão) ≈ $1 - e^{-n^2/(2N)}$: hash de 64 bits com 10^6 itens quase nunca colide, de 32 bits colide; use também para estimar quantas tentativas aleatórias bastam.

- **Esperança como fração mod p.** Calcule E = P/Q com inversos de inteiros pequenos (inv[1..n]); nunca use double se a saída é mod 998244353; DP "esperado do futuro" a partir do estado final.
- **Secretário.** Recuse os primeiros n/e e escolha o primeiro melhor que todos: sucesso ≈ 1/e.

Repetidas de outras seções:

- **Subsequências distintas** → dp[i] = 2·dp[i−1] − dp[última ocorrência − 1] (strings).
- **Parênteses/caminhos que não cruzam** → Catalan e reflexão (strings, grid).
- **Contar pares com gcd/condição de divisor** → Möbius de cima para baixo (teoria dos números).
- **n ≤ 20** → somar sobre subconjuntos com SOS; IE sobre máscaras.
- **Sequências com mínimo de prefixo ≥ −k** = C(2n, n) − C(2n, n+k+1) (reflexão); nº de palavras de Lyndon = (1/n) Σ μ(d) k^(n/d).
- **LGV (Lindström–Gessel–Viennot)** → número de famílias de caminhos disjuntos em vértices em DAG = det de matriz de caminhos (quando só a permutação identidade é possível para caminhos disjuntos).
- **Contar caminhos/ciclos de comprimento k** → potência da matriz de adjacência; ciclos de comprimento 3/4 por orientação em m√m.

Bits e XOR

Quase sempre: trate cada bit separadamente, do mais significativo para o menos.

- **Bit a bit:** soma de AND/OR/XOR sobre pares → conte, para cada bit, quantos têm 1 e multiplique por 2^b.
- **Maximizar XOR** → guloso do bit alto ao baixo; trie binária; base linear (eliminação de Gauss em GF(2)).
- **XOR de subarray** → prefix xor; xor de caminho em árvore → xor das raízes.
- **OR/AND de subarray** muda no máximo 30 vezes ao estender o fim (igual ao gcd).
- **a + b = (a xor b) + 2(a and b)**; a xor b ≤ a + b.
- **x xor y = 0 ⇔ x = y**: transforma pares em contagem de iguais.
- **Submáscaras** → enumere com s = (s − 1) & m (total 3^n); somas sobre subconjuntos → SOS.
- **Construtivo com XOR:** xor de 0..n tem padrão de período 4; use números grandes (2^29) para "ajustar" o xor final.
- **Paridade de popcount** e **número de 1s** podem ser o invariante.

adicionado em 2026-11-10 por agentes; revisado

- **Trie xor com contagem por nó.** "Quantos y têm x⊕y < k": desça pelos bits de k; quando o bit de k é 1, some o cnt do filho que dá xor 0 naquele bit e siga pelo que dá 1; quando é 0, siga pelo que dá 0. Com cnt também se faz delete (conjunto dinâmico).
- **Trie xor persistente / diferença de versões.** Max x⊕a_i com i ∈ [l, r]: guarde cnt por nó em root[i]; só entram ramos com cnt[root[r]] − cnt[root[l−1]] > 0.
- **Max xor de caminho / de subarray** = max de pref[i]⊕pref[j]; insira os prefixos na trie conforme varre (subarray) ou todos os px[v] da raiz (caminho; o LCA se cancela).
- **k-ésimo menor a_i⊕x** → desça na trie com cnt (se o ramo "xor 0" tem cnt ≥ k vá nele, senão subtraia). Mesma ideia dá o rank de um valor.
- **Mex de {a_i⊕x}** → trie sobre os distintos; um ramo "cheio" tem exatamente 2^b folhas; desça pelo ramo cujo xor é 0 se ele NÃO estiver cheio; se estiver cheio, o bit do mex é 1 e siga pelo outro ramo.
- **Ordem sob xor.** Comparar a⊕x com b⊕y é decidido pelo bit mais alto onde (a⊕x) e (b⊕y) diferem; para x fixo, o bit mais alto onde a e b diferem decide, e x só inverte a ordem se tiver 1 ali.
- **Min xor entre dois elementos** → o par ótimo é adjacente no array ordenado. Max xor global → trie.
- **Min do max a_i⊕x sobre x** → f(nó, b): se só um filho, f(filho); se dois, 2^b + min(f(esq), f(dir)). Recursão pela trie, O(n·bits).
- **Xor-MST (n números, peso = xor)** → divida por bit alto (ou trie): MST de cada metade + uma aresta mínima de xor entre as metades (achada por trie da outra metade). Clássico: CF 888G.
- **Update xor em todos os elementos / em intervalo** → trie com tag lazy de xor (troca filhos por bit) ou segtree com contagem de bits e flip por bit.
- **Range xor update + range xor query** → x só contribui na interseção se o tamanho for ímpar; use duas BITs separadas por paridade do índice.
- **Base linear (Gauss em GF(2)).** Insira vetores; rank r ≤ nº de bits. Nº de xors distintos de subconjuntos = 2^r, cada valor alcançável aparece 2^(n−r) vezes; x é alcançável ⇔ reduz a 0.
- **k-ésimo menor xor de subconjunto** → base reduzida (forma escalonada reduzida); os bits de k−1 escolhem quais vetores (do menor pivô ao maior) entram.

- **Base com prefixo (queries [l, r] de max xor de subconjunto)** → mantenha, ao varrer r, a base com o índice mais recente em cada pivô (troca ao inserir se o novo é mais recente); query usa só vetores com índice ≥ l.
- **Base com remoções** → segtree no tempo (cada vetor vive num intervalo) + cópia da base (≤ 30 inteiros) a cada nível da recursão, em vez de rollback.
- **n > nº de bits (ou n > 2^k com valores < 2^k)** → pigeonhole: existe subconjunto/subarray de xor 0 (n+1 prefixos em 2^k valores têm repetição). Use para provar que a resposta existe ou limitar o brute force.
- **Operação "a_i ← a_i⊕a_j"** → o span (espaço gerado) é invariante; classes de equivalência = mesmo span; o mínimo de passos para a forma canônica NÃO depende só de n − rank: depende de quais vetores são redundantes e de suas posições.
- **Soma de xor de todos os subconjuntos** = (OR de todos) · 2^(n−1) (cada bit presente em algum elemento aparece em metade dos subconjuntos).
- **Xor de todos os subarrays** → a_i (0-indexado) está em (i+1)(n−i) subarrays e entra se esse número é ímpar, ou seja, i par e n ímpar; se n é par, o resultado é 0.
- **Soma de a_i⊕a_j, a_i&a_j, a_i|a_j sobre pares** → por bit: $c_1 \cdot c_0$, $C(c_1,2)$, $C(n,2) - C(c_0,2)$, vezes 2^b.
- **Max AND de um par** → guloso do bit alto: teste cand | 2^b e conte elementos que contêm cand; se ≥ 2, fixe o bit.
- **Pares com a_i + a_j em que o bit k é 1 (ou xor das somas)** → reduza cada valor mod 2^(k+1), ordene e conte somas ∈ [2^k, 2^(k+1)) ∪ [2^(k+1)+2^k, 2^(k+2)) com two pointers. Xor de todas as somas = xor dos bits obtidos (CF 1322B).
- **a + b = (a⊕b) + 2(a&b) invertido.** Dados s = a+b e x = a⊕b: precisa s ≥ x, s − x par, e (s−x)/2 & x = 0; então o nº de pares (a, b) ordenados é 2^popcount(x). Também a+b = a⊕b ⇔ a&b = 0, e n+x = n⊕x ⇔ x ⊆ ~n.
- **Identidades úteis:** a+b = (a|b) + (a&b); a⊕b = (a|b) − (a&b); a|b ≥ max(a, b); a&b ≤ min(a, b).
- **AND é não-crescente, OR não-decrescente ao estender.** Contar subarrays com AND/OR ≥ k, = k → para cada fim, ≤ 30 pares (valor, menor início); ou two pointers com contador cnt[b] por bit (janela com OR: bit ligado ⇔ cnt[b] > 0).
- **Janela deslizante com AND/OR/gcd (não invertíveis)** → duas pilhas (fila com agregado), ou contador por bit; ambos O(n·bits) ou O(n).
- **Simulação de soma/carry.** "Quantos pares (a, b) ≤ A, B com a+b = ... / a&b = 0 / a⊕b ≤ K": digit DP bit a bit, estado [bit][tightA][tightB][carry].
- **Intervalo [0, A) = ≤ log A blocos alinhados** (prefixo fixo + sufixo livre). Contar x ≤ A com propriedade bit a bit, ou xor de dois intervalos, vira combinação de blocos.
- **Soma dos popcounts de 0..n** → por bit b: ⌊(n+1)/2^(b+1)⌋·2^b + max(0, (n+1) mod 2^(b+1) − 2^b).
- **Contar x ≤ n com popcount = k** → percorra os bits 1 de n: ao passar por um 1 na posição b, some C(b, k − usados); ao final, some 1 se popcount(n) = k (o próprio n).
- **n como soma de exatamente k potências de 2** ⇔ popcount(n) ≤ k ≤ n (quebre uma potência 2^b em duas 2^(b−1) para aumentar). Mínimo de termos com ±2^i = peso da NAF (forma não adjacente).
- **Gosper's hack.** Próxima máscara com o mesmo popcount: c = x&−x; r = x+c; nxt = (((r⊕x)>>2)/c)|r. Enumera C(n, k) máscaras em ordem crescente.
- **Contador binário** → custo amortizado O(1) por incremento (potencial = nº de 1s); vale só para incrementos puros, não para mistura ±1.
- **Xor de 0..n** → n mod 4 = 0: n; 1: 1; 2: n+1; 3: 0. Xor de a..b = F(b) ⊕ F(a−1).
- **Paridade de C(n, k)** → ímpar ⇔ k & (n−k) = 0 ⇔ k ⊆ n (Lucas mod 2). Nº de ímpares na linha n = 2^popcount(n).
- **Nim com limite / generalizações.** Tirar até k de uma pilha: Grundy = a mod (k+1). Moore's Nim_k (tirar de até k pilhas): escreva em binário e some cada bit mod (k+1); perde quem tem tudo 0.
- **Ordem de Gray.** g(i) = i⊕(i>>1); de i−1 para i muda o bit ctz(i). Percorre os 2^n subconjuntos alterando 1 elemento por passo → soma/xor/estado atualizado em O(1) por subconjunto (enumeração de subset sums sem recomputar).
- **Lowbit.** x & −x isola o menor bit; x & (x−1) remove-o; itere os bits ligados em O(popcount). Fenwick: na atualização, o pai de i é i + lowbit(i); na consulta de prefixo, o próximo bloco é i − lowbit(i).
- **Troca de dois bits** i, j: x ⊕= (((x>>i) ⊕ (x>>j)) & 1) · ((1<<i) | (1<<j)).
- **Zeta/Möbius sobre subconjuntos (SOS) como ferramenta de contagem.** Pares com a_i & a_j = 0 → para cada a_i, quantos b ⊆ ~a_i (soma sobre submáscaras do cnt). Möbius (inversa) recupera "exatamente" a partir de "⊆".
- **SOS guardando top-2.** "Max/min de a_j sobre j com a_j ⊇ máscara (ou ⊆)" e "dois melhores índices distintos" → SOS com 2 melhores por máscara (ex.: max a_i | (a_j & a_k) com i < j < k, CF 1208F).
- **SOS sobre superconjuntos** para AND ("existe elemento que contém m?"), sobre subconjuntos para OR; a ordem dos laços (bit externo) é essencial.
- **Convoluções AND/OR/XOR.** OR: zeta de subconjuntos, produto pontual, Möbius. AND: o mesmo com superconjuntos. XOR: Walsh–Hadamard.
- **Walsh–Hadamard (xor convolution).** Transformada em O(n log n) com +/−; a inversa é a mesma transformada dividida por 2^k (mod p: multiplique por inv(2)^k). Em Z ou com mod p o resultado é exato.

- **Potência/caminhos no hipercubo.** Contar sequências de k passos com xor total x (cada passo ∈ conjunto S) → WHT de [S], eleve ponto a ponto a k, WHT inversa. Também "nº de maneiras de escolher k elementos (com ordem) com xor = x".
- **DP de submáscaras 3^n.** dp[m] = f(dp[s], dp[m\s]): fixe lowbit(m) ∈ s para não contar partições ordenadas duas vezes (÷2 em constante). Para partições em blocos válidos, use bloco que contém o menor elemento.
- **Meet in the middle em xor.** n ≤ 40 e "subconjunto com xor = X / maior xor ≤ K": 2^20 de cada metade em dois arrays ordenados/hash; para "max xor" combine com trie. Se n grande e valores pequenos, prefira base linear.
- **Distância de Hamming pequena (≤ d, com ≤ 20 bits):** BFS multi-fonte no hipercubo (vizinho = flip de 1 bit) dá, para cada máscara, a distância ao conjunto mais próximo em O(2^k·k).
- **Soma de distâncias de Hamming** entre n strings binárias → por coluna $c_0 \cdot c_1$. Mediana/centro de Hamming: bit a bit pela maioria.
- **Xor de caminho em árvore.** px[v] = xor da raiz até v; xor(u, v) = px[u]⊕px[v] para valores em arestas; para valores em vértices, inclua a[lca].
- **Contar caminhos com xor = k em árvore** → pares de px com px[u]⊕px[v] = k (contagem por mapa); com valores em vértices, o a[lca] extra exige small-to-large ou centroide.
- **Ciclo base em grafo (xor de ciclos).** Faça DFS, px[v] pela árvore; cada aresta fora da árvore (u, v, w) gera ciclo c = px[u]⊕px[v]⊕w; insira todos na base linear. Xor de qualquer passeio u→v = px[u]⊕px[v]⊕(elemento do span); mínimo/máximo = reduza pela base. Dimensão do espaço de ciclos = m − n + (componentes).
- **Aresta a aresta em GF(2).** "Escolher arestas com paridade de grau dada em cada vértice" = sistema linear em GF(2) (T-join); numa árvore a solução é única, de baixo para cima.
- **Operação "inverter bits de um conjunto" (lights out).** Cada operação é um vetor em GF(2)^n; resolva por Gauss com bitset $O(n^3/64)$; o nº de soluções é 2^(n−rank) ou 0.
- **Bitset shift-and.** Padrão curto/média em texto, ou matching com classes de caracteres: mantenha o bitset de prefixos do padrão casados, D = ((D<<1)|1) & M[c]; O(n·m/w). Com k erros, k+1 bitsets.
- **Bitset: BFS/DFS em grafo denso** → unvisited &= ~adj[v] e extraia com _Find_first/_Find_next: cada vértice sai uma vez; $O(n^2/w)$ total.
- **Fecho transitivo/Floyd booleano com bitset** → para k, para i: if reach[i][k] reach[i] |= reach[k]; $O(n^3/w)$. Produto de matrizes booleanas e "vizinhos comuns" = popcount(adj[u] & adj[v]).
- **Bitset knapsack e variantes.** dp |= dp << w; reconstrução guardando por item ou usando divide e conquista. Para "soma alcançável com k itens" use dp[k] bitset (k·S/w).
- **LCS/LIS com bitset:** ver Strings (LCS em O(nm/w)).

Repetidas de outras seções:

- **Nim e Sprague-Grundy** → xor dos Grundy (teoria dos jogos).
- **Anagrama de palíndromo** → máscara de paridades de 26 bits; prefixos com máscara igual ou a 1 bit (strings).
- **Hash de conjunto por xor** de valores aleatórios (aleatorização).
- **C(n, k) ímpar** ⇔ k é submáscara de n (Lucas p=2); # ímpares na linha n = 2^{popcount(n)}.
- **Convolução XOR/OR/AND** → FWHT / zeta em subconjuntos; subset convolution $O(2^n\, n^2)$.

Strings

Primeiro teste: hashing resolve? Se sim, é o caminho mais curto até um AC.

Comparação e busca:

- **Comparar substrings** → hash polinomial (dois módulos ou mod $2^{61} - 1$); LCP de dois sufixos → binária + hash.
- **Menor/maior lexicográfico** → compare via LCP (hash + binária) em O(log n); guloso caractere a caractere do início.
- **Padrão em texto** → KMP/Z-function; muitos padrões → Aho-Corasick (+ DP no autômato para "string que não contém nenhum padrão").
- **Padrão com coringa '?'** → FFT: Σ $(a - b)^2 \cdot a \cdot b = 0$ por posição.
- **Período/borda** → função prefixo: menor período p = n − π[n−1]; é período inteiro se p divide n.
- **Concatenar com separador** (s + '#' + t) para transformar "busca de s em t" em Z/prefixo de uma string só.

Estruturas:

- **Substrings distintas, k-ésima substring, ocorrências** → suffix array + LCP ou suffix automaton.
- **LCP de dois sufixos quaisquer** → SA + sparse table no array LCP.
- **Maior substring comum de k strings** → SA da concatenação + janela deslizante sobre o LCP; ou SAM.
- **Prefixos de várias strings** → trie; contar pares com LCP ≥ k → profundidade na trie.

- **Palíndromos** → Manacher; todos distintos/contar → eertree; "menor número de palíndromos" → DP + eertree (séries em log).

Contagem e limites escondidos:

- **N strings com Σ|s| ≤ S** → no máximo √(2S) tamanhos distintos. Agrupe por tamanho; para cada tamanho faça um hash set. (Também vale em DP, arrays e queries: mesmo truque.)
- **Σ|s| ≤ S e uma string "curta"/"longa"** → curtas (≤ √S) por brute force, longas são ≤ √S strings.
- **Uma string de tamanho n** → só n substrings distintas por tamanho; O(n) palíndromos distintos; O(n) bordas.
- **Aho-Corasick com padrões de Σ ≤ S** → cada posição do texto casa com ≤ √(2S) tamanhos distintos de padrão.
- **Alfabeto pequeno (26)** → 26 no estado, frequências, máscara de paridades (anagrama de palíndromo = ≤ 1 bit ligado).
- **Anagramas** → ordene a string ou compare vetores de frequência; hash = Σ rand[c].
- **Contar subsequências** → DP com next[i][c]; subsequências distintas → dp[i] = 2·dp[i−1] − dp[última ocorrência − 1].

Transformações:

- **Rotação/cíclico** → duplique a string (s + s); mínima rotação → Booth ou SA de s + s.
- **Strings binárias e operações** → run-length (blocos iguais), número de trocas 01/10, balanço prefixo (+1/−1).
- **Parênteses** → prefixo ≥ 0 e total 0; mínimo de prefixo diz quantos consertar; Catalan para contar.
- **Operação "apague dois iguais adjacentes"** → pilha; resultado é canônico (invariante).
- **Reverter, ler de trás pra frente** → strings invertidas, palíndromo = s igual ao reverso (hash nos dois sentidos).
- **s se repete k vezes / s infinita** → não basta 2 ou 3 cópias em geral; para matching de um padrão p use ⌈|p|/|s|⌉+1 cópias e extrapole.
- **Jogo com strings** → ver Teoria dos jogos (quem escolhe a primeira letra geralmente decide).

adicionado em 2026-11-10 por agentes; revisado

- **Hash mod 2^61−1.** Base aleatória em [2, p−2], mulmod via __int128; colisão ≈ n/2^61 por comparação. É o padrão seguro; mod 2^64 (overflow) é quebrável.
- **Anti-hash por Thue–Morse.** Para hash com overflow em 2^64 (qualquer base), a string de Thue–Morse de ordem ≥ 11 e seu complemento colidem. Contra mod conhecido fixo, ataques de aniversário/reticulado; a defesa é base aleatória + mod grande (ou dois mods).
- **Concatenação e remoção por hash.** H(AB) = H(A) + B^|A|·H(B) (com H = Σ s_i B^i); prefixo/sufixo se subtrai. Permite treap/rope com hash para strings com inserção/remoção no meio.
- **Hash de string e do reverso** → palíndromo de substring em O(1); com updates de caractere, BIT/segtree guardando os dois hashes.
- **Hash 2D.** H[i][j] com duas bases (uma por eixo); hash de submatriz em O(1); busca de padrão 2D e "submatrizes iguais" por tamanho fixo com set de hashes.
- **Hash de árvore enraizada (isomorfismo).** id(v) = id do multiconjunto ordenado de ids dos filhos (AHU com mapa), ou hash H(v) = 1 + Σ f(H(filho)) com f pseudoaleatório. Sem raiz: enraíze nos 1–2 centroides e compare ambos.
- **Comparação lexicográfica por hash.** LCP por binária em O(log n); ordenar n sufixos/substrings com esse comparador em O(n log² n). Com SA + RMQ vira O(1) por comparação.
- **Hash de multiconjunto de letras** (anagrama) = Σ rand[c]; janela deslizante de tamanho |p| em O(1) por passo. Alternativa: manter cnt e um contador de letras "erradas".
- **Z-function para contar prefixos.** Ocorrências de s[0..L) em s = #{i : z[i] ≥ L} + 1; com array de diferenças, conta para todos os L em O(n). Soma de z = Σ LCP(s, sufixo).
- **Prefix function: contar ocorrências de cada prefixo.** cnt[π[i]]++ para todo i, depois propague de trás pra frente cnt[π[i−1]] += cnt[i]; cnt[k]+1 = nº de ocorrências do prefixo de tamanho k.
- **Autômato KMP.** aut[0][c] = (p[0] == c ? 1 : 0); para i > 0, aut[i][c] = i+1 se p[i] = c, senão aut[π[i−1]][c]; O(n·Σ). Serve para DP "string de tamanho L contendo/evitando p" e (com matriz) para L enorme.
- **KMP e rollback.** A amortização de π quebra se você desfaz caracteres; use o autômato completo, ou guarde a pilha de estados, ou faça DFS numa trie de strings com π por nó.
- **Todas as bordas** = cadeia π[n−1], π[π[n−1]−1], ...; todos os períodos = n − borda. Existe borda de tamanho b ⇔ período n − b; s é potência ⇔ p = n − π[n−1] divide n e p < n.
- **Lema de periodicidade (Fine–Wilf).** Se p e q são períodos e p + q − gcd(p, q) ≤ n, então gcd(p, q) é período. Consequência: as bordas de tamanho ≥ n/2 formam uma PA; todas as bordas se agrupam em O(log n) PAs.
- **Ocorrências de p em janela de tamanho ≤ 2|p|** formam uma PA (ou são ≤ 2): guarde (início, razão, quantidade) em O(1). Duas ocorrências a distância d < |p| fazem de d um período de p.

- **Runs theorem.** O nº de runs (repetições maximais com expoente ≥ 2) é < n; a soma dos expoentes é ≤ 3n; calcule todas via fatores de Lyndon + LCP/LCE em O(n log n). Número de quadrados distintos numa string é < 2n.
- **Quadrados (xx) / tandem repeats** → Main–Lorentz O(n log n) por D&C, ou via runs; "existe substring xx" em O(n).
- **Comutação.** xy = yx ⇔ x e y são potências da mesma palavra (Lyndon–Schützenberger); s^a = t^b ⇔ s·t = t·s. Para a raiz primitiva de s: p0 = n − π[n−1]; se p0 | n, a raiz é s[0..p0); senão a raiz é a própria s (s é primitiva).
- **Rotações e potências por s+s.** t é rotação de s ⇔ t ocorre em s+s (|t| = |s|). Menor período cíclico (raiz primitiva) = primeira posição i > 0 onde s ocorre em s+s.
- **Aho-Corasick: link tree.** Contar quantas vezes cada padrão ocorre no texto: marque cnt[nó] nos nós visitados e some a subárvore na árvore de links (processe em ordem inversa de BFS). O(|T| + Σ|p|).
- **AC em DP.** Marque nó como ruim se ele ou algum ancestral de link é terminal (propague na BFS). dp[i][nó] conta strings de tamanho i sem padrão; para i enorme → matriz (nós × nós) e exponenciação, O(S^3 log L).
- **AC com ganhos.** val[v] += val[link[v]] na BFS; max de soma de valores de padrões presentes numa string de tamanho L = dp[i][nó] com transições pelo autômato.
- **Dict link (exit link).** Para listar só os padrões que terminam numa posição, pule pelo próximo terminal via link: O(nº de ocorrências) em vez de O(profundidade).
- **AC com padrões inseridos dinamicamente** → técnica logarítmica: mantenha autômatos de tamanhos potências de 2 e junte como binário (Bentley–Saxe); consulta soma sobre O(log) autômatos.
- **Menor string que contém todos os padrões** (poucos padrões, ≤ 10–15): remova padrões que são substring de outros, calcule overlaps (KMP/Z) e faça DP[máscara][último] (superstring mínima = TSP); alternativa BFS em (nó AC, máscara).
- **Ocorrências de padrão i em prefixo s[1..r], por queries.** Ordene por r, percorra s no AC fazendo +1 no tin do nó (BIT sobre Euler da link tree); resposta = soma da subárvore do nó do padrão. Intervalo [l, r]: diferença de dois prefixos (r e l+|p| −2).
- **SA: substrings distintas** = n(n+1)/2 − Σ lcp. **k-ésima substring distinta:** o sufixo i do SA contribui n − sa[i] − lcp[i] novas; desça somando.
- **SA: maior substring que ocorre ≥ k vezes** → máximo, sobre janelas de k−1 LCPs consecutivos, do mínimo da janela (deque monotônico). k = 2: max lcp.
- **SA: menor substring única** (ocorre uma vez) começando em i → max(lcp[i], lcp[i+1]) + 1, se ≤ n − sa[i].
- **SA: soma de LCP sobre todos os pares de sufixos** = soma de min de todos subarrays do array lcp → pilha monotônica. Contar substrings comuns a duas strings: o mesmo, separando origens (conta(A#B) − conta(A) − conta(B)).
- **SA: maior substring repetida sem sobreposição** → binária em L: agrupe sufixos consecutivos com lcp ≥ L; vale se max(sa) − min(sa) ≥ L no grupo.
- **SA de múltiplas strings.** Concatene com separadores distintos (ou use um só e limite o LCP pelo tamanho de cada string); substring comum a todas as k → menor janela do SA com todas as cores, valor = min lcp da janela, maximizar.
- **SA: comparar substrings s[a..b) vs s[c..d)** em O(1): l = LCP(a, c) por RMQ; se l ≥ min(tamanhos) compare tamanhos, senão compare s[a+l] com s[c+l].
- **SA: ocorrências de p em s[l..r].** Intervalo [lo, hi] do SA por binária; conte sa[i] ∈ [l, r−|p|+1] com wavelet/segtree persistente ou BIT offline.
- **BWT.** BWT[i] = s[sa[i]−1] (com $); invertível por LF-mapping; agrupa letras iguais (compressão, run-length). FM-index: busca de padrão em O(|p|) por backward search com C[] e rank.
- **SAM: contagem de ocorrências.** cnt = 1 nos estados não-clone (prefixos), 0 nos clones; soma na subárvore da link tree. Substrings distintas = Σ (len[v] − len[link[v]]); online: ao adicionar um caractere, ans += len[last] − len[link[last]].
- **SAM: maximizar |t|·ocorrências(t)** → max sobre estados de cnt[v]·len[v]. Maior substring com ≥ k ocorrências: max len[v] com cnt[v] ≥ k.
- **SAM: substring comum de duas strings (ou k).** SAM de A; percorra B mantendo (estado, comprimento) com saltos de link ao falhar; para k strings, para cada string calcule best[v] = maior match no estado e propague para o link com best[link[v]] = max(best[link[v]], min(best[v], len[link[v]])) em ordem decrescente de len; após propagar cada string, atualize o mínimo global por estado (o menor best entre as strings).
- **SAM: link tree de s = suffix tree de reverse(s).** Logo, suffix tree de s = SAM de rev(s). Tamanho ≤ 2n−1 estados, ≤ 3n−4 transições.
- **SAM: substring t[l..r] → estado.** Pegue o estado do prefixo t[1..r] e suba pela link tree (binary lifting) até o ancestral de menor profundidade com len ≥ r−l+1; ali estão cnt e endpos.
- **SAM: k-ésima substring lexicográfica** → dp[v] = nº de caminhos a partir de v (distintas) ou Σ cnt (com multiplicidade); desça pelas transições em ordem de letra.
- **SAM generalizado (várias strings / trie).** Reinicie last = root a cada string e trate o caso em que a transição já existe (evite clone duplicado); ou construa por BFS na trie. Substrings distintas do conjunto = Σ (len − len[link]).

- **Menor string que NÃO é substring** (alfabeto k): no SAM dp[v] = 1 + min_c dp[go[v][c]] com dp = 0 se go não existe (o menor é 1 se falta uma letra). A lexicográfica mínima segue do dp.
- **Menor string que não é subsequência.** dp[i] = 1 + min_c dp[next[i][c]+1] (next = n se não existe, dp[n+1] = 0); reconstrua pegando a letra de menor dp (menor lexicográfica entre as de mesmo valor).
- **Suffix tree.** LCA das folhas = LCP; substring de tamanho ≥ L mais frequente, "ocorrências por nó" por small-to-large ou Euler tour; é a estrutura certa quando o problema fala de "árvore de sufixos" ou mistura strings com DP em árvore.
- **Manacher + diferenças.** Cada centro com raio r contribui, para o nº de palíndromos que cobrem cada posição, com uma função triangular; some tudo com array de segunda diferença e integre duas vezes. Para palíndromos que terminam/começam em cada índice, use range add (1ª diferença): cada centro soma +1 no intervalo correspondente. Dá "nº de palíndromos que cobrem" (2ª diferença) ou "que terminam/começam" (1ª diferença) em O(n).
- **Pares de palíndromos adjacentes / s = P_1P_2...** → conte palíndromos que terminam em i (L[i]) e começam em i+1 (R[i+1]) e some L·R; fatoração em k palíndromos com k ≤ 3 tem caracterização direta.
- **Eertree: ocorrências.** cnt[v] += cnt[filho em link] em ordem de criação invertida dá nº de ocorrências de cada palíndromo distinto; ≤ n palíndromos distintos. Rollback do último caractere por pilha de `last`.
- **Fatoração em palíndromos (série link).** diff[v] = len[v] − len[link[v]]; palíndromos-sufixos com mesmo diff formam PA, só O(log n) por posição → dp de partição mínima/contagem em O(n log n). Partição em palíndromos de tamanho par: intercale $s_1\ s_n\ s_2\ s_{n-1}$... e conte partições em palíndromos de tamanho par da nova string (CF 932G).
- **Menor palíndromo adicionando na frente/no fim.** Na frente: n − (maior prefixo palíndromo), e o maior prefixo palíndromo = π de s#rev(s). No fim: simétrico (maior sufixo palíndromo).
- **Lyndon / Duval.** Toda string se fatora de modo único em palavras de Lyndon não-crescentes, em O(n) (Duval). Lyndon = estritamente menor que todos os seus sufixos próprios (e que todas as rotações).
- **Rotação mínima** → início do último fator de Duval de s+s que começa antes de n; ou Booth; ou SA de s+s. Canonização de colar/rotação: hash da rotação mínima.
- **Nº de palavras de Lyndon de tamanho n em k letras** = (1/n) Σ_{d|n} μ(d)·k^(n/d). Colares: (1/n) Σ φ(d)·k^(n/d) (Burnside).
- **Pulseira / "string reversível".** Se s e rev(s) são equivalentes, canon = min(rotmin(s), rotmin(rev s)); com operação reversível, compare formas canônicas dos dois lados.
- **Strings de Fibonacci (S_n = S_{n−1}S_{n−2}) e SLPs.** Ocorrências cnt_n = cnt_{n−1} + cnt_{n−2} + cruzamentos; só os |p| −1 caracteres de borda importam, e as bordas estabilizam depois de poucos passos. Vale para qualquer string definida por concatenações recursivas.
- **Thue–Morse.** t_n = popcount(n) mod 2; livre de sobreposições (sem cubos nem xyxyx). Aparece como anti-hash, em construções "sem repetição" e em partição de {0..2^k−1} em dois conjuntos com mesmas somas de potências até k−1 (Prouhet).
- **de Bruijn B(k, n).** Sequência cíclica de tamanho k^n com toda string de tamanho n exatamente uma vez; construa por ciclo euleriano nos (n−1)-gramas ou concatenando palavras de Lyndon de comprimento divisor de n em ordem lexicográfica. Cadeado/senha mínima, construtivos.
- **Trie + small-to-large / merge de tries.** Mesclar duas tries recursivamente (juntando filhos de mesma letra) custa O(total de nós) amortizado; serve para DP sobre árvore onde cada nó tem um conjunto de strings/sufixos.
- **Trie persistente / de versões.** "Versão v + caractere c" cria um nó na trie global (árvore de prefixos); LCP = profundidade do LCA, hash do prefixo = hash do pai + c, e comparações de versões via binary lifting. Processar offline por DFS com rollback.
- **Ordenar strings por trie** em O(Σ|s|) (DFS em ordem de letra); LCP de duas strings ordenadas = min dos LCP adjacentes (RMQ).
- **Edit distance com limite k.** Banda |i − j| ≤ k → O(nk); Landau–Vishkin com LCE em O(1) → O(n + k^2) para distância ≤ k.
- **Kangaroo jumps.** Posições onde p casa em t com ≤ k erros → em cada deslocamento, k+1 saltos de LCE (hash+binária ou SA+RMQ): O(n·k) total.
- **Matching de Hamming por FFT.** Nº de coincidências por deslocamento = Σ_c conv(1[t=c], 1[p reverso = c]) (σ convoluções para alfabeto de tamanho σ); uma única convolução complexa com raízes de unidade ω^c não recupera a contagem para alfabeto > 2. Com '?' use Σ $(a-b)^2$·a·b e valores aleatórios mod p para evitar falso zero.
- **LCS com bitset** em O(nm/w): V = tudo 1; para cada caractere c de t, M = máscara de ocorrências de c em s, V = (V + (V & M)) | (V & ~M); LCS = nº de zeros em V (nos primeiros |s| bits).
- **LCS ↔ LIS.** Para permutações (ou strings com poucas ocorrências por letra), LCS = LIS em O(n log n) (Hunt–Szymanski, O(r log n)); LPS (subsequência palindrômica) = LCS(s, rev s); inserções mínimas p/ palíndromo = n − LPS.
- **Subsequência como gatilho.** "t é subsequência de s?" com muitas queries → next[i][c] ou listas de ocorrências + binária; contar quantas de várias strings t são subsequências de s → listas de espera por próxima letra, O(|s| + Σ|t|).
- **k-ésima subsequência distinta lexicográfica** → cnt[i] = 1 + Σ_c cnt[next[i][c]+1] (next = n se não existe; cnt[n] = 1, cnt[n+1] = 0) e desça escolhendo letras em ordem.

- **Maior borda que também aparece no meio** (prefix = sufixo = substring interior) → entre as bordas de π, a maior com ocorrência interna (use max de z/π no meio) (CF 126B).
- **Strings k-periódicas / período d.** s tem período d ⇔ s[i] = s[i+d]; classes i mod d via DSU. Menor nº de mudanças para período d = n − Σ_classes(max freq). Com palíndromo + período, DSU em i ~ n−1−i e i ~ i+d.
- **Anagramas e multiconjuntos.** Nº de permutações distintas = n!/Π c_i!; ranking lexicográfico de permutação de multiconjunto → soma sobre letras menores com BIT de contagens; "próxima permutação" em O(n).
- **Parênteses: balanço e mínimo de prefixo.** Válida ⇔ min prefixo ≥ 0 e total 0. Inserções mínimas = (−min) aberturas + (total − min) fechamentos; maior subsequência válida = 2·(nº de pares casados pelo guloso com contador/pilha), que pode ser menor que 2·min(#(, #)).
- **Parênteses em segtree.** Nó guarda (soma, min prefixo); merge = (sL+sR, min(mL, sL+mR)). Substring é válida ⇔ soma 0 e min ≥ 0. Updates por troca de caractere.
- **Contar sequências com mínimo de prefixo ≥ −k** (2n passos, terminando em 0) = C(2n, n) − C(2n, n+k+1) (reflexão). k = 0: Catalan.
- **Árvore ↔ Dyck.** DFS que escreve (ao entrar e) ao sair: subárvore = intervalo balanceado, LCA = posição de mínimo do balanço (RMQ), profundidade = balanço. Contar árvores com n nós = Catalan(n−1).
- **Parênteses de vários tipos** → pilha; troca mínima / "casamento" quando só existe um tipo ou tipos de mesma classe → guloso com contador.
- **Shunting-yard.** Operadores com precedência e associatividade (^ à direita); menos unário como token distinto com precedência alta; avalie com pilha de operandos. Para gramática recursiva, descent: expr → termo (±) → fator (×, /) → átomo/parênteses.
- **Parsing com módulo.** Ao avaliar expressões mod p, divisão = inverso modular; números grandes lidos como string e reduzidos mod p na hora; avalie com pilhas em O(n) sem recursão profunda.
- **Regex/glob simples.** '.' e '' → *DP O(nm); glob com '?' e* '' → guloso com retorno à última '*' (O(nm) no pior caso); NFA de Thompson simulado com bitset.
- **DP de dicionário.** dp[i] = nº de modos de dividir s[0..i) em palavras do dicionário: use trie parado em profundidade maxlen (O(n·maxlen)), ou ≤ √(2S) tamanhos distintos com hash set por tamanho, ou AC com dict link.
- **LZ78 / LZ77.** LZ78 = percorrer trie criando um nó por fator (≤ O(n/log n) fatores); LZ77 por SA/SAM. O nº de fatores é medida de compressibilidade e aparece como limite de complexidade em estruturas sobre strings repetitivas.
- **Construção lexicográfica mínima.** Escolha letra a letra e teste viabilidade com um dp pronto (SAM/AC/next[i][c]); o desempate é local, o teste é global.
- **Strings com operações e rollback** → pilha de estados (hash, nó do AC, nó da trie) em vez de recomputar; versionamento por árvore de prefixos.
- **Palíndromo em substring com updates** → segtree com (hash direto, hash do reverso); para contagem de palíndromos/qualquer centro, Manacher não serve com updates: use hash + binária no raio.

Repetidas de outras seções:

- **Tempo esperado até padrão** → Σ q^k sobre bordas do padrão (ABRACADABRA = $26^{11} + 26^4 + 26$).
- **Strings primitivas / períodos exatos** → Möbius sobre divisores de n: Σ_{d|n} μ(n/d)k^d.

Grafos

Pergunte: quais são os vértices de verdade? Muitas vezes o grafo útil é de estados, não o do enunciado.

- **Grafo de estados:** (vértice, quantos usos de algo, paridade, tempo mod k) → BFS/Dijkstra em camadas.
- **Pesos 0/1** → 0-1 BFS com deque. Pesos pequenos → BFS com várias filas.
- **Várias fontes** → super-fonte ligada a todas com custo 0; caminho de volta → grafo reverso.
- **Arestas de um vértice para um intervalo** → segtree de arestas (grafo sobre os nós da segtree).
- **Restrições x_j − x_i ≤ c** → diferença de potenciais, Bellman-Ford.
- **Restrições booleanas de 2 variáveis** → 2-SAT (SCC).
- **Direcionado com ciclos** → condense em SCC e vire DAG; no DAG, DP em ordem topológica.
- **Pontes e articulações** → árvore de componentes 2-aresta-conexas / block-cut tree.
- **"Adicione arestas" ou "conectividade"** → DSU; com remoções → offline ao contrário ou DSU com rollback + D&C no tempo.
- **MST:** Kruskal; "maior aresta mínima no caminho" → MST + LCA ou árvore de Kruskal.
- **Bipartido / 2-coloração** → paridade dos ciclos; ciclo ímpar = não bipartido.

- **Emparelhamento, cobertura, "atribuir"** → fluxo/matching; cortes mínimos para "escolher conjunto com dependências" (projeto de seleção / closure).
- **Funcional (cada vértice uma saída)** → ciclos com árvores penduradas; k passos → binary lifting.
- **Grafo denso ou complemento** → BFS no complemento com conjunto de não visitados.
- **Euleriano** → graus pares; caminho usando cada aresta uma vez → Hierholzer.
- **Grau ≤ 2** → caminhos e ciclos; grau alto com m pequeno → no máximo $2\sqrt{m}$ vértices com grau ≥ $\sqrt{m}$ (contar triângulos em $m\sqrt{m}$).

---

adicionado em 2026-11-10 por agentes; revisado

- **Dijkstra com pesos negativos só nas fontes/arestas de saída.** Se só há pesos negativos em arestas que saem de uma super-fonte, some constantes e rode Dijkstra; em geral, potenciais h(v) (Johnson) tornam $w'(u,v) = w + h(u) - h(v) \geq 0$ e dão todos-os-pares em $n \cdot (m \log n)$ com um Bellman-Ford inicial.
- **Dijkstra lazy vs. decrease-key.** Com $m \approx n^2$ (grafo denso) use Dijkstra $O(n^2)$ com vetor, sem heap; com m pequeno, heap com pares e ignore entradas velhas (d > dist[v]).
- **Dijkstra com custo "máximo" ou "bottleneck".** Minimizar o máximo de arestas no caminho (ou maximizar o mínimo) → Dijkstra com combinação max/min no lugar de soma (funciona porque é monotônica); equivale a caminho na MST (minimax).
- **Dijkstra com custo multiplicativo.** Maximizar produto de probabilidades → Dijkstra com $-\log$, ou direto com max-heap (probabilidades ≤ 1 mantêm a monotonicidade).
- **Dijkstra em grafo implícito.** Estado = (posição, algo) sem listar arestas: gere vizinhos na hora; se o grafo é denso e implícito (ex.: i → j com custo $|a_i - a_j|$), só os vizinhos na ordem por valor importam, ou use segtree/set para pegar o próximo não visitado.
- **Dijkstra com "k arestas grátis" / "k cupons".** Estado (v, j) com j ≤ k → $n \cdot k$ vértices; se o enunciado diz k pequeno é isto; se k grande e custo convexo em k, Aliens.
- **Dijkstra com tempo de espera / janelas.** Chegar mais cedo nunca piora (propriedade FIFO) → Dijkstra por tempo de chegada; sem FIFO, o problema fica NP-difícil em geral.
- **Dijkstra com contagem de caminhos mínimos.** Mantenha cnt[v]; ao relaxar com igualdade some cnt[u]; para "arestas que pertencem a algum caminho mínimo" use $d_s[u] + w + d_t[v] = d(s,t)$.
- **Segundo menor caminho / k-ésimo caminho (walks).** Dijkstra deixando cada vértice ser extraído até k vezes (walks, não simples); para caminhos simples use Yen ($k \cdot n$ Dijkstras) ou Eppstein para walks em $O(m + n \log n + k \log k)$.
- **Caminho mínimo com exatamente k arestas** → matriz min-plus elevada a k ($O(n^3 \log k)$) ou DP por camadas em $O(k \cdot m)$.
- **Caminho mínimo com paridade/cores.** Estado (v, paridade do número de arestas/uma cor) dobra o grafo; "caminho de comprimento ímpar" é BFS em (v, bit).
- **Caminho de mínimo custo que alterna tipos de aresta ou limita trocas** → estado (v, último tipo); se há "no máximo k trocas", camadas.
- **0-1 BFS com custo de "virar".** Grid com setas: custo 0 seguir a seta, 1 mudar → 0-1 BFS; também para "mínimo de arestas para inverter" e "mínimo de mudanças de direção" (estado (célula, direção)).
- **BFS multi-fonte em grid com "tempo de propagação".** Fogo que se espalha + pessoa que foge: BFS do fogo dá t_fogo[v]; pessoa só pode entrar em v se chega estritamente antes. Para o último vértice, relaxe a condição (≤).
- **Bellman-Ford em n passos detecta ciclo negativo.** Se a n-ésima iteração ainda relaxa, há ciclo negativo alcançável; para saber quais vértices têm dist = $-\infty$, propague dos relaxados na última rodada.
- **Ciclo negativo explícito.** Guarde pai ao relaxar; vértice relaxado na rodada n → ande n passos pelos pais para garantir cair dentro do ciclo, depois volte até repetir.
- **SPFA** é só heurística (pior caso $\Theta(nm)$); em problemas com anti-SPFA use Dijkstra com potenciais ou Bellman-Ford camadas.
- **Floyd-Warshall como "fecho" de qualquer semianel.** min-plus (menor caminho), or-and (fecho transitivo), max-min (gargalo), max-× (confiabilidade); a ordem dos laços k externo é essencial.
- **Floyd com adição incremental de vértices.** Adicionar vértice k e atualizar todas as distâncias em $O(n^2)$; "remova vértices numa ordem, soma das distâncias" → inverta e adicione ($O(n^3)$ total).
- **Floyd com ciclo mínimo.** Ao processar k, antes de relaxar pares via k, o menor ciclo cujo maior índice é k é $\min(d[i][j] + w(j,k) + w(k,i))$; gira ciclo mínimo (peso) em $O(n^3)$.
- **Ciclo mínimo em grafo não pesado (girth).** BFS de cada vértice e, para toda aresta não-da-árvore (u,v), candidato $d[u] + d[v] + 1$; $O(nm)$. Girth ≥ 3 e $m > n \cdot \sqrt{n}$ garante ciclo de comprimento ≤ 4 (Kővári–Sós–Turán: sem C4 ⇒ $m = O(n^{3/2})$).
- **Ciclo mínimo em grafo grande esparso com girth pequeno.** Se $m \geq n^{1+1/k}$ há ciclo de comprimento ≤ 2k (Bondy–Simonovits): BFS limitada a profundidade k acha.
- **Ciclo de comprimento ímpar** existe ⇔ não bipartido; ache por BFS: aresta entre dois vértices de mesma paridade de nível fecha ciclo ímpar via LCA na árvore BFS.

- **Ciclo par em grafo com min grau ≥ 3** sempre existe; todo grafo com grau mínimo ≥ k tem caminho de comprimento ≥ k e ciclo de comprimento ≥ k + 1 (DFS: último vértice da trilha maximal tem todos vizinhos na trilha).
- **Base de ciclos (espaço de ciclos GF(2)).** Dimensão m − n + c; cada aresta fora da árvore geradora dá um ciclo fundamental; "xor máximo de ciclo/caminho" → base linear dos xors dos ciclos fundamentais + xor na árvore.
- **Xor mínimo/máximo de passeio s→t em grafo qualquer** = xor da árvore s→t xor algum elemento do span dos ciclos fundamentais.
- **Quantas arestas remover para virar árvore/floresta** = m − n + c; arestas que fecham ciclo no Kruskal são as redundantes.
- **Ciclo único (m = n numa componente)** → pseudofloresta: ache o ciclo com DSU ou removendo folhas por grau; árvores penduradas viram DP.
- **Remover folhas iterativamente (peeling por grau 1)** → acha o ciclo único, o centro da árvore e vértices que não participam de ciclos; com grau ≤ k, k-core.
- **k-core / degenerescência.** Remova repetidamente o vértice de menor grau (fila de buckets, O(m)); a ordem dá orientação onde cada vértice tem ≤ d arestas de saída (d = degenerescência ≤ √(2m)); contagem de triângulos/cliques de tamanho fixo em O(m·d) (contar todas as cliques é #P-difícil).
- **Arboricidade ≤ √(2m)** e m ≤ a·(n − 1): orientar arestas do menor para o maior grau limita grau de saída a √(2m) → triângulos, C4 em O(m√m).
- **Contar triângulos** → oriente cada aresta do menor (grau, id) para o maior; para cada u marque saídas e percorra saídas das saídas: O(m√m); com bitset, $O(n^3/64)$ ou O(n·m/64).
- **Contar ciclos de comprimento 4** → para cada u, para cada v alcançável em 2 passos com ordem (grau, id) maior que u, some C(cnt[v], 2); O(m√m).
- **Contar caminhos de comprimento 2 e ciclos pelo traço** → $tr(A^3)/6$ = triângulos; $tr(A^4)$ contém C4 mais termos de grau; matriz de adjacência elevada a k conta walks de comprimento k.
- **Subgrafo induzido / conjunto independente máximo em grafo pequeno (n ≤ 40)** → meet in the middle ou Bron–Kerbosch com pivô (≈ 3^{n/3}); n ≤ 50 clique máxima → branch and bound com bitset.
- **Coloração de grafos: grau máximo Δ ⇒ guloso usa ≤ Δ + 1 cores; Brooks: Δ cores se é conexo, Δ ≥ 3, e não é clique nem ciclo ímpar; Vizing: arestas em Δ ou Δ + 1; bipartido: arestas em exatamente Δ cores (König).**
- **Coloração de arestas de bipartido em Δ cores** → construtivo: para cada aresta (u,v) escolha cor livre a em u e b em v e troque ao longo do caminho alternado a/b (O(nm)); também por decomposição euleriana em potências de 2.
- **Grafo de intervalos** (vértice = intervalo, aresta = interseção): é cordal e perfeito; cor mínima = clique máxima = máximo de sobreposição (sweep); MIS = guloso por menor fim; cobertura de cliques = guloso.
- **Grafo cordal** (sem ciclo induzido ≥ 4) → ordem de eliminação perfeita por MCS/LexBFS; coloração gulosa na ordem reversa é ótima; clique máxima = max(|vizinhos posteriores| + 1).
- **Grafos de permutação** (aresta i–j se i < j e p_i > p_j): conjunto independente máximo = LIS; clique máxima = subsequência decrescente máxima; cor mínima = LDS (por Dilworth/Mirsky).
- **Dilworth.** Antichain máxima = mínimo de cadeias que cobrem o poset = n − matching máximo no fecho transitivo (cobertura de caminhos com vértices repetíveis); para vértices disjuntos num DAG, n − matching direto nas arestas.
- **Mirsky (dual).** Cadeia mais longa = mínimo de antichains para particionar = nível do vértice no DAG (profundidade máxima); em problema "particione em o mínimo de grupos sem relações" é o comprimento da maior cadeia.
- **Antichain máxima a partir de König.** Dado o matching máximo no grafo bipartido (L, R) do fecho, tome a cobertura mínima C; antichain = vértices i com nem i_L nem i_R em C.
- **Cobertura mínima de caminhos em DAG** (disjuntos em vértices) → n − matching bipartido (out-copy, in-copy); com vértices podendo ser compartilhados, tome fecho transitivo antes.
- **König.** Matching máximo = cobertura mínima de vértices (bipartido); conjunto independente máximo = n − matching. Construção: BFS alternado a partir de L não casados; cobertura = (L não visitados) ∪ (R visitados).
- **Hall.** Existe matching saturando L ⇔ |N(S)| ≥ |S| ∀S ⊆ L; deficiência max_S(|S| − |N(S)|) = |L| − matching máximo. Aparece em "existe distribuição válida?" com restrições de vizinhança.
- **Kuhn com heurística.** Ordem aleatória das arestas, inicialização gulosa e reset de visited só quando o DFS acha caminho aumentante: praticamente O(m√n); para $n \le 10^5$ prefira Hopcroft–Karp O(m√n).
- **Matching bipartido em grafo denso com bitset.** Kuhn onde "próximo vizinho não visitado" usa bitset de não visitados: $O(n^3/64)$.
- **Matching em grafo bipartido com vizinhança = intervalo** (cada L conecta a [l_i, r_i] de R) → guloso: varra R em ordem, heap por menor r_i; também para "emparelhar tarefas/janelas".
- **Matching máximo ponderado em bipartido (atribuição)** → Hungarian $O(n^3)$; "maximize" → negar custos; para custo mínimo com k pares exatos use mincost flow com k augmentações (cada uma é caminho mínimo, custo convexo em k).

- **Casamento estável (Gale–Shapley).** Propostos pelo lado que propõe → ótimo para quem propõe, pior possível para o outro lado; dá solução única do lado dos pedintes mesmo com empates quebrados em ordem fixa.
- **Matching geral (não bipartido)** → Blossom $O(n^3)$ / Edmonds; quando o grafo é uma árvore, guloso de folhas; tamanho do máximo pela fórmula de Tutte–Berge ($n - 2\nu = \max_S(\text{odd}(G - S) - |S|)$).
- **Pareamento de arestas em caminhos de comprimento 2** → um grafo conexo com m arestas pode ser particionado em $\lfloor m/2 \rfloor$ caminhos de comprimento 2 (DFS pós-ordem emparelhando arestas pendentes no vértice); se m é par, tudo.
- **Orientação de arestas com grau de saída/entrada limitado** → fluxo (aresta → vértice com capacidade) ou Hall; "orientar para que todo grau de saída seja par/dado" → Euler após adicionar vértice auxiliar.
- **Orientar grafo 2-aresta-conexo para ficar fortemente conexo** (Robbins): orientação da DFS (árvore para baixo, retorno para cima); ponte ⇒ impossível.
- **Circulação com demandas** → fonte S → v com cap d_v para excedentes, v → T para déficits; viável ⇔ fluxo máximo satura tudo. Limite inferior l na aresta: demanda l, capacidade c − l.
- **Fluxo com limites inferiores, fluxo máximo s→t:** adicione aresta t→s de capacidade ∞, teste viabilidade, depois rode maxflow s→t no resíduo (sem a aresta t→s) e some o fluxo inicial (valor em t→s).
- **Menger.** Máximo de caminhos disjuntos em arestas (vértices) = corte mínimo de arestas (vértices); disjuntos em vértices → divida v em $v_{in} \to v_{out}$ de capacidade 1 para $v \notin \{s, t\}$ (s e t com capacidade ∞).
- **k caminhos disjuntos de custo mínimo** → mincost flow com k unidades (capacidade 1); "ir e voltar" visitando cada ponto uma vez = 2 caminhos disjuntos.
- **Corte mínimo em grid** (pintar célula preto/branco com custo de vizinhos diferentes, pixels, segmentação) → cada célula é vértice, penalidade de vizinhança como arestas bidirecionais; células fixas ligadas a S/T.
- **Seleção com custo de par "se escolher a e b juntos"** (projetos, lucro − custo) → closure: lucro positivo de S, custo para T, dependência com ∞; resposta = soma lucros − corte mínimo.
- **Seleção com bônus de pares (x_i, x_j diferentes custa c)** → corte mínimo onde variável é 0/1 = lado de S/T; custos tipo $c(x_i \neq x_j)$ são submodulares e se resolvem; $c(x_i = x_j)$ em geral não (a menos que bipartido: inverta um lado).
- **Atribuição com custo submodular em 3+ variáveis** → ainda dá corte mínimo se a função for submodular e grafo-representável (decomponível em termos; ex.: "ao menos um"); grafo com vértice auxiliar para cada termo.
- **Máximo closure / densidade máxima (Goldberg).** Subgrafo de densidade máxima $m_S/|S|$ → binária em g + corte mínimo ($m_S - g|S| \geq 0$); arestas viram vértices ou nó por vértice.
- **Fluxo máximo em unidade (capacidades 1)** → Dinic é $O(\min(V^{2/3}, E^{1/2}) \cdot E)$; em bipartido com capacidade 1 é $O(E\sqrt{V})$ (igual a Hopcroft–Karp).
- **Escolha em dois lados com capacidades (ex.: n alunos, m clubes, cada um ≤ k)** → S → alunos (1) → clubes (cap) → T; restrições de "no máximo/ao menos" por grupo dão camadas extras de vértices.
- **Min-cost flow com capacidades pequenas e custo ≥ 0** → SPFA/Dijkstra com potenciais; com fluxo F pequeno, F augmentações; custos negativos sem ciclo negativo: Bellman-Ford inicial.
- **Mincost flow com custo convexo por aresta** → divida a aresta em k cópias de capacidade 1 com custos crescentes (ou cap/custo por segmento linear).
- **Dualidade fluxo/corte para "mínimo de remoção".** Remover o menor número de vértices/arestas para desconectar s de t = corte mínimo; com pesos nos vértices, divida v em in/out.
- **2-SAT: contagem.** SCC dá uma atribuição; "quantas variáveis forçadas" = x forçado a falso se $x \to \neg x$, a verdadeiro se $\neg x \to x$ (ambos ⇒ insatisfatível); "menor lexicográfica" = tente x=0 e propague ($O(n \cdot m)$ ou com SCC tricks); "contar soluções" é #P em geral (só vale para casos especiais).
- **2-SAT com cardinalidade "no máximo uma de k"** → prefix variables: $p_i = x_1 \vee \ldots \vee x_i$, cláusulas $x_i \to p_i$, $p_i \to p_{i+1}$, $p_i \to \neg x_{i+1}$, usando $O(k)$ cláusulas em vez de $O(k^2)$.
- **2-SAT com escolha de 2 opções por item** (pintar/escolher uma de duas posições/horários) e conflitos entre pares → vértice por opção; conflito (a,b) vira $a \to \neg b$ e $b \to \neg a$.
- **2-SAT online/binária**: "maior t tal que as primeiras t restrições são satisfatíveis" (ou o menor t insatisfatível) → binária em t ou adicionar incremental com verificação.
- **SCC: ordem topológica de Tarjan** devolve componentes em ordem topológica reversa; condensação com fontes/sumidouros: mínimo de arestas para tornar fortemente conexo = max(#fontes, #sumidouros) (se há > 1 componente).
- **Alcançabilidade em DAG com $n \approx 5 \cdot 10^4$ e bitset** → fecho transitivo em $O(n \cdot m/64)$ ou, com queries, bitsets em blocos de 64 fontes por vez para caber na memória.
- **Contar pares (u, v) alcançáveis em DAG grande** → estimativa probabilística (min de hashes aleatórios por vértice, k rodadas) ou bitset por blocos; exato é difícil.
- **Dominator tree** (Lengauer–Tarjan / Cooper–Harvey): v é dominado por d se todo caminho s→v passa por d; em DAG, idom(v) = LCA dos predecessores na própria árvore, construindo em ordem topológica.

- **Condensação de componentes por pontes.** Contraia 2-aresta-conexas → árvore; "mínimo de arestas para tornar 2-aresta-conexo" = [folhas/2] (árvore de pontes com ≥ 2 vértices).
- **Block-cut tree.** BCCs são vértices-bloco ligados aos cortes que pertencem a eles; caminho entre u e v passa por articulações da árvore; "u e v separados se remover w" = w é vértice de corte no caminho do block-cut tree.
- **Contar pares desconectados ao remover vértice v** → tamanhos das componentes pendentes na DFS (low[child] ≥ tin[v]) e o resto n − 1 − Σ; pares desconectados = C(n−1, 2) − Σ C(sz_i, 2).
- **Articulação ≠ ponte para multiarestas.** Em ponte com arestas paralelas, compare por id da aresta, não por vértice pai; ao testar articulação, a raiz é articulação só se tiver ≥ 2 filhos na DFS.
- **Pontes dinâmicas (só adições de arestas)** → mantenha a árvore de 2-aresta-conexas com DSU: nova aresta (u,v) contrai o caminho da árvore; amortizado O(n α) + travessia.
- **Euler em não direcionado:** todos graus pares (circuito) ou exatamente 2 ímpares (trilha, comece num); grafo (ignorando isolados) conexo. Para completar com o mínimo de arestas, pareie os ímpares (menor emparelhamento: Chinese postman, via Floyd + emparelhamento perfeito mínimo em ≤ 20 ímpares com bitmask).
- **Euler em direcionado:** in = out em todo vértice (circuito); trilha com um vértice out − in = 1 (início) e um in − out = 1 (fim); conectividade fraca basta quando os graus fecham.
- **Sequência de de Bruijn** B(k, n) = circuito euleriano no grafo de vértices k^{n−1}; é cíclica de comprimento k^n; a string linear que contém todas as strings de comprimento n tem k^n + n − 1 caracteres; "senha" → de Bruijn.
- **Cobertura de arestas por mínimo de trilhas** = max(1, #vértices de grau ímpar / 2) por componente conexa com arestas; "desenhar sem levantar a caneta" = isto.
- **Decomposição de grafo em caminhos de comprimento par/ímpar** → Euler após adicionar vértice auxiliar ligado aos de grau ímpar, depois remova-o para cortar o circuito em trilhas.
- **Hamilton em n ≤ 20** → dp[mask][v] em O(2^n·n²) com bitmask; para contar ciclos hamiltonianos fixe o vértice 0; para n ≤ 25–30 e grafo esparso, meet in the middle.
- **Hamilton em torneio:** todo torneio tem caminho hamiltoniano (inserção por busca binária/merge sort com comparação "u→v"); torneio fortemente conexo tem ciclo hamiltoniano (Camion).
- **Torneio — score sequences (Landau).** Sequência de pontuações s_1 ≤ … ≤ s_n é válida ⇔ Σ_{i≤k} s_i ≥ C(k, 2) ∀k com igualdade em k = n; reconstrução gulosa (maior pontuação ganha dos de menor).
- **Torneio: número de triângulos cíclicos** = C(n,3) − Σ C(w_i, 2) (w_i = vitórias); rei do torneio = vértice com distância ≤ 2 a todos (vértice de maior pontuação serve).
- **Grafo funcional (outdeg = 1).** Componente = 1 ciclo + árvores; detecte ciclo com cores/tortoise–hare; k-ésimo sucessor → binary lifting ou "índice na rota + tamanho do ciclo"; contar quantos entram num ciclo → peeling por grau de entrada.
- **Permutação como grafo funcional** → ciclos disjuntos; "transformar p em q com mínimo de swaps" = n − #ciclos de $p^{-1}q$; p^k mantém ciclos de comprimento L em gcd(L, k) ciclos de L/gcd.
- **Grafos funcionais com pesos** → soma de k passos por binary lifting guardando soma; "primeira repetição / pré-período" = distância até o ciclo + posição.
- **Grafo bipartido por DSU com paridade** → relação "x e y diferentes/iguais" mantida com paridade ao pai; conflito = ciclo ímpar; serve para "adicionar arestas online e dizer se ainda é bipartido".
- **Bipartido completo vs. complemento:** se o complemento é bipartido (ou tem poucas arestas), BFS com conjunto de não visitados; componentes do complemento com m' arestas ausentes em O(n + m').
- **Complemento: clique/independente.** Conjunto independente máximo no complemento = clique; em grafo com m pequeno, o complemento é denso, ordem de grau e Turán dizem que existe clique ou independente grande (Ramsey: n ≥ C(2k, k) garante clique ou independente de tamanho k).
- **Fecho transitivo/BFS com bitset.** "Alcançável em ≤ 2 passos": BFS onde o próximo frontier = OR das linhas dos vértices da fronteira e conjunto de não visitados mascara; O(n^2/64) por camada total.
- **Contar componentes conexas após remover cada aresta/vértice (offline)** → segtree no tempo + DSU rollback (já conhecido); se só queries do tipo "subgrafo induzido por [l, r]" use DSU com ordenação por r e pesos de aresta como tempo de vida (MST por tempo).
- **Subgrafo induzido por índices [l, r] com queries de componentes** → árvore geradora de máxima "l" para cada r (LCT) ou Mo com DSU rollback (Mo com rollback: ordena por bloco de l, r crescente).
- **MST como "caminho minimax".** Menor valor M tal que u e v se conectam usando arestas ≤ M = maior aresta no caminho da MST; árvore de reconstrução de Kruskal (Kruskal reconstruction tree): cada junção cria nó com peso da aresta, então minimax(u,v) = peso do LCA e "componentes alcançáveis com peso ≤ x" = subárvore.
- **Kruskal reconstruction tree + k-ésimo.** Em queries "k-ésimo maior vértice alcançável de v usando arestas ≤ x", suba ao ancestral mais alto com peso ≤ x (binary lifting) e consulte a k-ésima folha da subárvore pelo Euler tour (persistente/wavelet).

- **Segundo MST** → para cada aresta fora da MST, troque pela maior aresta do caminho na MST (max por binary lifting); o menor aumento dá o 2º melhor. "MST sem uma aresta" é outra query: se a aresta está na MST, use a aresta de reposição que cruza o corte; se não está, a MST não muda.
- **MST com aresta forçada/proibida** → forçada: se e ∉ T, custo MST + w(e) − max do caminho (se e ∈ T, o custo não muda); ela pertence a alguma MST ⇔ w(e) = max do caminho na MST; a todas ⇔ é ponte no subgrafo das arestas com peso ≤ w(e).
- **Aresta está em alguma/toda MST** → agrupe arestas por peso; para cada grupo, com os componentes do DSU antes do grupo, a aresta é "em toda MST" se é ponte no multigrafo reduzido e "em alguma" se une componentes distintos.
- **MST único?** Sim ⇔ nenhuma aresta fora da árvore tem peso = maior do caminho. Contar MSTs → matrix-tree por classe de peso (determinante) em cada grupo de peso igual.
- **Matrix-tree (Kirchhoff).** Número de árvores geradoras = det de qualquer cofator da Laplaciana; com pesos, soma dos produtos; arestas "exatamente k" → polinômio em x; Cayley: n^{n−2}.
- **Boruvka** → O(m log n), útil quando as arestas são geradas implicitamente (MST de pontos com custo de par, MST com k "melhor aresta por componente" via estrutura); também serve para MST em O(n log n) com consulta de "vizinho mais próximo fora da componente".
- **MST de pontos no plano** → arestas só entre vizinhos de Delaunay (O(n)); Manhattan: para cada ponto, 4 octantes → vizinho mais próximo em cada (BIT) → 4n arestas candidatas e Kruskal.
- **MST com custo |a_i − a_j| ou a_i xor a_j** → xor: Boruvka + trie binária (ou divida & conquiste em bit alto); |a_i − a_j|: arestas entre vizinhos ordenados.
- **MST com custo a_i + a_j (ou min/max(a_i, a_j))** → ligue todos ao menor a_i (para a_i + a_j) ou analise por ordenação; para gcd(a_i, a_j): arestas via múltiplos com custo d (crivo + Kruskal só entre pares compartilhando divisor).
- **Árvore geradora com grau restrito** → em geral NP-difícil; com restrição em um único vértice (grau exato k) resolve-se via MST por multiplicador (Lagrange/Aliens nas arestas desse vértice).
- **MST com arestas "cor": exatamente k brancas** → Aliens/binária numa penalidade somada às arestas brancas; pela matroid, o custo é convexo em k.
- **Árvore geradora de custo "máximo mínimo dos buracos" / bottleneck** → MST já é ótima para minimizar o maior peso (basta o peso da maior aresta da MST).
- **MST dinâmico offline** (arestas entram/saem, query MST) → D&C no tempo com contração: em cada nível reduza o grafo às arestas que importam (O(k) por nível) → O(m log² n).
- **Grafo de intervalos/arestas implícitas por faixas** → arestas u→[l, r] por segtree/sparse table de vértices; Dijkstra/BFS usando "próximo não visitado" por DSU ou set removendo ao visitar (cada vértice sai uma vez).
- **BFS onde vizinhos são "todos não visitados com condição"** → mantenha set/DSU de não visitados e extraia quem satisfaz; custo amortizado O((n + m) log) mesmo em grafo denso implícito.
- **Planar duality.** Corte mínimo s–t em planar = caminho mínimo no dual (com ciclo/arestas de s a t); "número de faces" = m − n + 1 + c (Euler: V − E + F = 1 + c); em grid, componentes e buracos viram faces (4-vs-8 conectividade complementar).
- **Difference constraints com sistema grande** → grafo de x_j − x_i ≤ c; viável ⇔ sem ciclo negativo; solução mínima/máxima: distâncias de super-fonte; igualdade vira duas desigualdades; "x_i < x_j" com inteiros vira x_i − x_j ≤ −1.
- **Difference constraints + minimizar Σ ou max** → dual é fluxo de custo mínimo (LP dual); max x_n − x_1 = distância mais curta de 1 a n.
- **Contagem de componentes conexas de grafo aleatório/denso** → n ≤ 5000 com m grande: BFS com bitset de não visitados O(n²/64).
- **Grafos com vértice removido e queries "n − 1 vezes"** → ordem inversa + DSU, ou percorra por centroides; para "tudo exceto um" use prefixos/sufixos de estruturas (divide and conquer com rollback).
- **Distância em grafo com poucas arestas extras (m ≤ n + k)** → árvore geradora + ≤ 2(k + 1) vértices especiais (extremos das extras): BFS/Dijkstra de cada especial e distância em árvore por LCA; query = min(d_árvore, min_s d(u,s) + d(s,v)).
- **Grafo quase-árvore com consultas de distância** → idem; ou HLD/Euler para a árvore e 2(k + 1) Dijkstras.
- **Orientar para minimizar o maior grau de saída (pseudoarboricidade)** → binária no limite + fluxo, ou Hakimi; o mínimo possível é ⌈max_S m_S/|S|⌉.
- **Árvores geradoras e "mínimo de arestas para conectar"** → c − 1 arestas ligando componentes; ligar com custo mínimo = MST sobre componentes contraídos.
- **Grafos de "bola" / vizinhança em hipercubo e xor** → vértices = valores, aresta se diferem por 1 bit: BFS com 20·2^20; "vizinhos a distância de Hamming ≤ d" → BFS multi-fonte sobre máscaras.
- **Caminho com "no máximo k vezes" um tipo de aresta** → mais eficiente: BFS em camadas por k (O(k(n + m))); para k ≈ log, estado (v, j) cabe.
- **Grafo de duas camadas (ônibus/linhas)** → vértices de linha e de parada, entrada com custo de embarque, saída custo 0; reduz arestas de O(L²) a O(L).

- **Teorema de Ore/Dirac para Hamilton** → grau mínimo ≥ n/2 garante ciclo hamiltoniano; construtivos com "grafo denso" costumam ter isto: a construção é por inserção de vértices e rotação de caminho (Pósa).

Repetidas de outras seções:

- **Jogo em grafo com empate** → BFS retrógrada (teoria dos jogos).
- **Strings que viram umas nas outras por operação** → vértices = strings, BFS; operação reversível → componentes.
- **Operação reversível** → componentes conexas = classes de equivalência (construtivos).
- **Euler com vértice fantasma** orienta arestas com |in − out| ≤ 1 e constrói de Bruijn.
- **Kruskal reconstruction tree** responde "maior aresta mínima no caminho" e "primeiro instante de conexão".
- **Coloração de arestas de bipartido** com Δ cores; K_n em n − 1 rodadas (n par) pelo método do círculo.
- **BEST** → circuitos eulerianos dirigidos = t_w·∏(deg−1)!.
- **Graphical sequence** → Erdős–Gallai/Havel–Hakimi; torneios: Landau.
- **Torneio: 3-ciclos** → C(n,3) − Σ C(out_v, 2).
- **Número de 4-ciclos** → Σ_{u<w} C(cnt(u,w), 2)/2 com vizinhos comuns.
- **Ciclo base (xor de ciclos)** → px pela árvore geradora; cada aresta fora vira um ciclo; base linear dá max/min xor de qualquer passeio; dimensão m − n + c.
- **Sistema de paridades em arestas (T-join)** → GF(2); em árvore a solução é única, de baixo para cima.

Árvores

Primeira pergunta: a resposta depende da raiz? Se sim, pense em rerooting ou centroide.

- **Enraíze** em 1 (ou numa folha, ou no centroide, ou no centro do diâmetro) e faça DP de subárvore.
- **Para toda raiz** → rerooting (prefixo/sufixo dos filhos se a operação não é invertível).
- **Todos os pares de caminhos** → contribuição por aresta: sz·(n − sz); ou centroide.
- **Caminhos com condição (comprimento k, soma ≤ x)** → centroid decomposition; ou small-to-large de maps.
- **Diâmetro** → duas BFS (sem pesos) ou dois DFS/Dijkstra (pesos ≥ 0); distância máxima de cada vértice = até uma das pontas do diâmetro.
- **Queries de caminho** → LCA (binary lifting/Euler + sparse); com updates → HLD + segtree.
- **Queries de subárvore** → Euler tour (tin/tout) vira intervalo de array.
- **Contar cores/valores distintos em subárvore** → small-to-large (DSU on tree / sack).
- **Folhas e graus:** número de folhas, vértices de grau ≥ 3, cadeias de grau 2 costumam ser o que importa.
- **Remover vértice/aresta** → componentes = subárvores dos filhos + o resto (n − sz).
- **Pesado/leve:** caminho raiz-folha cruza ≤ log arestas leves; ≤ 2√n vértices com grau ≥ √n.
- **Paridade de profundidade** → árvore é bipartida; jogos em árvore costumam depender disso.
- **Virtual tree** quando cada query toca k vértices e Σk é pequeno.
- **Floresta/grafo quase árvore** → ache o ciclo único (m = n) e trate as árvores penduradas.

adicionado em 2026-11-10 por agentes; revisado

- **Euler tour de aresta (2n − 2 posições) ou de vértice (2n − 1); eventos tin/tout = 2n.** Registre o vértice na entrada e saída; subárvore = intervalo; caminho u–v em Mo ou em BIT de ±: guarde +x em tin e −x em tout para "soma de caminho raiz–v".
- **LCA com sparse table sobre ordem de DFS (n posições, sem 2n).** Para u ≠ v com tin[u] < tin[v], LCA = pai do vértice de menor profundidade em (tin[u], tin[v]]; meia memória e O(1).
- **LCA offline (Tarjan)** → DFS + DSU: ao sair de v, una v ao pai; para query (u, v) com u já visitado, LCA = find(u). Útil quando memória é restrita.
- **LCA em árvores dinâmicas (adicionar folha online)** → binary lifting preenchido na criação (O(log) por folha) ou skew-binary jump pointers (O(1) memória extra por vértice, O(log) por query).
- **k-ésimo ancestral** → binary lifting O(log); O(1) com ladder decomposition + binary lifting (long-path); offline: DFS com pilha de caminho atual (ancestral à profundidade d − k = stack[d − k]).
- **Distância em árvore** = d[u] + d[v] − 2·d[lca]; **k-ésimo vértice no caminho u–v** = sobe de u se k ≤ d[u] − d[lca], senão sobe de v o complemento.
- **Soma/xor/qualquer grupo abeliano em caminho** → prefixo da raiz (valores em vértices): f(u) + f(v) − 2·f(lca) + a[lca]; para xor, f(lca) aparece duas vezes e cancela: pre[u] ⊕ pre[v] ⊕ a[lca]. Com valores em arestas não há o termo a[lca].
- **Max/min em caminho sem updates** → binary lifting guardando (pai 2^k, max); ou HLD + sparse table nas cadeias; ou Kruskal reconstruction tree quando o peso está nas arestas e query é minimax.

- **Árvore com updates em aresta** → coloque o peso da aresta no filho; update em subárvore/caminho como Euler tour/HLD.
- **HLD com subárvore também por segtree** → numere filho pesado primeiro; subárvore = [pos[v], pos[v] + sz[v] − 1], cadeia = intervalo contíguo; ambos funcionam no mesmo array.
- **HLD: query não comutativa** → mantenha lado esquerdo e direito separados (u→lca e v→lca invertido) e combine na ordem do caminho.
- **HLD offline com 2 Fenwicks para "adicione x no caminho, some caminho"** → diferenças no array HLD; por contribuição do vértice × tamanho: Σ_{v em caminho} a_v = prefixos da raiz.
- **Link-cut tree** quando a árvore muda (link/cut) e ainda há queries de caminho; `access`/`makeroot`/`splay` → O(log n) amortizado; também mantém MST incremental (só inserções; MST totalmente dinâmico exige D&C/ETT) com arestas ordenadas por peso (max no caminho), "o grafo é floresta?", e subárvore virtual (info de subárvore).
- **Top tree / Euler tour tree** → estruturas para subárvore + dinâmico; ETT (árvore de Euler em balanced BST) deixa link/cut e consulta de componente/soma em O(log n); top tree é o caso geral (diâmetro dinâmico, centro dinâmico).
- **Diâmetro: propriedades.** Todos os diâmetros se cruzam no centro (1 ou 2 vértices); a distância máxima de v = max(d(v, a), d(v, b)) com (a, b) diâmetro; ao unir duas árvores por uma aresta (u, v), o novo diâmetro tem pontas entre os 4 candidatos {a1, b1, a2, b2}; vale também para união de conjuntos (distância máxima mergeable).
- **Two-sweep não vale com pesos negativos** (use DP); com pesos ≥ 0, dois DFS/Dijkstra valem (BFS só sem pesos); para "diâmetro após remover vértice", use prefixo/sufixo ou rerooting guardando os 2–3 maiores filhos.
- **Raio = ⌈diâmetro/2⌉** (só com arestas unitárias; com pesos, raio ≥ diam/2 e não há fórmula fechada); centro minimiza a excentricidade; ao adicionar uma folha por aresta de peso w, o diâmetro cresce no máximo w e atualiza-se comparando distâncias às pontas atuais.
- **Distâncias a k (todos os vértices com dist exata k)** → rerooting guardando contagens por profundidade; para k fixo, DP com dp[v][d] só até d ≤ k: O(nk) tempo/memória (O(n) só com long-path decomposition ou k constante).
- **dp[v][d] "profundidade" com merge O(n)** → long-path decomposition: herde o vetor do filho de maior altura e some os outros elementos em O(altura); resolve "contar vértices na subárvore com dist exata d" e "profundidade mais comum" em O(n).
- **Small-to-large por cadeia pesada (DSU on tree/sack)**: mantenha a estrutura do filho pesado, adicione todos os vértices das subárvores leves, limpe só nos leves; O(n log n) com contadores em vetor.
- **Small-to-large com sets e queries "k-ésimo na subárvore" ou "primeiro ≥ x"** → merge de sets/mapas; ou merge de segtrees dinâmicas (O(n log n) total) com consulta na raiz.
- **Merge de segtree dinâmica** para "para cada vértice, o mais frequente / soma de conjunto" em subárvore: cada operação cria O(log) nós e o merge custa O(total de nós) amortizado.
- **Centroide: propriedades.** Todo componente ao remover o centroide tem ≤ n/2 vértices; existem 1 ou 2 centroides (2 ⇔ aresta que divide n/2 e n/2); o centroide minimiza Σ dist (mediana da árvore): ele é a "mediana ponderada".
- **Árvore de centroides (centroid tree)** → altura ≤ log n; para query de distância ao vértice marcado mais próximo, guarde best[c] e, para cada ancestral c do vértice v, dist(v, c) + best[c]; pré-calcule dist(v, c) por nível (O(n log n) memória).
- **Centroid tree com soma de distâncias de v a todos os marcados** → em cada centroide c guarde (cnt, sum de dist ao c) e (cnt, sum de dist ao pai de c na centroid tree) para excluir o subcentroide; query em O(log n).
- **Centroid tree: caminho u–v passa por LCA na centroid tree** (o ancestral comum mais baixo) → distância(u, v) = dist(u, c) + dist(v, c) para c = LCA centroide (exato: c sempre está no caminho u–v).
- **Contar caminhos com soma de peso ≤ K ou comprimento = k via centroide** → para cada centroide, ordene as distâncias e use two pointers; subtraia o par da mesma subárvore filha (ou faça por subárvore com BIT). Com comprimento exato k e k pequeno, use vetor de contagem por profundidade e processe filhos do menor para o maior.
- **Contar caminhos com condição "produto/mod"** → centroide + hash map: guarde o valor prefixo de v até c e o sufixo para combinar (inverso modular) em O(n log n).
- **Contribuição por aresta** → número de caminhos através da aresta = sz·(n − sz); soma de distâncias entre todos os pares = Σ_e w_e·sz_e·(n − sz_e); valor esperado de distância em árvore aleatória idem.
- **Caminhos que passam por um conjunto de vértices (queries com k vértices)** → árvore virtual (auxiliary tree) ordenando por tin, adicionando LCAs de adjacentes, ligando por pilha; Σk ≤ $2 \cdot 10^5$ → O(Σk log).
- **Virtual tree + DP** → dp[v] só sobre vértices relevantes, com arestas ponderadas pela distância original; tipicamente "mínimo de cortes/custo para separar os especiais da raiz": defina dp[v] = custo de separar v do pai; se v é especial é obrigatório cortar acima dele (dp[v] = menor peso no caminho pai→v, não min com os filhos); senão dp[v] = min(menor peso no caminho pai→v, Σ dp filhos).
- **Rerooting com monoide não invertível** → prefixo/sufixo de filhos; com invertível, "total − contribuição"; para max, guarde os dois melhores filhos; para min/máx de duas coisas, combine por top-2.
- **Rerooting em árvore com estado "removi uma aresta"** → para cada aresta, resultado da componente de cima e de baixo: down[v] e up[v] e depois combine; para "melhor aresta para cortar" itere e combine as duas metades.

- **Árvore → intervalos.** Ordem de DFS de subárvores gera intervalos aninhados (laminar); duas subárvores ou são disjuntas ou uma contém a outra, então qualquer família laminar de intervalos é uma árvore (use para contar/conectar com "pilha de intervalos").
- **Família laminar de conjuntos** → árvore de inclusão (pilha após ordenar por início e fim decrescente); contar pares de subconjunto = DP na árvore.
- **Árvore cartesiana + small-to-large** → "para cada elemento como máximo, percorra o lado menor": total O(n log n) (usado em contar subarrays com condição em max e a resposta se parte).
- **Árvore de Huffman / merge de menores** → heap: junte os dois menores; custo total = Σ pesos × profundidade; com pesos já ordenados faça em O(n) com duas filas. Variantes com "k-ária": adicione 0s para (n − 1) mod (k − 1) = 0.
- **Árvore binária de busca com inserção aleatória** → altura esperada O(log n); treap: prioridade aleatória faz a forma ser a mesma de uma árvore cartesiana por prioridade, e alturas são independentes da ordem de inserção.
- **Contar árvores com restrição de grau** → fórmula multinomial de Prüfer + GF; para "árvores com a aresta (u,v) obrigatória": 2·n^{n−3}.
- **Hash de árvore enraizada (AHU).** Canônico = id do multiset ordenado de ids dos filhos (map global de vetores → id); O(n log n); isomorfismo de árvores sem raiz → compare os hashes enraizados nos 1–2 centros; ou hash aleatório f(v) = 1 + Σ f(filho)·rnd[altura] (hash por altura).
- **Subárvores isomórficas dentro da mesma árvore** → mesmo id canônico; contar classes de simetria de uma árvore = ids distintos; número de automorfismos = ∏ (grupos de filhos isomorfos)! · ∏ automorfismos dos filhos.
- **DP em árvore: dp[v][0/1]** → cobertura de vértices mínima, conjunto independente máximo (soma ponderada), dominating set mínimo (3 estados: escolhido, dominado por filho, ainda não dominado); em árvore também é guloso: folhas pelo pai.
- **Matching máximo em árvore** → guloso de baixo para cima: se filho livre e v livre, case (v, filho); equivale a König (cobertura mínima de vértices = matching máximo em bipartido/árvore).
- **Mochila em árvore (tree knapsack).** dp[v][j] com soma de tamanhos limitando j: merge O(sz_a·sz_b) soma total O(n·K) se j ≤ K (truncando em K); para k exato de "k vértices conectados", dp[v][k] com ordem DFS (o "dp de ordem de DFS": pule a subárvore inteira em O(1) por tin).
- **Tree knapsack com DFS order** → dp[i][j] sobre ordem de DFS: escolher o vértice i (vai para i + 1) ou pular sua subárvore (vai para i + sz): O(n·K) sem merge de vetores.
- **Merge de dp côncavas em árvore** → dp[v][j] côncava em j (ex.: escolher k vértices com custo) → guarde a lista de incrementos e junte por heap/merge small-to-large; "corte k arestas / selecione k caminhos disjuntos" caem em Aliens ou slope trick.
- **Cobrir árvore com k caminhos máximo de vértices distintos** → guloso por cadeias de altura longa (long-path decomposition): ordene comprimentos das cadeias e some as k maiores.
- **Escolher k folhas para maximizar união de caminhos até a raiz** → mesma long-path decomposition: as k cadeias mais longas (peso nas arestas), por exchange argument.
- **Dominating set de raio r em árvore** → guloso: processe de baixo para cima, o vértice mais profundo não coberto escolhe seu ancestral a r níveis acima; O(n) com dp (dist ao mais perto coberto/não coberto).
- **Facility location (p-mediana/p-centro) em árvore** → p-centro: binária no raio + guloso de baixo para cima; p-mediana: DP $O(n^2p)$ ou O(np) com Aliens.
- **Cortar a árvore em componentes com condição (tamanho ≥ k, soma ≤ S)** → guloso pós-ordem: acumule sz, se ≥ k corte a aresta e zere; para soma máxima por componente + "minimizar # de cortes", binária no limite e guloso (combine os menores filhos primeiro).
- **Dividir a árvore em componentes de tamanho exatamente k/igual** → só cortar arestas com sz múltiplo de k; isso é necessário, não suficiente (ex.: estrela K1,5, n=6, k=2). Se a partição existe (k | n), os cortes são exatamente os v ≠ raiz com sz_v ≡ 0 mod k, e há n/k − 1 cortes.
- **Remover aresta e comparar partes** → sz_v e n − sz_v; somas de xor/soma de subárvore equilibram: "remover duas arestas para xors iguais" → procure três partes com xor igual: se total ≠ 0, ou dois vértices incomparáveis com xor = total, ou um ancestral com xor 0 e um descendente com xor = total; se total = 0, dois vértices não-raiz com xor 0.
- **Ordenação topológica de árvore (ordem pai antes do filho) contada** → n!/∏ sz_v; "heap ordenado" idem; número de ordens de DFS = ∏ (filhos)! ; "permutações onde cada elemento aparece depois do pai" = n!/∏ sz.
- **Ordem de DFS livre (escolha da ordem dos filhos).** Dá intervalos de subárvores contíguos; para maximizar/minimizar algo (soma |pos_u − pos_v|, tempo de percurso) ordene filhos por uma chave com exchange argument (ex.: sz/custo).
- **Passeio de Euler e distância percorrida** → andar visitando todos e voltar = 2·Σ w; sem voltar = 2·Σ w − maior distância (do começo ao fim, diâmetro/distância do ponto de partida); ao visitar k vértices especiais, só vale a árvore virtual.
- **Árvore de pontes (2-edge-connected components)** → compressão por DSU nas arestas não-ponte; queries "adicione aresta (u, v): quantas pontes restam" = contrai caminho; "mínimo de arestas para 2-aresta-conexo" = ⌈L/2⌉.

- **DSU com peso (potencial) em árvore/grafo** → guarde diferença/xor/soma ao pai com compressão: relações "a_u − a_v = w" consistentes; contradição = ciclo; também resolve "pesos entre componentes" online em α(n).
- **DSU com merge de listas pequenas e queries "quem está no mesmo componente em tempo t"** → small-to-large guardando tempos; ou DSU sem compressão (por rank) + tempo de união: caminho até a raiz tem o máximo/minimax de tempos (árvore de Kruskal implícita).
- **Árvore de tempo de união (DSU sem compressão, união por tamanho)** → altura O(log n); para cada aresta guarde o tempo; "u e v conectados no tempo t?" = max dos tempos no caminho na árvore DSU ≤ t.
- **Queries offline em árvore: "quantos vértices na subárvore de v com valor ≤ x"** → Euler tour + BIT offline ordenado por valor; para "k-ésimo" persistente/wavelet; com updates em vértices → BIT de BIT/CDQ.
- **Queries "v está na subárvore de u?"** → tin[u] ≤ tin[v] < tout[u] em O(1); "u é ancestral de v em árvore dinâmica" idem com ETT.
- **Árvore enraizada e "reduzir o conjunto de vértices a ancestrais minimais/maximais"** → ordene por tin e use pilha; "remova vértices que têm ancestral no conjunto" → varra por tin mantendo o último intervalo ativo.
- **Conjunto máximo de caminhos disjuntos em vértices com LCAs** → ordene por profundidade do LCA decrescente; aceite se nenhum vértice está marcado (BIT em Euler tour: "soma na subárvore de u" ou "marca no caminho"); um caminho com LCA mais fundo "bloqueia" menos.
- **Cobertura de arestas da árvore por caminhos dados** (escolher o mínimo de caminhos para cobrir) → guloso: pegue a aresta mais funda não coberta e escolha o caminho cobrindo-a que sobe mais alto; DSU de "pulo" para cobrir.
- **Contar árvores rotuladas/enraizadas** → Cayley: n^{n−2} sem raiz, n^{n−1} enraizada, florestas enraizadas (n + 1)^{n−1}; árvores binárias não rotuladas = Catalan; ordenadas com n nós = Catalan(n − 1).
- **Número de subárvores conexas (subconjuntos conexos) de uma árvore** → dp[v] = ∏ (1 + dp[filho]) e soma sobre v; para contar por tamanho k, tree knapsack.
- **Probabilidade/esperança em árvore** → linearidade por vértice/aresta: E[tamanho do componente] = Σ P(u conectado a v); "sorteio de ordem aleatória" = ancestrais em ordem de remoção (v é "raiz" com prob. 1/(depth+1)): E[altura de treap/depth de nó] ≈ 2 ln n.
- **Subárvore com tamanho ≥ n/2 única, "heavy"** → o centroide é o vértice de menor profundidade cujo filho maior tem sz ≤ n/2 — desça sempre pelo filho com sz > n/2.
- **Aresta de custo c e "pintar caminhos"** → coloração de caminho em árvore offline: "último a pintar" → processe em ordem inversa com DSU pulando arestas já pintadas (O(n α)).
- **Atualizar caminho + ler ponto, sem HLD** → cada update (u, v, x): diff[u] += x, diff[v] += x, diff[lca] −= x, diff[pai(lca)] −= x; final: soma de subárvore = valor do vértice. Para arestas, use só diff[lca] −= 2x.
- **Quantos caminhos cobrem cada aresta/vértice (offline)** → mesma diferença em árvore com soma de subárvore; "arestas cobertas por exatamente/pelo menos k caminhos" = após somar.
- **Mediana de árvore (vértice que minimiza Σ dist)** → centroide; **centro** minimiza max dist (meio do diâmetro). Se pesos nos vértices, centroide ponderado.
- **Árvore com "todo vértice dentro de dist d de um escolhido"** → p-centro com raio dado = guloso de baixo para cima; com k escolhidos e minimizar o raio → binária no raio.
- **Soma de distâncias de todos a todos por vértice (Σ_u d(v, u))** → rerooting: res[child] = res[parent] + n − 2·sz[child] (arestas de peso 1); com pesos: res[child] = res[parent] + w·(n − 2sz).
- **Árvore com valores e "ganhos de remoção" (somas máximas de componentes conectados)** → dp[v] = a_v + Σ max(0, dp[filho]); resposta max_v dp[v]; rerooting para "máximo componente contendo v".
- **Árvore ordenada ↔ binária (filho esquerdo, irmão direito)** → bijeção; reduz queries de ordem entre irmãos e "DP sobre lista de filhos" a DP binária; também "árvores com n nós" ↔ Catalan.
- **Árvore de segmentos implícita sobre os filhos** → vértices com grau enorme e queries por intervalo de filhos: crie "árvore de prefixos" dos filhos (cadeia auxiliar) para reduzir arestas de $O(deg^2)$ para O(deg) em grafos derivados.
- **Árvore geradora como estrutura "ancestral–descendente" (DFS tree).** Em grafo não direcionado, toda aresta fora da árvore DFS liga ancestral–descendente (sem cross edges): usado para pontes, ciclos (cada aresta de retorno fecha um ciclo fundamental), caminhos longos (≥ k se grau ≥ k) e "cobrir com √n caminhos/ciclos".
- **DFS tree e coloração/independent set grande** → ou há caminho com ≥ √n vértices (profundidade da DFS), ou alguma camada grande (independência por níveis): "ache caminho de tamanho ≥ √n ou conjunto independente de tamanho ≥ √n" (par de níveis).
- **BFS tree e dual:** em BFS cada aresta liga níveis consecutivos ou iguais; ciclo ímpar ⇔ aresta entre vértices do mesmo nível; ciclo mais curto através de v = min sobre arestas (u, w) com lca(u, w) = v de d(u) + d(w) + 1 − 2·d(v) (d(u) + d(w) + 1 sozinho é passeio fechado).
- **Árvore e 2-coloração por profundidade** → qualquer árvore é bipartida; grafo = árvore + arestas extras: cada aresta extra decide a paridade; caminhos de comprimento par/ímpar entre u e v dependem de uma aresta extra de ciclo ímpar.

- **Árvores de menor caminho (SPT) e restrições adicionais** → "remova uma aresta e recalcule distâncias": usar a árvore de caminhos mínimos de s, e para a aresta (u, v) da SPT usar candidatos de arestas fora da árvore (substituição, "replacement path"); método clássico: Dijkstra com "lowest cross edge" ou LCT + sweep por distância.
- **Replacement path em árvore geradora mínima (substituir aresta removida)** → varra arestas fora da árvore em ordem crescente de peso e pinte o caminho ainda não pintado com DSU "pular caminhos pintados": cada aresta da MST recebe a menor aresta fora que a cobre.
- **Árvore binária cheia/heap implícito** → filhos 2i, 2i + 1; LCA(u, v) = desloque o maior até igualar; profundidade = $\lfloor \log_2 i \rfloor$; subárvore em nível d é intervalo [i·2^k, (i + 1)·2^k).
- **Trie binária como árvore de altura 30** → mesmas técnicas: DP em subárvore, small-to-large de tries (merge de tries é O(total de nós)), xor máximo = descer pelo bit oposto, e "mínimo xor MST" por D&C nos bits.
- **Árvore de sufixos / links** → árvore de links do SAM e de falhas do Aho-Corasick: operações de "subárvore = todas as strings que têm este sufixo/padrão" são Euler tour; propagar contagens por DP de baixo para cima.
- **Binary lifting guardando "primeiro ancestral com propriedade"** → monotonia por caminho (propriedade só piora/melhora subindo) → busca binária por lifting em O(log); cabe em "número de saltos para chegar à raiz" (greedy pulos).
- **Pulos gulosos em árvore** (de v sobe até o ancestral mais alto com custo ≤ L, repete) → defina próximo[v] = esse ancestral e conte saltos com binary lifting sobre próximo (grafo funcional/árvore de saltos); mesma ideia de "sparse table de saltos" em array.

Repetidas de outras seções:

- **Jogo em árvore** → paridade da profundidade, centro, folhas; Green Hackenbush (teoria dos jogos).
- **Trie é uma árvore** → LCP de duas strings = profundidade do LCA na trie (strings).
- **Árvore de sufixos / links do SAM** formam árvore → DP de subárvore conta ocorrências (strings).
- **Prüfer / Cayley** → graus e nós rotulados: (n−2)!/∏(d_i−1)!; ligar k componentes: n^{k−2}∏ s_i.
- **Rotulações pai < filho** → n!/∏ sz(v).
- **Hitting time numa árvore (passeio aleatório)** → E[u → v adjacente] = 2·|E(lado de u)| + 1.
- **Isomorfismo de árvores** → AHU (hash do multiconjunto ordenado dos filhos); sem raiz: centroide(s).
- **Hash de árvore enraizada** → H(v) = 1 + Σ f(H(filho)) com f pseudoaleatório (ou AHU com mapa de multiconjuntos); sem raiz, use os 1–2 centroides.
- **Árvore ↔ parênteses (Dyck)** → subárvore = intervalo balanceado; LCA = mínimo do balanço; contar árvores = Catalan.
- **Xor de caminho** → px[u]⊕px[v] (arestas) ou ⊕a[lca] (vértices).

Estruturas de dados e queries

Perguntas: dá para responder offline? Dá para ordenar as queries? A operação é associativa?

- **Offline + ordenar** queries por r (ou por valor) e varrer com BIT/segtree: resolve "distintos em [l, r]", "elementos ≤ x em [l, r]".
- **Mo** quando add/remove é O(1) e não há updates; com updates → Mo com tempo (n^(5/3)).
- **Segtree:** pense em qual informação do nó permite juntar dois filhos (max prefixo, max sufixo, soma, resposta). Esse é o problema inteiro.
- **Lazy** para update em intervalo; **segtree beats** para chmin/chmax; **persistente** para versões ou "k-ésimo em [l, r]".
- **a_i ← a_i mod x, a_i ← √a_i, a_i ← gcd** → cada elemento muda poucas vezes; pule os que não mudam. A poda por max no nó vale para mod/√; para gcd guarde o lcm do nó e pule quando lcm | x (senão O(nq)).
- **Sparse table** para RMQ estático em O(1).
- **Set de intervalos** (Chtholly / ODT) para "atribua valor em [l, r]".
- **Sqrt decomposition** quando a operação não é associativa ou é estranha; blocos de √n e reconstrução periódica.
- **Pesado/leve por frequência:** valores que aparecem > √n vezes são poucos → trate separado.
- **CDQ / D&C no tempo** para dominância 3D ou queries com updates offline.
- **Hash de multiconjunto** (soma de valores aleatórios) para "[l, r] é permutação" ou "conjuntos iguais".
- **DSU com informação extra** (paridade, tamanho, distância ao pai) para relações entre pares.

Combinações de estruturas (uma transformação + uma estrutura):

Problema	Transformação	Estrutura
Update/query em subárvore	Euler tour (tin, tout) → intervalo	Segtree / BIT (lazy se update em intervalo)
Update em vértice, query no caminho raiz–v	Euler tour com +x em tin[u] e −x em tout[u] + 1	BIT de prefixo

Problema	Transformação	Estrutura
Update no caminho raiz–v, query em vértice	Ponto em v, query = soma na subárvore	Euler tour + BIT
Query em caminho u–v com updates	HLD → O(log) intervalos	Segtree (lazy)
Query em caminho sem updates, operação invertível	Prefixo da raiz: f(u) + f(v) − 2f(lca) em arestas; em vértices, − f(lca) − f(pai do lca)	LCA
Query em caminho, min/max estático	Binary lifting guardando o agregado	Sparse por saltos
LCA em O(1)	Euler tour completo (2n − 1)	Sparse table de mínimo
Distintos em [l, r]	Ordene queries por r, guarde última ocorrência	BIT
k-ésimo menor em [l, r]	Compressão de coordenadas, versões por prefixo	Segtree persistente / wavelet tree
Contar a_i ≤ x em [l, r]	Offline por x, ou merge sort tree	BIT / merge sort tree
Distintos ou cores em subárvore	Euler tour vira "distintos em intervalo"	BIT offline, ou small-to-large
Queries em caminho de árvore, add/remove O(1)	Euler tour de 2n posições	Mo em árvore
Dominância 2D (i < j, a_i < a_j)	Ordene por uma coordenada	BIT na outra
Dominância 3D	D&C na 1ª, ordene a 2ª	CDQ + BIT
Retângulos / pontos em retângulo	Sweep em x, eventos	Segtree em y (área: contagem de cobertura)
Conectividade com remoção de arestas	Intervalo de vida de cada aresta	Segtree no tempo + DSU com rollback
Conectividade só com remoções	Inverta o tempo	DSU
Max xor de a_i com x em [l, r]	Prefixo de versões	Trie persistente
Ocorrências de padrão em substring s[l..r]	Suffix array → intervalo de sufixos	Segtree persistente / BIT offline
Ocorrências na subárvore do SAM/trie	Euler tour na árvore de links	BIT / small-to-large de sets
DP com transição min em intervalo de j	Índices de j como array	Segtree / deque monotônico
DP com transição sobre valores (LIS e variantes)	Indexe pelo valor comprimido	BIT de max
Distância em árvore para "vértice marcado mais próximo"	Centroid: dist(v, c) por ancestral	Centroid decomposition + min por centroide
Palíndromo/igualdade de substring com updates	Hash polinomial como soma	Segtree / BIT de hash

adicionado em 2026-11-10 por agentes; revisado

- **BIT range-update range-query.** Mantenha duas BITs B1, B2 sobre a diferença d: prefixo(x) = B1(x)·x − B2(x), com add(l, v): B1 += v em l, B2 += v·(l−1); add(r+1, −v) analogamente. Soma em [l, r] = prefixo(r) − prefixo(l−1).
- **BIT 2D e BIT de dimensão alta.** Update/query em O(log^d); memória n^d. Com coordenadas esparsas, comprima por offline (BIT de vetores ordenados) em vez de alocar n^2.
- **Busca binária na BIT.** Para "menor i com prefixo ≥ k" (k-ésimo de multiconjunto, valores ≥ 0) suba pelas potências de 2 de cima para baixo: O(log n), sem $\log^2$ da binária com query.
- **BIT só serve a grupos (operação invertível) para query de intervalo.** Para max/min use BIT apenas se os updates só aumentam (max) e a query é de prefixo/sufixo; senão segtree.
- **BIT com construção em O(n).** Comece com t[i] = a[i] e, para i crescente, faça t[i + lowbit(i)] += t[i]; evita n·log n na leitura de arrays grandes.
- **Segtree não comutativa.** Combine sempre esquerda·direita na ordem; em segtree iterativa (bottom-up) mantenha resl e resr separados. Exemplo: composição de funções afins ax+b, multiplicação de matrizes, concatenação de strings/hashes.
- **Nó como monóide de funções.** Se o update é "aplicar f em [l, r]" e a query é "soma", o lazy guarda a composição de f e o nó guarda (soma, tamanho) para saber aplicar f em O(1). Verifique que f∘g também pertence à classe (afim ∘ afim = afim).
- **Segtree de "autômato".** Cada nó guarda a função estado→estado para o bloco (tabela de k entradas, k pequeno); query = compor as tabelas. Serve para "processar string/sequência pelo autômato com updates pontuais" (ex.: contar subsequência "abc", parênteses com k tipos de estados).

- **Segtree de parênteses.** Nó guarda (soma, mínimo prefixo); sequência é balanceada ⇔ soma = 0 e mínimo ≥ 0. Merge: soma = a + b, min = min(a.min, a.soma + b.min). Mesmo nó resolve "primeiro prefixo negativo" descendo na árvore.
- **Segtree: máximo subarray.** Nó = (soma, melhor prefixo, melhor sufixo, melhor). Base de dezenas de variantes; com update de atribuição/negação guarde também o par com sinal invertido (max e min) para o lazy de "negar".
- **Segtree beats (chmin/chmax).** Guarde max, segundo max e contagem do max; chmin(x) só toca nós onde segundo max < x < max. Amortizado O(log n) por op (O($\log^2 n$) se há add em intervalo).
- **Segtree com descida em vez de binária+query.** "Primeiro i ≥ l com a_i ≥ x", "k-ésimo 1", "primeiro prefixo ≤ 0" → desça guardando max/soma no nó: O(log n) em vez de O($\log^2 n$).
- **Merge sort tree.** Cada nó guarda o intervalo ordenado; "contar < x em [l, r]" em O($\log^2 n$), com fractional cascading em O(log n). Não suporta update (use BIT de BITs/ordered set por nó se precisar).
- **Wavelet tree / wavelet matrix.** k-ésimo, contar < x, contar valores em [lo, hi] em [l, r], tudo em O(log σ) estático e com memória n log σ; mais leve que segtree persistente. Também dá quantil e "valor mais próximo de x" no intervalo.
- **Segtree persistente = versões por prefixo.** Query em [l, r] = (versão r) − (versão l−1) quando o nó é aditivo (contagens). Também resolve "k-ésimo em [l, r]", "contar distintos" (com a versão guardando a última ocorrência) e soma em retângulo online.
- **Persistência com lazy.** Não propague tags (marcação permanente / lazy sem push-down): o nó guarda a tag e a query soma as tags dos ancestrais; assim não precisa copiar filhos na leitura.
- **Segtree dinâmica (esparsa).** Domínio até 10^9 ou 10^{18} → crie nós sob demanda; memória O(q log U). Útil para DP com CHT/Li Chao sobre x grande e para BIT-de-segtree online em 2D.
- **Segtree de segtrees (2D online).** Update de ponto e query de retângulo com a camada externa em x e interna dinâmica em y: O($\log^2$) por op, memória O(q $\log^2$). Para queries só de max sem update de decremento, a camada interna pode ser BIT.
- **Segtree 2D com lazy é doloroso.** Prefira offline: sweep em x + segtree 1D em y, ou transforme retângulo em 4 queries de prefixo 2D por inclusão-exclusão.
- **Segtree kinetic (KTT) / kinetic segment tree.** Guarda retas m_i·t_i + c_i em cada nó para "max de m_i·t_i + c_i"; cada t_i só cresce (update soma δ > 0 a t_i no intervalo, o que equivale a c_i += m_i·δ); reconstrua o nó quando o tempo ultrapassa o ponto de troca. Amortizado polilog (O($\log^3 n$) na análise clássica) com δ > 0.
- **Segtree com merge de nós via small-to-large / merge de segtrees.** Em DP de subárvore com estruturas indexadas por valor, funda segtrees dinâmicas dos filhos em O(n log n) total (merge destrutivo; o custo amortizado é o número de nós criados).
- **Segtree iterativa bottom-up.** 2–3× mais rápida; funciona para operações não comutativas se tratar esquerda/direita, mas lazy exige a versão recursiva ou a variante "push do topo até a folha" antes da query.
- **Disjoint sparse table.** Query de qualquer operação associativa (não idempotente: soma, produto mod, xor, composição) em O(1) com preprocessamento n log n: para cada nível guarde prefixos/sufixos a partir do meio do bloco.
- **Sparse table também resolve idempotentes (min, max, gcd, and, or) em O(1)** pois pode sobrepor dois blocos de 2^k. Para gcd, a query é O(1) sem log do gcd só pela sobreposição.
- **Sparse table de índices.** Guarde o índice do mínimo em vez do valor para devolver "onde"; comparação por (valor, índice) mantém desempate estável (mais à esquerda).
- **Sqrt decomposition: escolha do bloco.** Se update custa O(1) e query O(B + n/B) ou ao contrário, B = √n; se o custo de reconstrução do bloco é O(B log B), ajuste B ≈ √(n / log n). Teste B por experimento: constante importa.
- **Sqrt em queries de tempo (rebuild a cada √q operações).** Mantenha estrutura estática reconstruída periodicamente + buffer das últimas ≤ √q mudanças verificadas à força bruta a cada query.
- **Sqrt com bloco que guarda estrutura auxiliar.** Bloco com array ordenado (contar ≥ x), bitset, ou tabela de contagem por valor (blocos × valores) com prefix sums por bloco para "k-ésimo/contar em [l, r]" em O(√n) com O(1) por bloco.
- **Sqrt no "pular" (Holes / jump).** Para cada i guarde onde cai após sair do bloco e quantos saltos deu; update reconstrói só o bloco (CF 13E "Holes").
- **Mo + rollback (sem remoção).** Quando só dá para adicionar (ex.: max, união sem desfazer), ordene por bloco de l e r crescente; para cada bloco refaça l a partir do fim do bloco e desfaça os adds de l após cada query ("Mo com rollback").
- **Mo com updates.** Terceira coordenada t; blocos de tamanho n^(2/3), ordem (⌊l/B⌋, ⌊r/B⌋, t); O(n^(5/3)). Update ao vivo: se a posição está dentro de [l, r], remova antigo e adicione novo.
- **Mo em árvore.** Euler tour de 2n posições; vértice aparece duas vezes; alterne add/remove a cada toggle; para query (u, v) com lca fora do segmento adicione o lca à parte. Ver "Combinações".
- **Mo + bitset.** Quando o add/remove é O(1) mas a resposta é difícil, mantenha o bitset do conjunto atual e responda em n/64 (ex.: "existe soma x = a + b em [l, r]"? use bitset e shifts).
- **Mo com ordenação de Hilbert.** Ordena queries pela curva de Hilbert em (l, r): mesma complexidade mas 1.5–2× menos movimento na prática; útil para Mo de 10^5–10^6 queries.

- **Mo e "ponteiros que andam"**: se add custa O(1) e remove custa O(log), prefira o Mo onde só a direção mais barata se repete mais.
- **Mo 2D (queries em retângulos) é raro; prefira offline sweep.** Mo é para queries onde o estado é multiconjunto de elementos de um intervalo.
- **CDQ para DP online.** dp[i] depende de dp[j], j < i, com condição 2D: resolva a metade esquerda, contribua da esquerda para a direita (ordenando por outra dimensão + BIT), depois resolva a direita. Vale para LIS ponderado 2D, CHT não monotônico, "dominância com DP".
- **CDQ para updates e queries no tempo.** Eventos em ordem temporal; D&C no tempo, a metade esquerda contribui para a direita, ordenada pelo espaço. Cuidado com ordem de desempate: update antes de query na mesma coordenada.
- **Offline sweep de valor.** "Contar pares (i, j) com a_i + a_j ≤ x e outra condição" → ordene por valor, insira conforme o limite cresce, mantenha estrutura nas posições.
- **Offline: ordene queries por limite e elementos por peso.** Padrão "quantos elementos de peso ≤ w têm propriedade P no intervalo/subárvore" = adicionar elementos em ordem de peso numa BIT.
- **Distintos em intervalo, versão com soma.** Soma dos valores distintos em [l, r]: mesmo sweep por r, BIT guarda valor em prev(i) apagado e i ativo. Versão com multiplicidade (≥ 2 vezes) usa prev de segunda ordem.
- **Moda em intervalo (range mode).** Sem update: frequência por valor + sqrt (candidatos = modas dos blocos mais valores avulsos, checando por vetor de posições + binária) em O(n√n + q√n log n); com tabela de contagens por bloco (O(n√n) de memória) a checagem é O(1) e o total cai para O((n+q)√n); resposta online pelo "pesado/leve" por frequência.
- **Range kth online.** Wavelet matrix ou persistente O(log n); com updates → BIT de segtrees dinâmicas, ou sqrt; binária paralela é offline (todas as queries conhecidas), não responde online.
- **Contar inversões em intervalo.** Mo + BIT (n√n log n); com blocos de contagem sobre valores, o update é O(1) mas a query é O(√n), e como são O(n√n) movimentos isso leva a $O(n^2)$. Para O(n√n) use Mo secundário (2nd offline) calculando via prefixos; alternativa: decomposição sqrt estática.
- **Inversões após swaps / updates pontuais.** Mantenha BIT de BITs, ou decomposição em blocos; ou CDQ (tempo, posição, valor), problema de dominância 3D.
- **MEX em intervalo.** Offline por r: para cada valor guarde a última ocorrência last[v] em segtree de mínimo; mex de [l, r] = menor v com last[v] < l (descida na segtree). Para mex com updates → contagem não indica presença em [l, r]: guarde as posições de cada valor (ordered set) e teste presença com estrutura 2D.
- **Xor máximo com x em [l, r].** Trie persistente por prefixo com contagem; query compara contagem(r) − contagem(l−1) em cada nível. Alternativa: trie com "maior índice" em cada nó e query com l.
- **Maior subarray de xor / par com xor máximo no intervalo.** Subarray: prefix xor + trie persistente (par de prefixos); base linear com "prefix linear basis" (mantém, para cada prefixo, base com timestamps mais recentes) responde "base de [l, r]" em O(log) por query. Par de elementos (a_i xor a_j, ambos em [l, r]): prefix xor não resolve; use trie sobre os valores com Mo + rollback (só adições) ou D&C offline.
- **Prefix linear basis (timestamp).** Na inserção de x no índice i, em cada bit, se o ocupante tem índice menor, troque (guarde o mais recente); então a base de [l, r] usa só vetores com índice ≥ l. Resolve xor máximo em subarray.
- **Maior retângulo em histograma/queries.** Estático: pilha monotônica. Com query de intervalo → Cartesian tree + sparse table (retângulo máximo que contém o mínimo) ou segtree com (min, melhor, prefixo/sufixo do nível).
- **Maior retângulo de 1s em matriz binária.** Histograma por linha + pilha: O(nm). Com updates → segtree por coluna é difícil; geralmente reduza.
- **Queries de gcd em intervalo.** Sparse table (idempotente) ou segtree; a contagem de elementos iguais ao gcd é segtree de pares (gcd, contagem). Em Mo, gcd só funciona com rollback ou "duas pilhas".
- **Duas pilhas (queue com agregado).** Fila com operação associativa sem inverso (gcd, min, matriz) via duas pilhas, cada uma guardando agregados; pop_front rebalanceia amortizado. Substitui deque monotônico quando a operação não é min/max. Janela deslizante com qualquer monóide.
- **Dividir blocos de k (janela deslizante de tamanho k).** Prefixos e sufixos por blocos de k dão qualquer monóide em janelas fixas em O(1) por query, sem inverso.
- **Processo ao contrário.** Se a sequência de operações destrói (remove, une, quebra) e a pergunta é sobre o estado final ou intermediário, rode do fim para o início: remove → add; cortar aresta → unir; "máximo de componente após cada remoção" → DSU reverso.
- **DSU com rollback.** Union por tamanho sem compressão; guarde pilha de (raiz filho, tamanho antigo); desfaça em O(1). Base de segtree-no-tempo e Mo com rollback; custo O(log n) por find.
- **DSU com paridade / potencial.** Cada nó guarda a diferença (xor, soma, mod k) até o pai; find acumula. Detecta inconsistência de equações x_a − x_b = w e bipartição online.
- **DSU "próximo livre".** pai[i] = próximo índice não processado ≥ i; resolve "pintar intervalo, cada posição só pintada uma vez" e "atribuir valores distintos" em quase O(n α).

- **DSU com merge de listas / small-to-large.** Mantenha em cada raiz um set/map e una pequeno no grande, $O(n \log^2 n)$; para consultas de componente mais frequente ou menor elemento.
- **DSU offline dinâmico (segtree no tempo).** Cada aresta vale em $[t_1, t_2)$: insira na segtree sobre o tempo em O(log) nós; DFS aplicando uniões com rollback; responde conectividade e bipartição. MST com remoções (mesmo max-spanning com pesos) não sai do rollback — use LCT/dynamic MST.
- **Treap/splay implícito.** Chave = posição (tamanho do subárvore); split/merge permitem inserir, remover, mover bloco, cortar-colar de substring e reverter intervalo com flag lazy de "reverse".
- **Treap com lazy de adição/atribuição e agregado.** Qualquer segtree com inserção/remoção no meio → treap implícito; custo ~5× mais lento, use só quando a operação precisa de estrutura mutável.
- **Splay: amortizado e "acesso recente".** Sequência de n acessos custa O(n log n) + pior caso sobre as chaves; usado em link-cut e em problemas com operações "mover para frente".
- **Chtholly / ODT.** Mapa de intervalos com valor constante; só funciona se há atribuição aleatória em intervalo (custo esperado O((n+q) log n) sob assigns aleatórios) ou o número total de splits é amortizado O(n + q). Pior caso O(nq).
- **Link-cut com informação de subárvore (virtual).** Para somas em subárvore mantenha "soma de filhos virtuais" além da soma da splay; permite update de ponto + query de componente.
- **Euler tour tree (ETT).** Floresta dinâmica com sequência de Euler guardada em treap/splay: link/cut/reroot em O(log n), query de subárvore como intervalo; alternativa ao LCT quando só precisa de subárvores.
- **Ordered set trick (pbds / treap).** Rank e k-ésimo em O(log n) com multiconjuntos dinâmicos; para valores com duplicatas insira (valor, id). Inversões online e "mediana dinâmica" se resolvem assim.
- **Mediana dinâmica com dois heaps.** Heap de max (metade inferior) + heap de min (metade superior), com remoção preguiçosa; contagens por lado separadas para soma de desvios absolutos.
- **Heap com remoção preguiçosa.** Dois heaps (ativos e deletados); ao topar, desempilhe enquanto os topos coincidem. Troque por set/multiset se precisar de iteração; por heap de pares (valor, versão) com array de versões se o item muda de valor.
- **Heap mergeable (leftist / skew / pairing).** União de duas filas de prioridade em O(log n): permite "juntar heaps de filhos" em árvore (ex.: slope trick em árvore, "escolher k mínimos da subárvore") sem small-to-large.
- **Li Chao.** Insira reta em O(log) e segmento em $O(\log^2)$; consulte o mínimo em x em O(log); sem exigir monotonicidade das inclinações; dinâmico (domínio 10^9) funciona. Persistente em árvore ("CHT em árvore": DP no caminho raiz, rollback na volta).
- **CHT em árvore.** dp[v] = min sobre ancestrais u de dp[u] + b_u·a_v (queries no caminho) → Li Chao com rollback (guarde os nós modificados, O(log) por inserção) e DFS; ou Li Chao persistente.
- **CHT com inclinações monótonas e queries monótonas** → deque O(n); queries arbitrárias → busca binária no hull; inserções arbitrárias → Li Chao ou "CHT dinâmico" com set ordenado de retas.
- **Logarithmic method (Bentley-Saxe).** Estrutura estática com merge O(n) + só inserções → mantenha ≤ log n estruturas de tamanhos 2^i e recombine como contador binário; query consulta todas. Torna estático insertion-only (hull, k-d tree, índice de SA) com inserção amortizada O(log n) e query O(log n · Q), onde Q é o custo da query estática ($O(\log^2)$ só se Q = O(log)).
- **Remoção com Bentley-Saxe offline.** Se o tempo de vida de cada item é conhecido, segtree no tempo + estrutura com rollback/estática por nó.
- **Contador binário e custo amortizado.** Incrementos custam O(1) amortizado (potencial = número de 1s); vale para "reconstruir quando metade foi mudado" (rebuild global em dobra de tamanho) e para analisar pilhas com undo.
- **Potencial para amortizar.** Defina Φ = tamanho da pilha/#bits/#componentes; custo real + ΔΦ ≤ constante. "Pop while condition" é amortizado O(1) pelo potencial.
- **Bitset como estrutura.** Matrizes booleanas, alcançabilidade em DAG com $n \le 5 \cdot 10^4$ (n/64 palavras por vértice; $n^2/64$ no total, tempo O(mn/64)), mochila, soma de subconjuntos, filtro de vizinhos em grafos densos; use _Find_first/_Find_next para iterar bits ligados.
- **Bitset + transições do tipo shift-or.** "Existe subconjunto/par com soma/diferença x" → dp |= dp << a; "x + a_i para todo i" → bitset de valores deslocado.
- **K-d tree.** Contagem em retângulo em O(√n) em 2D; vizinho mais próximo é rápido na média, mas O(n) no pior caso; reconstrução parcial (scapegoat) para inserções. Escolha mediana alternando eixos; poda por distância à caixa.
- **Quadtree/octree.** Para pontos muito agrupados com queries aproximadas ou colisão; no pior caso (pontos colineares) degenera; a k-d tree dá garantia.
- **Fractional cascading.** Busca do mesmo valor em k listas ordenadas em O(log n + k) em vez de k·log n; usada para tirar um log em merge sort tree e interseção de camadas.
- **Timestamp trick (visitado sem limpar).** Em vez de zerar array por query, guarde a "época" da última escrita; reset vira incremento. Útil em BFS/DFS multi-teste e Mo.
- **Versionamento / "timestamp" de atualização.** Para update "atribuir em tudo" (reset global) guarde (versão global, valor) por elemento e compare; evita O(n).

- **Parallel binary search (binária paralela).** Para q queries "primeiro tempo em que P(x) vale" com estrutura que só avança no tempo: todas as queries no mesmo nível mid são respondidas numa única passada de simulação, O(log n) passadas.
- **Tirar log por offline.** Se O(log) vem de busca em estrutura dinâmica e as queries são ordenáveis, ordene tudo e use ponteiros/DSU/vetor (ex.: pares com soma ≤ x ordenados → two pointers em vez de BIT).
- **Cache e constante.** Layout em SoA, loop interno em stride 1, blocos de 2^k na transposta de matriz, segtree iterativa com folhas contíguas, Eytzinger para busca binária; reduz 2–5× sem mudar a complexidade.
- **Árvore de segmentos para "soma de min de subarrays com update".** Combine pilha monotônica offline (contribuição de cada a_i) com segtree por tempo; ou use sweep em r mantendo segtree de "soma de min(l..r)" com lazy de atribuição em intervalo (históricos).
- **Segtree de soma histórica.** Para "soma dos valores de a_i em todos os tempos passados" guarde (valor atual, soma histórica, lazy de 'a' e de 'b') e componha tags 2D; clássico "historic sum" / sweep de r para "soma de max/min de todos os subarrays".
- **Sweep com segtree: máximo de cobertura.** Evento +1/−1 em intervalo, query = max global; "ponto coberto por mais retângulos", "máximo de sobreposição ponderada"; coordenada comprimida, lazy sem push-down.
- **Contagem de pares (i, j) com f(i) ≤ g(j) em estrutura de dois pontos.** Ordene f e g juntos num único conjunto, tipo de evento; BIT sobre o outro tipo (merge-sort-like). Evita D&C.
- **Árvore de Fenwick para máximos de prefixo em "DP LIS com peso".** dp[v] = max dp[u] com u < v e a_u < a_v → BIT de max por valor comprimido; reconstrução guardando o índice.
- **Binary lifting guardando agregado e "salto com condição".** Para "primeiro ancestral com valor ≥ x" use binary lifting com max por salto; ou sparse por saltos; ou HLD + descida na segtree.
- **Chunking temporal de queries (offline por blocos).** Quando as queries e updates são em √q lotes, as mudanças do lote são poucas → comprima a árvore/array nesses ≤ √q pontos relevantes e responda cada query em O(√q).
- **Persistência fat-node / path copy e rollback de memória.** Persistir uma estrutura é equivalente a árvore de versões; para "voltar a versão k" use DFS na árvore de versões com rollback quando os ops são desfazíveis (mais barato que persistência).
- **DSU com rollback em segtree + "Mo em tempo"**: para queries "conectividade no intervalo de arestas [l, r]" → ordene por bloco, rollback; ou offline: MST incremental com LCT.
- **Queries de "intervalo cujos elementos distintos estão todos em [lo, hi]".** Transforme em pontos (l, r) e use sweep: para cada elemento calcule prev e next; intervalo [l, r] contém todas as ocorrências de v ⇔ prev(l) < l e next(r) > r etc. Dualidade posição-intervalo.

Repetidas de outras seções:

- **Queries com strings de Σ|s| ≤ S** → ≤ √(2S) tamanhos distintos; hash set por tamanho (strings).
- **Comparar substrings em query** → hash de prefixo, ou segtree de hash se houver updates (strings).
- **Elemento majoritário em [l, r]** → sorteie posições e conte com vetor de posições + binária (aleatorização).
- **Descida em BIT/segtree** substitui binária + query e tira um log (primeiro prefixo com soma ≥ x, k-ésimo livre).
- **Pilha monotônica + estrutura**: processe por r e mantenha a pilha de candidatos (ex.: "subarrays em que i é máximo" atualiza um segtree).
- **Base linear com prefixo** (mantendo o vetor mais recente por pivô) → max xor de subconjunto em [l, r] offline por r ou online.
- **Trie xor persistente** → max x⊕a_i em [l, r] por diferença de cnt entre versões.
- **Ocorrências de padrão em prefixo por AC + BIT na link tree** → +1 no nó visitado, query soma de subárvore.
- **Mo em árvore (Euler de 2n)**: v aparece duas vezes; alterne add/remove a cada ocorrência (o vértice está ativo quando o número de ocorrências no segmento é ímpar); query (u, v) com tin[u] < tin[v] → [tin[u], tin[v]] se u é ancestral de v (sem lca extra), senão [tout[u], tin[v]] mais o lca explicitamente.
- **Union-find com rollback e Mo** → Mo com rollback (ordena por bloco de l, r crescente, desfaz o lado esquerdo) quando a operação só tem "adicionar".
- **Árvore de Fenwick em Euler tour** para update de subárvore + query de ponto/subárvore com dois BITs (range-update/range-query).
- **Hash de multiconjunto em intervalo** → soma de valores aleatórios por prefixo: [l, r] é permutação de 1..k ⇔ hash(r) − hash(l−1) = hash(1..k) (aleatorização).
- **Distintos em intervalo / mode / majoritário** → majoritário por sorteio de posições + vetor de ocorrências e binária (aleatorização).

Construtivos e interativos

Construtivos:

- **Ache a condição de impossível primeiro** (paridade, soma, contagem); a construção costuma provar que é a única.
- **Resolva n pequeno à mão**, ache o padrão, e faça indução: de n para n + 2 ou n + 4.
- **Respostas típicas:** 1, 2, 3, n, ordenado, ordenado ao contrário, alternado, i ↦ n + 1 − i, potências de 2.
- **Operação reversível** → leve as duas configurações para uma forma canônica.
- **"No máximo n operações"** → uma operação fixa um elemento de cada vez; "no máximo 2n" → duas por elemento.
- **Grid** → xadrez (paridade de i + j), cobrir linha por linha, cobra (zigue-zague).

Interativos:

- **Limite de queries ≈ log n** → binária; ≈ 2 log n → binária com duas perguntas por passo; ≈ n/2 → pares.
- **Adaptativo?** Pense no adversário: cada resposta deve dividir as possibilidades ao meio.
- **Teste com um interator local** antes de submeter.

adicionado em 2026-11-10 por agentes; revisado

- **Indução removendo extremos.** Remova dois elementos/vértices que se cancelam (maior e menor, folha e pai), resolva n − 2 e cole dos lados; checa-se bem com n = 1..8.
- **Pareamento de soma constante** (i, n + 1 − i) dá soma n + 1; use para permutações com |p_i − p_j| fixo, magia e "dividir em dois grupos de soma igual".
- **Sentinela/sacrifício.** Reserve um elemento para absorver o resíduo (ex.: último elemento = s − Σ resto, ou um vértice universal para consertar graus).
- **Construir de dentro para fora** (começa pelo centro/maior/único elemento forçado) funciona quando a condição é local em cada inserção.
- **Dividir [1..n] em dois conjuntos de mesma soma.** Precisa de n(n+1)/2 par; guloso do maior para o menor pegando se couber. Qualquer soma entre 0 e n(n+1)/2 é alcançável com subconjunto de {1..n}, pelo mesmo guloso.
- **Prouhet–Thue–Morse.** Separar {0..2^{k+1} − 1} pela paridade do popcount dá dois conjuntos com mesma soma de potências até k (soma, soma de quadrados, ...).
- **Base b / sistemas numéricos.** Todo n é soma de potências distintas de 2 (poucas moedas); ternário balanceado (dígitos −1,0,1) pesa qualquer valor com pesos 3^i nos dois pratos; Zeckendorf (Fibonacci sem consecutivos) representa qualquer inteiro; fatorial base dá a ordem de permutação (Lehmer).
- **Gray code.** g(i) = i ^ (i >> 1) passa por todas as máscaras mudando 1 bit por vez; usa-se para caminhar por subconjuntos, resolver Torre de Hanói/anéis chineses e passear no hipercubo (Hamiltoniano).
- **de Bruijn.** Sequência cíclica de tamanho k^n com todas as strings de tamanho n uma vez = ciclo Euleriano no grafo de k^(n−1) vértices (aresta = dígito); também sai da concatenação de palavras de Lyndon (FKM). Cofres/PINs e "menor string contendo todas as máscaras".
- **Euler tour como ferramenta de construção.** Cobrir todas as arestas de K_n com um passeio (n ímpar), orientar arestas com |in − out| ≤ 1 (pareie os ímpares com um vértice fantasma e ache o circuito), concatenar uma palavra.
- **Decompor arestas em caminhos de 2 arestas.** Grafo conexo com m par sempre decompõe; DFS, pareando de baixo para cima as arestas pendentes em cada vértice e passando a sobrante ao pai.
- **Grafo com graus dados.** Erdős–Gallai testa; Havel–Hakimi constrói (ligue o de maior grau aos próximos maiores). Árvore com graus d_i: Σd = 2(n − 1) e d_i ≥ 1 bastam (Prüfer).
- **Matriz 0/1 com somas de linhas e colunas dadas.** Gale–Ryser decide; construa por linha, ligando as colunas de maior soma restante. Com valores inteiros não negativos, canto noroeste: min(r_i, c_j) em cada passo, ≤ n + m − 1 entradas não nulas.
- **Matriz com condição em linha/coluna (todas soma k, xor 0, etc.).** Preencha o bloco (n−1)×(m−1) livremente e determine a última linha/coluna; cheque a consistência do canto (soma total de linhas = soma total de colunas).
- **Torneio com sequência de pontuação.** Landau decide: ordenada, Σ dos k menores ≥ C(k, 2) com igualdade no total; construa removendo o vértice de maior pontuação.
- **Round-robin (K_n em n − 1 rodadas, n par)** → método do círculo: fixe um time e gire os outros. Coloração de arestas de bipartido com Δ cores sempre existe (König).
- **Quadrado latino** L[i][j] = (i + j) mod n; dois ortogonais existem para n ≠ 2, 6. Forma a base para escalas, turnos e "cada linha e coluna com todos".
- **Strings sem padrão proibido.** 3 letras: "abcabc..." não tem palíndromo de tamanho ≥ 2; Thue–Morse não tem cubo nem overlap em 2 letras; existe sequência sem quadrados em 3 letras. Sem 'aaa'/'bbb' com contagens dadas: intercale grupos.

- **Rearranjar sem vizinhos iguais.** Possível sse maior frequência ≤ ⌈n/2⌉; preencha posições 0, 2, 4, ... com os caracteres mais frequentes primeiro e depois 1, 3, 5, ...
- **Conjunto de Sidon (somas de pares todas distintas).** $\{2p \cdot i + (i^2 \bmod p) : 0 \le i < p\}$ com p primo tem p elementos; também serve {2^i}. Útil para "subarrays/pares com somas distintas".
- **Permutação sem ponto fixo** → rotação cíclica; sem $p_i = i \pm 1$ ou com restrição de vizinhança → rotações e pares de blocos; teste n = 1..8 por brute force antes de fixar o padrão.
- **Ordenar com poucas operações.** Reverter prefixo → panqueca com ≤ 2n; reverter qualquer segmento → ≤ n − 1 (seleção); troca de qualquer par → n − ciclos; ordene em ≤ n operações fixando um elemento por vez.
- **Contar caminhos mínimos exatos N num grafo pequeno.** Cadeia de "losangos" (2 caminhos cada) constrói 2^k; N em binário usa k camadas e arestas de atalho por bit; ≈ 2 log N vértices.
- **Grafo com exatamente k triângulos / arestas.** Clique de tamanho t dá C(t, 3); acrescente vértices ligados a pares do clique (cada um soma uma quantidade de triângulos); guloso do maior t.
- **Parênteses com profundidade/pares dados.** Aninhe a quantidade de profundidade necessária e concatene o resto como "()"; contagens de "()" vs "(())" dão controle fino.
- **Arranjo com soma de |diferenças| adjacentes alta** → intercalar menor e maior (ordem pico-vale); como candidato inicial para maximizar e depois testar com brute force.
- **Local search com monovariante.** "Dividir vértices em dois grupos com cada um tendo ≥ metade dos vizinhos no outro": mova quem viola; o corte cresce a cada passo, então termina.
- **Passeio do cavalo / Hamiltoniano em grid** → heurística de Warnsdorff + backtracking; com número ímpar de casas não existe tour fechado (a coloração xadrez prova).
- **Ladrilhar com L-triminós** um tabuleiro 2^k com um buraco → indução dividindo em 4 quadrantes e colocando um L no centro para criar mais buracos. Coloração (módulo 3, xadrez) prova as impossibilidades de dominós.
- **Armadilha: construtivo com limite no tamanho da resposta.** Verifique o pior caso do número de operações/vértices na construção (ex.: 2n + 5) antes de implementar.
- **Armadilha: múltiplas soluções.** Se há checker, não force a resposta de exemplo; se o exemplo parece "estranho", ele costuma mostrar a construção do autor.
- **Busca binária interativa.** Cada pergunta de resposta binária resolve 1 bit: $\lceil \log_2 N \rceil$ queries; para $N = 10^{18}$ são 60. Perguntas de 3 respostas (<, =, >) dão $\log_3$.
- **Flush e protocolo.** Imprima com flush após cada query, leia a resposta (−1 ou erro = abortar), conte queries localmente e teste com interator caseiro; misturar cin e scanf sem sync causa deadlock.
- **Achar máximo/pico com queries** → compare a[m] e a[m+1] (binária); posição do segundo maior: consulte o max de [1, n] e localize a posição p do máximo por binária (o lado cujo max é o global contém p); o segundo maior é o maior entre o max de [1, p−1] e o de [p+1, n], em O(log n) queries.
- **Min e max juntos** em $\lceil 3n/2 \rceil - 2$ comparações (pareie e compare vencedores com vencedores); **segundo maior** em $n + \lceil \log_2 n \rceil - 2$ com torneio; **ordenar com mínimo de comparações** → inserção binária (n log n) ou Ford–Johnson (merge insertion), ótimo para n ≤ 22 mas não em geral: há algoritmos com menos comparações em certos n (por exemplo, n = 47) e ele não alcança $\lceil \log_2 n! \rceil$ para todo n.
- **Reconstruir permutação com queries de soma/xor/and de pares.** Três queries de soma (a+b, b+c, a+c) determinam três números; com and não (bit com um único 1 é indistinguível) e com xor só se determinam diferenças (a menos de um xor global); para n posições fixe a_1 por 3 queries e depois n − 3 queries "$a_1 + a_i$".
- **Permutação com queries $p_i \bmod p_j$.** Compare (i, j) nos dois sentidos: o maior dos dois resultados é $\min(p_i, p_j)$ (se $p_i < p_j$, $p_i \bmod p_j = p_i$); o menor é só o resto de $p_j \bmod p_i$; mantenha o candidato a maior e descubra todos os outros em ≤ 2n queries.
- **Permutação com queries "quantos antes de i são menores".** Reconstrua de trás para frente com k-ésimo livre num BIT (inverso da codificação de Lehmer).
- **Queries em árvore: achar vértice escondido.** Se a resposta diz de que lado do centroide está o alvo, $\lceil \log_2 n \rceil$ queries no total (uma por centroide, a cada nível). Com queries de distância, d(u, x), d(v, x) e d(u, v) dão a projeção de x no caminho u–v: dist de u = (d(u,v) + d(u,x) − d(v,x)) / 2.
- **Reconstruir árvore escondida com distâncias.** d(r, ·) dá as camadas; para cada vértice, o pai é um vértice da camada anterior que está a distância 1 (query d(v, candidato)); busque por binária entre candidatos quando possível, ou use pares de folhas para reduzir.
- **Mentiras limitadas.** Se o juiz mente até k vezes (ou em até uma de duas consecutivas), repita a query em janelas de k + 1 ou mantenha intervalo candidato com folga e reduza apenas quando houver consenso; calcule o número de queries com a folga.
- **Quente/frio.** Perguntas "mais perto de x que antes?" → compare dois palpites, o ponto médio entre eles divide o espaço; reduz a binária no ponto médio.

- **Interativo em jogo.** Decida qual jogador ser (primeiro ou segundo) se o enunciado permite; a estratégia de espelho/pareamento costuma ser a resposta e o programa só responde ao movimento do juiz.

Repetidas de outras seções:

- **Simetria/espelho** → construção pelo centro; mesmo truque do jogo espelho (teoria dos jogos).
- **Pares (i, n+1−i), (x, x xor 1)** → construir em pares resolve muitos construtivos e jogos.
- **Construir string** → palíndromo, repetição de um período, alternância 0101 (strings).
- **Construir árvore com graus dados** → soma dos graus = 2(n − 1) e todos ≥ 1 é suficiente; construção por Prüfer ou ligando maior grau a folhas (ordem decrescente).
- **Construir grafo com sequência de graus dada (Erdős–Gallai / Havel–Hakimi)** → ordene decrescente, ligue o maior aos próximos d vértices de maior grau, repita; condição $\Sigma_{i\le k} d_i \le k(k-1) + \Sigma_{i>k} \min(d_i, k)\ \forall k$ e soma par.
- **Construir árvore com distâncias dadas / diâmetro dado** → cadeia do diâmetro + folhas pendentes a profundidade ≤ ⌊diâm/2⌋ ao meio; verifique soma e matriz de distâncias (aditiva/four-point).
- **Grafo bipartido com graus dados (Gale–Ryser)** → ordene um lado, condição de dominância por conjugado; construção gulosa por maior grau.
- **Circuito de Euler dá construção de orientação balanceada, "ordem de dominós/linhas", de Bruijn e "desenho sem levantar".**
- **Torneio com pontuações dadas (Landau)** → construção gulosa; também "torneio com mínimo de ciclos 3".

Teoria dos jogos

Primeiro passo sempre: brute force de vitória/derrota para estados pequenos e imprima a tabela. O padrão quase sempre aparece em n ≤ 20.

Heurísticas de estratégia:

- **Imite o adversário (espelho).** Tabuleiro ou configuração simétrica → o segundo jogador copia o lance do outro do lado oposto e nunca fica sem jogada. Se o primeiro pode criar a simetria com um lance (tirar o meio, dividir em dois iguais), ele ganha.
- **A primeira escolha é a que mais importa.** Muitos jogos se decidem no lance 1 (escolher a ponta, a primeira letra, a cor, a raiz); depois disso a resposta segue um padrão fixo. Teste as n primeiras jogadas e veja se o resto é forçado.
- **Roubo de estratégia.** Se "passar a vez" ou um lance inofensivo existe, o primeiro jogador nunca perde (se o segundo tivesse estratégia, o primeiro roubaria).
- **Paridade do número de lances.** Se o jogo sempre dura o mesmo número de lances, só a paridade importa. Procure o que é fixo: total de peças, de casas, de inversões.
- **Quem controla o último lance** ganha; conte lances forçados e lances "livres".
- **Pares (pairing strategy).** Agrupe os lances em pares (i, i+1), (i, n+1−i), (x, x xor 1): sempre responda com o par do lance do adversário.
- **Reduzir para um estado perdedor conhecido.** Ache uma família de estados P (perdedores) fechada: de P só se vai para N, e de N sempre existe lance para P.
- **Lance que mantém o invariante** (xor = 0, soma ≡ 0 mod k+1, todas as pilhas pares).
- **Ganância falha em jogos.** O lance que maximiza agora raramente é o certo; pense no que sobra para o adversário.

Ferramentas:

- **Jogos independentes somados** → Sprague-Grundy: xor dos Grundy; Grundy de um estado = mex dos sucessores.
- **Nim** → xor ≠ 0 ganha. **Tirar 1..k** → n mod (k+1). **Misère Nim** → igual, exceto quando todas as pilhas são 1.
- **Staircase Nim** → só as posições ímpares importam. **Nim em árvore (Green Hackenbush)** → Grundy(v) = xor de (Grundy(filho) + 1).
- **Jogo em grafo com ciclos (empate possível)** → BFS retrógrada a partir dos estados finais, com contador de saídas.
- **Jogos de soma de pontos (minimax)** → DP em intervalo (pegar das pontas), dp = melhor diferença para quem joga.
- **Dois jogadores escolhem itens alternadamente com valores a_i (Alice) e b_i (Bob)** → ordene por a_i + b_i em ordem decrescente e pegue nessa ordem.
- **Grundy grande demais** → calcule pequenos, encontre periodicidade, prove depois.

Gatilhos por tipo:

- **Jogo em array** → pontas e paridade das posições; quando n é par, o primeiro pode garantir todas as pares ou todas as ímpares.
- **Jogo em string** → a primeira letra/escolha costuma decidir; palíndromo e simetria sugerem espelho.

- **Jogo em árvore** → paridade da profundidade, centro do diâmetro, número de folhas.
- **Jogo em grid** → paridade de i + j (xadrez), espelho pelo centro.
- **Jogo com números** → divisores, potências de 2, n mod k, representação binária.

adicionado em 2026-11-10 por agentes; revisado

- **Moore's Nim_k.** Pode tirar de até k pilhas por lance → escreva as pilhas em binário e some cada coluna de bit mod (k+1); posição perdedora ⇔ todas as colunas ≡ 0. Para k = 1 é o Nim comum.
- **Fibonacci Nim.** Uma pilha; o 1º lance tira de 1 até n−1; depois cada um tira de 1 até 2× o que o outro acabou de tirar. Perdedor para quem começa ⇔ n é Fibonacci (Zeckendorf: tire o menor termo da representação).
- **Nim com conjunto de subtração S.** Grundy de uma pilha = mex{g(n − s)}; é eventualmente periódico para S finito. Calcule 2–3 mil valores e detecte o período; "tirar 1..k" é o caso n mod (k+1).
- **Subtraia só quadrados / primos / potências.** Sem fórmula: g(n) por DP em O(n·|S|) com mex; se $n \le 10^6$, é só isso. Se $n \le 10^{18}$, não presuma período: periodicidade de Grundy só é garantida para S finito; com quadrados/primos/potências (S infinito) não há garantia — para quadrados é problema aberto.
- **Kayles e Dawson.** Kayles (derrubar 1 pino ou 2 adjacentes numa fileira; vira soma de duas fileiras) tem Grundy periódico de período 12 após exceções iniciais; Dawson's Kayles (só 2 adjacentes) tem período 34. Qualquer "remover bloco e partir a fileira em duas" → soma de jogos e Grundy por DP em n^2.
- **Lasker's Nim.** Pilha: tirar qualquer quantidade OU dividir em duas não vazias → g(4k+1) = 4k+1, g(4k+2) = 4k+2, g(4k+3) = 4k+4, g(4k+4) = 4k+3. Mostra que "dividir" pode tornar o Grundy quase o tamanho.
- **Grundy limitado pelo grau.** g(v) ≤ grau de saída de v, e num grafo com m arestas g ≤ O(√m) (um vértice com g = x precisa de sucessores com 0..x−1, e eles de sucessores próprios). Vetor de "valores vistos" pode ser pequeno.
- **Vários tokens num DAG.** Cada token é um jogo independente: xor dos Grundy das posições; vence quem começa se ≠ 0. Se tokens podem ocupar o mesmo vértice, ainda vale (jogos somados, não interagem).
- **Lances reversíveis são ignoráveis.** Se o jogo permite também aumentar uma pilha, o vencedor pode desfazer o aumento (remove o que foi somado), então a classificação P/N é a do jogo sem esse lance, desde que o jogo termine.
- **Coin-turning games.** Virar moedas com regra "a moeda mais à direita vira de cara para coroa": o jogo é soma dos jogos de cada cara, g(posição) = xor de g(i) de cada cara na posição i. Turning Turtles (virar 1 ou 2 moedas) → g(i) = i (Nim). Ruler game (virar qualquer bloco contíguo) → g(i) = maior potência de 2 que divide i.
- **Tartan (produto de jogos).** Coin-turning em 2D que é produto de dois jogos 1D → g(x, y) = nim-produto $g_1(x) \otimes g_2(y)$. Raro, mas explica Grundy de "virar retângulo de moedas" (usa nim-multiplicação, que se calcula recursivamente em potências de 2 de Fermat).
- **Green Hackenbush geral.** Ciclos colapsam: **fusion principle** — todo ciclo pode ser contraído num único vértice (laços são removidos) sem mudar o Grundy; depois é uma árvore. Chão = raiz.
- **Colon principle.** Vários ramos pendurados no mesmo vértice → só importa o Grundy de cada ramo (xor); pode substituir um ramo por uma haste Nim de mesmo Grundy. Dá a fórmula g(v) = xor(g(filho) + 1).
- **Hackenbush azul-vermelho (partizan).** Hastes só azuis valem +n inteiros, só vermelhas −n; ao trocar de cor o valor binário acrescenta ±1/2, ±1/4... Soma dos valores > 0 ⇒ Esquerda ganha começando ou não; = 0 ⇒ quem começa perde; < 0 ⇒ Direita ganha.
- **Jogos partizan: compare com 0.** Quando cada jogador tem lances próprios, defina o valor do jogo (número dyadic, ou jogo quente) e some; o sinal do total decide. "Cada um só pode mexer nas suas peças" → valores numéricos, soma, sinal.
- **Jogos quentes (switches).** {a | b} com a > b: quem joga primeiro ganha (a−b)/2 em relação à média; média (a+b)/2 e temperatura (a−b)/2. Em jogos de pontuação em que mover é bom, jogue primeiro o de maior temperatura (heurística exata nos casos simples; confirme por brute force).
- **Cram e Domineering.** Cram (colocar dominó sem sobrepor, ambos com os mesmos lances): as duas dimensões pares → o 2º espelha pelo centro; exatamente uma ímpar → o 1º coloca um dominó central simétrico e espelha; ímpar×ímpar não é coberto pela simetria central (3×3, 5×5, 1×5 são P; 3×5, 1×3 são N) → Grundy/brute. Em Domineering (partizan) use contagem de lances seguros de cada lado.
- **Strategy stealing em Chomp.** Chomp com tabuleiro retangular m×n: para m·n ≥ 2 o 1º sempre ganha (se o lance de pegar o canto perdedor perdesse, o 2º teria uma resposta que o 1º poderia ter feito já); 1×1 é a exceção (o 1º é obrigado a comer a casa envenenada e perde). Construtivo em casos especiais: n×n → tire (2,2) e espelhe o L; 2×n → deixe a 1ª linha 1 maior que a 2ª.
- **Hex lemma.** Tabuleiro cheio de Hex tem exatamente um vencedor (nunca empate) ⇒ por stealing o 1º ganha. "Bloquear o outro = conectar eu" também vale em jogos de conexão em grade/triangular.
- **m,n,k-games.** Em jogos de alinhar k (tic-tac-toe generalizado) o 2º nunca ganha por stealing; a pergunta real é empate vs vitória do 1º → brute force com poda.

- **Geography em grafo não-direcionado.** Token percorrendo arestas sem revisitar vértices: o jogador que move a partir de v ganha ⇔ v está em todo emparelhamento máximo (calcule matching máximo com e sem v). Em direcionado geral o problema é PSPACE-completo, então o enunciado tem estrutura (DAG, árvore, bipartido).
- **Euclid game.** (a, b) com a > b: tirar múltiplo de b de a. Se ⌊a/b⌋ ≥ 2 quem joga ganha (escolhe entre terminar em a mod b ou a mod b + b); senão o lance é forçado → simule Euclides e conte os passos forçados; a paridade decide, mas depende do terminal: quociente ≥ 2 (ganha quem move) ou b = 0 (perde quem move).
- **Trie de palavras (jogo de prefixos).** Jogadores alternam acrescentando uma letra mantendo prefixo de alguma palavra; quem não pode perde. Calcule em cada nó win(v) e lose(v) (pode forçar perder?). Com k partidas em que o perdedor começa a próxima: se pode forçar ganhar e perder → 1º ganha; só ganhar → vence ⇔ k ímpar; nenhum → 2º.
- **Jogo com bitmask de itens usados.** "Escolher números distintos de 1..n até a soma passar T" → estado = máscara de usados (a vez vem do popcount, e a soma da própria máscara). n ≤ 20, dp[mask] ∈ {vence, perde}.
- **Memoização com hash em jogos de busca.** Tabela de transposição (tabuleiro → resultado) e ordenar lances (melhor primeiro) fazem alpha-beta cortar a maior parte; ainda é exponencial, então só vale com poucos lances ou profundidade ≤ ~20.
- **Alpha-beta e minimax.** minimax com profundidade fixa e retorno em "diferença de pontos"; alpha-beta reduz b^d para ≈ b^(d/2) com boa ordenação. Em problemas de ICPC o estado costuma ser pequeno o bastante para memo puro.
- **Pegar das pontas com n par.** O 1º pode garantir todas as posições pares ou todas as ímpares → nunca perde (empata ou ganha); decida pelo maior somatório. Para n ímpar ou regra diferente → dp[l][r] em $O(n^2)$, com array 1D rolando se a memória for curta.
- **Diferença como DP.** dp[l][r] = max(a[l] − dp[l+1][r], a[r] − dp[l][r−1]): a mesma recorrência vale para qualquer jogo de soma zero com dois jogadores alternados (subtraia o valor do adversário). Evite guardar dois valores.
- **Jogos de soma não zero / multi-jogador.** Sem Sprague-Grundy; cada estado vira vetor de payoffs e cada um maximiza a própria coordenada (backward induction). Empates de desempate entram na ordem da comparação, defina-os no enunciado.
- **Jogos probabilísticos.** dp[estado] = probabilidade de vencer quem joga; em estados com ciclo e só acaso (sem escolhas) vira sistema linear (Gauss); com escolhas max/min a equação de Bellman não é linear → iteração de valores ou LP. Se sem ciclo, DP de trás pra frente. Ambos escolhem lances: expectiminimax.
- **Loopy Grundy (Smith).** Com ciclos o valor de um estado é um nimber ou ∞ com conjunto de sucessores; para soma de jogos cíclicos: se algum componente é ∞ o resultado vira empate/ganho conforme os sucessores finitos. Geralmente basta BFS retrógrada e, em somas, evite o caso.
- **Misère além de Nim.** Último a mover perde: a classificação difere da normal só perto do fim; teste por brute force e compare com a normal. Em Nim com pilhas todas 1, ganha quem tem número par de pilhas. Se o jogo é "tame" (comporta-se como Nim) o misère se resolve com a normal exceto quando todas as componentes são pequenas; Kayles é tame (quociente de misère de ordem 40, período 12), mas Dawson's Kayles não é conhecido como tame (comportamento wild; análise de misère em aberto): use brute force.
- **Jogo com passe ou lance livre.** Se alguém pode passar, ninguém pode ficar em zugzwang; vence quem consegue empatar a contagem de lances seguros → conte lances seguros por jogador.
- **Jogo interativo adversarial.** O interator escolhe a resposta mais cruel consistente; mantenha o conjunto de candidatos possíveis, e a melhor pergunta é a que divide o pior caso ao meio (ou em três, se a resposta tem 3 valores). Prove limite inferior contando folhas da árvore de decisão: $\lceil \log_3 N \rceil$ ou $\lceil \log_2 N \rceil$.
- **Alice/Bob em strings.** Se cada um escolhe um caractere para formar a string final (um quer a menor, outro a maior), o ótimo é guloso dos extremos, com uma virada (CF 794C): enquanto o menor de Alice < o maior de Bob, cada um põe o seu melhor na primeira posição livre; quando deixa de valer, cada um passa a pôr o seu pior na última posição livre. Se há simetria entre posições → espelho.
- **Nim com pilha inicial grande e k pequeno.** Tabule g(n) para n ≤ 200 e olhe a parte periódica; o que escapa do período vem do início (pré-período).

Repetidas de outras seções:

- **Estratégia de pareamento/espelho** em construtivos interativos: se você escolhe ser o segundo jogador, responda ao par do lance do juiz.
- **Wythoff** → posições perdedoras ($\lfloor k\varphi \rfloor$, $\lfloor k\varphi^2 \rfloor$), $\varphi = (1+\sqrt{5})/2$; Fibonacci Nim: retirada ≤ 2× a anterior, perdedores = Fibonacci (Zeckendorf).
- **Moore's Nim_k e Nim com limite.** Tirar até k de uma pilha: Grundy = a mod (k+1); tirar de até k pilhas: some cada bit mod (k+1), posição perdedora se todos 0.

Geometria

Use inteiros sempre que puder: produto vetorial e escalar resolvem quase tudo sem ponto flutuante.

- **Orientação, interseção, área** → produto vetorial (cross); área de polígono → shoelace.
- **Pontos extremos, maior distância** → fecho convexo + rotating calipers.
- **Muitos pontos e uma reta/direção** → ordene por ângulo (sweep angular).
- **Manhattan** → rotação (x + y, x − y); **pontos em grid** → Pick (A = I + B/2 − 1).
- **Pares mais próximos** → divide e conquista ou grid de hash.
- **Interseções de muitos retângulos/segmentos** → sweep line com segtree.
- **Precisão:** compare com eps, evite sqrt e atan2 quando der, use long double.

adicionado em 2026-11-10 por agentes; revisado

- **Cross como orientação.** cross(b − a, c − a) > 0 ⇒ anti-horário; = 0 ⇒ colinear; o sinal decide tudo sem divisões. Com coordenadas até 10^9 as diferenças chegam a $2 \cdot 10^9$ e o cross a $2 \cdot (2 \cdot 10^9)^2 \approx 8 \cdot 10^{18}$, que ainda cabe em int64 (perto do limite); com 10^{18} use __int128.
- **Dot como ângulo / projeção.** dot > 0 agudo, = 0 reto, < 0 obtuso; projeção de p em reta ab devolve o parâmetro t = dot(p − a, b − a)/$|b - a|^2$; pé foot = a + t·(b − a); reflexão de p = 2·foot − p.
- **Cross e dot juntos dão ângulo orientado.** atan2(cross, dot) ∈ (−π, π]; evite atan2 como chave de ordenação: use meio-plano (y > 0 ou y = 0 e x > 0) + cross como comparador exato.
- **Ordenação por ângulo exata.** Divida em semiplanos e compare por cross > 0; pontos colineares na mesma direção: desempate por distância. Evita erro de atan2 em coordenadas grandes.
- **Interseção de segmentos (inteira).** Segmentos se cruzam se o1·o2 ≤ 0 e o3·o4 ≤ 0, onde o1, o2 são as orientações de (a, b, c) e (a, b, d) e o3, o4 as de (c, d, a) e (c, d, b); quando algum oi = 0 cheque onSegment (ponto sobre o outro segmento); caso colinear: cheque sobreposição das caixas (intervalo de projeção).
- **Ponto de interseção de retas** como a + t·d com t = cross(c − a, e)/cross(d, e); mantenha frações (num/den) em inteiros para comparar sem erro.
- **Ponto em polígono.** Convexo: busca binária por ângulo a partir de v_0 em O(log n); qualquer polígono: contagem de cruzamentos de raio (cuide de vértices no raio com a regra "y_a ≤ y < y_b") ou winding number.
- **Área e sinal do polígono.** Shoelace dá área sinalizada: positivo ⇒ anti-horário. Área de triângulo = |cross|/2; guarde 2·área em inteiros.
- **Centroide de polígono.** C = Σ (p_i + p_{i+1})·cross(p_i, p_{i+1}) / (6·A); decomponha em triângulos com área sinalizada. Centroide de pontos com massas = média ponderada.
- **Teorema de Pick.** A = I + B/2 − 1, com B = Σ gcd(|dx|, |dy|) sobre as arestas; resolve "pontos interiores de polígono em grid" e "contar pontos inteiros num segmento" (gcd + 1).
- **Pontos de grid em triângulo/reta.** Σ ⌊(a·i + b)/c⌋ em O(log) (floor sum) conta pontos inteiros abaixo de uma reta; use para pontos em triângulo rectângulo/trapézio sem iterar.
- **Fecho convexo (Andrew monotone chain).** Ordene por (x, y), construa inferior e superior com pop enquanto cross ≤ 0 (≤ para remover colineares, < para manter). Cuidado com todos os pontos colineares e duplicados.
- **Fecho convexo é a estrutura do "máximo de função linear".** max a·x + b·y sobre pontos = vértice do hull; sempre reduza o conjunto aos vértices do hull. Queries de direção → busca binária no hull.
- **Rotating calipers.** Diâmetro (par mais distante), largura mínima, retângulo envolvente de área/perímetro mínimo, distância mínima entre dois polígonos convexos; dois ponteiros que giram monotonicamente em O(n).
- **Diâmetro de nuvem de pontos = diâmetro do hull.** Só existem O(h) candidatos; com hull pequeno (pontos aleatórios) o hull tem O(log n)–O(n^(1/3)) vértices em média; vale a aproximação por brute force.
- **Soma de Minkowski de convexos.** Mescle as arestas ordenadas por ângulo começando pelos vértices mais baixo-esquerdos: O(n + m). Dois convexos intersectam ⇔ origem ∈ A ⊕ (−B); distância entre convexos idem.
- **Minkowski e "mover o objeto".** Colisão de polígono convexo em translação = ponto em Minkowski(A, −B); "alcançável com k passos de vetores" = soma de Minkowski de k segmentos (zonótopo: arestas ordenadas por ângulo).
- **Interseção de semiplanos (HPI).** Ordene por ângulo, mantenha deque; O(n log n). Resolve "kernel de polígono estrelado", "feasibilidade de LP 2D", "região onde todas as condições valem"; trate regiões ilimitadas com uma caixa grande e vazias via checagem do deque.
- **LP em 2D / 3D por randomização.** Seidel: O(n) esperado em dimensão fixa; ou HPI em 2D. Gatilho: "maximize ax + by sujeito a desigualdades lineares" com poucas variáveis.

- **Par de pontos mais próximo.** D&C por x e tira de largura 2d ordenada por y (cada ponto checa ≤ 7 vizinhos), O(n log n); ou sweep com set ordenado por y; ou grid de hash com tamanho d em O(n) esperado (reconstrói o grid quando d cai, amortizado com ordem aleatória).
- **Círculo mínimo envolvente (Welzl).** Embaralhe os pontos; incremental com 3 laços aninhados "se p fora do círculo atual, refaça com p na fronteira": O(n) esperado. Círculo por 2 pontos = diâmetro; por 3 = circuncentro.
- **Circuncentro e incentro.** Circuncentro: resolver 2 equações lineares de mediatrizes (não bissetoras); use denominador D = 2·cross e mantenha numeradores; compare raios com frações para evitar ponto flutuante. Incentro = (a·A + b·B + c·C)/(a + b + c) com a, b, c = lados opostos (raízes), em geral não inteiro.
- **Teste de ponto em círculo circunscrito (in-circle).** Determinante 4×4 (ou 3×3 após subtrair) dá sinal exato com inteiros; base de Delaunay e da checagem de "círculo vazio".
- **Delaunay / Voronoi.** Maior círculo vazio, vizinho mais próximo para todos os pontos, MST euclidiano (MST está contido em Delaunay), O(n log n); em contestes normalmente se evita implementando a MST euclidiana via Boruvka+k-d tree; os 8 octantes são da MST Manhattan, não da euclidiana.
- **MST Manhattan.** Para cada ponto, em cada um dos 4 octantes só o vizinho mais próximo importa (sweep + BIT de min por x+y); 4n arestas candidatas + Kruskal.
- **Manhattan e Chebyshev.** $|dx| + |dy| = \max(|dx + dy|, |dx - dy|)$; maior distância Manhattan entre pontos = max sobre as 4 combinações de sinais (x ± y). Em dimensões d use 2^(d−1) máscaras de sinais.
- **Distância Manhattan com sweep.** Para o ponto mais próximo/discos Manhattan, separe os 4 quadrantes por simetria (troca de sinal) e reduza para "max/min de (x + y) com restrição de ordem" → BIT/segtree. Discos Manhattan viram quadrados rotacionados.
- **Distância ponto-segmento.** Projete: $t = \text{dot}(p - a, b - a)/|b - a|^2$, limite t em [0, 1]; distância entre dois segmentos disjuntos = mínimo das 4 distâncias ponto-segmento (se não se cruzam).
- **Distância mínima entre segmentos e polígonos convexos.** Mínimo das 4 distâncias ponto-aresta é suficiente quando não se tocam; para convexos, calipers.
- **Círculo ∩ reta / círculo ∩ círculo.** Reduza a distância centro-reta $h^2 = r^2 - d^2$; círculo-círculo: $a = (d^2 + r_1^2 - r_2^2)/(2d)$, $h = \sqrt{(r_1^2 - a^2)}$. Teste de interseção com inteiros comparando d^2 com $(r_1 \pm r_2)^2$.
- **Tangentes a círculos.** Ponto externo: ângulo asin(r/d) em torno da direção; tangentes comuns de dois círculos: externas via mesmo raio deslocado, internas via soma de raios ($|r_1 \pm r_2|$).
- **Área da união de círculos / de interseção círculo-polígono.** Teorema de Green: some a integral de cada arco da fronteira que não está coberto + termos cross; interseção círculo-triângulo com origem no centro como soma de setores/triângulos sinalizados.
- **União de retângulos (área).** Sweep em x, segtree em y com (min, contagem de mínimo) ou (cobertura, comprimento coberto); O(n log n). Com perímetro: some também variação de comprimento coberto nas bordas.
- **Perímetro da união de retângulos.** Sweep horizontal e vertical separadamente: perímetro = Σ |Δ comprimento coberto| ao longo do sweep em cada eixo; ou 2×(variações). Cuidado com a ordem de eventos (abre antes de fechar nas coordenadas iguais).
- **Área da união de polígonos.** Para cada aresta, calcule o comprimento não coberto pelos outros polígonos (projeção em parâmetro t) e some cross das partes visíveis (Green); $O(n^2 \log n)$.
- **Sweep line + set (Shamos–Hoey).** Detectar se existe interseção entre n segmentos: mantenha set ordenado por y no x atual; eventos de abertura checam vizinhos; interseção só pode surgir entre vizinhos. Bentley–Ottmann reporta todas em O((n + k) log n).
- **Comparador de segmentos no set.** Compare a y em x corrente (via fração), nunca guarde float; segmentos verticais tratados à parte; desempate por id para não apagar errado.
- **Sweep angular em torno de um ponto.** Ordene os outros pontos por ângulo e use dois ponteiros para contar "quantos em semiplano" ou "triângulos que contêm a origem" = C(n, 3) − Σ C(k_i, 2), onde k_i = pontos no semiplano à esquerda de i.
- **Contar triângulos que contêm um ponto.** Complemento: triângulos que NÃO contêm = os 3 vértices num mesmo semiplano aberto; para cada ponto i some C(cnt_i, 2) onde cnt_i = pontos em (θ_i, θ_i + π). Cuidado com colinear com a origem.
- **Contar triângulos de área não nula/ retângulos / paralelogramas.** Paralelogramas: conte pares de pontos por ponto médio (hash de soma de coordenadas); retângulos: hash por (ponto médio, comprimento da diagonal).
- **Quadrados / retângulos axis-alinhados em pontos.** Fixe duas colunas/linhas e conte: O(n√n) usando pesado/leve (colunas com muitos pontos vs poucos) ou $O(\Sigma k^2)$.
- **Sweep angular com rotação de eixo: pares com cross sinal.** Número de pares de pontos separados por uma reta que passa por dois pontos = contagem por ângulo; o "rotating sweep" (rotação contínua) mantém a ordem de projeção e troca vizinhos nos ângulos dos pares (n^2 eventos).

- **Rotating sweep (ordem de projeção).** Ordene os n pontos pela projeção numa direção que gira; só muda a ordem quando a direção fica perpendicular a um par: processe os $O(n^2)$ eventos por ângulo trocando posições vizinhas; resolve "maior triângulo/área em k pontos" e "k-ésimo pelo qual passa" em $O(n^2 \log n)$.
- **Maior triângulo num conjunto.** Está no hull; para cada vértice fixe e avance dois ponteiros (calipers) em $O(n^2)$ no hull; ou O(n) com técnicas mais sofisticadas.
- **Dualidade ponto-reta.** (a, b) ↔ $y = a \cdot x - b$ preserva "acima/abaixo" e incidência: o hull superior de pontos = envelope inferior (CHT) de retas; queries de "reta mais próxima/ponto mais alto numa direção" ↔ CHT/Li Chao.
- **Dualidade para arranjos de retas.** Pontos em posição geral ↔ retas; "quantos pontos acima da reta pelos pontos i e j" vira contagem de interseções e vértices do arranjo; k-nível de arranjo.
- **Inversão de círculo.** Transforma círculos que passam pelo polo em retas e preserva tangência; útil para problemas de tangentes múltiplas (Apollonius, cadeia de Steiner) e para transformar interseção de círculos em retas (reduz a casos lineares).
- **Geometria 3D: produto misto.** Volume do tetraedro = |dot(a − d, cross(b − d, c − d))|/6; sinal dá orientação (lado do plano). Ponto em plano: dot(normal, $p - p_0$) = 0. Distância ponto-plano = |dot(n, $p - p_0$)|/|n|.
- **Fecho convexo 3D.** Incremental $O(n^2)$ (ou n log n com D&C/randomizado); conta faces via Euler V − E + F = 2 (F = 2V − 4 para triangulação). Distância mínima à origem entre poliedros também usa isso.
- **Transformações lineares/afins.** Escala não uniforme preserva razão de áreas; rotação por matriz; "alongar eixo" transforma elipse em círculo e depois resolva como círculo (problemas de distância em elipse).
- **Razões de áreas e cevianas.** Teorema de Ceva/Menelaus e massas pontuais em problemas de razões; coordenadas baricêntricas para ponto em triângulo e interpolação (λ_i = área sub-triângulo / área total).
- **Ponto em triângulo.** Três sinais de cross com o mesmo sinal (ou zero na borda); ou baricêntricas. Para muitos pontos e muitos triângulos → sweep/estrutura offline (contagem por triângulo = diferença de contagens de 3 trapézios).
- **Visibilidade de ponto entre obstáculos.** Sweep angular com set de segmentos ordenado pela distância no raio atual; ou grafo de visibilidade $O(n^2 \log n)$ (vértices dos obstáculos + sweep por vértice) e caminho mínimo euclidiano em obstáculos poligonais.
- **Caminho mais curto com obstáculos poligonais.** Só passa por vértices dos obstáculos: grafo de visibilidade + Dijkstra. Para um polígono simples, "funnel algorithm".
- **Convex layers (camadas convexas).** Aplique o hull repetidamente (O(n log n) total com estrutura dinâmica ou $O(n^2)$ simples); a profundidade de um ponto = "mínimo de pontos a remover/disparos para isolá-lo". Gatilho: "retirar pontos extremos repetidamente".
- **Pontos extremos repetidos e dominância.** Pontos que podem ser o maior para alguma função linear = vértices do hull; pontos "não dominados" em 2D = frente de Pareto (sweep + max corrente); em 3D → CDQ/sweep + set.
- **Point location.** Subdivisão planar → sweep persistente (set de segmentos persistente por x) ou trapezoidal map aleatorizado; em contests: offline com sweep e segtree/BIT por segmento. Para polígonos convexos: busca binária angular.
- **Maior retângulo vazio (maximum empty rectangle).** Pontos num retângulo: $O(n^2)$ por enumeração de pares de bordas com pilha, ou D&C/sweep em $O(n \log^2 n)$; canto inferior-esquerdo + histograma sobre colunas ordenadas. Gatilho: "maior retângulo sem pontos".
- **Maior círculo vazio.** Centro está num vértice de Voronoi (dentro do hull) ou na interseção da borda com arestas de Voronoi; vem do Delaunay.
- **Número de interseções de segmentos, pares que se cruzam.** Pares de cordas de círculo cruzam ⇔ extremos se intercalam: contagem de inversões/BIT; "interseções entre diagonais" = pares com (a < c < b < d).
- **Cordas e pontos no círculo.** Ordem circular + intercalação; para geometria com ≥ 3 pontos num círculo, indexe por ângulo (ordem) e problemas viram parênteses/intervalos (Catalan para não cruzamento).
- **Alturas/ângulos inteiros: tangente como fração.** Compare ângulos via cross em vez de atan2; ângulo entre direções ≤ 90° ⇔ dot ≥ 0; "ângulo do vetor em relação a outro" por cross/dot sem funções trigonométricas.
- **Contagem de pontos dentro de círculos/ retângulos por consultas offline.** Retângulo = 4 prefixos 2D (inclusão-exclusão), com sweep + BIT; círculos de raio fixo → grade de células de lado r e examinar 3×3 vizinhanças.
- **Geometria com float: epsilon relativo.** Compare $|a - b| \le \text{eps} \cdot \max(1, |a|, |b|)$; para ordens use comparador por cross inteiro quando as coordenadas são inteiras; evite acos fora de [−1, 1] (clamp) e sqrt de negativo pequeno (max(0, x)).
- **Geometria em inteiros: ordem de grandeza.** Coordenadas até 10^9: cross cabe em int64 ($\le 2 \cdot (2 \cdot 10^9)^2 \approx 8 \cdot 10^{18}$, perto do limite), mas produtos de cross (in-circle) exigem __int128 ou long double com cuidado; para interseção de retas em frações use __int128 em num/den.
- **Orientação robusta com long double.** Quando as coordenadas são float, o sinal de cross com |cross| < eps·|a||b| é duvidoso; reformule como inteiros escalados (multiplique por 10^k) se a entrada tem poucas casas decimais.
- **Áreas e probabilidade geométrica.** "Escolha pontos aleatórios em região" → área da região favorável / total; integral com Simpson para áreas de interseção de formas curvas (adaptativo se não for suave).

- **Simpson / integração numérica.** Função convexa/unimodal em um intervalo → ternária; áreas de união de círculos/elipses como integral ao longo de x do comprimento coberto (Simpson adaptativo).
- **Ternária e geometria.** "Distância mínima de um ponto em movimento" ou "posição do ponto em reta que minimiza soma de distâncias" é convexo em t → ternária (ou ponto de Fermat); quando a função é dist linear por partes, vale busca de ponto ótimo nos vértices.
- **Soma de distâncias mínima.** Ponto que minimiza $\Sigma|p - p_i|_1$ = mediana por eixo; $\Sigma|p - p_i|_2$ (Fermat/geometric median) → Weiszfeld ou ternária aninhada; Chebyshev (L∞) → centro da caixa envolvente alinhada (não rotacionada; a rotação de 45° apenas relaciona L1 e L∞).
- **Menor caixa/quadrado envolvendo pontos após rotação.** O ótimo ocorre quando um lado do retângulo é colinear com uma aresta do hull → calipers, O(n) após o hull.
- **Menor distância entre duas curvas/pontos convexos.** Funções convexas → ternária aninhada; polígonos → calipers; distância de polígonos ao ponto → min de pontos-segmento.
- **Cobertura por intervalos angulares.** "Um ponto vê quantos objetos" / "o laser cobre todos?" → cada objeto vira um intervalo angular; ordene eventos e faça sweep circular (duplique a lista em 2π para wrap).
- **Pontos visíveis do lado de dentro de uma torre / skyline.** Pilha monotônica sobre inclinações ou hull superior do perfil; problema "quais prédios são visíveis" = inclinação crescente (cross) em varredura.
- **Skyline / envelope superior de segmentos.** Sweep com set ou D&C unindo duas envoltórias; para retas, hull superior das retas (CHT).
- **Raio atravessa polígono convexo / segmento corta convexo.** Clipping em O(log n) por busca binária por tangente, ou O(n) clip por semiplanos; Cyrus–Beck para linha contra polígono convexo.
- **Recorte de polígono (Sutherland–Hodgman).** Clipar polígono por semiplano: percorra arestas e emita interseções; vale para polígono convexo contra semiplano e HPI incremental.
- **Triangulação de polígono (ear clipping / monótono).** Qualquer polígono simples tem triangulação com n − 2 triângulos; área = soma dos triângulos. Contagem de triangulações de polígono convexo = Catalan(n − 2).
- **Euler para planar e subdivisões.** V − E + F = 1 + C; contar regiões criadas por retas/segmentos: regiões = 1 + n + (nº de interseções) para retas em posição geral (R = 1 + n + I).
- **Retas e regiões.** n retas em posição geral dividem o plano em 1 + n + C(n, 2) regiões; n círculos em posição geral em $n^2 - n + 2$. Para cordas num círculo: 1 + C(n, 2) + C(n, 4) para n pontos em posição geral (diagonais).
- **Contagem de pontos de grid dentro de círculo (Gauss).** $\Sigma \lfloor\sqrt{(r^2 - x^2)}\rfloor$ em O(r); versão sublinear existe, mas a ideia padrão é varrer x.
- **Mínimo de $x^2 + y^2$ sob restrição de grid / retas inteiras.** Reduza com Euclides estendido para achar solução paramétrica $(x_0 + t \cdot b/g, y_0 - t \cdot a/g)$ e minimize função convexa em t.
- **Áreas de cruz de vetores e determinantes.** O sinal de determinante 2×2 testa dependência linear (paralelismo); em 3D o triplo produto = 0 ⇔ coplanares. Dimensão d: rank por Gauss.

Repetidas de outras seções:

- **Pontos de rede num segmento** → gcd(|dx|, |dy|) + 1; Pick com isso; círculo de Gauss O(r).
- **Dualidade ponto-reta ⇔ CHT / Li Chao** (hull superior de pontos = envelope inferior de retas); uso em DP (estruturas).
- **Dominância 2D/3D e frente de Pareto** → BIT/CDQ por coordenada (estruturas); frente de Pareto (não dominados) ≠ camadas convexas: um ponto do hull pode ser dominado (ex.: (0,0) com (1,1) presente).
- **Área da união de retângulos/segmentos** → sweep + segtree com (cobertura, comprimento coberto) (estruturas).

Aleatorização

Gatilho: "um elemento aparece em pelo menos metade" ou "a resposta é compartilhada por muitos" → sorteie e verifique.

- **Elemento frequente** (≥ n/2, ≥ n/k): escolha aleatória acerta com probabilidade ≥ 1/2 (≥ 1/k no caso n/k); repita 30–60 vezes (k vezes mais no caso n/k).
- **gcd de pelo menos metade** → sorteie um a_i, teste cada divisor dele.
- **Hashing aleatório de conjuntos/multiconjuntos**: atribua um número aleatório de 64 bits a cada valor; soma identifica o multiconjunto; xor só identifica conjunto/paridade ({a, a} = ∅), não multiplicidade.
- **XOR hashing**: "todo valor aparece um número par de vezes em [l, r]" → prefix xor dos hashes = 0.
- **Shuffle da entrada** para quebrar casos ruins (inserção em BST, incremental de círculo mínimo em O(n) esperado).
- **Color coding**: caminho simples de k vértices → pinte com k cores aleatórias e faça DP em máscara; a chance de um conjunto fixo ficar colorido é k!/k^k ≈ e^{−k}, então repita Θ(e^k) vezes.
- **Matriz/polinômio** → teste de Freivalds ou Schwartz-Zippel avaliando em ponto aleatório.

- **Anti-hack:** use mt19937_64 com semente de chrono e hash customizado no unordered_map.

adicionado em 2026-11-10 por agentes; revisado

- **Karger (corte mínimo global).** Contraia arestas aleatórias até restar 2 vértices; sucesso ≥ 2/(n(n − 1)); repita $O(n^2 \log n)$ vezes (ou Karger–Stein em $O(n^2 \log^3 n)$). Hoje Stoer–Wagner O(nm) é determinístico e mais simples para n ≤ 500.
- **Treap.** Prioridade aleatória no BST dá altura O(log n) esperada independentemente da entrada; split/merge por prioridades. É a base de sequências implícitas.
- **Hash polinomial seguro.** Módulo $2^{61} - 1$ com base aleatória em [2, p − 2]; dupla base mod $10^9 + 7$ não basta (colisão por par $\sim(n/p)^2 \approx 10^{-8}$ para $n = 10^5$, e em n^2 comparações isso colide); probabilidade de colisão por par ≤ n/p por módulo (grau do polinômio sobre o corpo). Para n^2 comparações use $2^{61} - 1$ com duas bases (ou dois primos grandes).
- **Anti-anti-hash.** Nunca use base fixa, mod 2^{64} natural (Thue–Morse quebra qualquer base), nem base pequena previsível; ler a base de chrono::steady_clock e de /dev/urandom/time. Em Codeforces sempre há hacks.
- **Hash de unordered_map customizado.** splitmix64 com semente aleatória: o padrão do GCC é atacável (primos de bucket); alternativamente use gp_hash_table com hash customizado ou map/sort.
- **Pollard rho.** Fatora $n \leq 10^{18}$ em O(n^(1/4)) esperado com ciclo de Floyd/Brent e gcd em lotes; combine com Miller–Rabin recursivo. Paradoxo do aniversário explica o √p.
- **Freivalds.** Verificar A·B = C em $O(n^2)$: sorteie $r \in \{0,1\}^n$ (ou do corpo) e compare A(Br) = Cr; erro ≤ 1/2 por rodada. Também verifica identidade de matrizes inversas e produtos de polinômios.
- **Schwartz–Zippel.** Polinômio não nulo de grau d em corpo F se anula em ponto aleatório com prob ≤ d/|F|. Testa identidade de polinômios, igualdade de multiconjuntos (Π(x − a_i)), determinante nulo (matching).
- **Matching por determinante (Tutte/Edmonds).** Matriz de Tutte com variáveis aleatórias mod p: grafo tem matching perfeito ⇔ det ≠ 0 (com alta prob., n par); o rank da matriz de Tutte é 2ν(G), logo o matching máximo é ν = rank/2, em $O(n^3)$ para grafos gerais.
- **Reservoir sampling.** Escolha uniforme de 1 em stream de tamanho desconhecido: o i-ésimo substitui com prob 1/i; k itens: i-ésimo entra com prob k/i substituindo um aleatório do reservatório.
- **Amostragem uniforme em estruturas.** Elemento uniforme de conjunto com remoções → vetor + swap-pop; uniforme em segtree → desça escolhendo filho com prob proporcional à soma; sorteio ponderado → prefix sums + binária ou alias method.
- **Las Vegas vs Monte Carlo.** Las Vegas: sempre certo, tempo aleatório (quicksort aleatório, Welzl, seleção); Monte Carlo: tempo fixo, erro com prob. pequena (Freivalds, Miller–Rabin). Amplifique Monte Carlo repetindo; converta para Las Vegas se há verificador barato.
- **Probabilidade de falha e repetições.** Se cada tentativa acerta com prob p, k = ⌈ln(1/ε)/p⌉ tentativas dão falha ≤ ε. Elemento que cobre ≥ 1/2: 30 tentativas ⇒ 2^{-30}; cobre ≥ 1/k: k·ln(1/ε) tentativas.
- **Sorteio por pares (amostragem de pares).** "Existe par/triple com propriedade comum a muitos pares" → sorteie pares e teste; se a fração de pares bons é δ, ~1/δ amostras bastam. Ex.: reta com ≥ n/2 dos pontos → sorteie 2 pontos sem reposição: a reta por eles acerta com prob (n−2)/(4(n−1)) < 1/4 (com reposição, ≥ 1/4).
- **Maioria/mediana aproximada por amostragem.** A mediana de uma amostra de k elementos está dentro de ε·n do verdadeiro com alta prob. se $k = O(1/\varepsilon^2 \cdot \log(1/\delta))$; serve para quantis, pivôs de seleção e para estimar contagens.
- **Shuffle em guloso e DP aleatório.** Quando a ordem de processamento arbitrária tem pior caso adversarial (ex.: inserções em BST, círculo mínimo, LP incremental), embaralhe a entrada: esperado O(n) ou O(n log n) pela análise de "elemento de trás" (probabilidade 3/i de mudar o ótimo).
- **Análise "backward".** O i-ésimo ponto muda a solução com prob ≤ d/i (d = tamanho da base); o custo esperado total é Σ d/i · custo(refazer) — sustenta Welzl, Seidel, closest pair randomizado e quicksort.
- **Randomized incremental.** Insira em ordem aleatória, repare quando o novo elemento viola: círculo mínimo, LP, hull 2D/3D (conflict graph), Delaunay, trapezoidal map, closest pair em grade, contagem de componentes.
- **Simulated annealing.** Soluções vizinhas por swap/perturbação; aceite pior com prob exp(−Δ/T), T decai geometricamente até o tempo limite; funciona em problemas de "maximize a pontuação" com TL 2–10 s e sem solução exata (AtCoder Heuristic, CF Marathon). Use avaliação incremental O(1) do delta.
- **Hill climbing com restarts.** Para otimização sem mínimos locais ruins: comece de ponto aleatório, melhore até estagnar, repita; guarde o melhor. Ótimo para problemas de geometria/continuo com função suave (centro de círculo mínimo, ponto ótimo).
- **Heurística de "gradiente por perturbação".** Para otimizar em $\mathbb{R}$^d sem derivada: passo de tamanho decrescente com tentativas aleatórias, redução do passo pela metade quando não melhora (útil p/ Fermat, círculo mínimo, ponto que minimiza o máximo).
- **Tempo como parâmetro.** Em heurísticas, itere "enquanto tempo < limite" com chrono; deixe 10% de folga. Pré-calcule tabelas de exp/log/rand para ganhar iterações, e use xorshift em vez de mt19937 no laço interno.

- **Local search para cobertura/coloração.** Mudar um elemento por vez com delta de custo O(grau), tabu list pequena ou "kick" aleatório quando preso; solução inicial gulosa aleatória.
- **Beam search.** Quando o espaço é uma árvore de decisões com função de avaliação razoável: mantenha as K melhores configurações por nível (hash para deduplicar); típico em heurísticas de AtCoder.
- **Greedy randomizado + melhor de N.** Execute guloso com desempates aleatórios N vezes e fique com o melhor; funciona para construtivos com TL alto e para aproximar problemas NP-difíceis pequenos.
- **Polynomial identity para string/permutação.** Checar "a e b são permutações um do outro" → compare $\Pi(x - a_i)$ em x aleatório mod p; ou soma de f(a_i) com f aleatória (hash aditivo). "Sequência é permutação de 1..k em [l, r]" → soma de hash = soma esperada com hash de prefixos.
- **Hash de árvores (isomorfismo).** Hash de subárvore = f(multiconjunto de hashes dos filhos) com f aleatória (p. ex. $\Pi(x + h_{filho}) \bmod p$ ou soma de $\varphi(h)$); duas árvores enraizadas iguais ⇔ hashes iguais com alta prob.; para não enraizadas, enraize no centro.
- **Hash de grafos / conectividade por xor.** Cada aresta recebe valor aleatório; vértice guarda xor das arestas incidentes; um conjunto S é uma componente fechada / corte vazio ⇔ xor dos vértices = 0 (se é corte o xor é sempre 0; a recíproca vale com alta probabilidade, não deterministicamente). Detecta corte e conectividade com remoções de arestas por xor de cortes.
- **Xor de valores aleatórios em árvore (cut hashing).** Dê a cada aresta que não está na árvore geradora um valor aleatório e a cada aresta da árvore o xor das de não-árvore que a cobrem; então arestas formam corte ⇔ xor = 0 com alta probabilidade (mincut com ≤ k arestas, pontes).
- **Contagem aproximada (Monte Carlo).** Estimar |S| amostrando e extrapolando; para contar elementos distintos em stream → HyperLogLog/k-mínimos (mantenha os k menores hashes; estimativa = k/h_k).
- **Seleção randomizada (quickselect).** Pivô aleatório → O(n) esperado para k-ésimo; mediana das medianas dá O(n) determinístico mas é pesado. std::nth_element já é introselect.
- **Sorteio com viés e "prefixo ponderado".** Para escolher índice com prob ∝ w_i com updates → BIT/segtree com descida; para o resultado só em O(1) amortizado → alias method com tabela reconstruída em lotes.
- **Aleatorização em grafos: coloração / separação.** Particionar vértices em dois lados aleatoriamente captura qualquer aresta/par fixo com prob 1/2 (ou 1/4 para dois); use para max-cut ≥ m/2, "MAX-3SAT ≥ 7/8" (MAX-2SAT ≥ 3/4; cláusula k: $1 - 2^{-k}$), e para color coding com repetição.
- **Probabilistic method (existência).** Se a esperança de uma contagem é ≥ k, existe configuração com ≥ k: "max-cut ≥ m/2", "tournament com caminho hamiltoniano", "sub-conjunto independente de tamanho ≥ n/(d + 1)". Desrandomize pelo método das esperanças condicionais.
- **Desaleatorização por esperança condicional.** Fixe uma variável por vez escolhendo o valor que mantém a esperança ≥ alvo: transforma o argumento probabilístico em algoritmo guloso determinístico.
- **Estimar com poucas amostras: aproximação por tamanho.** Se a resposta desejada é "existe vértice com grau ≥ d" e há muitos candidatos, amostrar por aresta aleatória escolhe vértices proporcionalmente ao grau.
- **Aleatorização em strings: mínima rotação / igualdade.** Igualdade de substring por hash; compare duas strings gigantes comparando poucos índices aleatórios só serve se diferem em muitas posições (teste de proximidade); para "diferem em ≥ δ n posições" uma amostra de O(1/δ) acha uma diferença.
- **Probabilidade e antichain: erro de "hash por soma".** Hash aditivo de multiconjunto falha para somas coincidentes; use 64 bits e valores aleatórios independentes (não valor = índice), nunca $f(x) = x^2$ etc.
- **Cuckoo / hash perfeito offline.** Para tabelas hash com acesso O(1) pior caso; em contest, use sort + binária, ou gp_hash_table; relevante só para constante.

Repetidas de outras seções:

- **Método probabilístico** → atribuição aleatória ≥ esperança; derandomize por esperanças condicionais.
- **Paradoxo do aniversário** → colisão com ~√N itens; Tutte matrix aleatória → emparelhamento máximo.
- **Treap e skip list** como estruturas de altura esperada log (estruturas).
- **Hash de subárvores/isomorfismo** → árvores (hash de multiconjunto de filhos).
- **Simulated annealing/hill climbing para ótimo contínuo** → geometria (círculo mínimo, Fermat, ponto que minimiza o máximo).

Álgebra, polinômios e FFT

adicionado em 2026-11-10 por agentes; revisado

Quando a contagem vira produto de polinômios, recorrência linear ou sistema linear, a ferramenta é álgebra: pense em coeficientes, não em objetos.

- **Convolução de somas.** "Quantos pares (i, j) com a_i + b_j = s para todo s" → polinômios A(x) = Σ x^{a_i}, B(x) = Σ x^{b_j}, A·B. Diferenças a_i − b_j → inverta B (x^{M−b_j}). Triplas/PAs de 3 termos: a_i + a_j = 2a_k → A^2 e olhe nos pontos 2a_k.
- **NTT: primos amigáveis.** 998244353 = 119·2^{23} + 1 (g = 3); 167772161 = 5·2^{25} + 1 (g = 3); 469762049 = 7·2^{26} + 1 (g = 3); 1004535809 = 479·2^{21} + 1 (g = 3); 754974721 = 45·2^{24} + 1 (g = 11). Tamanho máximo = 2^k do primo; convolução de tamanho > 2^{23} não cabe em 998244353.
- **Módulo arbitrário (MTT).** Três NTTs com primos distintos + CRT (Garner), pois o coeficiente real pode chegar a n·mod² ≈ 10^{24} (n = 10^6), que cabe no produto de três primos ≈ 10^{26}; ou FFT em double dividindo cada coeficiente em 15 bits (4 FFTs com o truque de empacotar parte real/imaginária).
- **Precisão da FFT em double.** Regra: n·(maior coeficiente)² $\lesssim 10^{14}$–10^{15} passa sem dividir (ex.: coeficientes ≤ 10^4 com n ≈ 10^6, ou ≤ 10^5 com n ≈ 10^5); coeficientes ≤ 10^6 com n ≈ 10^6 já não cabem em double; coeficientes até 10^9 exigem split ou NTT. Use raízes calculadas por cos/sin diretos (ou long double), não por multiplicação acumulada.
- **Convolução cíclica / tamanho não potência de 2.** Para mod $x^n - 1$ com n arbitrário: convolução linear e dobre; para DFT de tamanho primo ou arbitrário → Bluestein (chirp-z) usa ij = C(i+j,2) − C(i,2) − C(j,2).
- **Chirp-z.** Avaliar P em a·w^k, k = 0..m−1 (progressão geométrica) em O(n log n) com uma convolução: usa w^{jk} = w^{C(j+k,2)}/(w^{C(j,2)}·w^{C(k,2)}). Aparece em "avalie o polinômio em todas as raízes da unidade de ordem n qualquer".
- **Inverso de série.** B ← B·(2 − A·B) mod x^{2k} dobra a precisão (Newton); exige $a_0 \neq 0$. Custo total O(n log n) com constante ≈ 3 multiplicações (convolução de tamanho 2n).
- **ln, exp, sqrt, potência.** ln A = ∫ A′/A (a_0 = 1); exp (exige A(0) = 0): B ← B·(1 − ln B + A); sqrt: B ← (B + A/B)/2 (a_0 quadrado, normalmente 1); A^k = exp(k·ln A) para a_0 = 1; senão fatore x^t e a_0^k (k grande: guarde k mod p e k mod (p−1) separados).
- **Divisão de polinômios.** rev(Q) = rev(A)·$\text{rev}(B)^{-1}$ mod x^{n−m+1}, depois R = A − QB. É o bloco básico de multipoint e de "x^N mod f(x)".
- **Produto de muitos polinômios.** ∏(x − r_i) ou ∏(1 + a_i x) por D&C com pares do mesmo tamanho: O(n $\log^2$ n). Em problemas de "escolher subconjunto com peso", o produto ∏(1 + x^{w_i}) agrupa por peso e usa potência binomial.
- **Knapsack de contagem via ln/exp.** Subconjuntos de soma s (cada uma das cnt_w cópias entra 0/1): ∏(1 + x^w)^{cnt_w} = exp(Σ cnt_w·ln(1 + x^w)); mochila ilimitada (multiconjunto, cópias ilimitadas de cada peso): exp(Σ cnt_w Σ_k x^{wk}/k) = ∏(1 − x^w)^{−cnt_w}; limitada a cnt_w cópias por peso: ∏(1 − x^{w(cnt_w+1)})/(1 − x^w) — soma harmônica S log S para montar o argumento.
- **Partições.** p(n) para todos até N: 1/∏(1 − x^k) por pentagonal (Euler) em O(N√N): p(n) = Σ (−1)^{k+1}[p(n − k(3k−1)/2) + p(n − k(3k+1)/2)]. Partições em partes distintas = ∏(1 + x^k); em partes ≤ k: 1/∏_{i≤k}(1 − x^i). Valores: p(100) = 190 569 292.
- **Transformada de Euler.** Contagem de estruturas "multiconjuntos de componentes conexas" (não rotuladas): exp(Σ c_k·Σ_j x^{kj}/j); para rotuladas (EGF): todas = exp(conexas), conexas = ln(todas). Ex.: grafos conexos rotulados = n!·[x^n] ln(Σ 2^{C(n,2)} x^n/n!).
- **Funções geradoras como ferramenta.** Ordinária (OGF) para escolhas não ordenadas, exponencial (EGF) para arranjos rotulados (produto de EGFs = escolher quais rótulos vão em cada parte). Restrição "cada tipo aparece em {a,b,c} cópias" → polinômio por tipo e produto.
- **Lagrange inversion.** Se f(g(x)) = x, [x^n]g = (1/n)x^{n−1}; mais geral: [x^n]H(g) = (1/n)[x^{n−1}] H′(x)(x/f)^n. Dá árvores enraizadas (n^{n-1}), Catalan; série inversa composicional: por coeficiente com (x/f)^n via ln/exp custa O(n² log n) no total — para O(n $\log^2$ n), use Newton + composição rápida.
- **Composição e power projection.** f(g(x)) mod x^n em O(n $\log^2$ n) (Kinoshita–Li); a transposta é "power projection" [x^k] h·g^j para todo j. Gatilho: "aplique f repetidas vezes", "inversa composicional", "contagem com F(G(x))". Casos especiais são mais baratos: f(e^x), f(x + c) (shift de Taylor), f(a·x).
- **Taylor shift.** P(x + c) em uma convolução: b_j = (1/j!)·Σ_i (a_i·i!)·c^{i−j}/(i−j)!. Usos: converter base de potências ↔ falling factorials, Stirling de primeira espécie por dobra (x(x+1)…(x+n−1) → duplicar n usando shift).
- **Multipoint evaluation.** Avaliar P (grau n) em n pontos: árvore de produtos ∏(x − x_i), remainder tree descendo — O(n $\log^2$ n) com divisão por série inversa. Gatilho: "avalie f em x_1..x_m" para m ≈ 10^5.
- **Interpolação rápida.** Dados (x_i, y_i): P = Σ y_i/M′(x_i) · M(x)/(x − x_i); M′(x_i) sai de multipoint em M′; a soma sobe pela árvore de produtos (D&C). O(n $\log^2$ n). Em pontos 0..d é O(d) por Lagrange direto.
- **Lagrange em pontos consecutivos.** P(n) = Σ_i y_i · ∏_{j≠i}(n − j)/(i − j) com denominador i!(d−i)!(−1)^{d−i}; prefixos/sufixos de ∏(n − j) dão O(d) sem inverso por termo. Se n ≤ d, devolva y_n direto (evita divisão por 0).
- **Grau d → d+1 pontos.** Uma sequência é polinomial de grau ≤ d iff a (d+1)-ésima diferença finita é 0. Se f(n) = Σ_{i≤n} g(i) com g polinomial de grau d, f tem grau d+1 → d+2 pontos. Gatilho: "n ≤ 10^{18} mas a resposta parece polinomial": calcule d+2 valores por brute force e interpole.
- **Σ k^d.** Σ_{k=1}^n k^d é polinômio de grau d+1 em n → Lagrange com d+2 pontos em O(d) (d ≤ 10^6, n ≤ 10^{18}). Faulhaber: Σ_{k=0}^{n−1} k^d = (1/(d+1)) Σ_j C(d+1, j) B_j n^{d+1−j} (B_1 = −1/2).
- **Bernoulli por série.** Σ B_j x^j/j! = x/(e^x − 1) → inverso de série de (e^x − 1)/x; dá todos B_0..B_n em O(n log n).

- **$\sum k^d \cdot r^k$.** Para $r \neq 1$: $\sum_{k<n} P(k)r^k = r^n \cdot Q(n) - Q(0)$ com $\deg Q = \deg P$; ache Q por Lagrange ($d+1$ pontos, $Q(0)$ desconhecido via somas parciais). Aparece em "soma de $i \cdot x^i$" generalizada.
- **Potência de soma binomial.** $\sum_i C(n,i) \cdot i^k = \sum_j S(k,j) \cdot n^{\underline{j}} \cdot 2^{n-j}$ (Stirling de segunda espécie): n enorme, $k \leq 10^5$ → $S(k, \cdot)$ por convolução.
- **Stirling por FFT.** $S(n,k)$ para um n e todo k: $S(n,k) = (1/k!) \sum_j (-1)^{k-j} C(k,j)\, j^n$ → convolução de $j^n/j!$ com $(-1)^m/m!$. Primeira espécie (linha n): $x(x+1)\ldots(x+n-1)$ por D&C ou dobra com shift. Coluna k: $S(n,k) = n![x^n]\, x^k/k!$, via exp/ln.
- **Newton–Girard.** $k \cdot e_k = \sum_{i=1}^{k} (-1)^{i-1} e_{k-i} p_i$; em série: $\sum e_k x^k = \exp(\sum (-1)^{k-1} p_k x^k/k)$. Converte somas de potências ↔ coeficientes do polinômio com raízes dadas; p_k de raízes de uma recorrência = traço de M^k.
- **Convolução online (relaxed).** $f_n = \sum_{i<n} f_i \cdot g_{n-i}$ (+ termo): CDQ — resolva [l, m), contribua [l, m) × g[0..r−l) em [m, r) — $O(n \log^2 n)$. Quando ambos f e g são online, contribua nos dois sentidos com cuidado no bloco inicial. Cobre exp, inverso e DPs "$dp[n] = \sum dp[i] \cdot w[n-i]$".
- **Berlekamp–Massey.** De 2L termos de uma recorrência linear de ordem L (mod p primo) recupera os coeficientes em $O(L^2)$. Gatilho: "n-ésimo termo de algo definido por DP de estado pequeno" → calcule ~2·(estados) termos, BM, depois Kitamasa. Também recupera polinômio mínimo de matriz esparsa (Wiedemann).
- **Kitamasa / Bostan–Mori.** $[x^n]P(x)/Q(x)$ (exige $Q(0) \neq 0$; fatore x^d antes): multiplique por $Q(-x)$; denominador vira par, numerador seleciona paridade de n; $O(k \log k \log n)$ com FFT ($O(k^2 \log n)$ ingênuo). Vence matriz k×k ($k^3 \log n$) quando $k \geq 50$.
- **Recorrência linear ↔ matriz.** Termo n de recorrência de ordem k: $x^n \bmod (x^k - c_1 x^{k-1} - \ldots - c_k)$, depois combine com os primeiros k termos. Ordem k pequena e várias queries n: precompute $x^{2^j} \bmod f$.
- **Polinômio característico.** $\det(xI - A)$ por Hessenberg em $O(n^3)$ mod p. Cayley–Hamilton: A^n = polinômio de grau $< n$ em A; permite $A^n \cdot v$ com Kitamasa em $O(n^2 \log n)$ usando só produtos matriz-vetor (matriz esparsa).
- **Matriz de adjacência e passeios.** $(A^k)_{uv}$ = nº de passeios de comprimento k; $\operatorname{tr}(A^k)$ = passeios fechados. Soma de comprimentos 0..k: matriz aumentada [[A, I],[0, I]] elevada a k+1. Contar ciclos de comprimento 3 = $\operatorname{tr}(A^3)/6$; ≤ 4 por M^k com estado (anterior).
- **Exponenciação min-plus / semi-anel.** Mesmo princípio em (min, +) ou (max, ×) para caminhos mínimos com exatamente k arestas; custo $n^3 \log k$. Se a matriz é circulante, o produto vira convolução (min,+) — não há FFT para esses semi-anéis; continue $O(n^2)$ por produto (ou explore estrutura especial).
- **Gauss mod p.** Sistema Ax = b, determinante, posto, inversa em $O(n^3)$ com pivô por busca de não-zero (mod primo sempre vale). Det = produto dos pivôs × $(-1)^{\text{trocas}}$. Mod composto: use Euclides entre linhas (como gcd).
- **Gauss em GF(2) com bitset.** n ≤ 2000–5000 variáveis: $O(n^3/64)$. Número de soluções = $2^{n - \text{posto}}$ (se consistente). Lights-Out, "xor de subconjunto = alvo", "quantos xors distintos" (2^{posto}) são o mesmo problema.
- **Sistema com módulo composto ou reais.** Reais: pivô parcial (maior |a|) e eps; módulo m composto: Gauss com troca por gcd; módulo 2^k: pivô de menor valuação 2-ádica.
- **Determinante como contagem.** Kirchhoff: nº de árvores geradoras = det de qualquer menor principal do Laplaciano (peso = produto de pesos; multigrafo ok). K_n: n^{n-2}; $K_{a,b}$: $a^{b-1} b^{a-1}$. Árvores contendo arestas fixas: contraia; arborescências: Laplaciano de entrada/saída.
- **Emparelhamento via determinante.** Bipartido: perm(A) é #P-difícil mas det mod 2 = perm mod 2; existência de perfeito = det da matriz de Edmonds com variáveis aleatórias ≠ 0 (Schwartz-Zippel). Geral: posto da matriz de Tutte aleatória = 2·ν(G) mod p grande ($O(n^3)$, mais simples que blossom).
- **Contagem de emparelhamentos perfeitos em planar.** FKT / Pfaffian com orientação Pfaffian — gatilho: "contar dominós / tilings em grid com buracos" → Pfaffian ou, em grid retangular, determinante com sinais; ou DP de perfil se a largura é pequena.
- **Permanente.** Ryser: $\operatorname{perm}(A) = \sum_S (-1)^{n-|S|} \prod_i \sum_{j \in S} a_{ij}$ em $O(2^n \cdot n)$ (com Gray code); n ≤ 20–25.
- **Determinante com parâmetro.** $\det(A + xB)$ é polinômio de grau ≤ n em x → avalie em n+1 pontos e interpole; ou resolva generalizado (B inversível: $\det B \cdot \det(B^{-1}A + xI)$ = polinômio característico). Gatilho: "árvores geradoras com exatamente k arestas vermelhas".
- **Atualização de posto 1.** $\det(A + uv^T) = \det A \cdot (1 + v^T A^{-1} u)$; Sherman–Morrison: $(A + uv^T)^{-1} = A^{-1} - A^{-1}uv^T A^{-1}/(1 + v^T A^{-1} u)$. Troca de uma linha/coluna → $O(n^2)$ por atualização em vez de $O(n^3)$.
- **Cauchy–Binet.** $\det(AB) = \sum_S \det(A_{\cdot,S}) \cdot \det(B_{S,\cdot})$ (A é n×m, B m×n, $m \geq n$). Prova Kirchhoff e converte "soma sobre subconjuntos de colunas de determinantes" em um único determinante.
- **Base de espaço vetorial (xor basis) generalizada.** Inserir vetores mod p numa base escalonada: dependência = reduz a 0; mantenha os "mais recentes" (base com timestamp) para consultas em janela [l, r] (prefix linear basis). Também vale para matroide linear.
- **Resultante.** $\operatorname{Res}(f,g) = a_n^m \prod g(\alpha_i)$ é 0 sse há raiz comum; discriminante = $(-1)^{n(n-1)/2} \cdot a_n^{-1} \cdot \operatorname{Res}(f,f')$. Em competições: eliminar uma variável de duas equações polinomiais, número de raízes comuns, "f e g têm fator comum?" (gcd de polinômios) — gcd rápido por half-gcd $O(n \log^2 n)$; Euclides $O(n^2)$ basta para $n \leq 5000$.

- **Zeta/Möbius de subconjuntos.** Soma sobre subconjuntos (ou superconjuntos) em $O(2^n n)$; Möbius é a inversa (troca + por −). OR-convolução = zeta → produto ponto a ponto → Möbius; AND-convolução = zeta de superconjuntos.
- **Walsh–Hadamard (XOR-convolução).** Transformada H de tamanho 2^n em $O(2^n n)$; produto ponto a ponto; inversa divide por 2^n. k-ésima potência (nº de maneiras de escolher k valores ordenados, com repetição, com xor x): eleve cada ponto à k (k grande). Mod p ímpar: divisão por 2^n ok.
- **Convolução de gcd/lcm e Dirichlet.** h(n) = Σ_{gcd(a,b)=n} f(a)g(b): zeta sobre múltiplos, produto, Möbius sobre múltiplos; lcm: sobre divisores. "Soma sobre divisores/múltiplos" em O(N log log N) fazendo uma passada por primo.
- **Coloração por IE de subconjuntos.** Cobertura de V por k conjuntos independentes = Σ_S $(-1)^{n-|S|}\cdot i(S)^k$ onde i(S) = nº de subconjuntos independentes de S (zeta de [independente]); a soma conta coberturas com sobreposição, não k-colorações próprias, mas χ(G) ≤ k iff o valor é > 0. Para #k-colorações próprias, use subset convolution de [independente] ou IE sobre arestas: Σ_{F⊆E} $(-1)^{|F|}\cdot k^{c(F)}$ (c(F) = nº de componentes de (V, F)). n ≤ 20–23.
- **Polinômio cromático e de grafos.** P(G, k) tem grau n; calcule em n+1 valores de k (DP exponencial ou IE) e interpole; contagem de orientações acíclicas = |P(G, −1)|. Polinômio de Tutte é #P-difícil, mas é fácil em largura de árvore pequena.
- **Holonômico (P-recursivo).** Sequências com recorrência de coeficientes polinomiais (Catalan, desarranjos, Motzkin): "adivinhe" a recorrência com álgebra linear em ~poucos termos e rode em O(n); para $n \approx 10^{12}$ mod p use passos de √n (n! mod p em O(√p log p) por avaliação deslocada de ∏(x + i)).
- **Hash de polinômios / verificação.** Dois polinômios de grau ≤ d iguais se coincidem em d+1 pontos; teste de identidade por avaliação em ponto aleatório mod p grande (erro ≤ d/p). Serve para "produto de matrizes está certo?" (Freivalds) e "duas expressões são iguais?".
- **Soma em k-ésimas raízes.** Para contar subconjuntos com soma ≡ r (mod m) com m pequeno: polinômio em $Z[x]/(x^m - 1)$ elevado por produto de n termos $(1 + x^{a_i})$ — multiplicação cíclica $O(m^2)$ ou NTT quando m | p − 1.
- **Contagem de pares em disco/soma com limite.** Σ_{i<j} [a_i + a_j ≤ T] em valores pequenos: convolução do histograma e prefix sum; para valores grandes, ordene + two pointers (não precisa de FFT).
- **Casamento com curinga e variantes.** Σ_i $(p_i - t_{s+i})^2 p_i t_{s+i} = 0$ (curinga = 0) usa 3 convoluções; casamento "≤ k erros por símbolo" faz uma convolução por símbolo (alfabeto pequeno) → |Σ| FFTs.
- **Convolução de funções em grid 2D / multidimensional.** Mapeie (i, j) → i·W' + j com W' ≥ 2W − 1 para convolução 2D com uma FFT 1D; vale para k dimensões com strides.
- **Matriz de Vandermonde e interpolação.** Resolver Σ c_j x_i^j = y_i (sistema de Vandermonde) é interpolação: $O(n^2)$ ou $O(n \log^2 n)$; transposto (Σ c_j x_i^j dado c) é multipoint. Sistema "somas de potências p_1..p_k conhecidas → valores" = Newton + fatorar polinômio mod p.
- **Fatorar polinômio mod p.** Raízes em F_p: gcd(f, $x^p - x$) e Cantor–Zassenhaus aleatório (gcd com $(x + a)^{(p-1)/2} - 1$); gatilho: "ache x tal que f(x) ≡ 0 (mod p)" com f de grau pequeno, ou raiz quadrada/k-ésima mod p (Tonelli–Shanks, Cipolla).
- **Logaritmo e raiz discreta.** BSGS para $a^x \equiv b$ em O(√m); k-ésima raiz módulo primo: ache gerador g, resolva log, divida por gcd(k, p−1). Gatilho: "resolva $x^k \equiv a$ mod p".
- **Matriz diagonalizável e Fibonacci.** F_n por dobra: $F_{2k} = F_k(2F_{k+1} - F_k)$, $F_{2k+1} = F_{k+1}^2 + F_k^2$; gcd(F_a, F_b) = F_{gcd(a,b)}; Pisano $\pi(10^k) = 15\cdot 10^{k-1}$ (k ≥ 3).
- **Sparse + Wiedemann.** Det, posto ou solução de sistema esparso $n \approx 10^5$ com nnz pequeno: BM em $u\cdot A^i\cdot v$ (2n termos, cada um é um produto matriz-vetor de custo nnz) → O(n·nnz); BM escalar só dá o polinômio mínimo da projeção — det/posto exigem pré-condicionamento aleatório ou block Wiedemann.

Matemática de olimpíada e math.stackexchange

adicionado em 2026-11-10 por agentes; revisado

Muita prova de olimpíada é, no fundo, um algoritmo ou uma garantia de existência: ache o argumento extremal, o invariante ou a desigualdade que corta a busca.

- **Princípio extremal.** Pegue o elemento máximo/mínimo/mais longo/mais à esquerda: ele é forçado e reduz o caso a n − 1. Ex.: torneio sempre tem caminho hamiltoniano (Rédei) — insira vértices por varredura linear na ordem (O(n) por inserção, $O(n^2)$); Sylvester–Gallai; "o pior vértice não pode piorar".
- **Menor contraexemplo.** Para provar P(n) para todo n, assuma o menor n que falha e derive contradição; algoritmicamente: o primeiro erro de um guloso é o contraexemplo mínimo, e vira uma checagem local.
- **Pombal (pigeonhole).** n+1 prefixos módulo n → algum bloco contíguo tem soma ≡ 0 (mod n); Erdős–Ginzburg–Ziv: em 2n − 1 inteiros há n com soma ≡ 0 (mod n). Gatilho: "sempre existe" → a resposta é só construir (DP em restos).
- **Pombal por limiar.** Se há mais de k·m itens em m categorias, alguma tem > k; se Σ ≥ T então max ≥ T/m. Usado para limitar o número de iterações ("resposta ≤ √n" ou "≤ log n").

- **Erdős–Szekeres.** Toda sequência de rs + 1 números distintos tem crescente de tamanho r + 1 ou decrescente s + 1; $n^2 + 1$ distintos → monótona de tamanho n + 1. Gatilho: "LIS ≥ k ou LDS ≥ k" é garantia, só construa.
- **Dilworth / Mirsky.** Min cadeias que cobrem o poset = max antichain; min antichains = maior cadeia. Cobertura mínima por cadeias em DAG transitivo = n − emparelhamento máximo; antichain máxima = n − emparelhamento máximo no fecho (König). Em permutações: cobrir por crescentes = LDS.
- **Invariantes clássicos.** Coloração de tabuleiro (xadrez, 3 cores em diagonais), soma mod k, xor total, paridade de inversões (puzzle de 15: solucionável iff paridade da permutação + linha do vazio batem), gcd, soma de pesos ponderados. Invariante completo = classificação em classes de equivalência.
- **Monovariantes.** Uma quantidade inteira limitada que muda em cada passo (soma de quadrados, número de inversões, número de pares ruins) prova término e limita o nº de passos: operação "troque se vale a pena" termina em ≤ C(n,2) trocas se cada troca reduz inversões.
- **Descida infinita e Vieta jumping.** $a^2 + b^2 = k(ab + 1)$ com inteiros positivos ⇒ k é quadrado (IMO 1988/6): de uma solução (a, b) com a ≥ b, a outra raiz em a é $a' = kb - a = (b^2 - k)/a$, menor; descendo chega-se a $a' = 0$. Gatilho: equação quadrática simétrica em duas variáveis; algoritmicamente é Euclides.
- **Indução forte e estrutura recursiva.** Se o problema para n sai de n/2, n − 1 ou "tirar um vértice extremal", a construção/DP tem forma recursiva; ótimo para provar que um guloso ou construção funciona (e dá o algoritmo).
- **AM-GM para otimizar.** Σ x_i fixa, ∏ x_i máximo quando todos iguais; inteiros: iguais ±1. Partir n em parcelas para maximizar produto → use 3s (com resto 4 = 2 + 2 ou 2): n mod 3 = 0: 3^{n/3}; 1: 4·3^{(n−4)/3}; 2: 2·3^{(n−2)/3}.
- **Cauchy–Schwarz e Engel.** $(\Sigma a_i b_i)^2 \le \Sigma a_i^2 \cdot \Sigma b_i^2$; $\Sigma a_i^2/b_i \ge (\Sigma a_i)^2/\Sigma b_i$. Minimizar Σ c_i/x_i com Σ x_i = S → $x_i \propto \sqrt{c_i}$ (igualdade da desigualdade); aloca recursos com retornos decrescentes.
- **Rearrangement e Chebyshev.** Σ a_i b_{σ(i)} é máximo com ordens iguais e mínimo com ordens opostas; para f convexa, Σ f(a_i − b_σ(i)) é mínimo com ordens iguais e máximo com ordens opostas (direção oposta à do produto). Gatilho: "emparelhe dois arrays minimizando Σ|a_i − b_σ(i)|" → ordene ambos.
- **Jensen e distribuição igual.** f convexa: Σ f(x_i) com Σ x_i fixa é mínima quando todos iguais (inteiros: ⌊S/k⌋ ou ⌈S/k⌉). Ex.: minimizar pares Σ C(g_i, 2) ao dividir n em k grupos; Mantel/Turán por convexidade.
- **Telescopia.** 1/(i(i+1)) = 1/i − 1/(i+1); Σ i·i! = (n+1)! − 1; Σ $(i^3 - (i-1)^3)$; produtos ∏(1 + 1/i) = n + 1. Gatilho: termo que "cancela com o vizinho": procure f(i+1) − f(i).
- **Vieta e Newton.** Raízes de um polinômio mônico: soma = $-a_{n-1}$, produto = $(-1)^n a_0$; relações simétricas se escrevem por e_k; p_k satisfaz a recorrência linear do polinômio, logo p_k de raízes algébricas = $\mathrm{tr}(M^k)$ da matriz companheira.
- **Identidades binomiais úteis.** Vandermonde Σ_i C(a,i)C(b,k−i) = C(a+b,k); hockey stick Σ_{i=r}^n C(i,r) = C(n+1,r+1); Σ_k k·C(n,k) = n·2^{n−1}; Σ_k $C(n,k)^2$ = C(2n,n); C(n,k)C(k,j) = C(n,j)C(n−j,k−j). Gatilho: "soma sobre subconjuntos" → reduza o somatório antes de implementar.
- **Recorrências lineares: solução fechada.** Característica r^n: raízes r_i distintas → Σ c_i r_i^n; raiz dupla → $(c_1 + c_2 n) r^n$; a_n = c·a_{n−1} + d: desloque pelo ponto fixo. Útil para cortar um expoente: Fibonacci mod m tem período (Pisano), a_n mod m é periódico com período ≤ m^2.
- **Quadrado de Fibonacci e identidades.** Cassini $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$; $F_{m+n} = F_{m+1}F_n + F_m F_{n-1}$; Σ_{i≤n} F_i = F_{n+2} − 1; Zeckendorf: todo inteiro é soma única de Fibonacci não consecutivos (guloso). Gatilho: jogo de Fibonacci Nim.
- **Floor sum.** f(n,m,a,b) = Σ_{i=0}^{n−1}⌊(ai + b)/m⌋ em O(log m) por Euclides: se a ≥ m ou b ≥ m, extraia (a/m)·n(n−1)/2 + (b/m)·n; senão troque papéis: y_max = ⌊(a(n−1) + b)/m⌋, soma = (n−1)·y_max − f(y_max, a, m, m − b − 1). Versões com Σ i·⌊·⌋ e Σ $\lfloor\cdot\rfloor^2$ também em log.
- **Pontos de rede sob reta.** #{(x,y) inteiros: 1 ≤ x ≤ n, 1 ≤ y ≤ ax/m + b} = floor sum; área e polígonos de rede com Pick e floor sum fecham sem iterar. Gatilho: n ≈ 10^9–10^{12} com "quantos pontos sob/sobre a reta".
- **Hermite e floor úteis.** Σ_{k=0}^{n−1}⌊x + k/n⌋ = ⌊nx⌋; ⌊⌊x/a⌋/b⌋ = ⌊x/(ab)⌋; ⌈a/b⌉ = ⌊(a + b − 1)/b⌋; Legendre: $v_p(n!)$ = Σ_k ⌊n/p^k⌋ = (n − s_p(n))/(p − 1) com s_p = soma dos dígitos em base p.
- **Beatty.** Se 1/α + 1/β = 1 com α irracional > 1, as sequências ⌊nα⌋ e ⌊nβ⌋ particionam os naturais. Posições P do Wythoff Nim: (⌊nφ⌋, ⌊$n\varphi^2$⌋) com φ = (1+√5)/2. Gatilho: duas progressões "quase aritméticas" que cobrem tudo sem sobreposição.
- **Sequências equidistribuídas.** {nα} é equidistribuída para α irracional; teorema dos três intervalos: os pontos {kα}, k < N, dividem o círculo em intervalos de ≤ 3 comprimentos distintos. Aproximação de Dirichlet: ∃ q ≤ N com |qα − p| < 1/N. Gatilho: "gire por α a cada passo" → frações contínuas.
- **Frações contínuas / Stern–Brocot.** Melhor aproximação racional p/q com q ≤ N: convergentes e semiconvergentes; busca por p/q entre dois racionais com menor denominador: desça Stern–Brocot em O(log) com saltos. Gatilho: "menor fração em (a/b, c/d)".
- **Funções aritméticas.** Σ_{d|n}φ(d) = n; Σ_{d|n}μ(d) = [n = 1]; Σ_{i≤n}gcd(i, n) = Σ_{d|n} d·φ(n/d); Σ_{i≤n}τ(i) ≈ n ln n; densidade de pares coprimos = $6/\pi^2$ ≈ 0,608. Multiplicativa → crivo linear; Dirichlet: (f∗1) é soma sobre divisores.

- **Soma de dois quadrados.** $n = a^2 + b^2$ iff todo primo $\equiv 3 \pmod 4$ tem expoente par; $r_2(n) = 4(d_1(n) - d_3(n))$ (divisores $\equiv 1$ e $3 \bmod 4$). Gaussianos inteiros: norma multiplicativa. Gatilho: "pontos de rede a distância $\sqrt{n}$ da origem".
- **Números complexos como rotações.** Multiplicar por e^{iθ} rotaciona; z^k/ potências de i para movimentos periódicos; vetores 2D como complexos simplificam rotação/escala. Trigonometria exata: use racionais (vetores inteiros), nunca graus.
- **Esperança por indicador e linearidade.** $E[X] = \Sigma P(\text{evento}_i)$, mesmo dependentes. Registros em permutação aleatória: H_n; ciclos de permutação aleatória: H_n; pontos fixos: 1; cupom: $n \cdot H_n$; profundidade média em BST aleatório $\approx 2 \ln n$ (altura $\approx 4{,}3 \ln n$); E[comprimento da LIS] $\approx 2\sqrt{n}$.
- **Cadeias e martingais.** Ache potencial Φ com $E[\Phi(\text{próximo}) - \Phi(\text{atual})] = -1$ por passo: então E[tempo] = Φ(início) − Φ(fim). Passeio aleatório em linha de 0..n: E[tempo] = i(n − i); ruína do jogador: P(chegar a n) = i/n (caso justo). Gatilho: "esperança de passos até terminar" com estrutura linear.
- **Independência: Caro–Wei.** $\alpha(G) \geq \Sigma_v 1/(\deg(v) + 1)$ — guloso (tire o vértice de menor grau e seus vizinhos) alcança. Turán: $\alpha \geq n/(\bar{d} + 1)$. Gatilho: "achar conjunto independente de tamanho ≥ n/(d+1)" ou "ache vértices sem arestas entre si".
- **Mantel / Turán e limites de m.** Sem triângulo $\Rightarrow m \leq \lfloor n^2/4 \rfloor$; sem $K_{r+1} \Rightarrow m \leq (1 - 1/r)n^2/2$. Sem C_4 (ou: dois vértices têm ≤ 1 vizinho comum) $\Rightarrow m = O(n\sqrt{n})$ e $\Sigma_v C(\deg v, 2) \leq C(n, 2)$: iterar pares de vizinhos é $O(n^2)$. Gatilho: "grafo sem ciclo de comprimento 4".
- **Grafos planares.** $m \leq 3n - 6$ e há vértice de grau ≤ 5 (degenerescência ≤ 5): triângulos/K_4 em O(n), orientação com grau de saída ≤ 5, coloração com 6 cores em O(n). Euler: V − E + F = 1 + C. Fluxo s–t = corte mínimo; no planar, corte mínimo ↔ ciclo mínimo no dual que separa s e t. Se s, t estão na mesma face, divida a face com uma aresta st e o corte vira o caminho mínimo no dual entre as duas faces novas (sem usar a aresta dual de st).
- **Euler em mapas.** V − E + F = 1 + C: contar componentes/buracos de uma figura em grid pelas células, arestas e vértices (ou pelo número de Euler local por 2×2) dispensa BFS; vale para qualquer desenho planar.
- **Ramsey.** R(3,3) = 6: em qualquer 2-coloração de K_6 há triângulo monocromático; R(4,4) = 18. Se n é ≥ ~6 a resposta "existe" é sempre sim; também, um grafo ou seu complemento é conexo — gatilho de construção em duas cores.
- **Dirac / Ore.** Grau mínimo ≥ n/2 ⇒ hamiltoniano; Ore: deg u + deg v ≥ n para não adjacentes. Construção $O(n^2)$ por rotações de Pósa. Gatilho: "grafo denso, ache ciclo hamiltoniano" → sempre existe, só constrói.
- **Hall e König.** Perfeito bipartido existe iff $|L| = |R|$ e $|N(S)| \geq |S|$ para todo $S \subseteq L$ (ou Hall nos dois lados); deficiência: $\nu = |L| - \max_S(|S| - |N(S)|)$. König: cobertura mínima = emparelhamento máximo; indep. máximo = n − ν. Grafo bipartido k-regular decompõe-se em k emparelhamentos perfeitos; arestas colorem-se com Δ cores (König).
- **Tutte / Tutte–Berge.** Perfeito iff para todo S, componentes ímpares de G − S ≤ |S|; $\nu = (n - \max_S(\text{odd}(G - S) - |S|))/2$. Gatilhos: árvore tem perfeito iff, removido cada vértice, sobra exatamente uma componente de tamanho ímpar (greedy a partir das folhas); grafo conexo com número par de arestas decompõe-se em caminhos de 2 arestas.
- **Gallai e coberturas.** α + τ = n (indep. + cobertura de vértices); ν + ρ = n (emparelhamento + cobertura de arestas, sem isolados). Cobertura de caminhos disjuntos em DAG = n − ν da bipartição entrada/saída.
- **Sequências gráficas.** Erdős–Gallai: $d_1 \geq \ldots \geq d_n$ é gráfica iff Σ par e $\Sigma_{i \leq k} d_i \leq k(k-1) + \Sigma_{i>k} \min(d_i, k)$; construção por Havel–Hakimi (ligue o maior aos próximos maiores). Torneios: Landau — ordenadas $s_1 \leq \ldots \leq s_n$, $\Sigma_{i \leq k} s_i \geq C(k,2)$ com igualdade em k = n. Bipartido: Gale–Ryser.
- **Helly (1D e geral).** Intervalos que se intersectam dois a dois têm ponto comum: $\max(l_i) \leq \min(r_i)$. Em R^d convexos: basta d + 1. Gatilho: "todos os pares de intervalos se cruzam" ⇒ existe um ponto comum; verifique só $\max(l_i) \leq \min(r_i)$. No plano, segmentos que se cruzam dois a dois não têm ponto comum em geral.
- **Extremal / Sperner / EKR.** Antichain de subconjuntos de [n] tem ≤ $C(n, \lfloor n/2 \rfloor)$ (Sperner); k-subconjuntos que se intersectam par a par têm ≤ C(n−1, k−1) (Erdős–Ko–Rado, n ≥ 2k). Gatilho: "família máxima sem inclusão / com interseção" → resposta fechada.
- **Hipergrafos de intervalos e grafos de intervalo.** Cor mínima = clique máxima = máxima sobreposição (grafo de intervalos é perfeito); cordal e perfeito; ordem de eliminação simplicial. Escalonamento de k máquinas = sweep com heap.
- **Teorema de Pick e variantes.** A = I + B/2 − 1 com B = Σ gcd(|Δx|, |Δy|) sobre arestas do polígono. Triângulo de rede sem pontos internos nem de fronteira (além dos vértices) tem área 1/2; contagem de I a partir de A e B dispensa varredura.
- **Geometria analítica útil.** Reta ax + by = c com gcd(a, b) = 1; distância ponto–reta $|ax_0 + by_0 - c|/\sqrt{(a^2 + b^2)}$; pontos de rede em segmento = gcd(|Δx|, |Δy|) + 1; área = (1/2)|cross|; compare quadrados (inteiros) em vez de distâncias.
- **Desigualdade triangular como poda.** $d(u, w) \leq d(u, v) + d(v, w) \Rightarrow$ em árvore, o ponto mais distante de qualquer v é um dos extremos do diâmetro (exato); em métrica geral isso falha, mas $\text{ecc}(v) \leq 2 \cdot \max(d(v, a), d(v, b))$ para a, b extremos de um diâmetro (2-aprox). Em Manhattan, soma de distâncias é separável por coordenada; mediana minimiza.
- **Schwartz–Zippel e grau ≤ d.** Polinômio não nulo de grau d em n variáveis tem raiz em S^n com probabilidade ≤ d/|S| (teste de identidade); em 1 variável, ≤ d raízes: dois polinômios de grau ≤ d concordantes em d + 1 pontos são iguais. Fundamento de hashing de polinômio e verificação de matrizes.

- **Ordem de grandeza.** $H_n \approx \ln n + 0{,}5772$; $n! \approx \sqrt{(2\pi n)}(n/e)^n$; $\log_2(n!) \approx n \log_2 n - 1{,}44n$ (cota inferior de comparações/queries); $C(2n,n) \approx 4^n/\sqrt{(\pi n)}$; Catalan $\approx 4^n/(n^{3/2}\sqrt{\pi})$; $\pi(x) \approx x/\ln x$ ($\pi(10^6) = 78\,498$, $\pi(10^9) = 50\,847\,534$); $p(100) = 190\,569\,292$; $\text{lcm}(1..43) \approx 9{,}42 \cdot 10^{18}$ (estoura int64 com sinal; cabe em uint64).
- **Cota de informação.** Se há M respostas possíveis e cada query tem b respostas, são necessárias $\geq \lceil \log_b M \rceil$ queries; em interativos o limite do enunciado é quase sempre isso — descubra a estratégia que divide M igualmente.
- **Limites de crescimento.** $\Sigma_{i \leq n}\ 1/i$ gera $n \cdot H_n \approx n \ln n$ para laços "para cada i, múltiplos de i"; soma de divisores $\sigma(n) \leq n \cdot H_n$; nº de divisores de n é $O(n^{\varepsilon})$; $\Sigma\ \tau(i)$ para $i \leq 10^7 \approx 1{,}6 \cdot 10^8$.
- **Ponto flutuante.** Evite subtrair números próximos (use fórmulas alternativas: hypot, atan2, log1p, expm1); compare por razão cruzada em inteiros (__int128 para até $10^{18} \times 10^{18}$); binária em reais: itere 60–100 vezes em vez de usar eps; √ inteira: s = sqrtl(x) e ajuste nos dois sentidos (s++ enquanto $(s+1)^2 \leq x$; s−− enquanto $s^2 > x$), com s^2 em __int128; soma de muitos termos: Kahan ou ordene por magnitude.
- **Provar guloso por desigualdade.** Para checar uma ordem: escreva custo de trocar adjacentes i, j e reduza a uma comparação de pares (ex.: $a_i \cdot b_j$ vs $a_j \cdot b_i$ → ordene por a/b; a+b vs b+a em strings). Se a comparação é transitiva e total, ordenar é correto.
- **Truques de somatório.** Troque a ordem $\Sigma_i \Sigma_{j|i} = \Sigma_j \Sigma_{k \leq n/j}$; use $\Sigma_{i<j} f = (1/2)(\Sigma_{i,j} - \Sigma_i f(i,i))$ para simetria; $(\Sigma x)^2 = \Sigma x^2 + 2\Sigma_{i<j} x_i x_j$ para tirar pares; expanda produtos e agrupe por potência $(\Sigma_i \Sigma_j (a_i - a_j)^2 = 2n\Sigma a^2 - 2(\Sigma a)^2)$.
- **Variância e pares.** $\Sigma_{i<j}(a_i - a_j)^2 = n\Sigma a_i^2 - (\Sigma a_i)^2$: soma de quadrados das diferenças em O(n); para absoluta $|a_i - a_j|$, ordene e use prefixos.
- **Convexidade e extremos.** Máximo de função convexa sobre politopo ocorre num vértice; mínimo de função côncava idem. Gatilho: "função convexa em intervalo fechado, ache o máx" → só extremos; minimizar côncava = testar vértices.
- **Subseqüências e somas de 2 classes.** Sempre que uma operação preserva "soma mod k" e o alvo exige outro resto → impossível; se a operação gera mod k × restos de uma base, procure classes de equivalência por gcd (ex.: valores alcançáveis = múltiplos de gcd dentro dos limites).
- **Moedas / Frobenius.** Com a, b coprimos, o maior valor não representável é $ab - a - b$ e há $(a-1)(b-1)/2$ não representáveis. Muitas moedas: Dijkstra nos restos módulo a menor moeda (O(a_min·n log)); valores acima de max_r dist[r] (o maior dos menores representáveis por resíduo) são todos alcançáveis; com gcd > 1, só múltiplos do gcd.
- **Prova por colorir.** Dominós em tabuleiro: cobertura perfeita exige #brancas = #pretas; coloração 2×2 ou por linhas mostra impossibilidade de L-triominós/tetraminós. Gatilho: "cobrir com peças de forma fixa" → teste invariantes de coloração antes de procurar construção.
- **Contagem dupla.** Conte pares (a, b) de duas formas: $\Sigma_v \deg(v) = 2m$; $\Sigma_v C(\deg(v), 2)$ = #caminhos de comprimento 2; $\Sigma_S |S \cap T|$. Útil para limites (existe v com grau ≥ média) e para identidades.
- **Pareamento de somas e simetria (Gauss).** $\Sigma_{i=0}^{n-1} f(i) + f(n-1-i)$ em pares; função simétrica $f(x) = f(n - x)$ → metade dos casos. Gatilho: P(x) com simetria → dobre e use combinatória central.
- **Soma sobre permutações.** Média de um estatístico sobre permutações = E[X] por linearidade e simetria: P(i antes de j) = 1/2; P(i é máximo entre k posições) = 1/k; E[#pares invertidos] = n(n−1)/4.
- **Parcial → global por escada.** "Escada" de desigualdades $a_{i+1} \geq a_i + \delta$ implica $a_n \geq a_1 + (n-1)\delta$; use para limitar tamanhos (Fibonacci-like: valores crescem exponencialmente, logo ≤ log passos).

Otimização, programação linear e matroides

adicionado em 2026-11-10 por agentes; revisado

Quando o problema pede "o melhor de uma família enorme", pergunte se é LP/fluxo, matroide, submodular ou convexo — cada um tem um algoritmo exato.

- **Dualidade em LP.** max $c^T x$ s.t. $Ax \leq b$, $x \geq 0$ ↔ min $b^T y$ s.t. $A^T y \geq c$, $y \geq 0$; valores iguais no ótimo; folga complementar: $x_j > 0$ ⇒ restrição dual j justa. Gatilho: provar otimalidade com um certificado (corte = fluxo, cobertura = emparelhamento, potenciais = Hungarian).
- **Fluxo e corte como LP.** Max flow = min cut; emparelhamento máximo = cobertura mínima (bipartido); matrizes totalmente unimodulares (incidência de grafo bipartido, intervalos consecutivos, redes) fazem o LP ter ótimo inteiro — então relaxação resolve o problema inteiro.
- **Simplex.** Para variáveis/restrições $\lesssim$ 100–300 em double: use Bland ou maior coeficiente com eps 1e-9; problemas de mistura, dieta, jogos de soma zero, max flow pequeno. Cuidado com degenerescência (ciclo) e inviabilidade inicial (fase 1).
- **Jogo de soma zero.** Valor do jogo matricial (estratégias mistas) = LP: max v s.t. $\Sigma_i p_i A_{ij} \geq v\ \forall j$, $\Sigma\ p_i = 1$; minimax de von Neumann. Antes: elimine linhas/colunas dominadas; 2×2 tem fórmula fechada.
- **LP em poucas variáveis.** 2 variáveis e n restrições → interseção de semiplanos O(n log n); d fixo → Seidel aleatório O(d!·n). Gatilho: "maximize ax + by sujeito a n desigualdades".

- **Lema de Farkas.** $Ax = b, x \geq 0$ tem solução OU $\exists y: A^T y \geq 0, b^T y < 0$ (exatamente um). Em grafos: restrições de diferença inviáveis ⇔ ciclo negativo (o certificado é o ciclo); Hall ⇔ Farkas para emparelhamento.
- **Problema de atribuição (Hungarian).** $n \leq 500$ ($O(n^3)$): potenciais $u_i + v_j \leq c_{ij}$ são exatamente a dual. Para "atribuir k tarefas a pessoas com custo" também min-cost flow. Para $n \leq 20$ DP em máscara.
- **Transporte em linha e círculo.** Mover unidades em linha com custo $|\Delta|$: custo = $\Sigma_i |P_i|$ com P = prefixo da (oferta − demanda). Em círculo: escolha constante c minimizando $\Sigma|P_i - c| \rightarrow c$ = mediana dos P_i. Gatilho: "balancear pessoas/moedas em fila ou mesa redonda".
- **Fluxo com custos convexos.** Aresta com custo convexo $c(f) \rightarrow$ substitua por arestas paralelas de capacidade 1 com custos marginais $c(k) - c(k-1)$ crescentes (ou capacidades maiores por trechos). Custo ótimo como função do fluxo total F é convexo: dá "resposta para todo k" pelas sucessivas augmentações.
- **Min-cost flow para escalonamento.** Intervalos em k máquinas; "escolha subconjunto de intervalos com peso máximo sem sobrepor mais de k": linha com arestas $i \rightarrow i+1$ (cap k, custo 0) + arco por intervalo (cap 1, custo $-w$) — fluxo k mínimo custo. Também "k caminhos disjuntos de custo mínimo".
- **Fluxo com limites inferiores.** Aresta com $low \leq f \leq high$: cap = high − low e excessos em nós; adicione S′, T′ e aresta $T \rightarrow S$ infinita; viável iff fluxo $S' \rightarrow T'$ satura. Circulação viável e max-flow com lower bounds.
- **Custo negativo e ciclos.** Para min-cost com ciclos negativos: sature as arestas negativas (empurre fluxo, corrija demanda) ou use Bellman-Ford/SPFA com potenciais iniciais; no DAG basta DP para potenciais iniciais.
- **Dinic em redes especiais.** Capacidades unitárias: O(E·min(V^{2/3}, E^{1/2})); bipartido unitário: Hopcroft–Karp O(E√V). Gatilho: "$n = 10^5$ emparelhamento bipartido" → HK, não Kuhn simples.
- **Fecho máximo (project selection).** max Σ w(S) com S fechado por dependências → fonte→v (w > 0), v→sink (−w), v→u de ∞; resposta = Σ positivos − min cut. Densidade máxima (Goldberg): binária em λ + fluxo.
- **Rotulagem convexa (Ishikawa / min-cost tension).** min $\Sigma f_i(x_i) + \Sigma w_{ij} \cdot g(x_i - x_j)$ com f_i convexas, g convexa, $w_{ij} \geq 0$ e rótulos ordenados $x_i \in \{0, \ldots, k-1\}$ → corte mínimo em camadas (cada x_i em k níveis) ou dual de min-cost flow. Caso $g = |\cdot|$: a dual é fluxo de custo mínimo (potenciais).
- **Binário submodular.** Termo $\theta(x_i, x_j)$ binário é representável por corte iff $\theta(0,1) + \theta(1,0) \geq \theta(0,0) + \theta(1,1)$ (submodular); tudo o que não é, precisa inverter alguns rótulos (grafos bipartidos). Gatilho: "escolher 0/1 por vértice com custos por par".
- **Funções submodulares.** $f(A) + f(B) \geq f(A \cup B) + f(A \cap B)$ (retornos decrescentes). Exemplos: cobertura, cortes, rank de matroide, concavidade de |A|. Maximização monótona com cardinalidade k: guloso dá $(1 - 1/e)$; minimização é polinomial para n arbitrário (Lovász/Fujishige); os algoritmos genéricos é que só são viáveis na prática para instâncias pequenas.
- **Guloso em matroide.** Se os conjuntos viáveis formam matroide, ordenar por peso e adicionar quando mantém independência dá ótimo (inclusive "menor peso" e "maior peso"). Gatilho: "escolher itens com restrição de independência (sem ciclo, ≤ c por cor, linearmente independentes)".
- **Matroides do dia a dia.** Gráfica (florestas) → Kruskal; linear (vetores independentes, base xor) → insira por peso decrescente numa base linear; uniforme (≤ k itens); de partição (≤ c_i do grupo i); transversal (emparelhável); de escalonamento (jobs unitários com deadline, via DSU/heap).
- **Escalonamento com prazos.** Jobs unitários com deadline e lucro: ordene por lucro e coloque no último slot livre (DSU) — é matroide. Moore–Hodgson: minimizar nº de atrasados com tempos p_i e prazos d_i: ordene por prazo, mantenha heap de p, remova o maior quando estoura. Smith: min $\Sigma w_i C_i \rightarrow$ ordene por p_i/w_i; EDD: min lateness máxima.
- **Interseção de matroides.** Máximo conjunto comum independente por caminhos aumentantes no grafo de trocas ($O(r^2 n)$ consultas de oráculo; com pesos: Bellman-Ford mínimo em arestas). Reduz: emparelhamento bipartido (partição ∩ partição), árvore geradora arco-íris (gráfica ∩ partição por cor), arborescência (gráfica ∩ in-grau ≤ 1), "floresta + emparelhamento".
- **União de matroides.** Particionar arestas em k florestas (arboricidade) = união de k matroides gráficas = interseção via transformação; max conjunto que cabe em k florestas. Gatilho: "dividir em k grupos sem ciclo".
- **Árvore geradora com restrição de cor.** Exatamente k arestas vermelhas: o custo é convexo em k (matroide) → WQS (adicione λ ao peso das vermelhas) + Kruskal com desempate; ou matroid intersection / interpolação Kirchhoff para contagem.
- **Troca de base.** Em matroide, para bases B_1, B_2 e $x \in B_1 \setminus B_2$ existe $y \in B_2 \setminus B_1$ com $B_1 - x + y$ base. Gatilhos: segunda melhor MST (trocar uma aresta), "melhorar ótimo trocando 1 elemento", árvore de mínima mudança.
- **Lagrangiana / WQS (Aliens).** Se f(k) é convexa, otimize $f(k) + \lambda k$ por λ real/inteiro; busca binária em λ, resposta = $g(\lambda) - \lambda \cdot k_{alvo}$; desempate consistente (menor ou maior k) e cuidado com platôs: use λ inteiro se f inteiro e guarde "contagem mínima". Aplicações: k intervalos disjuntos, k árvores/ciclos, k grupos.
- **Convexidade comprovada.** f(k) é convexa se vem de fluxo de custo mínimo ou de matroide ponderado, ou se a matriz de custo é Monge; sem uma dessas provas, teste numericamente com brute force pequeno antes de apostar em Aliens.
- **Min-plus de convexas = merge.** $(f \oplus g)(k) = \min_{i+j=k} f(i) + g(j)$ com f, g convexas: mescle as diferenças ordenadas (soma de Minkowski). Mochila onde cada grupo tem custo convexo em itens → guloso por inclinação (heap).

- **DP em árvore convexa.** dp[v][k] convexa em k para tarefas do tipo "escolher k vértices com distância" → merge de filhos por Minkowski com small-to-large (heap) ou long-path decomposition: O(n log n) em vez de $O(n^2)$.
- **Minimização de função convexa.** $\Sigma|x - a_i|$ → mediana; $\Sigma(x - a_i)^2$ → média; $\Sigma w|x - a_i|$ → mediana ponderada; $\max|x - a_i|$ → ponto médio do intervalo; $\max_i f_i(x)$ com f_i convexas é convexa → ternária ou binária no subgradiente.
- **Ternária 2D.** Minimizar convexa g(x, y): $\min_x \min_y g$ é convexa (minimização parcial preserva convexidade) → ternária aninhada $O(\log^2)$ avaliações; ou descida coordenada/gradiente/Newton em R^d suave. Conferir convexidade (Hessiana semi-definida) antes.
- **Gradiente e hill climbing.** Para convexa suave em poucas dimensões: passo decrescente ("anel": tente 4–8 direções, reduza o passo pela metade se não melhora) converge em ~100 iterações; funções não convexas → vários inícios aleatórios.
- **Otimização inteira pequena.** ≤ 3–5 variáveis limitadas → enumere as duas menores e resolva o resto por fórmula; ax + by = c com minimização → Euclides estendido + deslocamentos; "melhor inteiro perto do ótimo real" → teste $\lfloor x/e \rfloor$.
- **Mochila: LP e poucos fracionários.** Relaxação linear da mochila 0/1 tem no máximo 1 item fracionário (guloso por v/w); o ótimo inteiro não é "quase" o guloso — o guloso puro pode ser arbitrariamente ruim e max(guloso, melhor item) é só 1/2-aproximação; use branch & bound ou DP só ao redor do ponto de corte (O(n·W_max)).
- **Mochila por valor pequeno / peso pequeno.** Se peso ≤ W: DP [peso]; se valor total ≤ V: DP [valor] = menor peso; ambos grandes → meet in the middle. Itens com muitos pesos iguais: agrupe e use mochila limitada por deque monotônico (mod w) ou binária.
- **Mochila 2D / com conflitos.** Conflitos formam floresta/árvore → DP em árvore; conflitos bipartidos de peso → corte mínimo.
- **Monge e SMAWK.** Custo Monge: w(a,c) + w(b,d) ≤ w(a,d) + w(b,c) ⇒ opt monótono: D&C DP O(n log n); Knuth em intervalos $O(n^2)$; SMAWK mínimo de linhas O(n); LARSCH para DP 1D/1D online. Monge côncavo ⇒ f(k) convexa ⇒ Aliens.
- **Alocação ótima com retornos decrescentes.** Distribuir k recursos entre itens com ganho marginal decrescente → heap pelo ganho marginal (guloso exato). Se os ganhos não são decrescentes → DP/mochila de grupos.
- **Binária + guloso, binária no λ.** Min-max de partição em k blocos → binária no limite + guloso; razão Σa/Σb → Dinkelbach (converge em poucas iterações, mais rápido que binária) ou binária; fração ótima de rótulo → $\lambda_{k+1} = \Sigma a(S_k)/\Sigma b(S_k)$.
- **Programação por restrições: 2-SAT / difference constraints.** Sistemas $x_j - x_i \le c$ são LP dual de menor caminho; se há função objetivo linear $\Sigma w_i x_i$ → min-cost flow dual. Com somas, gatilho "$x_i \le x_j + c$ e minimize Σ".
- **Problema de transporte entre classes.** Distribuir mercadorias de depósitos a clientes com custo por par e capacidades → min-cost flow; grande se custo = $|p_i - q_j|$ em linha (sorted matching) → greedy.
- **Cobertura / hitting set pequeno.** Cobertura mínima de vértices com parâmetro k pequeno → FPT: ramifique (escolha u ou todos os vizinhos), $O(2^k \cdot m)$; em bipartido, exato por fluxo; árvore: guloso nas folhas.
- **Corte mínimo global.** Stoer–Wagner $O(n^3)$ / $O(nm + n^2 \log n)$; Karger aleatório; k-corte de arestas em grafo de poucos vértices; árvore de Gomory–Hu para todos os pares (n − 1 fluxos). Gatilho: "corte mínimo entre todos os pares" ou "mínimo corte global".
- **LP inteiro e arredondamento.** Se o LP é totalmente unimodular, ótimo inteiro; se não, arredondamento aleatório dá aproximação (e.g., cobertura de vértices por 2-aproximação; 1 − 1/e em cobertura máxima). Em ICPC, só se o enunciado pede aproximação; caso contrário procure estrutura especial.

Amostras de problemas e padrões de ICPC

adicionado em 2026-11-10 por agentes; revisado

ICPC raramente cobra uma técnica sozinha: o problema médio combina 2–3 estruturas conhecidas, então aprenda os pares que se encaixam.

- **Binária + fluxo.** "Minimizar a carga máxima / maximizar o mínimo" em atribuição ou transporte → binária no limite e checagem por max flow; minimizar a carga máxima usa capacidade = limite, mas maximizar o mínimo exige lower bounds/demandas (circulação com limites inferiores); o grafo é sempre o mesmo, só mudam capacidades e limites.
- **Binária + 2-SAT.** Escolher 1 de 2 posições/cores por item, todo par escolhido deve estar a distância ≥ D → binária em D e 2-SAT onde cada par proibido vira cláusula ($O(n^2)$ cláusulas se n pequeno ou segtree de arestas se grande).
- **Binária + MST/Kruskal.** Caminho que minimiza o máximo → MST mínimo (bottleneck); com consultas, árvore de reconstrução de Kruskal + LCA; com condição extra (conjunto de vértices conectado) → binária no peso + DSU.
- **Binária + Dijkstra.** "Maximize o mínimo/limite de combustível/ruído em caminho" → binária no limite e BFS/Dijkstra só em arestas permitidas; se o limite é do vértice, remova vértices no check.
- **Binária + geometria.** Raio mínimo de cobertura, círculo com condição, margem máxima: binária no raio + interseção de discos/semiplanos; checar com varredura angular e eps; para o maior círculo vazio, Voronoi ou binária + hull.

- **HLD + segtree + lazy.** Queries e updates de caminho em árvore com agregado não invertível (max, soma + atribuição, "primeiro vértice com valor ≥ x"): HLD vira O(log) intervalos, segtree guarda o agregado; ordem cuidadosa em caminhos não comutativos (esquerda/direita).
- **HLD + estruturas de aresta.** Pesos em arestas → guarde no filho; query de caminho exclui o LCA. Com update de subárvore + caminho misto, use Euler tour para subárvore e HLD para caminho sobre a mesma segtree (tin da HLD).
- **Link-cut / Euler tour tree.** Quando a floresta muda (link/cut) e o agregado é de caminho ou componente → LCT; quando só componentes e subárvore → ETT; offline, segtree no tempo + DSU rollback é mais simples.
- **Bitset + SCC / DAG.** Alcançabilidade em DAG com $n \approx 5 \cdot 10^4$ e $m \approx 10^5$: condense SCC, processe em ordem topológica com bitsets de n/64 palavras em blocos (memória) → O(nm/64). Também "transitive closure" e "quantos vértices alcança cada v".
- **Bitset + BFS denso.** $n \leq 3 \cdot 10^4$ com grafo denso: BFS mantendo bitset de não visitados e iterando por `_Find_next` → $O(n^2/64)$ em vez de O(m).
- **Bitset + DP.** Mochila booleana, alcançabilidade em grid com obstáculos, LCS de strings $5 \cdot 10^4$ (Hyyrö, n·m/64), distância de edição em bits; sempre que o DP é "OR de deslocamentos".
- **Centroide + FFT.** Contar, para todo comprimento d, o número de pares de vértices a distância d: em cada centroide, polinômio das profundidades, A^2 menos soma dos A_filho^2 (remove pares do mesmo filho) → $O(n \log^2 n)$.
- **Centroide + estrutura.** Caminhos com condição em soma e comprimento (ex.: soma ≤ K): em cada centroide, ordene as distâncias e use two pointers com subtração do mesmo filho (ou BIT/ordenação por filho).
- **Mo + DSU rollback.** Queries "o grafo com arestas l..r é bipartido/conexo/ #componentes": Mo sem remoções (Mo com rollback / blocos): adições ordenadas e desfazer; $O((n + q)\sqrt{m} \cdot \log)$.
- **D&C no tempo + DSU rollback.** Arestas com tempo de vida, queries offline (conectividade, bipartição por paridade, nº de componentes): insira cada aresta em O(log) nós da segtree do tempo e faça DFS com rollback.
- **Mo em árvore / Mo com tempo.** Consulta de caminho com add/remove O(1): Euler de 2n posições (paridade de ocorrências); com updates: Mo 3D, n^{5/3}.
- **SAM + DP.** DP no DAG do autômato: nº de substrings distintas, k-ésima lexicográfica, contagem de ocorrências (soma nos links), substrings que ocorrem em exatamente k strings (generalizado) ou em todas.
- **SAM / SA + segtree.** Ocorrências de padrão em substring s[l, r]: intervalo de sufixos no SA + estrutura 2D (BIT offline/persistente); no SAM, segtree merging em endpos (pergunta "existe ocorrência com início em [l, r]").
- **SA + DSU por LCP.** Para todo k, nº de pares de sufixos com LCP ≥ k, ou maior grupo com LCP ≥ k: processe LCP em ordem decrescente fundindo vizinhos no SA (Kruskal sobre o array LCP) com DSU que guarda agregados.
- **Aho-Corasick + matriz.** Contar strings de comprimento $\leq 10^{18}$ sem padrões proibidos → matriz de transição do autômato (≤ 100–200 estados) elevada; menor string que contém todos os padrões → Aho + máscara de padrões (≤ 10).
- **Aho-Corasick + digit DP.** Contar números ≤ N cuja representação contém/evita substrings → DP[pos][estado AC][tight]. Idem com DP de KMP para um padrão só.
- **Fail tree + BIT.** Queries "quantos padrões i ocorrem em prefixo/segmento do texto j": árvore de falhas do AC, Euler tour; cada posição do texto soma +1 no nó, query = soma na subárvore do padrão (BIT offline).
- **Hash + trie / ordenação.** Comparar caminhos de árvore (isomorfismo) → hash AHU canônico (ordenar filhos); isomorfismo não enraizado: enraíze nos centros (≤ 2) e compare. Subárvores iguais em árvore → contagem por hash.
- **DP + CHT + persistência.** dp[v] = min_{u ancestral} dp[u] + b_u·a_v (DP em caminho raiz→folha): CHT persistente (Li Chao persistente ou pilha com rollback + busca binária no hull) para voltar ao pai; O(n log n).
- **CDQ + CHT / DP 1D/1D online.** dp[i] depende de dp[j], j < i, mas o hull precisa de ordem: CDQ — resolva esquerda, construa o hull de [l, m], contribua para [m, r] ordenando queries; $O(n \log^2 n)$ ou log com merge.
- **DP + segtree + pilha monotônica.** dp[i] = min_j dp[j] + max(a[j+1..i]) (ou soma de máximos): a pilha monotônica diz quais j têm o mesmo máximo → atualizações de intervalo (+ constante) na segtree sobre dp + consulta de min em [l, i−1].
- **DP + SOS + máscara.** DP[mask] com transição sobre submáscaras por soma/condição → SOS; "escolher k de n ≤ 20 com restrição de pares" → DP[mask] com popcount; n ≤ 40 → meet in the middle (item seguinte).
- **Meet in the middle + ordenação.** n ≤ 40 em subset sum / contar subconjuntos com soma ≤ T / maximizar soma ≤ T: duas metades, ordenar e two pointers; com restrição extra de xor ou paridade → mapa por classe.
- **Persistent segtree + árvore.** k-ésimo na caminho u–v: versão em cada nó = pai + valor; resposta com versões ver[u] + ver[v] − ver[lca] − ver[parent(lca)] e descida simultânea.
- **Wavelet / persistente + binária.** Contar valores em [x, y] no subarray [l, r], k-ésimo, mediana: wavelet tree em O(log σ) por query; com updates → BIT de segtrees ou sqrt; offline → BIT + varredura.
- **Offline varredura + BIT.** "Quantos pares (i, j) em [l, r] com a_i < a_j" é contagem de pares em subarray (inversões/não inversões): use Mo + BIT, D&C offline (CDQ) ou estrutura 2D/wavelet; já "soma de max/min em subarrays" usa pilha monotônica para definir intervalos de contribuição e varredura por r com a segtree histórica do item seguinte.

- **Segtree histórica.** Soma de f(l..r) para todos os subarrays dentro de [L, R], "versão do array a cada tempo": segtree com lazy de "soma acumulada ao longo do tempo" (histórico), usada com varredura por r e pilha monotônica.
- **Sweep angular + BIT/two pointers.** Contar triângulos que contêm a origem ou que não contêm: ordene por ângulo e some C(k, 2) por ponto (complemento); fecho convexo + pontos dentro: binária no hull.
- **Fecho convexo + binária/CHT.** Queries de reta extrema, tangente a partir de ponto, maior produto escalar com direção → hull + busca binária; muitas direções ofensivas → rotating calipers.
- **Geometria + sweep + segtree.** Área da união de retângulos (segtree com contagem de cobertura + comprimento coberto, ou min da cobertura + contagem do mínimo), pontos em círculos/retângulos (sweep + BIT), interseção de segmentos (sweep com set ordenado por y).
- **Geometria + DSU/flow.** Discos que se tocam → grafo de interseção + DSU; "cortar caminho com círculos" → caminho mínimo no dual ou componentes conexas; fluxo planar ↔ dual.
- **Dijkstra + segtree de arestas.** Vértice → intervalo de vértices com custo: segtree de grafo com duas árvores (in/out) e arestas dirigidas de custo 0 (uma para cima, outra para baixo) permite Dijkstra com O(n log n) arestas; uma única árvore com arestas 0 bidirecionais colapsa as distâncias.
- **Dijkstra em camadas.** (vértice, k usos de bônus, paridade, tempo mod c) como estado; K ≤ 20 → n·K estados; muitas arestas com peso 0/1 → deque.
- **Floyd incremental / reverso.** Remoções de vértices em sequência, "soma de todas as distâncias após cada remoção" → inverta: adicione vértices e atualize $O(n^2)$ cada: $O(n^3)$ total.
- **Fluxo em grid (xadrez).** Paridade de i + j faz bipartição para dominós e para "escolher células sem vizinhas" → matching/max independent set (n − ν); com custos → min-cost; corte mínimo para cercar/separar. Torres que não se atacam → matching entre linhas e colunas (células proibidas viram arestas ausentes); rainhas com bloqueios é NP-difícil.
- **Cobertura mínima por caminhos / cadeias.** Min path cover em DAG vértice-disjunto = n − ν; com sobreposição, feche transitivamente primeiro. Min antichains/ chains → Dilworth via matching; "mínimo de guardas".
- **Euler tour + flow/matching.** Decompor arestas de multigrafo euleriano em caminhos/ciclos, ou orientar aresta a aresta: circuito euleriano; "pintar arestas com 2 cores balanceadas" → Euler tour alternando cores (grafo bipartido regular em potência de 2 → colorir por D&C).
- **Postman chinês.** Percorrer toda aresta custo mínimo: adicione emparelhamento perfeito mínimo entre vértices de grau ímpar (≤ 20 → DP em máscara sobre distâncias de Floyd).
- **TSP e variantes.** n ≤ 20: DP[mask][last] $O(2^n n^2)$; caminho hamiltoniano contagem: DP ou IE (2^n n); n ≤ 40 só com estrutura especial (meet in the middle em ciclos de comprimento fixo); métrica e n grande → aproximação (MST × 2) só se pedido.
- **Ciclo/ rho + fast-forward.** Simulação de 10^{18} passos de função determinística → detecte ciclo (Brent/Floyd ou mapa de visitados) e extrapole; com estado grande, hash do estado.
- **Árvore de Steiner.** Poucos terminais (k ≤ 10): DP[mask][v] = custo mínimo para conectar o conjunto mask e terminar em v, combinando submáscaras e Dijkstra por máscara: $O(3^k n + 2^k m \log n)$.
- **Circular: corte e duplicação.** Estrutura em círculo → fixe um ponto de corte (teste as 2–3 candidatas), ou duplique o array e use janela de tamanho n; DP circular: fixe o primeiro estado e rode DP linear.
- **Geradores de problemas "somas de produtos".** $\Sigma_{i<j} a_i a_j f(i, j)$ com f polinomial/exponencial em i − j → convolução (FFT/NTT) ou prefixos com potências; f = gcd → Möbius + múltiplos.
- **Contagem em árvores com FFT.** DP de árvore com polinômio por vértice (nº de maneiras como função do tamanho/grau): produto de filhos por D&C + NTT: $O(n \log^2 n)$; com peso por profundidade, long-path decomposition.
- **Enumerar caminhos curtos em árvores/grafos esparsos.** Caminhos de comprimento ≤ k em grafo com grau limitado: BFS limitado; ciclos de comprimento ≤ 6 em grafo planar usam a 5-degenerescência: oriente as arestas do menor para o maior rank para limitar o out-degree a 5 na enumeração — grau médio < 6 sozinho não limita Σ C(grau, 2), como na estrela.
- **Jogos em ICPC.** Soma de jogos independentes → Grundy: calcule pequeno por brute force, ache período; jogos em grafo/ árvore → Grundy com DP; "primeiro ou segundo vence" geralmente é uma condição local (xor ≠ 0 ou paridade de profundidade).
- **Simulação com estruturas.** "Ordem de eventos com tempo contínuo" → heap de eventos com cancelamento preguiçoso; "fila de impressão/CPU" → heap + sort por tempo; "elevador" → simulação com map; não faça passo a passo se T é grande — pule para o próximo evento.
- **Grafos funcionais + estrutura.** Cada vértice aponta para um só → ciclos com árvores penduradas: binary lifting para k passos, ciclo + prefixo de somas para aplicar f^k; "colorir a menor quantidade" → ciclos independentes.
- **Árvores de sufixo e DSU.** Contagem de substrings distintas em intervalo [l, r] (queries) → SA + segtree/BIT offline, ou SAM com LCT (heavy) — se a questão é mais simples, prefira hash com restrição.
- **Raiz quadrada de queries.** Rebuild a cada B updates: queries de menor caminho com poucas arestas modificadas (Dijkstra a cada √q) ou contagem de coisas com updates em lote; combine com "pesado/leve por grau".

- **Reconstrução da resposta.** Em ICPC, conferir o formato: se o problema exige "uma solução qualquer", rode o DP/fluxo e extraia os escolhidos (reverso de predecessores / arestas saturadas); teste o checker próprio no brute force.
- **Ler como grafo de dependências.** Pré-requisitos, "A deve vir antes de B" → ordenação topológica (lexicograficamente menor = heap; contagem de ordens = DP em máscara ou fórmula do gancho em floresta, no item seguinte).
- **Contagem de linearizações de floresta.** Nº de ordens topológicas de uma floresta enraizada = n!/∏_v sz(v) (fórmula do gancho para árvores); com condições extras → DP de intercalação com C(a+b, a).
- **Ranges + conjuntos de intervalos.** Atribuição em intervalo (range assign) e "conte valores distintos": ODT (set de intervalos) com amortização; já range add/consultas de soma podem ser O(nq) no ODT — use segtree com lazy (ou ODT só para decompor quando houver assigns frequentes).
- **Escaneamento duplo (two-pointer em estrutura).** Subarrays com "max − min ≤ k" ou "k distintos" → janela + deque/multiset; contagem de subarrays ótimos → número de janelas × saídas por início (soma n − L).
- **Meet in the middle em grafo.** Caminhos de comprimento 2k → conecte metades em vértice do meio (k passos de cada lado), contagem por mapa; caminho de soma S em subconjunto.
- **Fluxo + DP.** Primeiro DP calcula quais pares são compatíveis (ex.: janelas de tempo), depois fluxo/matching sobre o grafo resultante; ou fluxo dá estrutura e DP otimiza custos (redes em camadas por tempo).
- **Expandir o grafo no tempo.** Tempo discreto pequeno (T ≤ 100): vértice (v, t) com arestas de espera (v, t) → (v, t + 1); fluxo em rede temporal (evacuação) com capacidade por unidade de tempo.
- **Interativo em ICPC.** Poucos problemas, mas padrões: busca binária com perguntas de comparação, "descobrir permutação com ≤ n log n queries" (merge sort/insertion com binária), localizar vértice em árvore por centroide ou por descida no caminho pesado.

Quando nada funciona

O brute force é a ferramenta mais subestimada: ele gera dados, e dados geram observações.

- **Imprima a resposta para n = 1..15** e procure padrão, fórmula fechada ou a sequência na OEIS.
- **Imprima a estrutura ótima** (qual escolha o brute force fez), não só o valor: o guloso costuma aparecer ali.
- **Stress test** contra o brute force quando tiver WA: gere casos aleatórios pequenos e compare.
- **Chute a resposta** entre as candidatas usuais (0, 1, n, n − 1, 2^n, C(n, 2), soma, max − min) e teste.
- **Releia o enunciado** procurando a frase que você ignorou; quase todo "impossível" vem de uma condição esquecida.
- **Pergunte o que o autor queria testar:** se a solução óbvia é $O(n^2)$ e $n = 2 \cdot 10^5$, a ideia está em tirar exatamente essa dimensão.

Sobre forçar ideias: os dois lados estão certos em momentos diferentes. Forçar técnica cedo atrapalha, porque você encaixa o problema numa ferramenta sem entendê-lo. Forçar depois de observar ajuda, porque cada pergunta do catálogo vira um experimento barato no seu exemplo pequeno. A ordem que funciona é observar primeiro, catálogo depois.

Depois de cada problema, escreva uma linha nova aqui: o gatilho que você deveria ter visto e a ideia que ele pedia. O catálogo que mais ajuda é o que cresce com os seus próprios erros.

---

adicionado em 2026-11-10 por agentes; revisado

- **Diagnóstico por veredito.** WA 1: leitura/formato/sample. WA em teste médio: caso de borda ou guloso errado. WA em grande: overflow, precisão ou lógica errada. TLE: complexidade alta (n×n em vez de n log n), pior caso (estrela, caminho, todos iguais), recursão profunda, constante. RE: índice, stack, divisão por 0. MLE: arrays em 2D de tamanho cheio.
- **Casos de borda padrão.** n = 1, n = 2, todos iguais, já ordenado, ordenado ao contrário, valores máximos/mínimos, 0 e negativos, árvore caminho e estrela, grafo completo, componente isolada, resposta 0 / −1.
- **Gerador de testes.** Escreva um gerador com seed e parâmetros (n, valor máx); 1000 testes pequenos contra o brute force acham 95% dos bugs; reduza o contraexemplo (menor n, menor valor) até caber à mão.
- **Overflow.** a·b com a, b ≤ 10^9 em long long; produto mod p: multiplique em ll antes do %; INF perto de $9 \cdot 10^{18}$ estoura em INF + INF (use 1e18 ou 4e18 e cheque antes de somar); 1 << 40 precisa de 1LL; __int128 para produto de $10^{18} \times 10^{18}$ mod m.
- **Mod negativo.** (x % m + m) % m; subtração modular sempre soma m; cuidado com `-` e `unsigned` (size() − 1 quando vazio vira $2^{64} - 1$).
- **TLE vs constante.** Antes de otimizar assintótica, troque map por array/sort, vetor 2D por plano, endl por \n, sync_with_stdio(false) + cin.tie(0), evite divisões mod no laço interno, acesse memória em ordem, reserve vector.
- **Zerar arrays por teste.** memset de MAXN com t = 10^5 testes é O(MAXN·t); zere só n usados. Estruturas globais devem ser limpas entre testes; esquecer isso é a causa nº 1 de WA em multitestes.
- **Ordem de leitura.** Ao dar `return` cedo (n = 1, caso trivial), leia primeiro o resto do teste; senão os testes seguintes viram lixo.

- **Recursão profunda.** DFS em caminho de $2 \cdot 10^5$ → stack overflow; faça iterativo ou aumente stack (ulimit/#pragma); BFS/ordem topológica quando possível.
- **Comparador inválido.** `<=` em sort/set derruba o programa ou dá resultado errado; deve ser ordem fraca estrita. Em set/map de pares com ordem customizada, igualdade = nenhum menor que o outro.
- **Armadilhas do C++.** map[x] insere; multiset.erase(valor) apaga todos (use erase(find)); iterador invalidado ao inserir em vector; ponto flutuante igual com `==`; char com sinal; endl misturado com printf; uso de variável não inicializada (use -fsanitize=memory ou Valgrind; ASan/UBSan/_GLIBCXX_DEBUG não detectam).
- **Conferir leitura e saída.** Imprima o que leu; confira 0/1-indexado; confirme que a resposta é a pedida (índice? valor? quantidade?). Muitos WA são problema bem resolvido com a pergunta errada.
- **Cheque a hipótese do guloso.** Escreva o contraexemplo de pelo menos 3 tentativas de ordem; se nenhuma falha, enumere exaustivamente instâncias pequenas ($n \le 6$–7, valores pequenos) — 1000 testes aleatórios não validam o guloso — e ainda assim exija um argumento de troca.
- **Se o brute force é muito lento.** Faça brute force em duas camadas: um exponencial pequeno ($n \le 8$) para testar WA e um polinomial lento para testar TLE de ideias.
- **Se não acha a recorrência.** Calcule $2d$ termos por brute force e rode Berlekamp–Massey (ordem $\le d$); funciona para recorrências lineares, e confirme com termos extras.
- **Se a estrutura é grande demais.** Imprima o estado ótimo para $n = 5, 6, 7$ e desenhe; procure uma bijeção com algo contável ou um padrão de bits/paridade.
- **Estratégia de equipe (ICPC).** Leia todos os problemas nos 15–20 primeiros minutos; ordene por dificuldade; um computador por vez (quem pensa escreve no papel); depure no papel ou numa tela de outro, não no computador do time; penalidade de 20 min por WA ⇒ revise antes de submeter.
- **Quando submeter.** Rode samples, os 3 casos de borda e um stress rápido se há brute force; submeter com leve dúvida custa 20 min, enquanto 2 minutos de verificação raramente custam.
- **Treinamento.** Resolva problemas ~200–300 de rating acima do seu; passe 30–45 min tentando antes de ver o editorial; depois reimplemente sem olhar. Anote por problema o gatilho que faltou; refaça os que errou após 1–2 semanas.
- **Pós-contest (checklist).** 1) Upsolve todos os problemas que tentou. 2) Classifique cada erro: leitura / ideia / implementação / tempo. 3) Escreva a linha do gatilho no catálogo. 4) Reimplemente o que errou limpo, com brute force. 5) Leia o código de 2–3 soluções melhores para ver o truque. 6) Registre quanto tempo gastou em cada problema e se a ordem foi boa.
- **Biblioteca de ICPC.** Um template que compila em 1 s, com macros mínimas, DSU, segtree, BIT, grafo, Dinic, matching, hash, NTT, fórmula de mod; depois do contest, adicione o que você ainda reimplementou na hora.

Índice da lib (técnica → arquivo)

Índice da biblioteca

Gerado em 2026-11-11 a partir dos cabeçalhos de cada arquivo. Total: **206 trechos**. Status `compila` = `g++ -std=c++17 -Wall -fsyntax-only` com o prelude do template; `testado` = conferido contra brute force/casos conhecidos; `com binom_mod` = precisa colar `code/combinatorics/binom_mod.cpp` antes.

Pastas ainda vazias: `code/bits/`, `code/games/`, `code/primitives/` (Lote 1 e 2 de `docs/PLANO_LIB.md`).

Arquivo	Nome	Descrição	Seção do catálogo	Status
algebra/				
`code/algebra/berlekamp_massey.cpp`	Berlekamp-Massey + n-esimo termo de recorrencia linear (Kitamasa O(k^2 log n))	dado s[0..m) de uma recorrencia linear de ordem L (mod primo), acha os	Álgebra, polinômios e FFT	compila
`code/algebra/bostan_mori.cpp`	Bostan-Mori: [x^n] P(x)/Q(x) mod 998244353	n-esimo coeficiente de uma serie racional P/Q (Q[0] != 0)	Álgebra, polinômios e FFT	compila
`code/algebra/conv_mod.cpp`	Convolucao com modulo arbitrario por 3 NTT + CRT (Garner)	c = a*b mod M para M qualquer (ex.: 1e9+7), exato (sem erro de ponto flutuante)	Álgebra, polinômios e FFT	compila
`code/algebra/fft.cpp`	FFT complexa (double) e convolucao com modulo arbitrario por split	conv(a,b) convolui vetores de double	Álgebra, polinômios e FFT	compila
`code/algebra/lagrange.cpp`	Interpolacao de Lagrange em pontos consecutivos (0..d)	dado y[i] = P(i) para i = 0..d (P de grau <= d), calcula P(n) para n qualquer	Álgebra, polinômios e FFT	compila
`code/algebra/ntt.cpp`	NTT (transformada teorica dos numeros) e convolucao mod 998244353	multiplica dois polinomios com coeficientes mod 998244353 (g = 3)	Álgebra, polinômios e FFT	testado (brute, 2026-11-11)
`code/algebra/poly.cpp`	Polinomios e series formais mod 998244353 (inv, ln, exp, sqrt, pow, divmod, multipoint, interpolacao)	operacoes de series formais por Newton, divisao de polinomios, avaliacao em	Álgebra, polinômios e FFT	compila
`code/algebra/power_sum.cpp`	Soma de potencias sum_{k=1}^n k^d mod p (Faulhaber via Lagrange)	calcula S(n,d) = 1^d + 2^d +	Álgebra, polinômios e FFT	compila
combinatorics/				
`code/combinatorics/binom_general.cpp`	Binomial modulo composto qualquer (e potencia de primo)	C(n,k) mod m para m arbitrario, n,k ate 1e18	Combinatória, contagem e probabilidade	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/combinatorics/binom_mod.cpp	Fatoriais e binomiais modulo primo	pre-calcula fact, ifact e inv_[1..N]	Combinatória, contagem e probabilidade	compila
code/combinatorics/burnside.cpp	Burnside, colares, pulseiras, strings primitivas	(1) Burnside generico: dado um grupo de permutacoes (lista de	Combinatória, contagem e probabilidade	compila
code/combinatorics/catalan_family.cpp	Familia de Catalan e contagens de caminhos/selecoes classicas	Catalan, ballot (estrito e com empate), Narayana, Fuss-Catalan,	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/counting_misc.cpp	Contagens diversas: hook length, floresta, graficidade, torneios, torres, Ryser	# SYT por hook length	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/grid_paths.cpp	Caminhos em grid: obstaculos, faixa 0<=y<=h (reflexao alternada)	(1) # caminhos (direita/cima) de (0,0) a (X,Y) evitando k pontos	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/inclusion_exclusion.cpp	Inclusao-exclusao: utilitarios	(1) contar inteiros em [1,N] nao divisiveis por nenhum de uma lista	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/lgv.cpp	Lindstrom-Gessel-Viennot (caminhos disjuntos em vertices)	dado e[i][j] = # caminhos de a_i ate b_j em um DAG, devolve	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/lucas.cpp	Lucas e Kummer	C(n,k) mod p para p primo PEQUENO (p ate ~1e6) e n,k ate 1e18	Combinatória, contagem e probabilidade	compila
code/combinatorics/matrix_tree.cpp	Matrix-Tree (Kirchhoff), determinante mod p e mod composto	det de matriz mod primo (Gauss) e mod qualquer m (eliminacao tipo	Combinatória, contagem e probabilidade	compila
code/combinatorics/partitions.cpp	Particoes de inteiros (numeros pentagonais de Euler e DPs em sqrt)	p(0..N) por pentagonais	Combinatória, contagem e probabilidade	compila
code/combinatorics/perm_counts.cpp	Contagens de permutacoes especiais	desarranjos D_n, perms com exatamente k pontos fixos, involucoes,	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/prob_expect.cpp	Probabilidade e esperanca modulo p: utilitarios	cadeia de Markov absorvente (E[tempos] = (I-Q)^-1 1 por Gauss mod p),	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/stirling_bell.cpp	Stirling 1a e 2a especie, Bell, Eulerianos (tabelas O(n^2) e valor unico)	tabelas S2[n][k], S1[n][k] (sem sinal), Eulerianos A[n][k],	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/tree_counting.cpp	Contagens de arvores rotuladas e estruturas classicas (Cayley, Prufer, componentes)	Cayley n^{n-2}	Combinatória, contagem e probabilidade	compila (com binom_mod)
dp/				

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/dp/bitset_subset_sum.cpp	Subset sum com bitset e multiplicidades	dado multiconjunto de pesos, diz quais somas 0..W sao alcancaveis	DP	compila
code/dp/broken_profile.cpp	Broken profile DP (plug DP simples): ladrilhar grade com dominos	conta (mod) o numero de ladrilhamentos de uma grade n x m com dominos 1x2, com celulas bloqueadas '#'	DP	compila
code/dp/coins.cpp	Mochila de moedas: minimo de moedas para um valor	dp[i] = minimo de moedas (uso ilimitado) para somar i	DP	compila + testado
code/dp/comb.cpp	Fatoriais e binomiais modulo M (versao antiga e simples)	pre-calcula fact/invFact e responde C(n,k) modulo M	Combinatória, contagem e probabilidade	compila
code/dp/dc_opt.cpp	Divide and conquer optimization (D&C DP)	calcula dp[k][i] = min_{j<i} dp[k-1][j] + C(j,i) para k=1..K quando opt(i) e monotono	DP	compila
code/dp/digitdp.cpp	Digit DP (soma dos digitos de 0..k)	conta/otimiza sobre os digitos de 0..k com estado (posicao, soma, tight)	DP	compila + testado
code/dp/edit_distance.cpp	Edit distance (Levenshtein) com reconstrucao e versao em faixa	edit_dist: distancia minima (inserir/remover/trocar) entre a e b e uma sequencia de operacoes	DP	compila
code/dp/frog.cpp	Frog: custo minimo pulando de 1 a k posicoes	dp[i] = min_{1<=j<=k} dp[i-j] + |a[i]-a[i-j]|	DP	compila + testado
code/dp/frog3_cht.cpp	Frog 3 com CHT (exemplo completo, standalone)	resolve o DP dp[i] = c + h[i]^2 + min_j (dp[j] + h[j]^2 - 2*h[i]*h[j])	DP	compila (programa completo)
code/dp/interval_dp.cpp	DP em intervalo: exemplos classicos (baloes, impressora estranha, jogo em intervalo, circular)	moldes de dp[l][r] O(n^3)	DP	compila
code/dp/knapsack.cpp	Mochila 0/1	valor maximo com peso total <= w, cada item no maximo uma vez	DP	compila + testado
code/dp/knapsack_bounded.cpp	Mochila limitada (bounded knapsack): fila monotona e binary splitting	maximiza valor com itens (peso w_i, valor v_i, quantidade c_i), capacidade W	DP	compila
code/dp/knuth.cpp	Knuth optimization (DP em intervalo)	dp[l][r] = min_{l<=k<r} dp[l][k] + dp[k+1][r] + w(l,r) em O(n^2) usando opt[l][r-1] <= opt[l][r] <= opt[l+1][r]	DP	compila
code/dp/lcs.cpp	LCS classica O(n*m) (subsequencia comum maxima)	dp[i][j] = LCS dos prefixos a[0..i) e b[0..j)	DP	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
`code/dp/lcs_bitset.cpp`	LCS com bitset (Hyyro / Allison-Dix)	comprimento da LCS de a e b em O(n*m/64), usando vetor de palavras de 64 bits com soma e carry	DP	compila
`code/dp/lis.cpp`	LIS em O(n log n)	tamanho da maior subsequencia estritamente crescente	DP	compila + testado
`code/dp/slope_trick.cpp`	Slope trick	mantem uma funcao convexa linear por partes f(x) com dois heaps (pontos de quebra a esquerda L e a direita R)	DP	compila
`code/dp/sosdp.cpp`	SOS DP recursivo (soma sobre submascaras)	gen_submask(b, mask) = soma de F[s] para todas as submascaras s de mask	Bits e XOR	compila
`code/dp/subsetsum.cpp`	Maior soma de subconjunto <= t (subset sum otimizado)	dado w (pesos inteiros >= 0) e limite t, devolve o maior valor <= t	DP	compila + testado
`code/dp/tree_knapsack.cpp`	Knapsack em arvore: merge O(n^2) e DP sobre ordem DFS O(nK)	escolher exatamente k vertices formando subarvore conexa que contem a raiz (se pega v, pega o pai),	Árvores	compila
`code/dp/tsp_bitmask.cpp`	DP em bitmask: TSP, atribuicao e particao em subconjuntos	(1) tsp: menor ciclo hamiltoniano, dp[mask][ultimo]	DP	compila
ds/				
`code/ds/aliens.cpp`	Aliens trick / WQS binary search (lambda optimization) generico	se f(k) = melhor custo usando EXATAMENTE k itens eh CONVEXA	Busca binária e otimização	compila
`code/ds/bit.cpp`	BIT (Fenwick) 1D + busca binaria na BIT + BIT de maximo	soma de prefixo/intervalo com update pontual	Estruturas de dados e queries	compila
`code/ds/bit2d.cpp`	BIT 2D	update pontual (soma) e soma em retangulo numa grade n x m	Estruturas de dados e queries	compila
`code/ds/bit_rr.cpp`	BIT range-update range-query	soma em intervalo de v (add em [l,r]) e soma de intervalo,	Estruturas de dados e queries	compila
`code/ds/busca_binaria.cpp`	Busca binaria: templates (inteira, real, exponencial, k-esimo, mediana)	colecao de templates de binaria na resposta	Busca binária e otimização	compila
`code/ds/busca_binaria_paralela.cpp`	Busca binaria paralela (por niveis)	para Q queries "primeiro tempo t em que pred(query) vale", com	Busca binária e otimização	compila
`code/ds/busca_ternaria.cpp`	Busca ternaria (real, inteira) e Dinkelbach	acha o minimo de f unimodal/convexa	Busca binária e otimização	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/ds/cdq.cpp	CDQ (divide e conquista offline) - template com BIT: dominancia 3D / DP 2D	exemplo completo de CDQ: res[i] = val[i] + max(res[j]) para	Estruturas de dados e queries	compila
code/ds/cht_dinamico.cpp	Line container (CHT dinamico, estilo KACTL)	insere retas y=k*x+m em qualquer ordem e consulta o MAXIMO em x	Estruturas de dados e queries	compila
code/ds/cht_monotono.cpp	Convex Hull Trick monotono (deque)	mantem envoltoria inferior de retas y=m*x+b inseridas com	Estruturas de dados e queries	compila
code/ds/conectividade_dinamica.cpp	Conectividade dinamica offline (segtree no tempo + DSU com rollback)	cada aresta existe em um intervalo de tempo [l, r)	Estruturas de dados e queries	compila
code/ds/disjoint_sparse.cpp	Disjoint sparse table	query de qualquer operacao ASSOCIATIVA (soma, produto mod, xor,	Estruturas de dados e queries	compila
code/ds/distinct_range.cpp	Distintos em intervalo (offline, sweep por r + BIT)	para cada query [l,r] devolve (a) quantidade de valores	Estruturas de dados e queries	compila
code/ds/dsu_paridade.cpp	DSU com paridade (bipartido online / equacoes xor)	mantem, para cada no, a paridade (xor) da distancia ate a raiz	Estruturas de dados e queries	compila
code/ds/dsu_potencial.cpp	DSU com pesos / potencial (soma)	mantem w[x] = pot[x] - pot[raiz]	Estruturas de dados e queries	compila
code/ds/dsu_proximo_livre.cpp	DSU "proximo livre"	nxt(i) = menor j >= i ainda nao removido	Estruturas de dados e queries	compila
code/ds/dsu_rollback.cpp	DSU com rollback	uniao-busca sem compressao de caminho, com desfazer (undo) das	Estruturas de dados e queries	compila
code/ds/fila_monotonica.cpp	Fila/janela monotonica + fila com agregado (duas pilhas)	(1) deque monotono: min de janela deslizante em O(n)	Estruturas de dados e queries	compila
code/ds/heap_lazy.cpp	Heap com remocao preguicosa	priority_queue que aceita erase(x) de um valor presente,	Estruturas de dados e queries	compila
code/ds/heap_mergeavel.cpp	Heap mergeavel (leftist) de minimo	fila de prioridade com uniao de dois heaps em O(log n)	Estruturas de dados e queries	compila
code/ds/kdtree.cpp	K-d tree 2D (estatica): vizinho mais proximo e contagem em retangulo	constroi arvore balanceada alternando eixos (mediana por	Estruturas de dados e queries	compila
code/ds/lichao.cpp	Li Chao tree (minimo de retas / segmentos em x)	insere retas y=a*x+b (ou segmentos) e consulta o minimo em x	Estruturas de dados e queries	testado (brute, 2026-11-11)
code/ds/merge_sort_tree.cpp	Merge sort tree	cada no guarda seu intervalo ordenado	Estruturas de dados e queries	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
`code/ds/odt.cpp`	ODT / Chtholly (set de intervalos com valor constante)	mantem a particao de [0,n) em intervalos de valor constante	Estruturas de dados e queries	compila
`code/ds/ordered_set.cpp`	Ordered set / multiset (pb_ds): rank e k-esimo	set ordenado com order_of_key(x) (quantos < x) e	Estruturas de dados e queries	compila
`code/ds/ps2d.cpp`	Soma de prefixo 2D (prefix sum em matriz)	prefix[i][j] = soma do retangulo [0..i] x [0..j]	Estruturas de dados e queries	compila + testado
`code/ds/seg2d.cpp`	Segtree de segtrees (2D online): update de ponto, soma em retangulo	camada externa iterativa em x, cada no externo tem uma segtree	Estruturas de dados e queries	compila
`code/ds/seg_beats.cpp`	Segtree beats (chmin em intervalo + add em intervalo; soma e max)	a[i] = min(a[i], x) em [l,r), a[i] += v em [l,r), query soma e	Estruturas de dados e queries	compila
`code/ds/seg_descent.cpp`	Segtree com descida: primeiro i>=l com a[i]>=x, ultimo i<=r, k-esimo 1	segtree de max (point update) com first_ge/last_ge em O(log n),	Estruturas de dados e queries	compila
`code/ds/seg_dyn.cpp`	Segtree dinamica (esparsa) com soma, k-esimo e merge de segtrees	segtree sobre dominio [0, U) enorme (ate 1e18), nos criados sob	Estruturas de dados e queries	compila
`code/ds/seg_iter.cpp`	Segtree iterativa (bottom-up) com monoide generico (nao comutativo ok)	update pontual e query de intervalo com qualquer operacao	Estruturas de dados e queries	compila
`code/ds/seg_lazy.cpp`	Segtree com lazy generica (estilo ACL) + descida max_right	update em intervalo (aplica tag F) e query de intervalo (monoide S),	Estruturas de dados e queries	testado (brute, 2026-11-11)
`code/ds/seg_nodes.cpp`	Nos prontos para segtree (max subarray, parenteses, funcao afim)	structs de no + merge para usar com Seg (code/ds/seg_iter.cpp)	Estruturas de dados e queries	compila
`code/ds/seg_persist.cpp`	Segtree persistente (contagem) - k-esimo em [l,r] e contagem <= x	versao i = prefixo a[0..i] inserido numa segtree sobre os valores	Estruturas de dados e queries	testado (brute, 2026-11-11)
`code/ds/seg_t.cpp`	Segtree iterativa (ponto + consulta de intervalo)	arvore de soma com update pontual O(log n) e query de intervalo O(log n)	Estruturas de dados e queries	compila + testado
`code/ds/segtree.cpp`	Segtree lazy (soma em intervalo + soma em range)	arvore recursiva com lazy para somar val em [l, r] e consultar a soma	Estruturas de dados e queries	compila
`code/ds/sparse_min.cpp`	Sparse table para minimo em intervalo estatico	pre-processa um array e responde min(a[l..r]) em O(1)	Estruturas de dados e queries	compila + testado

Arquivo	Nome	Descrição	Seção do catálogo	Status
`code/ds/sparse_table.cpp`	Sparse table (RMQ O(1) para operacoes idempotentes) com indice	min/max estatico em O(1)	Estruturas de dados e queries	compila
`code/ds/sqrt_decomp.cpp`	Sqrt decomposition (exemplo: add em intervalo + contar >= x em intervalo)	blocos de tamanho B	Estruturas de dados e queries	compila
`code/ds/treap_implicito.cpp`	Treap implicito (sequencia) com split/merge, reverse e soma/add lazy	sequencia dinamica	Estruturas de dados e queries	testado (brute, 2026-11-11)
`code/ds/wavelet.cpp`	Wavelet matrix	sobre array estatico de inteiros >= 0: k-esimo menor, contar < x,	Estruturas de dados e queries	testado (brute, 2026-11-11)
flow/				
`code/flow/blossom.cpp`	Blossom (Edmonds) - emparelhamento maximo em grafo geral	matching maximo em grafo nao bipartido, encolhendo ciclos impares (blossoms)	Grafos	compila
`code/flow/dag_path_cover.cpp`	Cobertura minima de caminhos em DAG (disjuntos em vertices, ou com repeticao)	minimo de caminhos que cobrem todos os vertices de um DAG	Grafos	compila
`code/flow/dinic.cpp`	Dinic (fluxo maximo) + recuperacao do corte minimo	fluxo maximo s->t	Grafos	testado (brute, 2026-11-11)
`code/flow/flow_lower_bounds.cpp`	Fluxo com limites inferiores: circulacao viavel e fluxo maximo s->t	arestas com low <= f <= high	Grafos	compila
`code/flow/gale_shapley.cpp`	Gale-Shapley - casamento estavel	n homens e n mulheres com listas de preferencia completas	Grafos	compila
`code/flow/gomory_hu.cpp`	Arvore de Gomory-Hu (Gusfield) - corte minimo entre todos os pares	dado grafo NAO direcionado com capacidades, constroi arvore com n-1 arestas	Grafos	compila
`code/flow/hopcroft_karp.cpp`	Hopcroft-Karp - emparelhamento bipartido maximo	matching maximo em bipartido L x R em fases (BFS de niveis + DFS)	Grafos	testado (brute, 2026-11-11)
`code/flow/hungarian.cpp`	Hungarian - problema de atribuicao de custo minimo	dada matriz de custos n x m (n <= m), atribui cada linha a uma coluna	Grafos	testado (brute, 2026-11-11)

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/flow/konig.cpp	Konig - cobertura minima de vertices e conjunto independente maximo em bipartido	a partir do matching maximo (Hopcroft-Karp embutido) constroi cobertura	Grafos	compila
code/flow/kuhn.cpp	Kuhn - emparelhamento bipartido maximo (com heuristicas)	matching maximo em bipartido L x R por caminhos aumentantes	Grafos	compila
code/flow/matroid_intersection.cpp	Interseccao de matroides (cardinalidade maxima e peso maximo)	maior conjunto S do universo {0..n-1} independente em DUAS matroides, via	Grafos	compila
code/flow/mcmf.cpp	Min-cost max-flow (Dijkstra com potenciais; custos negativos ok se sem ciclo negativo)	fluxo de custo minimo de s a t, com limite opcional de fluxo	Grafos	testado (brute, 2026-11-11)
code/flow/mcmf_spfa.cpp	Min-cost max-flow simples com SPFA (aceita custos negativos sem ciclo negativo)	igual a mcmf.cpp, porem mais curto (bom para colar rapido, grafos pequenos)	Grafos	compila
code/flow/project_selection.cpp	Fecho maximo / selecao de projetos (closure) via corte minimo	escolhe conjunto S de itens maximizando soma dos lucros p[i] (podem ser	Grafos	compila
code/flow/push_relabel.cpp	Push-relabel (highest label + gap) - fluxo maximo	fluxo maximo	Grafos	compila
code/flow/simplex.cpp	Simplex (programacao linear) - estilo KACTL	maximiza c.x sujeito a A x <= b, x >= 0	Grafos	compila
general/				
code/general/annealing.cpp	Simulated annealing e hill climbing	template de recozimento simulado (maximizar score) com T geometrico e limite de tempo	Aleatorização	compila
code/general/buscaternaria.cpp	Busca ternaria (minimo de funcao convexa/unimodal)	tern_double(l, r) acha o x que minimiza func em [l, r] (250 iteracoes)	Busca binária e otimização	compila
code/general/compressao.cpp	Compressao de coordenadas	mapeia valores arbitrarios para 0..k-1 preservando a ordem	Arrays e guloso	compila
code/general/debruijn.cpp	Sequencia de de Bruijn	string ciclica de tamanho k^n contendo toda string de tamanho n sobre alfabeto {0..k-1} exatamente uma vez	Construtivos e interativos	compila
code/general/debug.cpp	Macro de debug e timer	dbg(x, y, ...) imprime nome e valor em cerr (so com -DLOCAL)	Aleatorização	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/general/diff2d.cpp	Prefixos 2D, diferencas 2D e maior submatriz de soma maxima	soma de retangulo em O(1)	Estruturas de dados e queries	compila
code/general/fastio.cpp	Fast IO (leitura/escrita manual com buffer)	leitura de inteiros/strings via fread e escrita via buffer	Como usar quando travar	compila
code/general/gale_ryser.cpp	Matriz 0/1 com somas de linhas e colunas dadas (Gale-Ryser) e canto noroeste	constroi matriz 0/1 com somas r[i] (linhas) e c[j] (colunas), ou diz que nao existe	Grafos	compila
code/general/gen_exemplo.cpp	Gerador de testes (exemplo para stress.sh / stress.bat)	gera um caso aleatorio pequeno determinado pela seed (argv[1])	Amostras de problemas e padrões de ICPC	compila (programa completo)
code/general/grau_seq.cpp	Sequencias de graus: Erdos-Gallai, Havel-Hakimi, arvore com graus dados	testa se uma sequencia e grafica	Construtivos e interativos	compila
code/general/gray_code.cpp	Codigo de Gray	g(i) = i ^ (i>>1) percorre todas as mascaras mudando 1 bit por passo	Construtivos e interativos	compila
code/general/guloso_heap.cpp	Gulosos com heap e arrependimento	(1) jobs com prazo e lucro (1 unidade cada)	Arrays e guloso	compila + testado
code/general/hash_anti.cpp	Hash customizado anti-hack (unordered_map / gp_hash_table)	splitmix64 com semente aleatoria para unordered_map/unordered_set e pb_ds gp_hash_table	Aleatorização	compila
code/general/int128_io.cpp	IO de __int128	le e imprime __int128 (cin/cout nao suportam)	Como usar quando travar	compila
code/general/interativo.cpp	Template interativo + buscas binarias interativas + interator local	funcao de query com flush, busca binaria por query (valor ate 1e18 em ~60 queries),	Construtivos e interativos	compila (standalone)
code/general/intervalos.cpp	Intervalos: sweep de eventos, salas, furar, cobrir, k maquinas	maxima sobreposicao (com tratamento de empates), atribuicao de salas, minimo de pontos para furar todos,	Arrays e guloso	compila
code/general/inversoes.cpp	Contagem de inversoes (merge sort e BIT) e pares com condicao	numero de pares i a[j]	Arrays e guloso	compila
code/general/maioria.cpp	Elemento majoritario: Boyer-Moore, > n/k, e versao aleatorizada	acha candidatos a elemento com frequencia > n/k em O(1)·k de memoria	Arrays e guloso	compila
code/general/mediana_janela.cpp	Mediana dinamica em janela deslizante (dois multisets)	mantem mediana e soma |a_i - mediana| de um multiconjunto com inserir/remover	Arrays e guloso	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/general/meet_middle.cpp	Meet in the middle (subconjuntos, n <= 40)	gera somas das duas metades e combina com ordenacao + two pointers	Arrays e guloso	compila
code/general/perm_ciclos.cpp	Ciclos de permutacao, potencia, paridade, trocas minimas	decompoe permutacao em ciclos	Arrays e guloso	compila
code/general/permutacoes.cpp	Geracao de permutacoes (backtracking)	generatePermutations(nums) devolve todas as permutacoes dos n elementos,	Construtivos e interativos	compila
code/general/pilha_monotonica.cpp	Pilha monotonica (proximo maior/menor, contribuicao, histograma, lexicografico)	prox/ant maior e menor	Arrays e guloso	compila
code/general/rng.cpp	RNG para gerar testes e stress	random_a(n) gera n inteiros uniformes em [l, r]	Aleatorização	compila
code/general/subsets.cpp	Geracao de todos os subconjuntos (por mascara)	subsets(nums) devolve os 2^n subconjuntos de nums (cada um como vetor)	Construtivos e interativos	compila
code/general/template.cpp	Template base para colar no inicio do arquivo	includes, macros (FORI, FORJ, endl, pb, all, pii), solve() vazio e main	Como usar quando travar	compila (programa completo)
code/general/torneio_landau.cpp	Torneio com pontuacoes dadas (Landau)	decide se uma sequencia de pontuacoes e de um torneio e constroi o torneio	Construtivos e interativos	compila
code/general/two_pointers.cpp	Two pointers / janela deslizante (templates)	(1) conta subarrays "bons" quando a propriedade e monotonica (encolher nunca piora)	Arrays e guloso	compila
geometry/				
code/geometry/angle_sort.cpp	Sweep angular: contar pontos em semiplano e triangulos que contem a origem	ordena pontos por angulo em torno da origem (exato, com cross) e usa dois	Geometria	compila
code/geometry/calipers.cpp	Rotating calipers: diametro, largura minima, retangulo envolvente de menor area	sobre um fecho convexo CCW (de hull_andrew.cpp) acha o par mais distante,	Geometria	compila
code/geometry/circles.cpp	Circulos: interseções, tangentes, area de interseção com poligono, area da uniao	circulo-reta, circulo-circulo, tangentes de ponto e comuns, area de interseção	Geometria	compila
code/geometry/closest_pair.cpp	Par de pontos mais proximo (sweep + set)	menor distancia^2 entre dois pontos (e os indices)	Geometria	compila
code/geometry/convex_extreme.cpp	Vertice extremo de poligono convexo numa direcao (busca binaria O(log n))	dado poligono convexo CCW e direcao dir, devolve o indice do vertice que maximiza	Geometria	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/geometry/convex_layers.cpp	Camadas convexas (convex layers / onion peeling) e frente de Pareto 2D	layer[i] = indice da camada do ponto i (0 = fecho externo)	Geometria	compila
code/geometry/convexhull.cpp	Fecho convexo (monotone chain)	convexHull(pontos) devolve o hull em ordem anti-horaria, sem pontos	Geometria	compila
code/geometry/geometry3d.cpp	Geometria 3D basica + fecho convexo 3D (incremental O(n^2))	struct P3 (cross, dot), volume de tetraedro (produto misto), distancia ponto-plano,	Geometria	compila
code/geometry/half_plane.cpp	Interseção de semiplanos (HPI) O(n log n)	acha o poligono convexo (vertices CCW) = interseção de semiplanos "a esquerda de p->p+d"	Geometria	compila
code/geometry/hull_andrew.cpp	Fecho convexo (Andrew monotone chain) - versao correta	devolve os vertices do fecho convexo em ordem anti-horaria, comecando pelo	Geometria	compila
code/geometry/min_circle.cpp	Circulo minimo envolvente (Welzl iterativo) e circuncentro	menor circulo contendo todos os pontos	Geometria	compila
code/geometry/minkowski.cpp	Soma de Minkowski de poligonos convexos	A + B = {a+b}	Geometria	compila
code/geometry/point_dbl.cpp	Ponto/vetor em ponto flutuante (long double) + primitivas	struct Pd com operadores, rotacao, projecao, reflexao, interseção de retas,	Geometria	compila
code/geometry/point_int.cpp	Ponto/vetor inteiro + primitivas exatas	struct P (ll) com cross, dot, orientacao, ordenacao angular exata,	Geometria	compila
code/geometry/polygon.cpp	Poligonos: area, centroide, Pick, ponto em poligono (geral e convexo O(log n)), pontos de rede	area*2 sinalizada (shoelace), centroide, contagem de pontos de rede (Pick),	Geometria	compila
code/geometry/polygon_cut.cpp	Corte de poligono por semiplano (Sutherland-Hodgman)	devolve a parte do poligono que fica a esquerda (ou sobre) da reta orientada a->b	Geometria	compila
code/geometry/rect_union.cpp	Area e perimetro da uniao de retangulos (sweep + segtree)	area da uniao de retangulos alinhados aos eixos	Geometria	compila
code/geometry/seg_any_inter.cpp	Existe interseção entre n segmentos? (Shamos-Hoey, sweep + set)	acha um par de segmentos que se intersectam (inclui toque), ou {-1,-1}	Geometria	compila
graph/				

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/graph/bcc_block_cut.cpp	BCC (componentes biconexas por vertices) e Block-Cut Tree	separa o grafo em blocos (BCCs)	Grafos	compila
code/graph/bellman_ford.cpp	Bellman-Ford (ciclo negativo, -infinito, ciclo explicito)	distancias com pesos negativos	Grafos	compila
code/graph/bfs.cpp	BFS (busca em largura)	percorre o grafo nao ponderado a partir de start, marcando visitados e	Grafos	compila
code/graph/bfs01.cpp	0-1 BFS e BFS com pesos pequenos (Dial)	menor distancia com pesos em {0,1} usando deque (bfs01), ou com pesos	Grafos	compila
code/graph/bipartite.cpp	Teste de bipartido / 2-coloracao com ciclo impar	colore o grafo com 2 cores por BFS	Grafos	compila
code/graph/bridges_2ecc.cpp	Pontes, articulacoes e componentes 2-aresta-conexas (bridge tree)	acha pontes e vertices de articulacao	Grafos	compila
code/graph/ciclos.cpp	Deteccao de ciclos: orientado, nao orientado, girth (menor ciclo nao ponderado)	find_cycle_dir devolve um ciclo orientado	Grafos	compila
code/graph/dijkstra.cpp	Dijkstra (menores caminhos de fonte unica, pesos nao negativos)	devolve dist[] com a menor distancia de start a todos os vertices	Grafos	compila
code/graph/dijkstra_estados.cpp	Dijkstra em grafo implicito / de estados (grid, k cupons, camadas)	Dijkstra generico onde os vizinhos sao gerados por uma lambda	Grafos	compila
code/graph/dijkstra_variantes.cpp	Dijkstra: variantes (denso O(n^2), gargalo/minimax, contagem de caminhos, arestas em	dij_dense (sem heap)	Grafos	compila
code/graph/dsu.cpp	DSU (union-find com path compression + union by size)	find(u) com compressao	Grafos	compila
code/graph/euler.cpp	Caminho/circuito euleriano (Hierholzer) - nao direcionado e direcionado	devolve a sequencia de vertices de um circuito/trilha que usa cada aresta	Grafos	compila
code/graph/floyd.cpp	Floyd-Warshall (distancias, ciclo negativo, caminho, ciclo minimo)	todos-os-pares em O(n^3)	Grafos	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/graph/grafo_funcional.cpp	Grafo funcional (cada vertice tem exatamente uma aresta de saida)	acha os ciclos, e para cada vertice: se esta no ciclo, id do ciclo, posicao,	Grafos	compila
code/graph/johnson.cpp	Johnson (todos-os-pares com pesos negativos, esparso)	Bellman-Ford de super-fonte para achar potenciais h, reponderar	Grafos	compila
code/graph/kruskal.cpp	Kruskal: arvore geradora minima	calcula a MST de um grafo nao direcionado e diz se ele e conexo	Grafos	compila
code/graph/kth_walks.cpp	K menores caminhos (walks) de s a t	devolve os k menores comprimentos de walks (vertices/arestas podem repetir)	Grafos	compila
code/graph/range_graph.cpp	Grafo com arestas para intervalos (segtree de arestas) + Dijkstra	permite arestas u -> [l,r] e [l,r] -> v com custo w usando duas arvores	Grafos	compila
code/graph/scc_kosaraju.cpp	SCC (Kosaraju iterativo)	componentes fortemente conexas	Grafos	compila
code/graph/scc_tarjan.cpp	SCC (Tarjan iterativo) + condensacao	componentes fortemente conexas	Grafos	testado (brute, 2026-11-11)
code/graph/spfa.cpp	SPFA (Bellman-Ford com fila)	menor caminho com pesos negativos	Grafos	compila
code/graph/toposort.cpp	Ordenacao topologica (Kahn)	devolve uma ordem topologica de um DAG	Grafos	compila
code/graph/two_sat.cpp	2-SAT	decide satisfatibilidade de clausulas de 2 literais e devolve uma atribuicao	Grafos	testado (brute, 2026-11-11)
math/				
code/math/bsgs.cpp	Logaritmo discreto (BSGS) para modulo qualquer	menor x >= 0 com a^x = b (mod m), ou -1 se nao existe	Teoria dos números e divisores	compila
code/math/crt.cpp	CRT geral (modulos nao coprimos) e Garner	crt_merge combina x=a1 (m1) e x=a2 (m2) em x=r (lcm)	Teoria dos números e divisores	compila
code/math/divisores.cpp	Divisores: listar, formulas, altamente compostos, serie geometrica	divisores de n por sqrt	Teoria dos números e divisores	compila
code/math/exgcd.cpp	Euclides estendido, diofantina linear, contagem de solucoes em caixa	exgcd	Teoria dos números e divisores	compila
code/math/floor_sum.cpp	Floor sum (algoritmo tipo Euclides)	floor_sum = sum_{i=0}^{n-1} floor((a*i+b)/m) exato (cabe em ll se	Teoria dos números e divisores	compila
code/math/inv_lote.cpp	Inversos modulares em lote	inv[1..n] mod p linear	Teoria dos números e divisores	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/math/miller_rabin.cpp	Miller-Rabin deterministico (64 bits)	testa primalidade de n < 2^64 de forma exata	Teoria dos números e divisores	compila
code/math/mint.cpp	mint: struct de inteiro modular	aritmetica mod MOD (primo) com operadores, pow e inverso	Teoria dos números e divisores	compila
code/math/mod.cpp	Aritmetica modular e utilidades de inteiros	gcd, lcm, soma/subtracao/multiplicacao mod MOD, exponenciacao e inverso	Teoria dos números e divisores	compila
code/math/ntt.cpp	NTT (Number Theoretic Transform) modulo 998244353	convolucao/multiplicacao de polinomios com raiz primitiva 3	Álgebra, polinômios e FFT	compila
code/math/pollard_rho.cpp	Pollard rho (Brent) + fatoracao completa	fatora n < 2^63 (~9e18	Teoria dos números e divisores	testado (brute, 2026-11-11)
code/math/primitive_root.cpp	phi, lambda de Carmichael, ordem multiplicativa, raiz primitiva	phi(n) e lambda(n) por trial division	Teoria dos números e divisores	compila
code/math/sieve.cpp	Crivo de Eratostenes	marca is_prime[0..MAXN] e guarda todos os primos em primes	Teoria dos números e divisores	compila
code/math/sieve_linear.cpp	Crivo linear (menor fator primo, phi, mu, tau, sigma)	em O(N) calcula lp[i] (menor fator primo), lista de primos, phi, mu,	Teoria dos números e divisores	compila
code/math/sieve_seg.cpp	Crivo segmentado (primos e phi em [L, R])	primos em [L,R] e phi de cada numero do intervalo, usando so os	Teoria dos números e divisores	compila
code/math/tonelli.cpp	Raiz quadrada modular (Tonelli-Shanks) e simbolo de Jacobi	x com x^2 = a (mod p), p primo	Teoria dos números e divisores	compila
string/				
code/string/aho_corasick.cpp	Aho-Corasick	automato de varios padroes	Strings	compila
code/string/bitset_match.cpp	Matching e LCS com bitset	(1) ocorrencias de p em t por bitset (util quando ha variacoes tipo	Strings	compila + testado
code/string/eertree.cpp	Eertree (arvore de palindromos) + particao em palindromos (serie link)	no por palindromo distinto (<= n nos)	Strings	compila
code/string/hash.cpp	Hash polinomial de string (prefix hash)	StringHash(s) monta o hash dos prefixos	Strings	compila
code/string/hash2d.cpp	Hash 2D de submatrizes (mod 2^61-1)	hash de qualquer submatriz em O(1)	Strings	compila
code/string/hash61.cpp	Hash polinomial mod 2^61-1 (substrings, LCP, palindromo, concatenacao)	hash de substring em O(1) com base aleatoria	Strings	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/string/kmp.cpp	Prefix function (KMP), automato KMP, bordas, periodos	pi[i] = maior borda propria de s[0..i]	Strings	testado (brute, 2026-11-11)
code/string/lyndon.cpp	Duval (fatoracao de Lyndon) e rotacao minima	duval(s) fatora s em palavras de Lyndon nao-crescentes	Strings	compila
code/string/manacher.cpp	Manacher (palindromos)	d1[i] = nro de palindromos de tamanho impar centrados em i (raio incl	Strings	testado (brute, 2026-11-11)
code/string/suffix_array.cpp	Suffix array + LCP + RMQ de LCP	sa (sufixos ordenados), rnk, lcp[i] = LCP(sa[i-1], sa[i]) (lcp[0]=0),	Strings	testado (brute, 2026-11-11)
code/string/suffix_automaton.cpp	Suffix automaton (SAM), incluindo generalizado	automato minimo dos sufixos	Strings	testado (brute, 2026-11-11)
code/string/trie.cpp	Trie (arvore de prefixos) para minusculas	insert(p) insere a palavra p	Strings	compila + testado
code/string/zfunc.cpp	Z-function	z[i] = LCP de s e s[i..]	Strings	testado (brute, 2026-11-11)
tree/				
code/tree/centroid.cpp	Centroid decomposition (arvore de centroides + distancias por nivel)	constroi a arvore de centroides (altura <= log n)	Árvores	compila
code/tree/centroid_paths.cpp	Contar pares (u<v) com dist(u,v) <= K via centroide (arestas ponderadas >= 0)	para cada centroide junta as distancias da componente, conta pares com	Árvores	compila
code/tree/hld.cpp	Heavy-Light Decomposition + segtree sobre monoide (caminho e subarvore)	decompoe a arvore em cadeias pesadas	Árvores	testado (brute, 2026-11-11)
code/tree/hld_bit.cpp	HLD com BIT range-add / range-sum (soma e add em caminho e subarvore)	add x em todos os vertices de um caminho / subarvore e soma sobre	Árvores	compila
code/tree/lca_lifting.cpp	LCA com binary lifting (+ k-esimo ancestral, distancia, k-esimo no caminho)	pre-processa pai 2^k de cada vertice	Árvores	testado (brute, 2026-11-11)
code/tree/lca_sparse.cpp	LCA O(1) com sparse table sobre a ordem de DFS (n posicoes)	LCA em O(1) por query	Árvores	compila
code/tree/lca_tarjan.cpp	LCA offline de Tarjan (DFS + DSU)	responde um lote de queries (u,v) de LCA sem tabela de saltos	Árvores	compila